

Software Analysis

Program Slicing

Gordon Fraser
Lehrstuhl für Software Engineering II

Outline

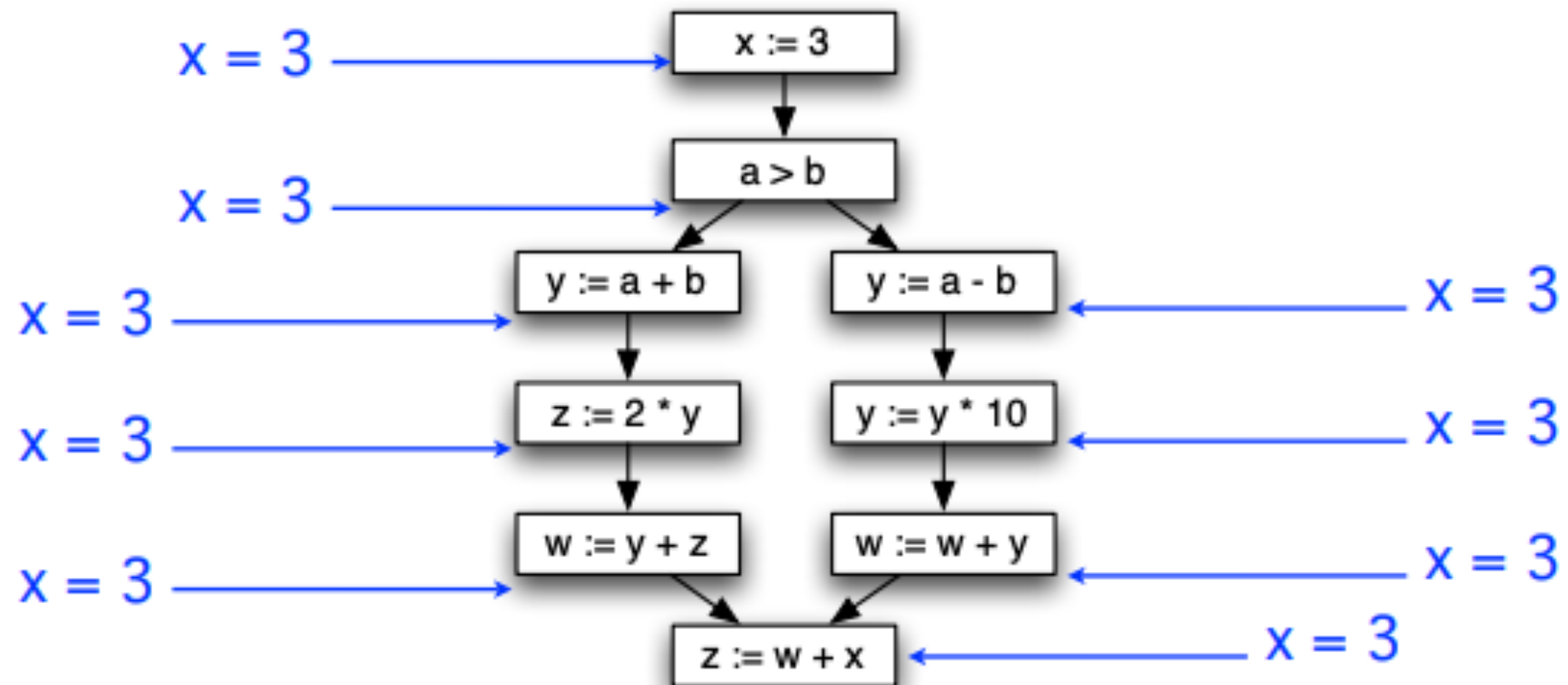
1. Static Single Assignment Form
2. Slicing
3. Interprocedural Slicing
4. Dynamic Slicing

Static Single Assignment

Motivation

- Data flow analysis needs to represent facts at every program point
- Each analysis needs several passes over the CFG
- What if
 - There are a lot of facts and
 - There are a lot of program points?
 - → potentially takes a lot of space/time
- Most likely, we're keeping track of irrelevant facts

Example



Sparse Representation

- A **sparse** representation can propagate facts about a variable x only where they're needed
- The **static single assignment** form is such a sparse representation for which each **use** of variable has a **single** definition
- Each variable is assigned exactly once but may be used multiple times

Another Example

$y := 1$

$y := 2$

$x := y$

Is the **first** assignment necessary?

Another Example

$y := 1$

$y := 2$

$x := y$

Is the **first** assignment necessary?

A program would have to perform reaching definitions analysis to determine that it is not.

Another Example

$y := 1$

$y := 2$

$x := y$

$a := 1$

$b := 2$

$c := b$

Is the **first** assignment necessary?

A program would have to do a simpler analysis to determine that.

Another Example

$y := 1$

$y := 2$

$x := y$

$y_1 := 1$

$y_2 := 2$

$x_1 := y_2$

$a := 1$

$b := 2$

$c := b$

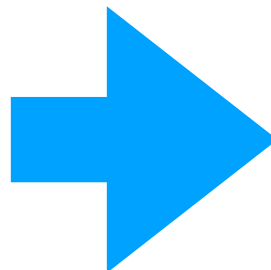
Is the **first** assignment necessary?

A program would have to do a simpler analysis to determine that.

Static Single Assignment (SSA)

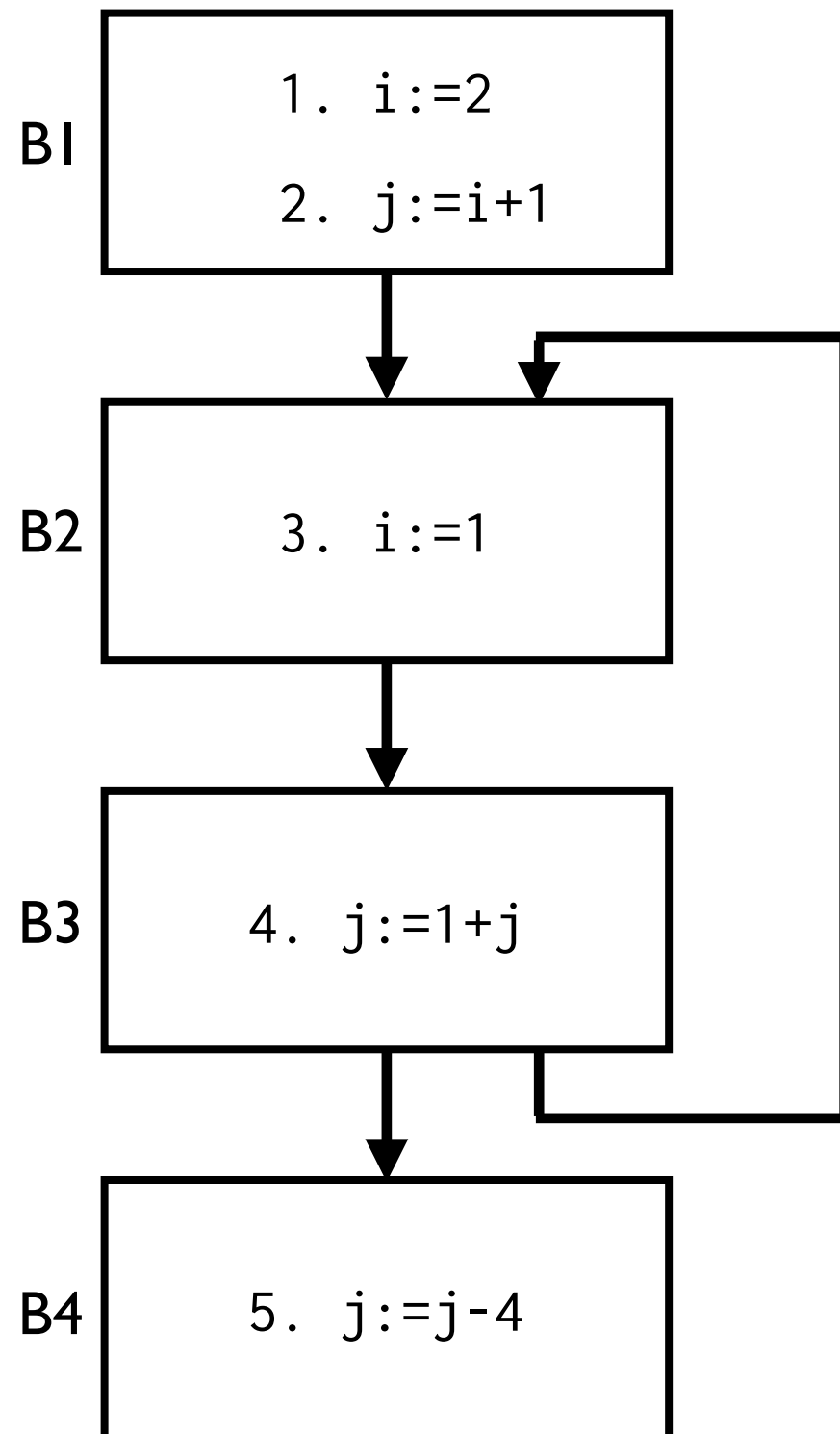
- An intermediate representation in which every variable assigned a value occurs as the target of only one assignment.
- Makes some **optimizations** more effective.
- For basic blocks is easy:
 - Assign to a **fresh** variable at each stmt.
 - Each use uses the **most recently** defined var.

```
a := x + y
b := a + x
a := b + 2
c := y + 1
a := c + a
```

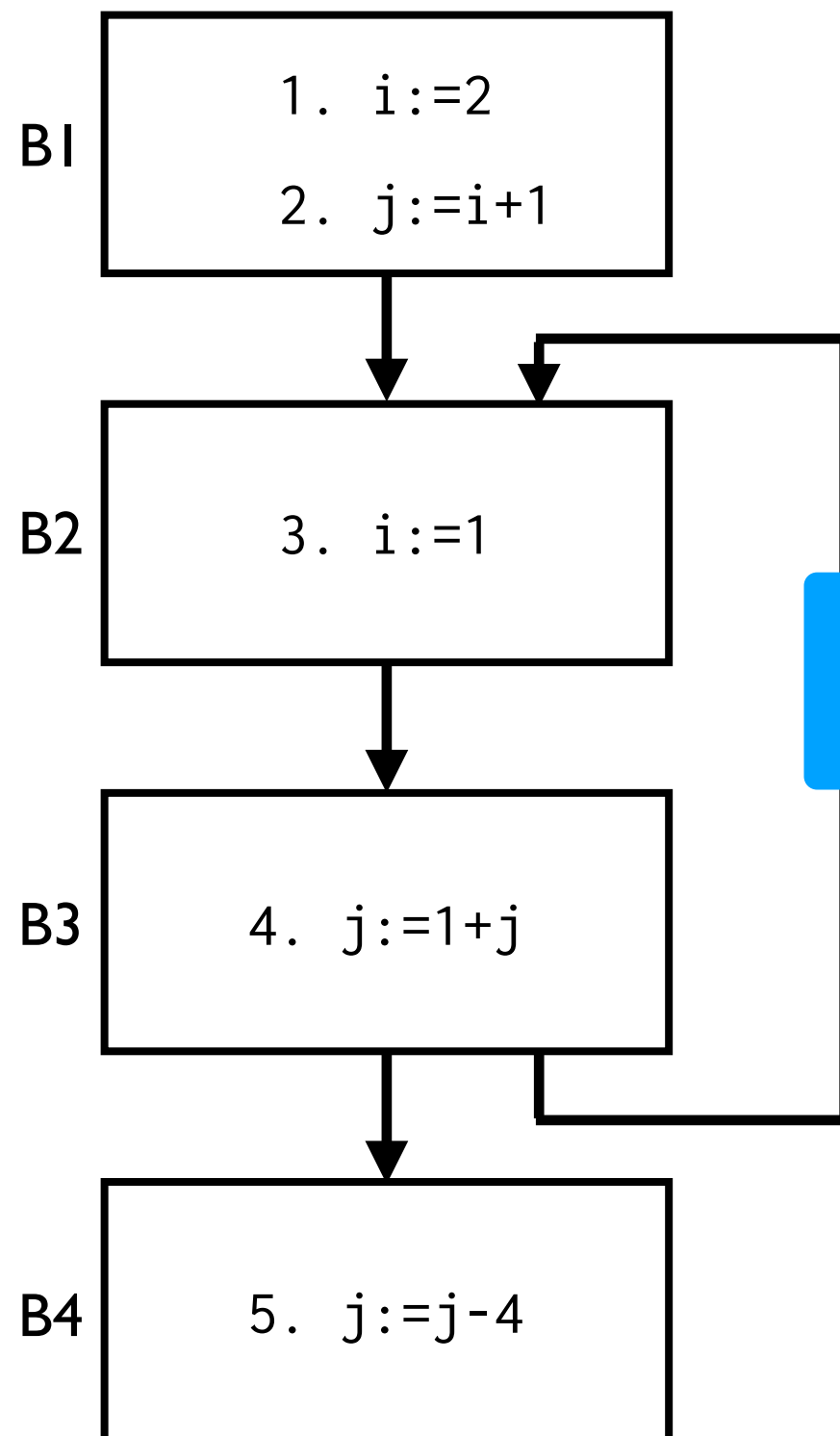


```
a1 := x + y
b1 := a1 + x
a2 := b1 + 2
c1 := y + 1
a3 := c1 + a2
```

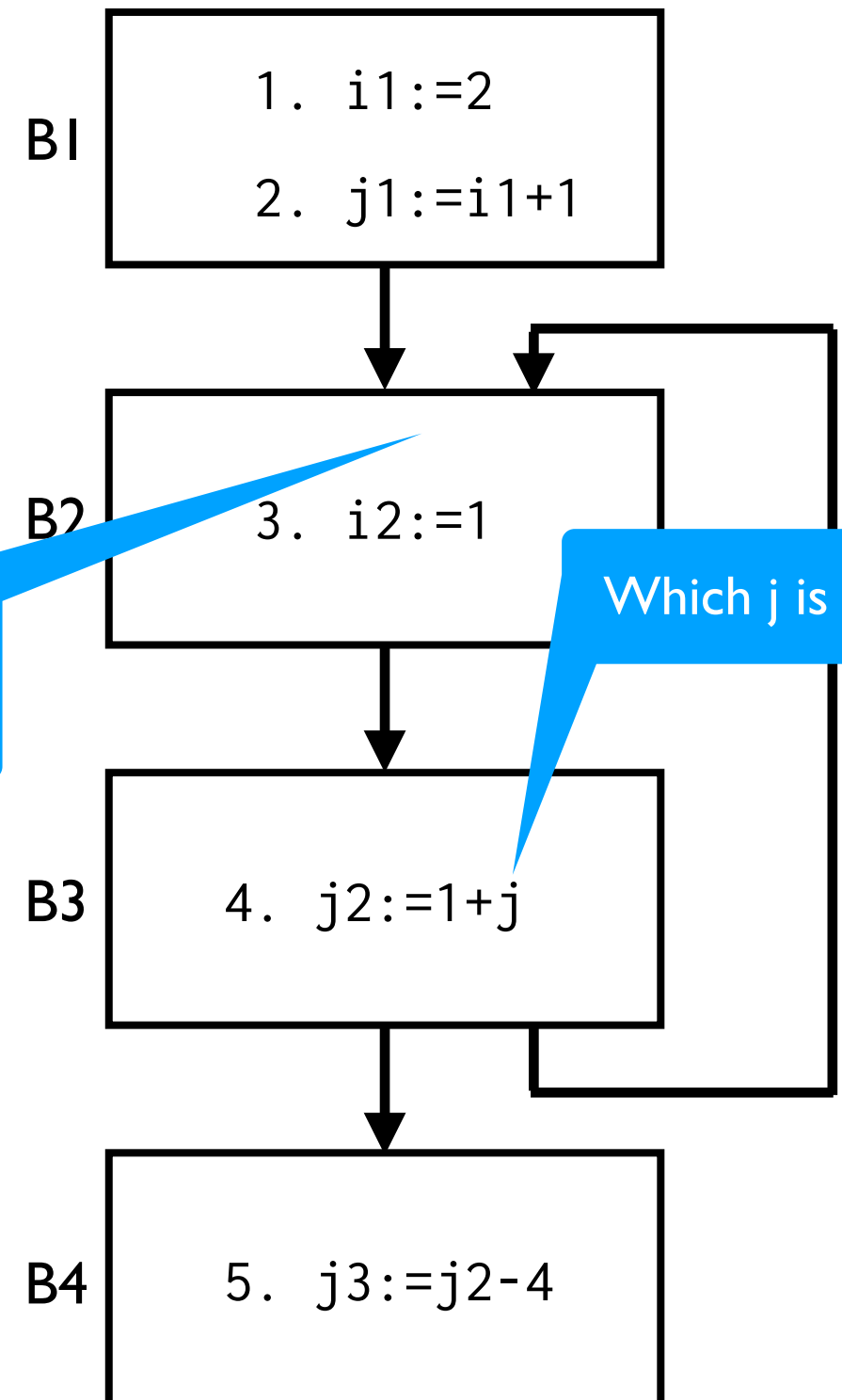
Static Single Assignment (SSA)



Static Single Assignment (SSA)

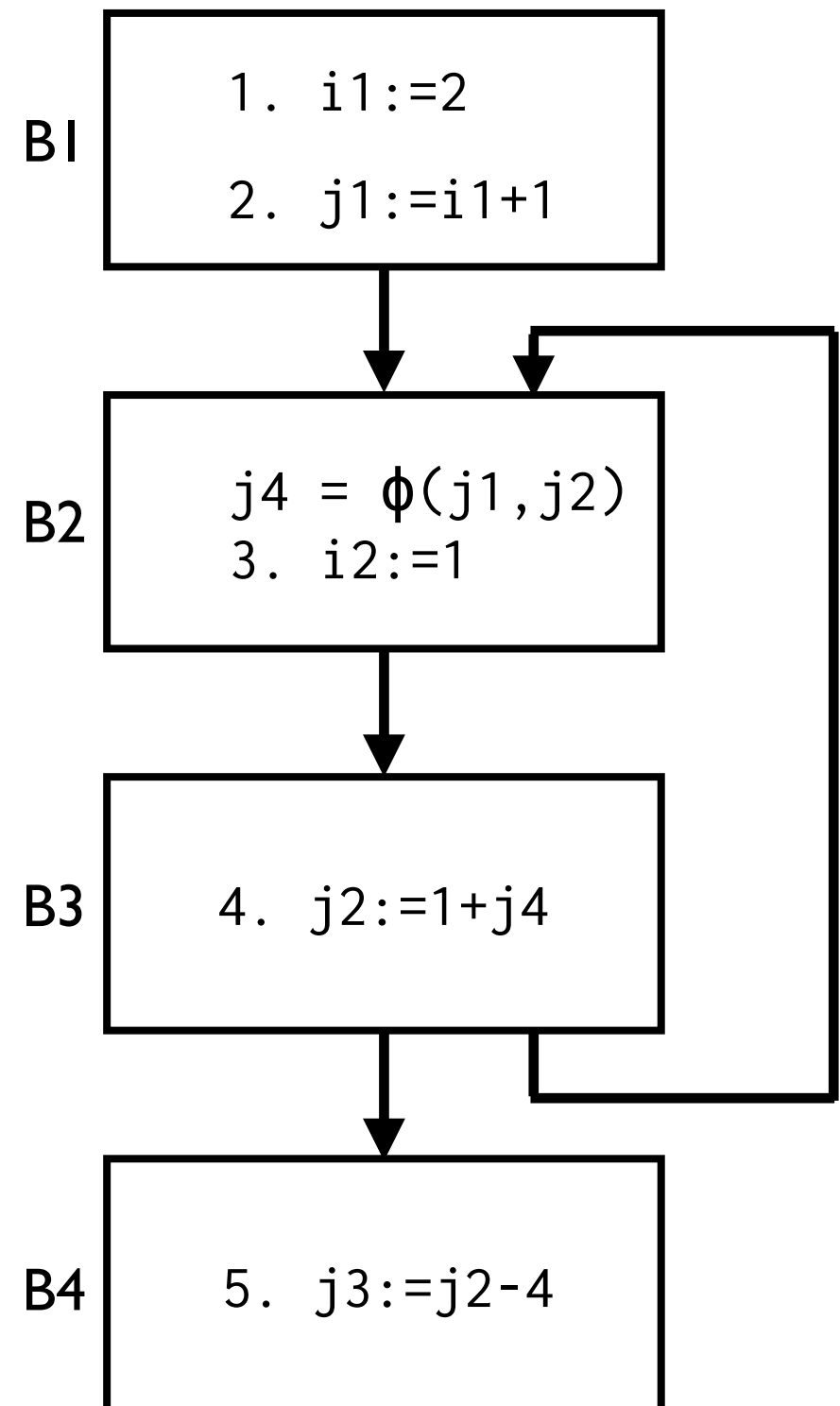
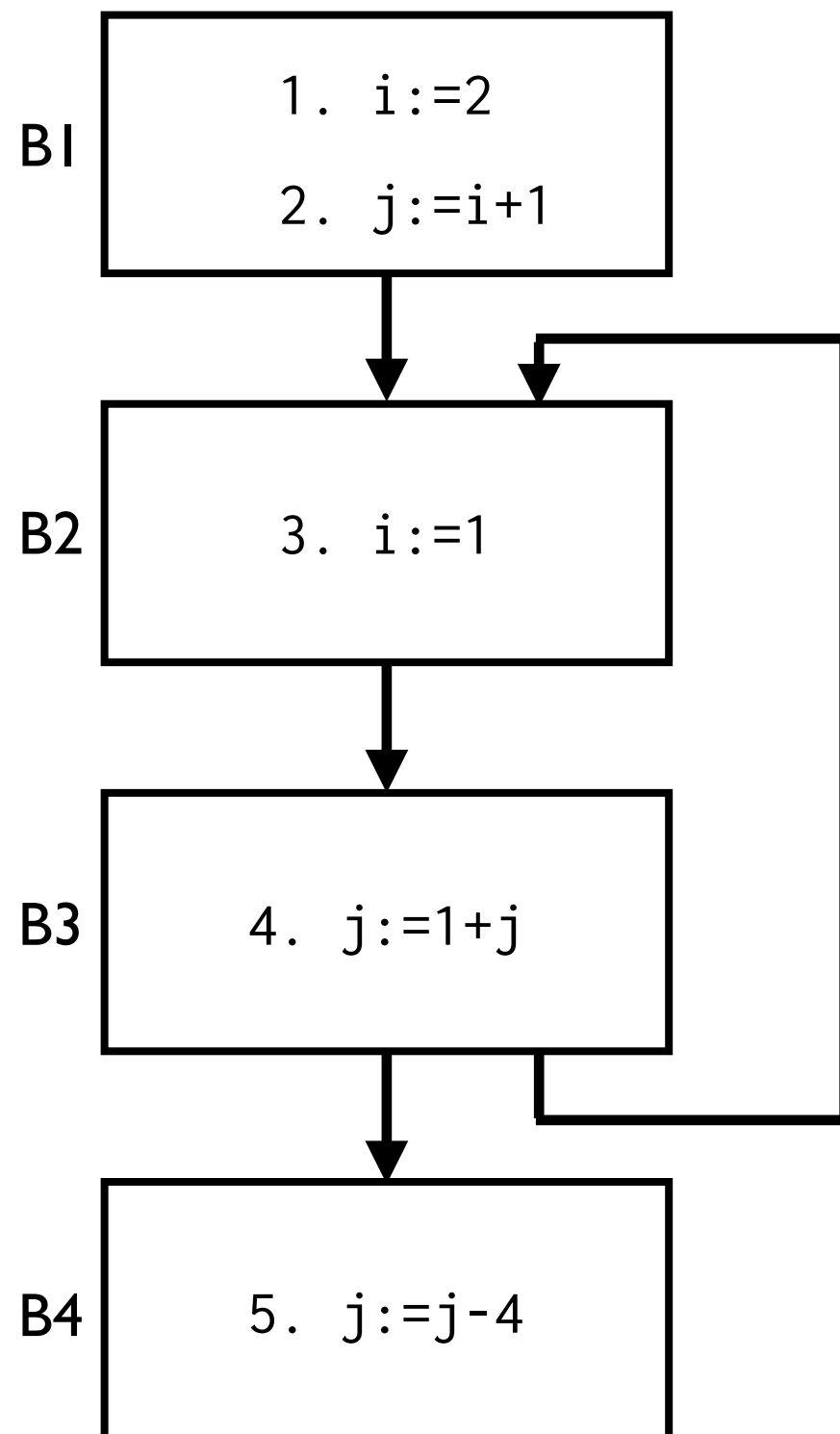


Need a ϕ function

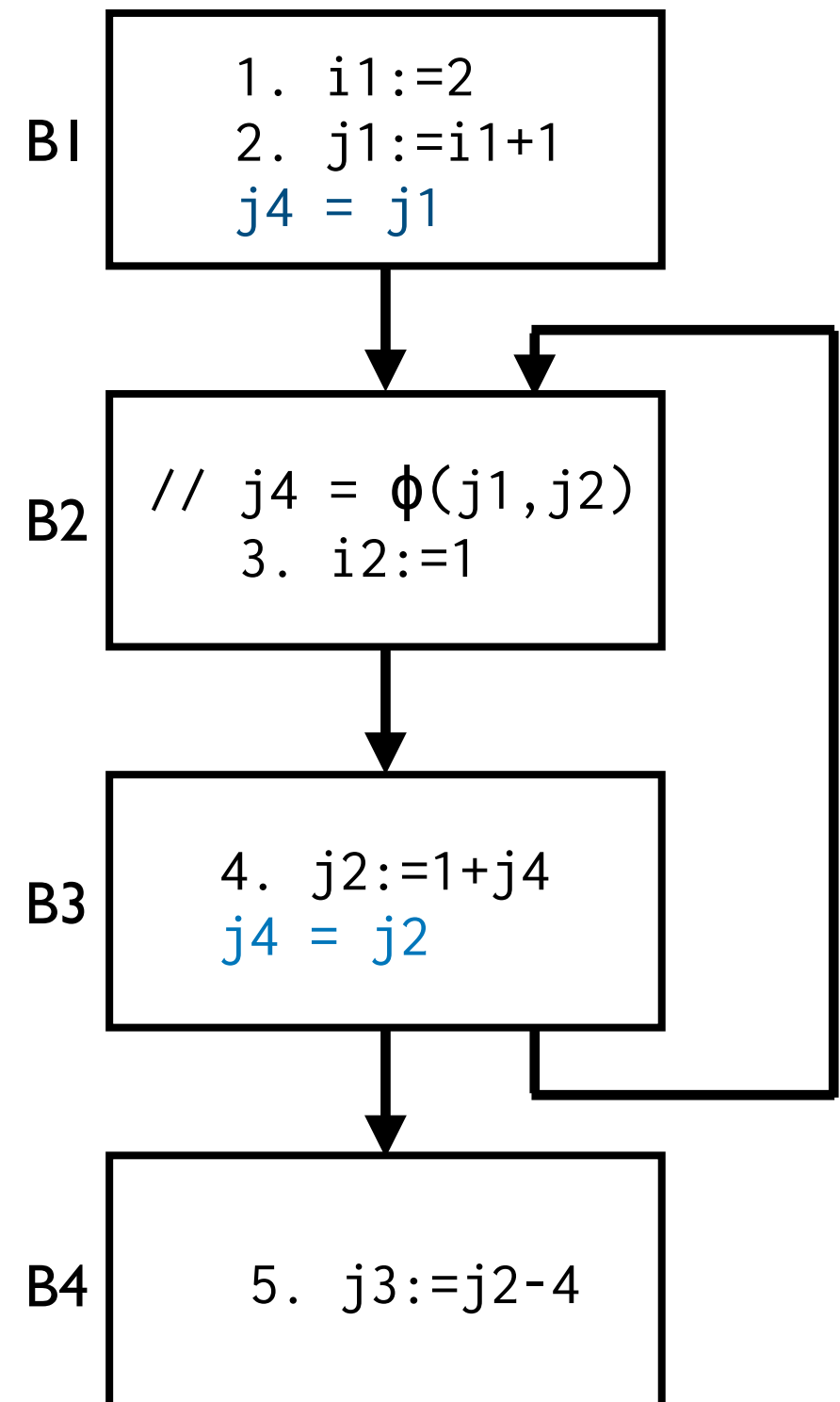
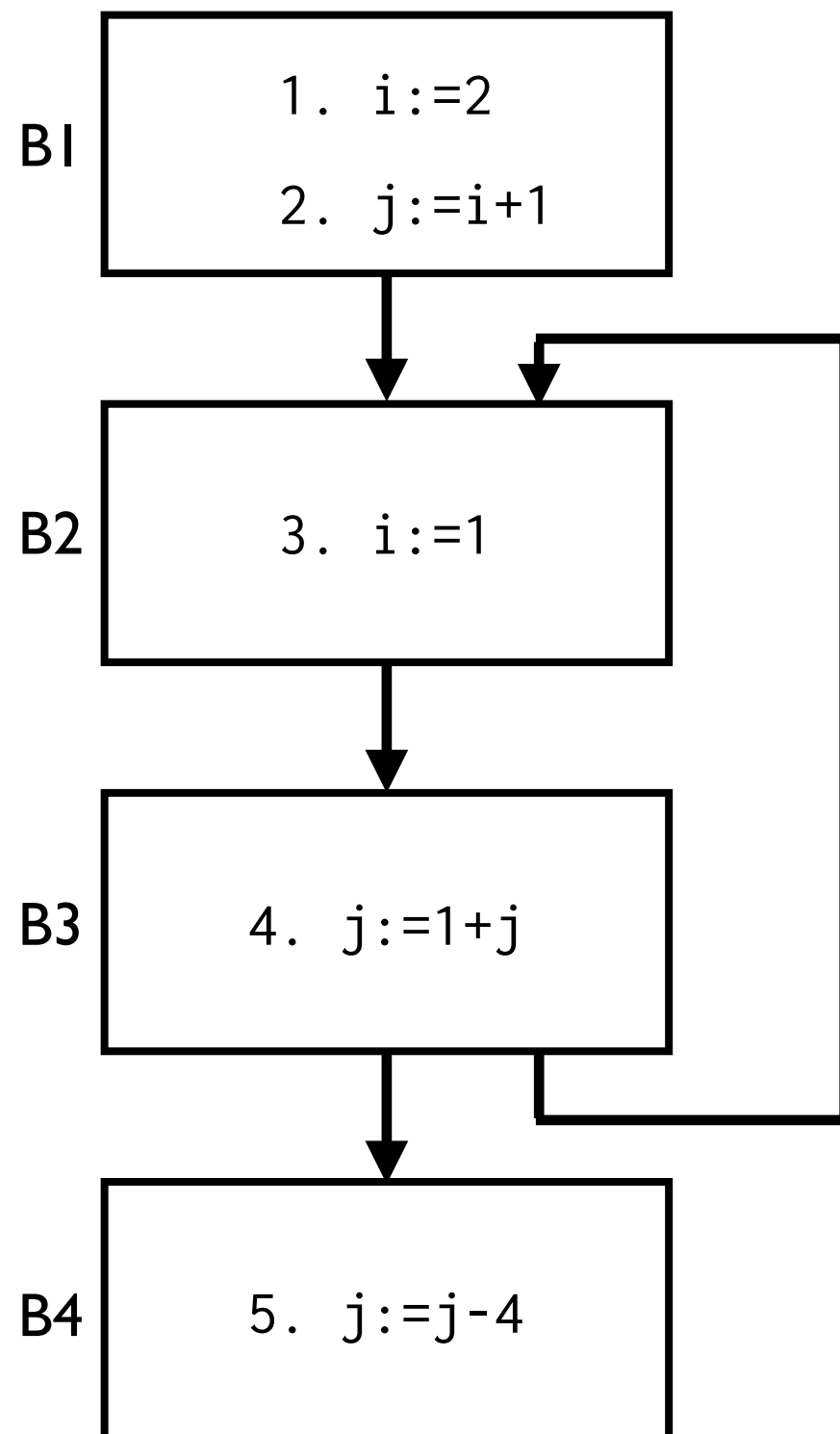


Which `j` is this?

Static Single Assignment (SSA)



Static Single Assignment (SSA)

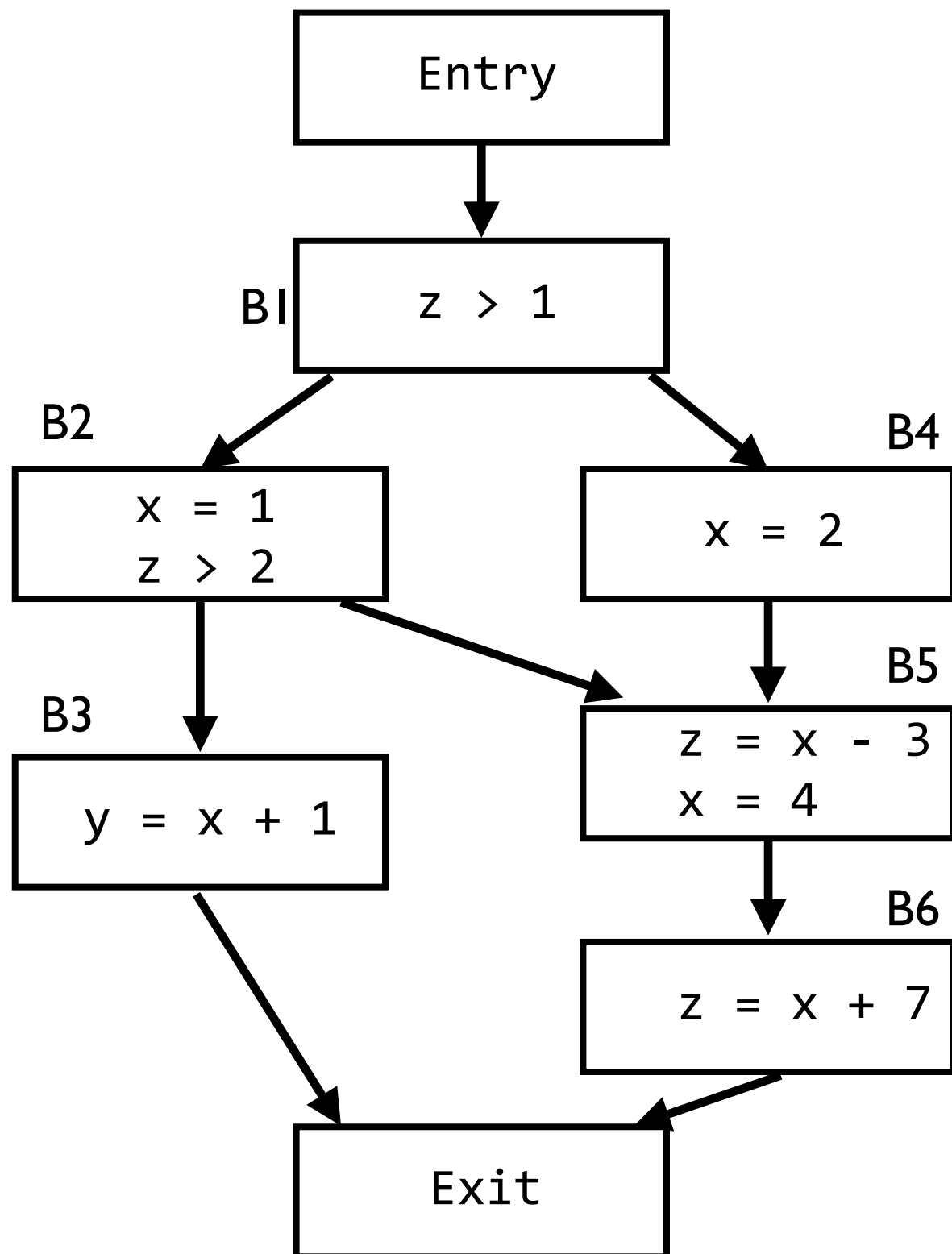


Rewrite program so that at runtime, it will use the correct definition of `j`

Computing SSA Form

- Step 1: Place Φ nodes
- Step 2: Rename variables so only one definition per name

Where to Place ϕ Functions



Use dominance frontier

Computing SSA Form

- Step 1: Place Φ nodes
 - 1a: Compute the dominance frontier
 - 1b: Place a Φ function for variable $\underline{a}$ in dominance frontier of node X if X contains an assignment to $\underline{a}$
- Step 2: Rename variables so only one definition per name

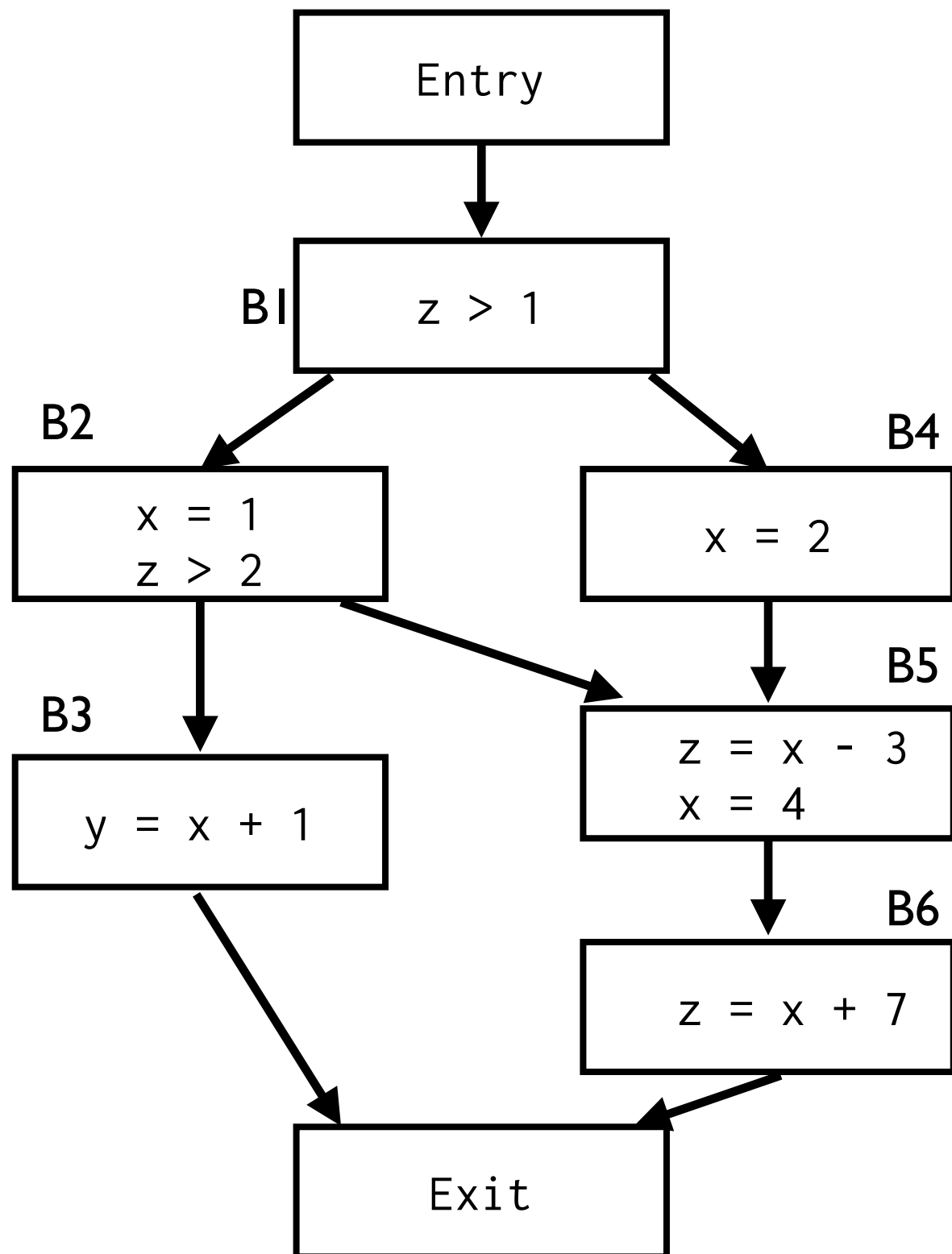
Computing SSA Form

- Step 1: Place Φ nodes
 - 1a: Compute the dominance frontier
 - 1b: Place a Φ function for variable $\underline{a}$ in dominance frontier of node X if X contains an assignment to $\underline{a}$
- Step 2: Rename variables so only one definition per name

Dominance Frontiers

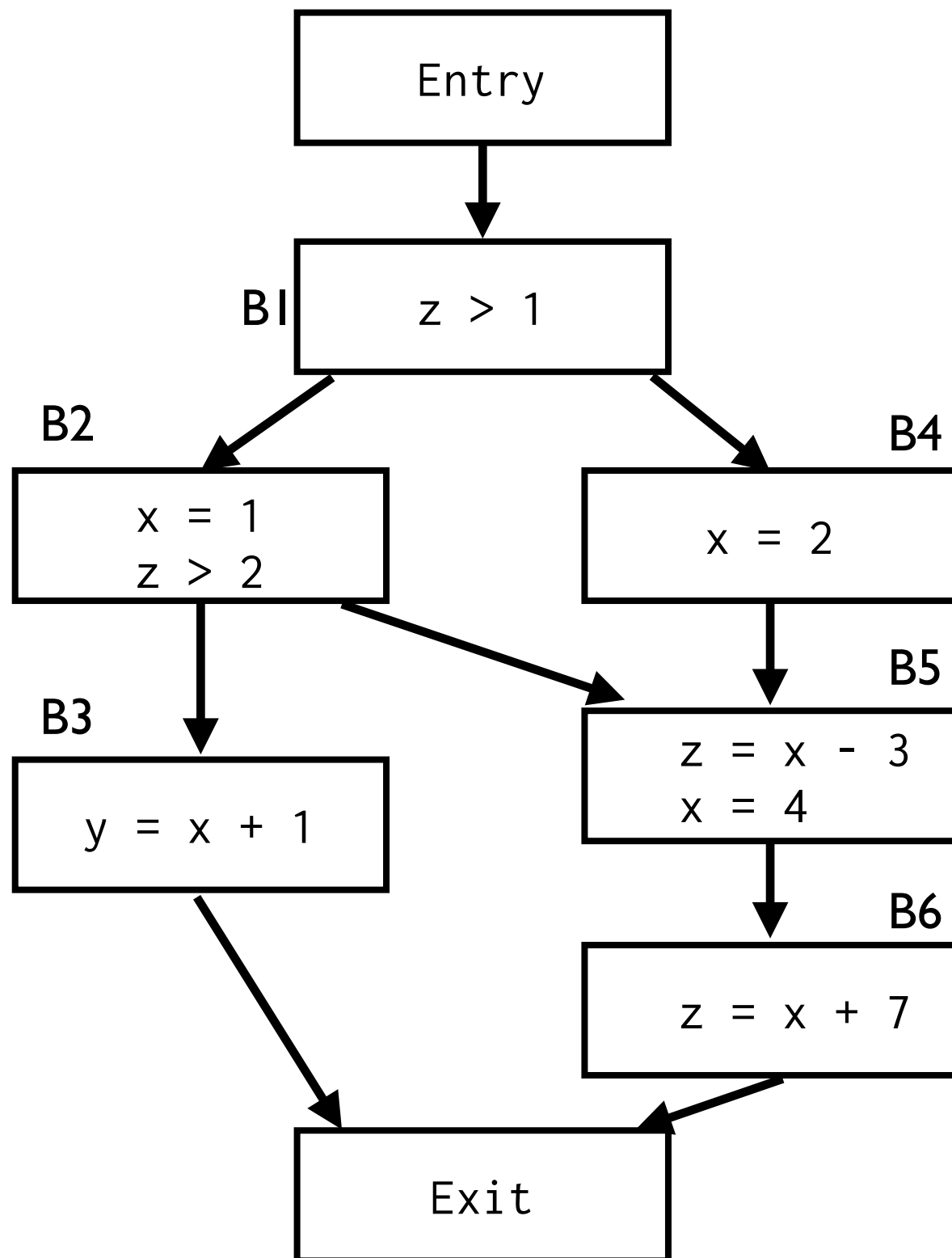
- Dominance frontier $DF(X)$ is the set of all nodes Y in the CFG such that X dominates an immediate predecessor of Y but does not strictly dominate Y
- X is a dominator of Y ($X \in Dom(Y)$), if X appears on all paths from `Entry` to Y
- Being a dominator is **reflexive** so $Y \in Dom(Y)$
- X is a **strict dominator** of Y if $X \in Dom(Y) - \{Y\}$

Dominance Frontiers



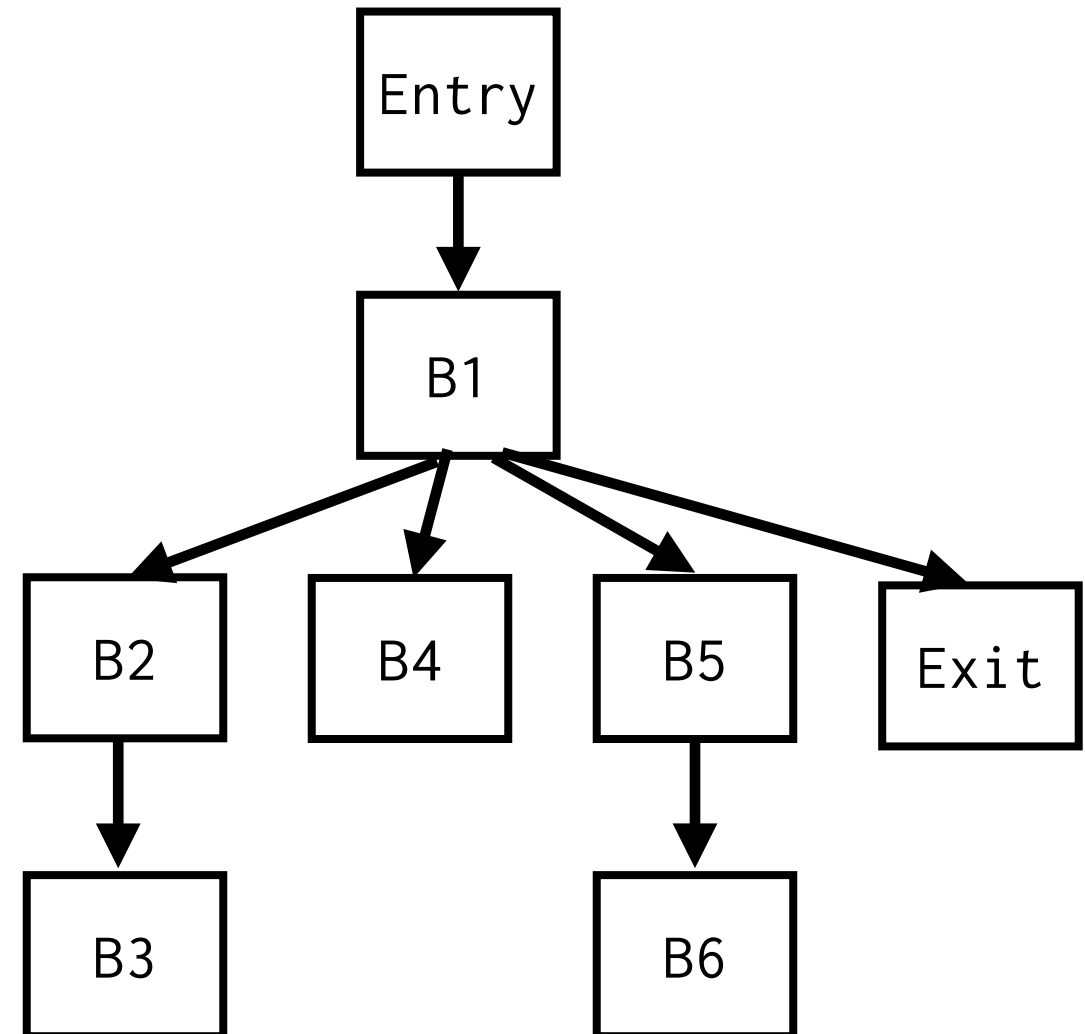
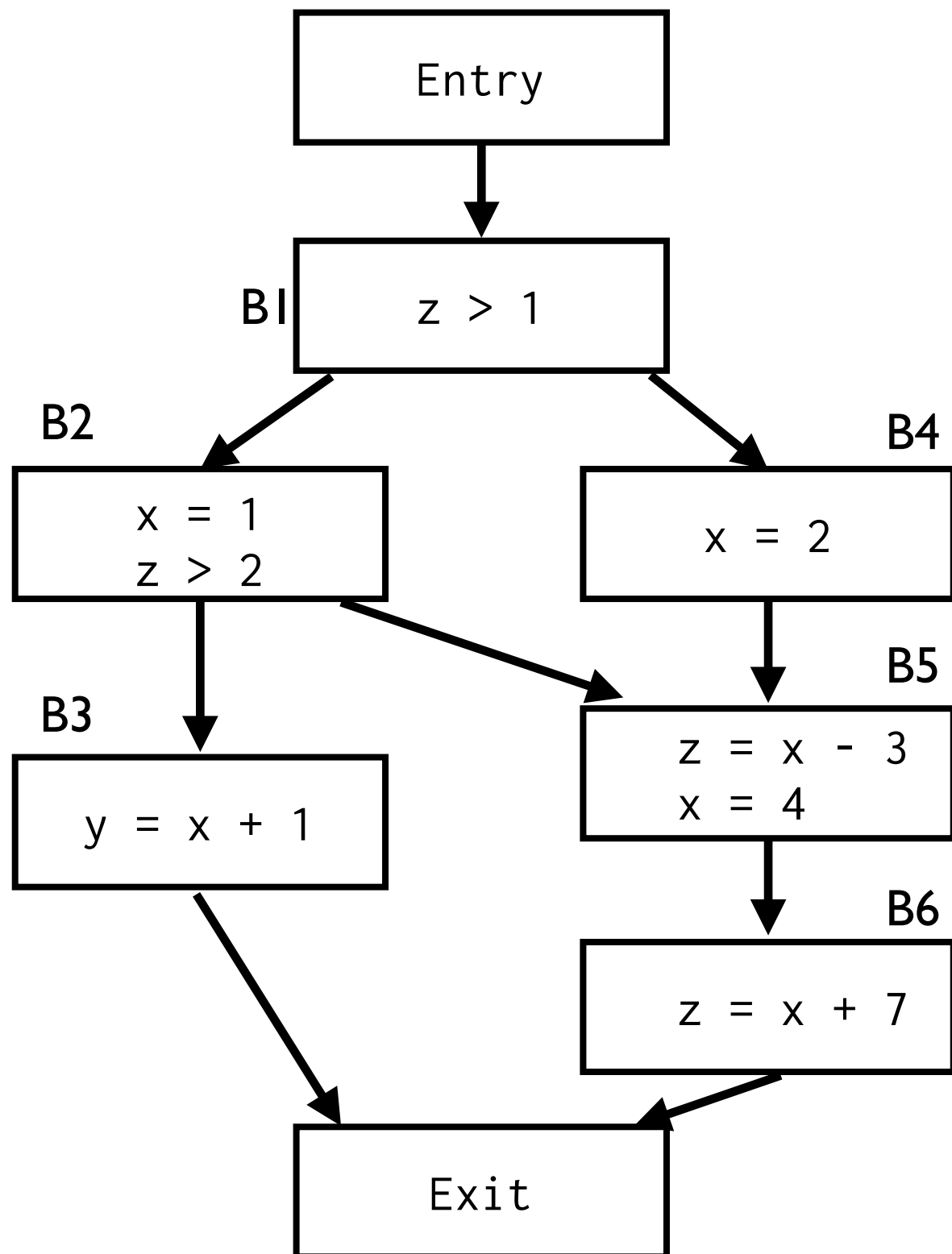
For a CFG and node x , the dominance frontier $DF(x)$ is the set of all nodes y in the CFG such that x dominates an immediate predecessor of y but does not strictly dominate y

Dominance Frontiers

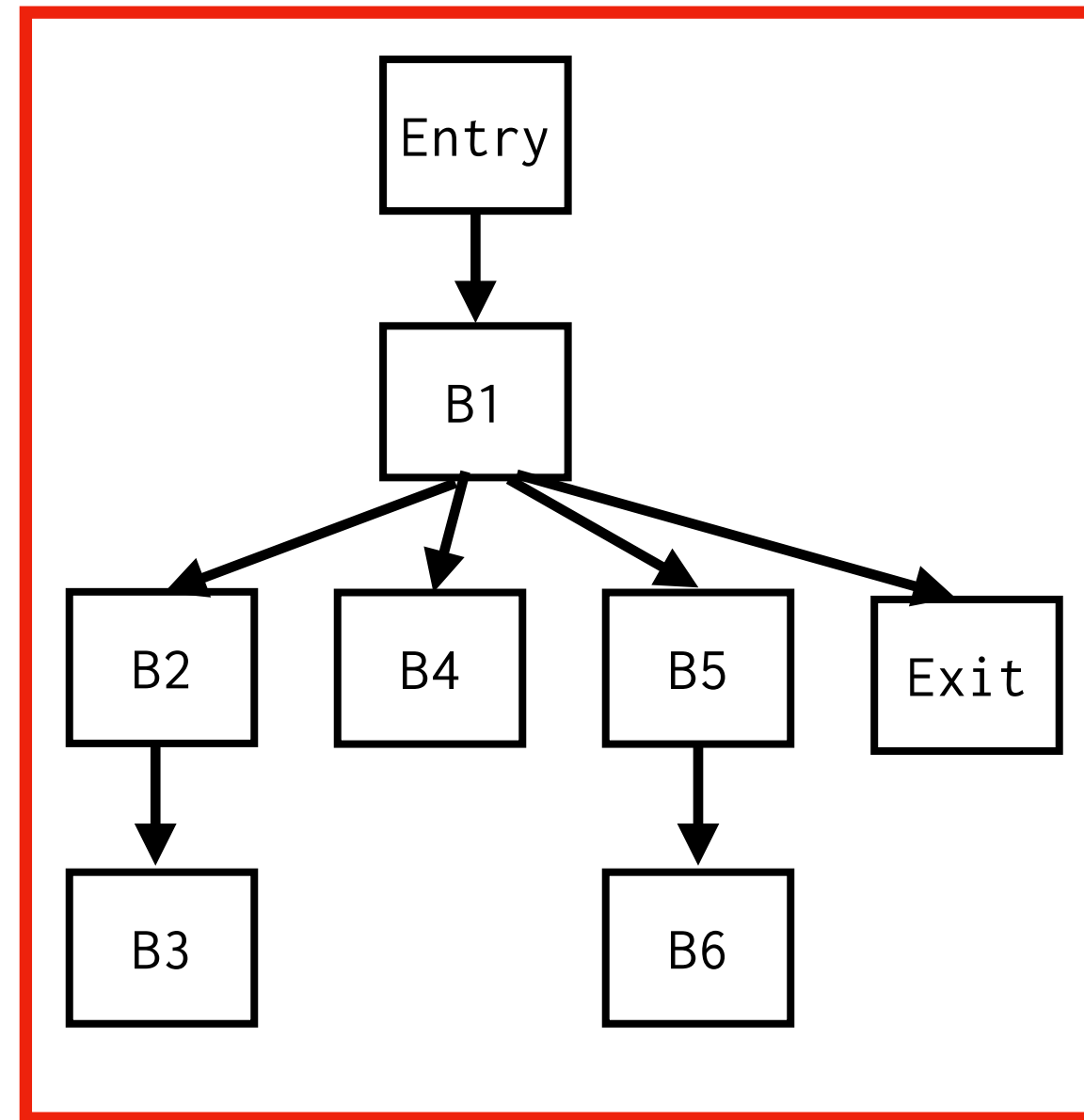
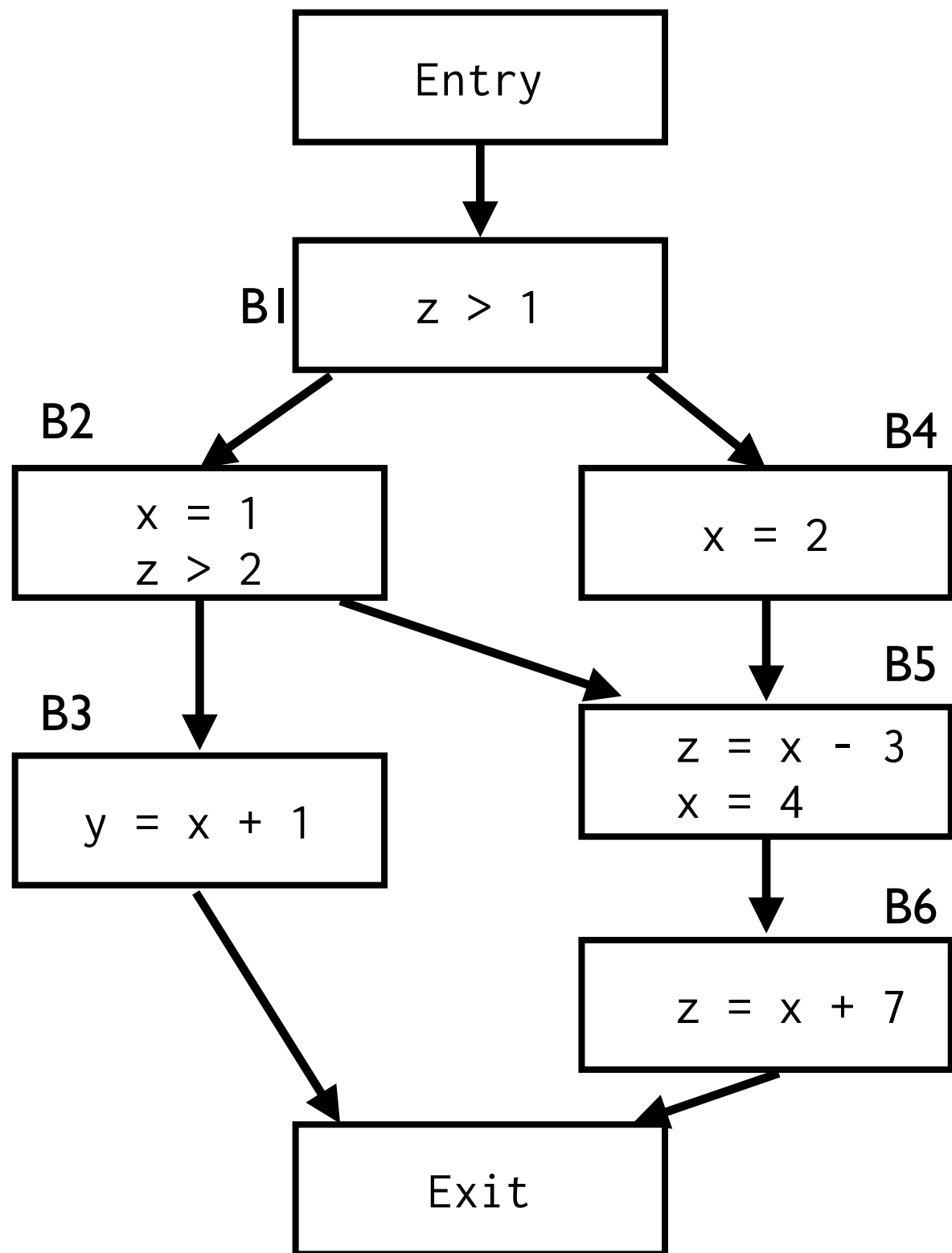


- What are the dominance frontiers for basic blocks Entry, 1, ..., 6, Exit?
- First get dominator tree

Dominance Frontiers

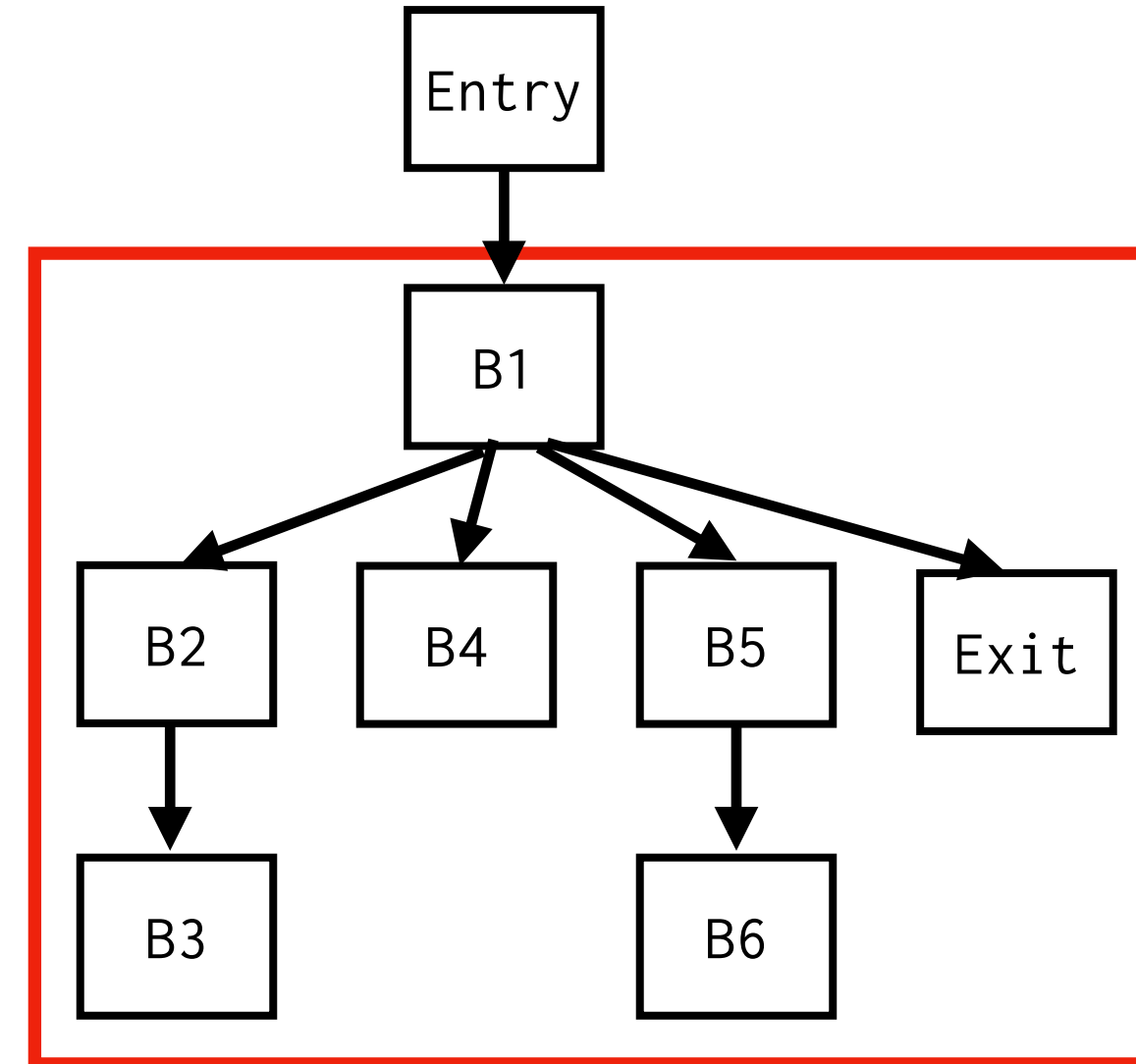
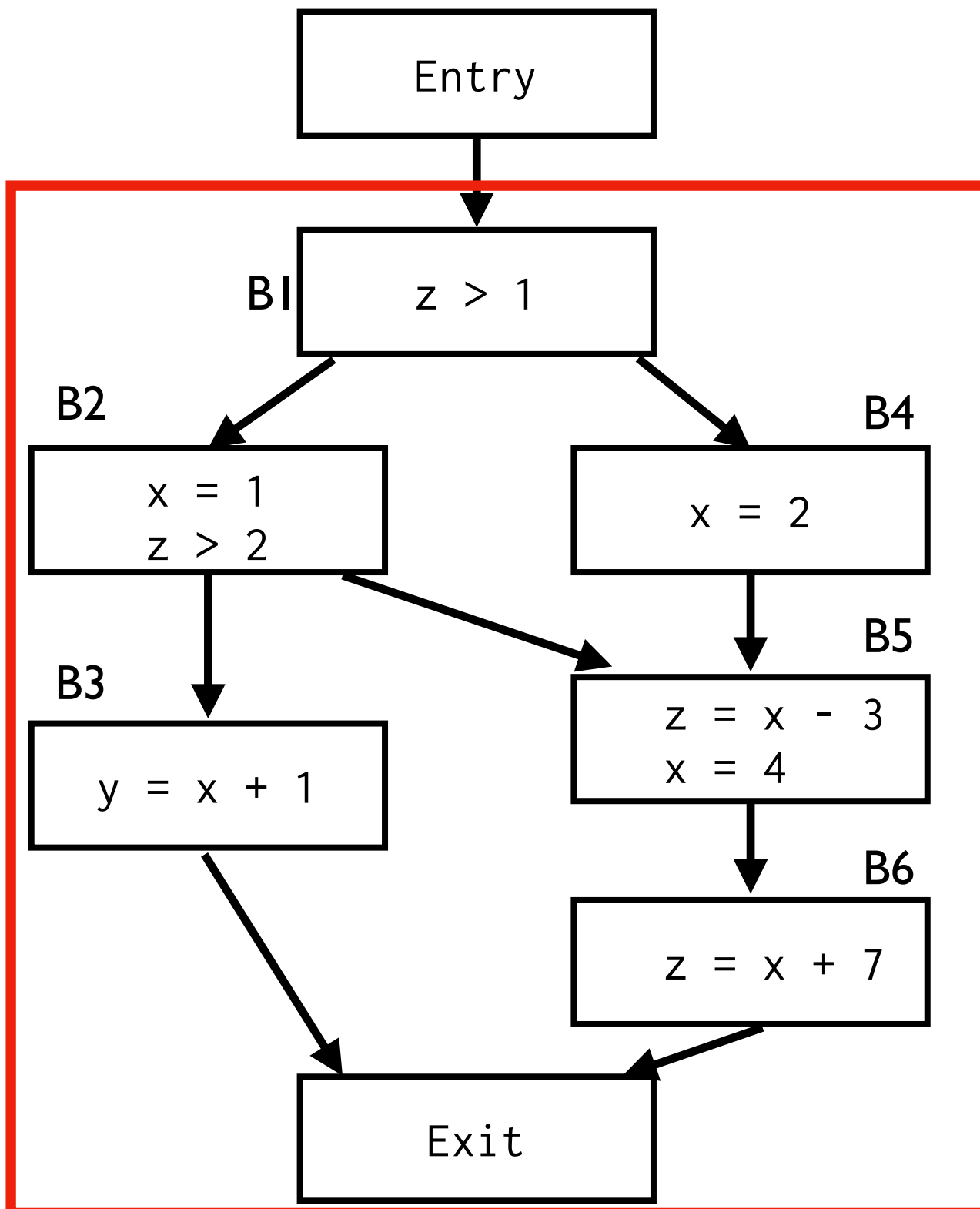


Dominance Frontiers



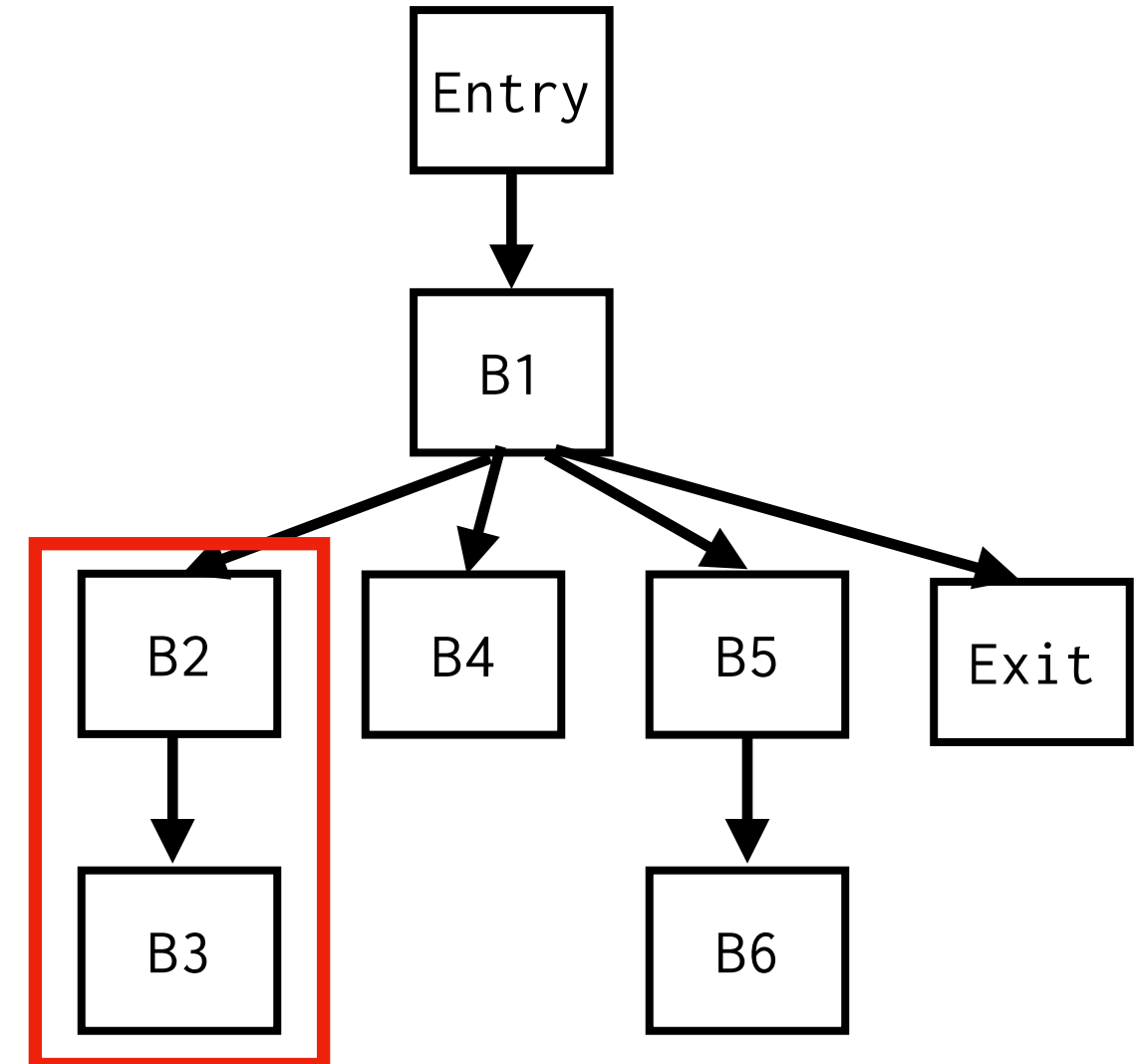
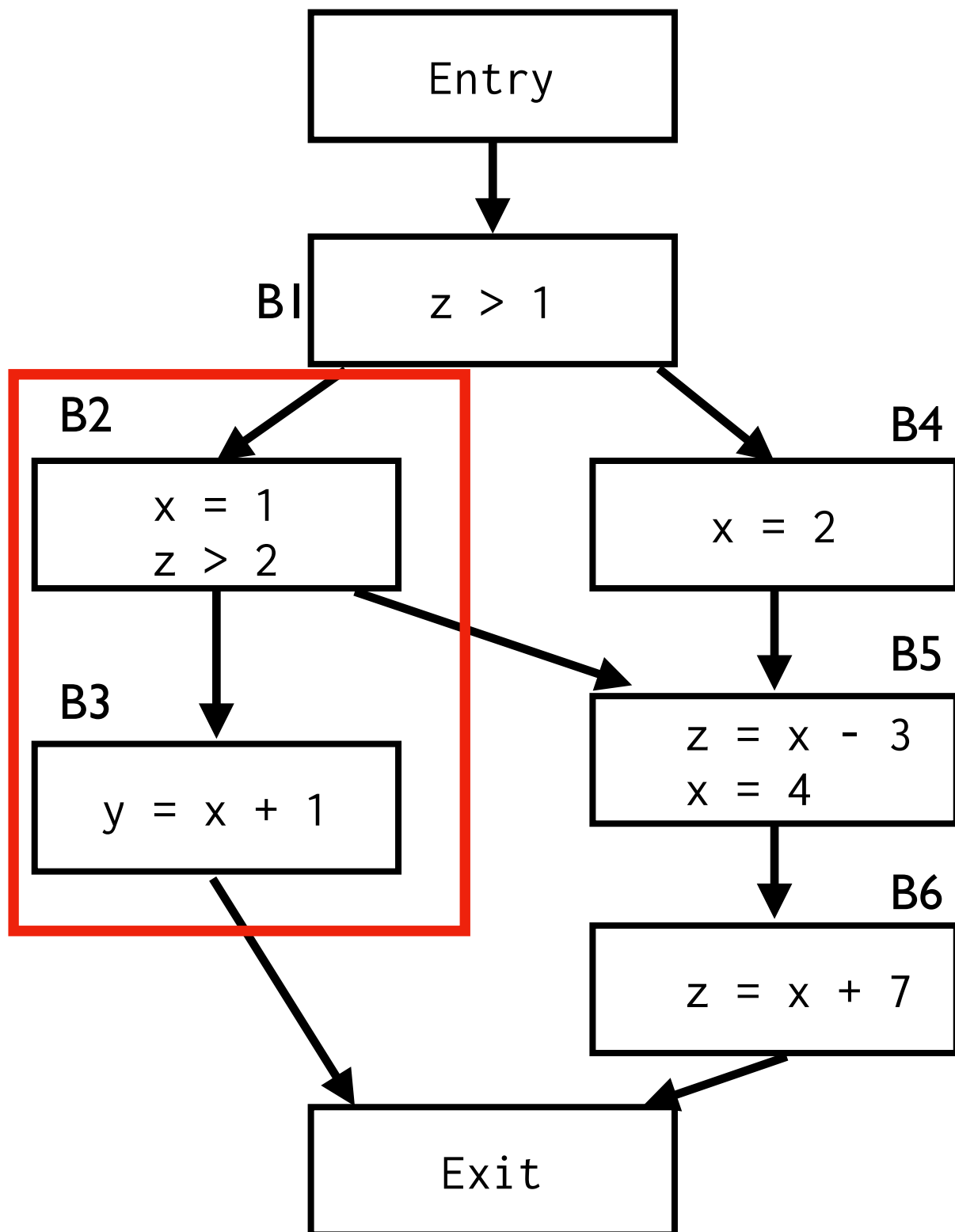
- Entry: Dominates all nodes
- $DF(Entry) = \{\}$

Dominance Frontiers



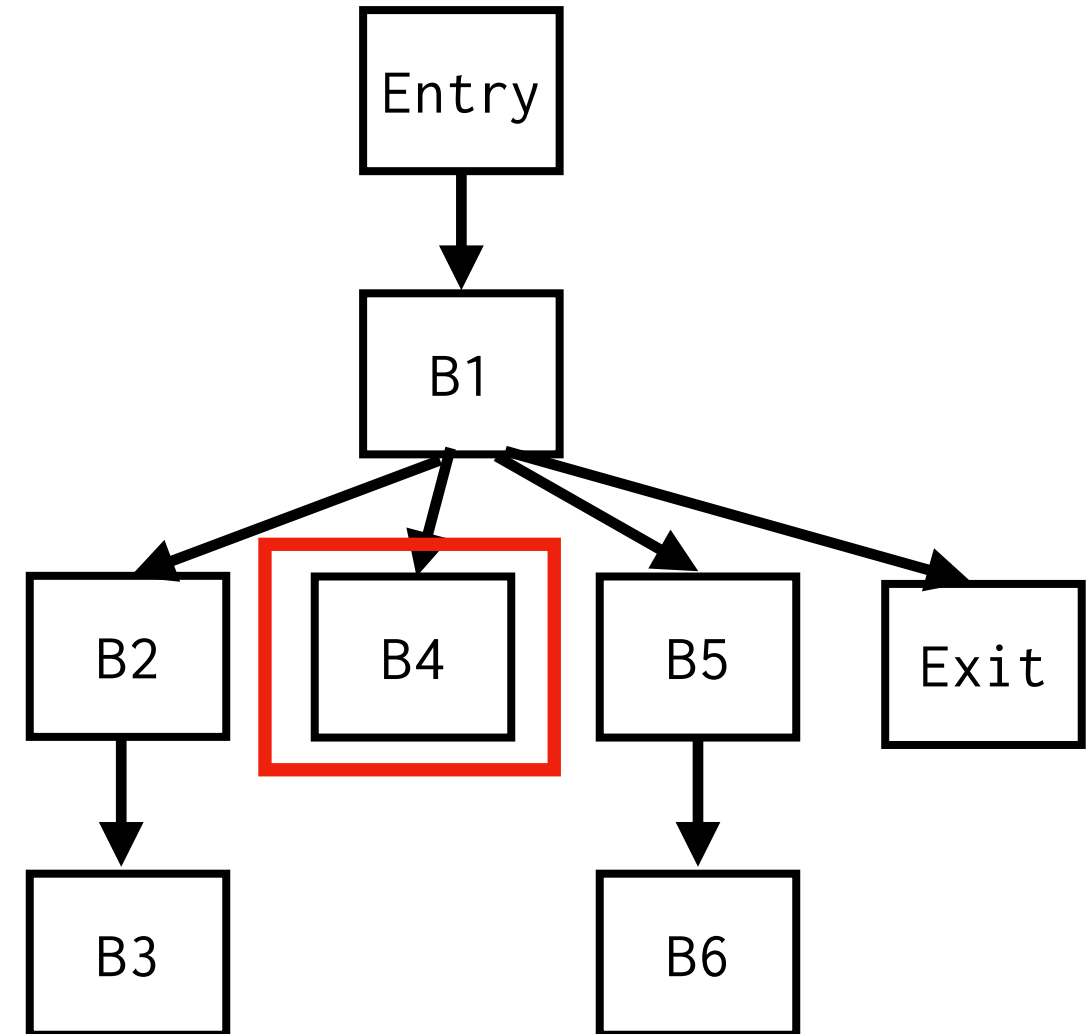
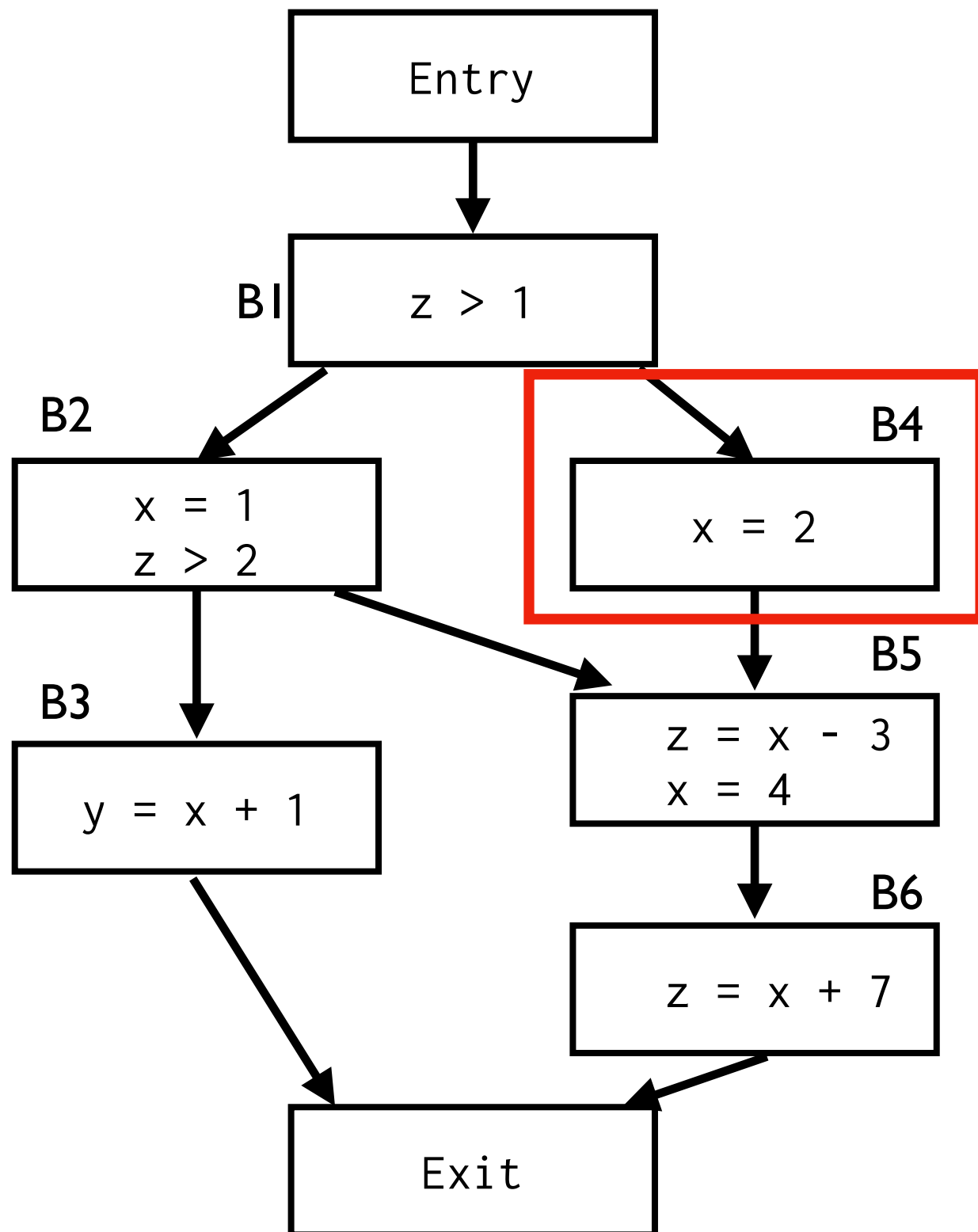
- **B1**: Dominates B1...Exit
- $DF(B1) = \{\}$

Dominance Frontiers



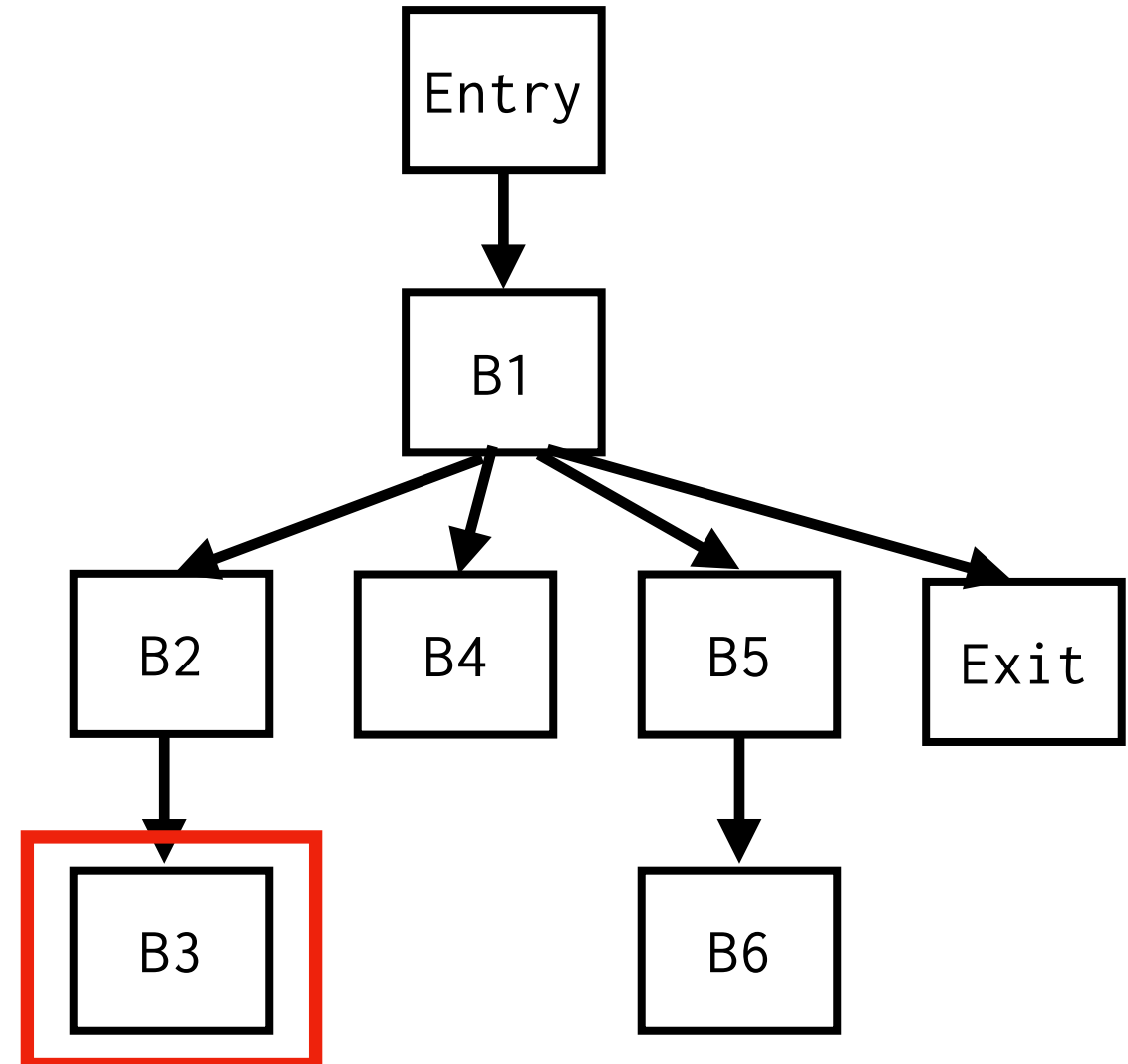
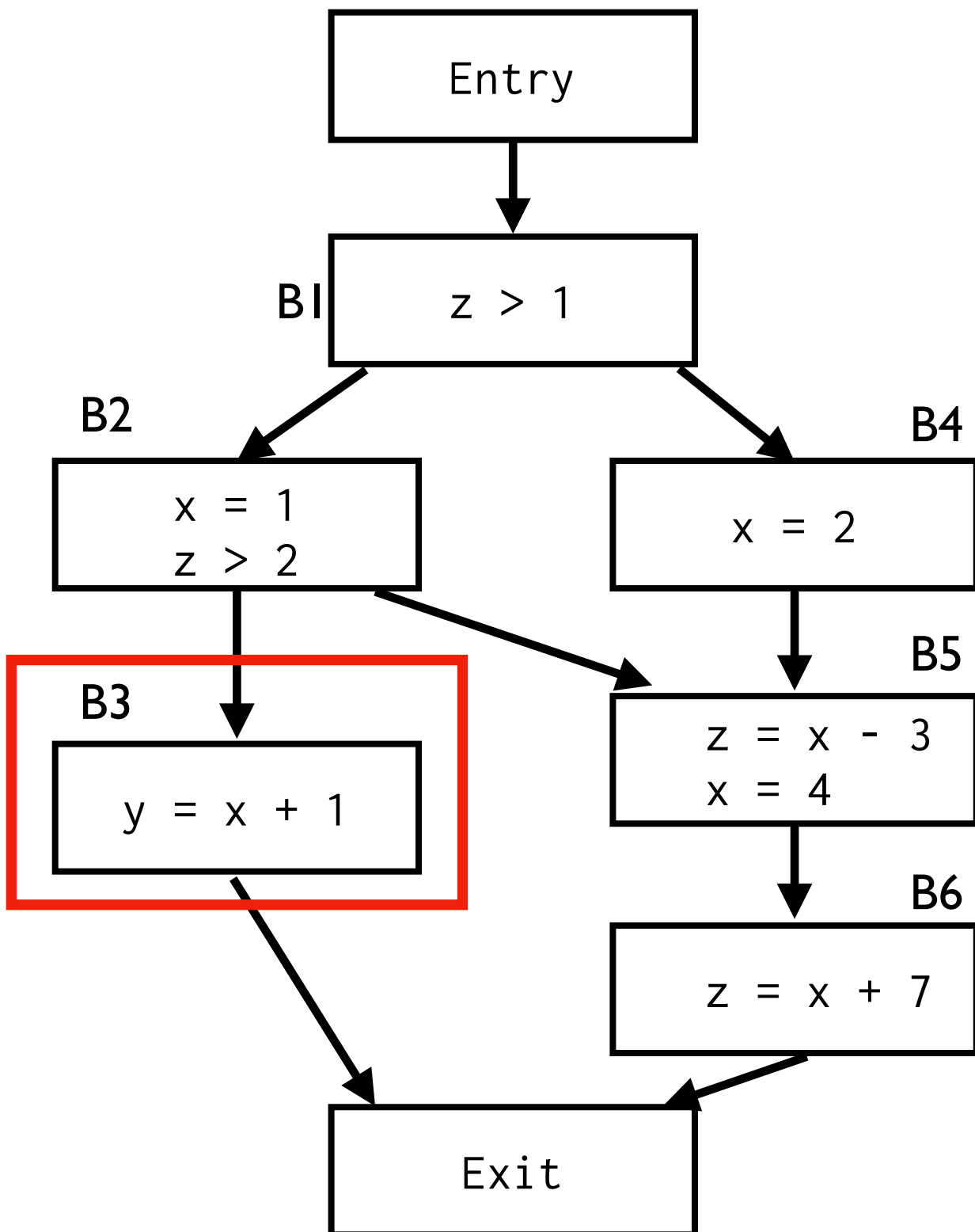
- B2: Dominates B2, B3
- Successor nodes outside this region are Exit, B5
- $DF(B2) = \{\text{Exit}, B5\}$

Dominance Frontiers



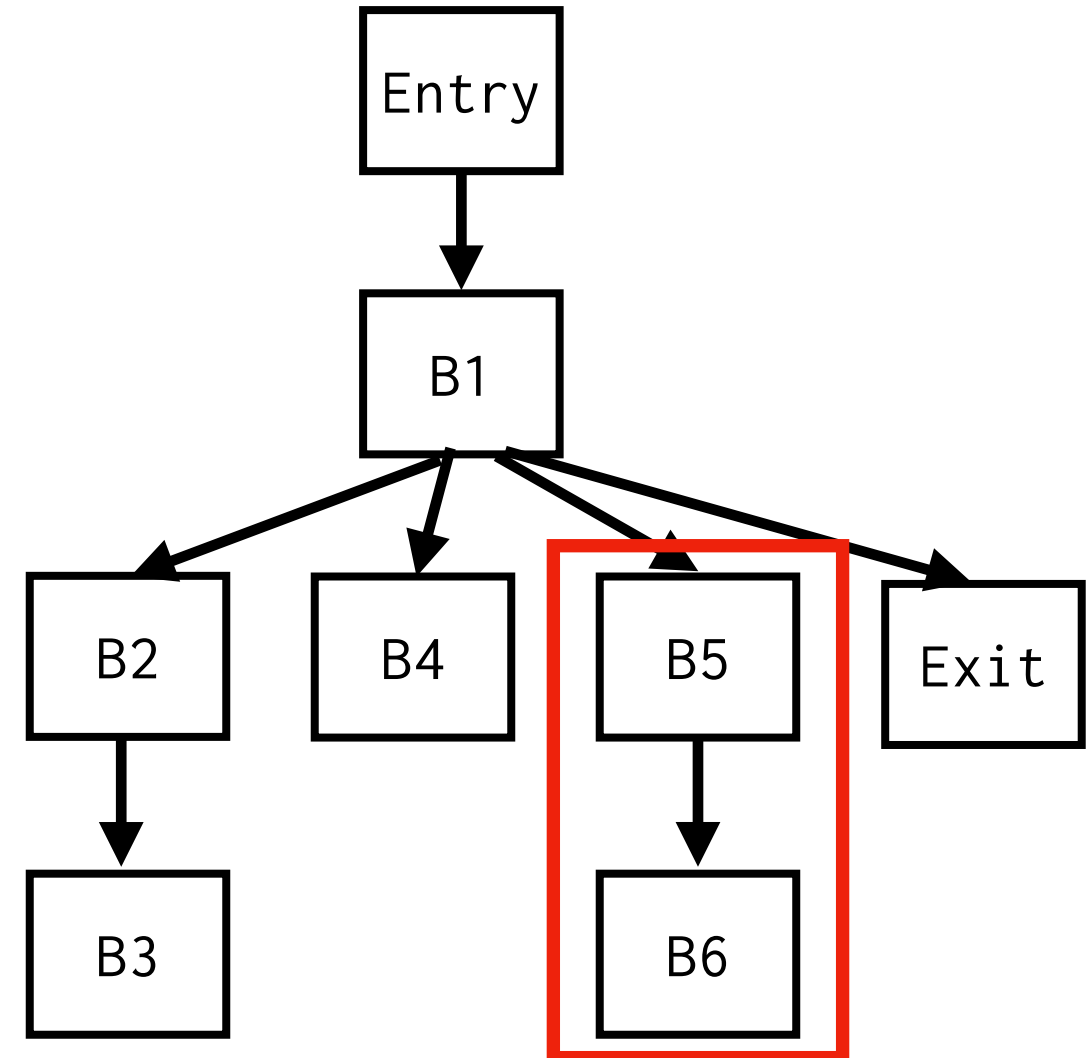
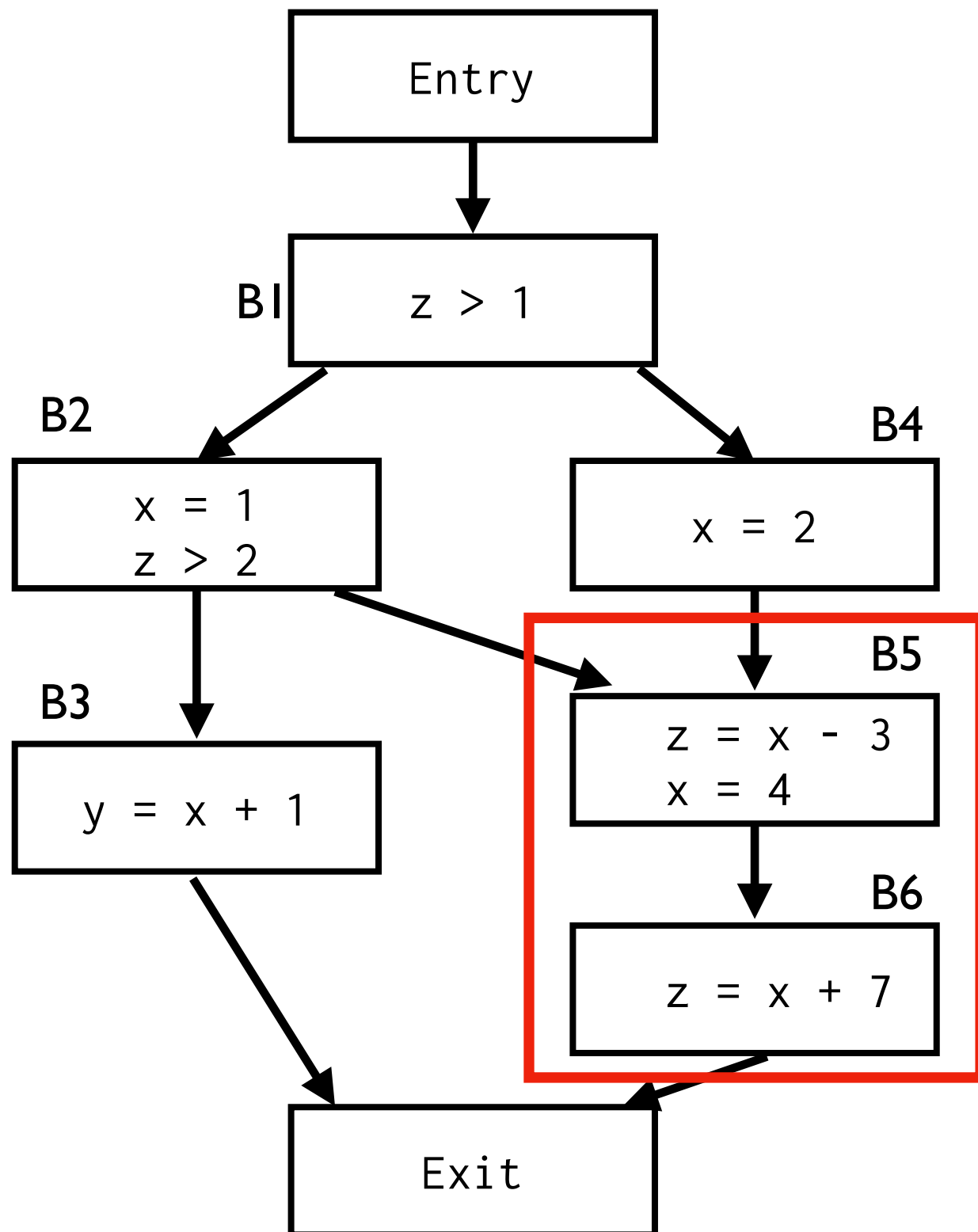
- B4: Dominates B4
- Successor node outside this region is B5
- $DF(B4) = \{B5\}$

Dominance Frontiers



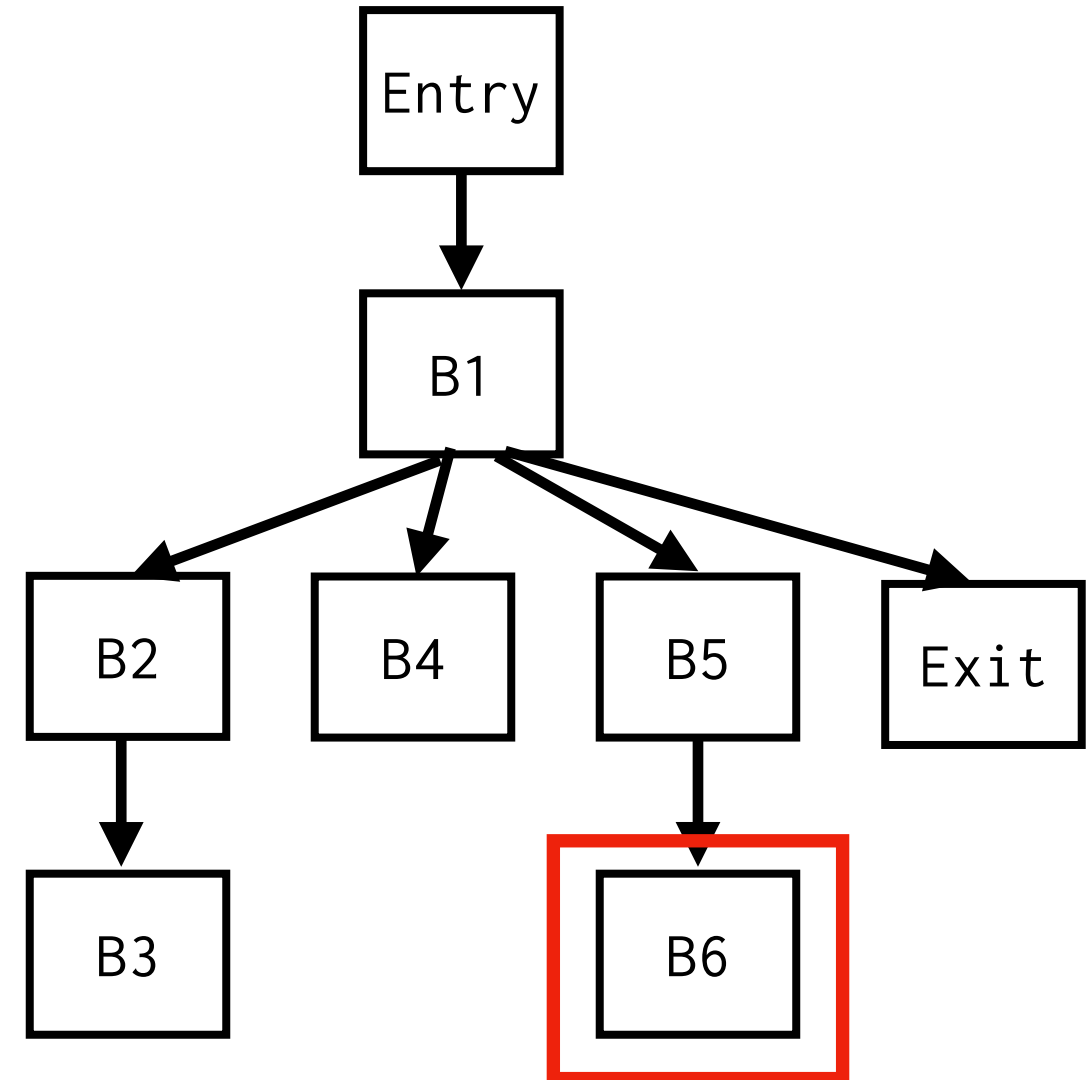
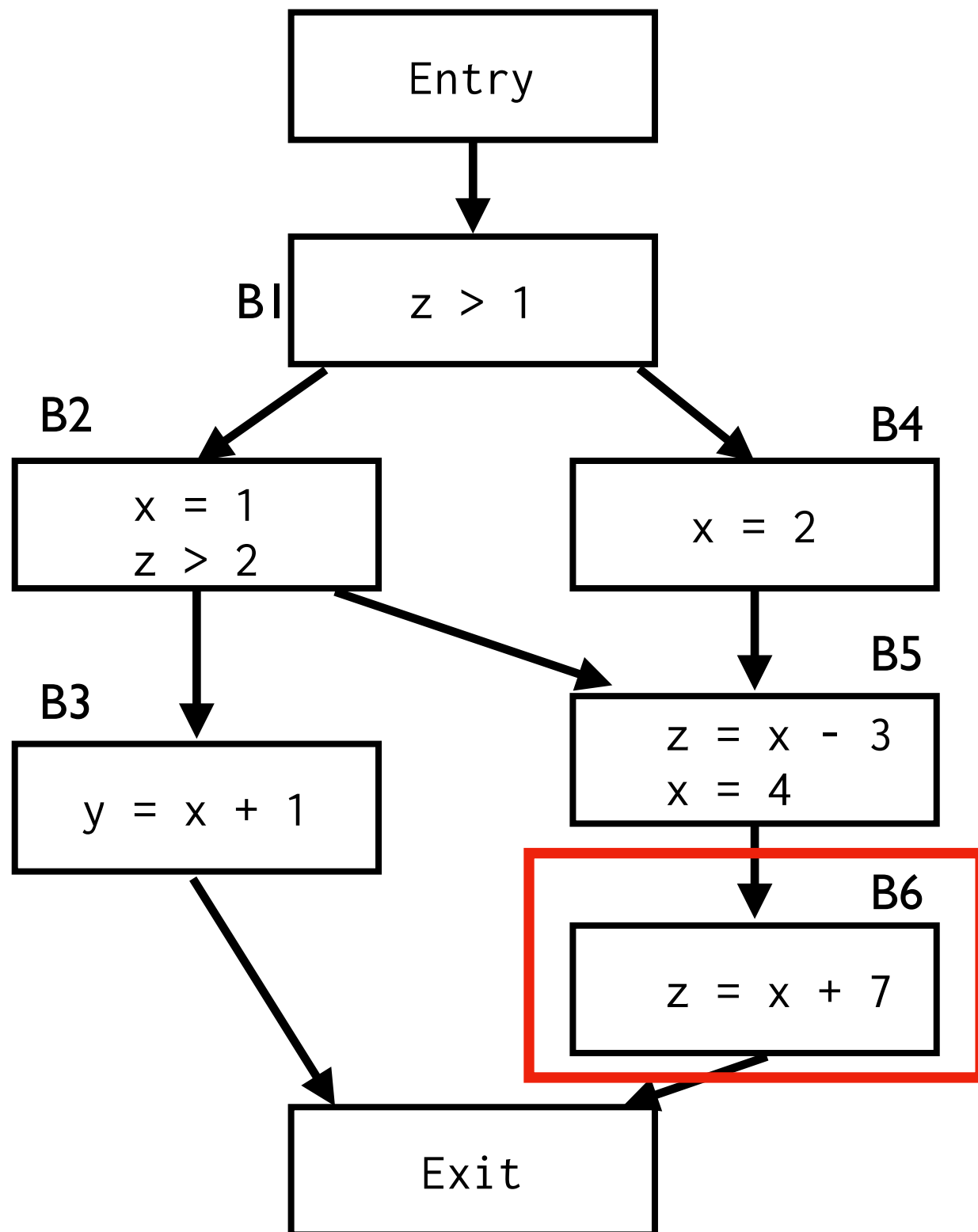
- B3: Dominates B3
- Successor node outside this region is Exit
- $DF(B3) = \{Exit\}$

Dominance Frontiers



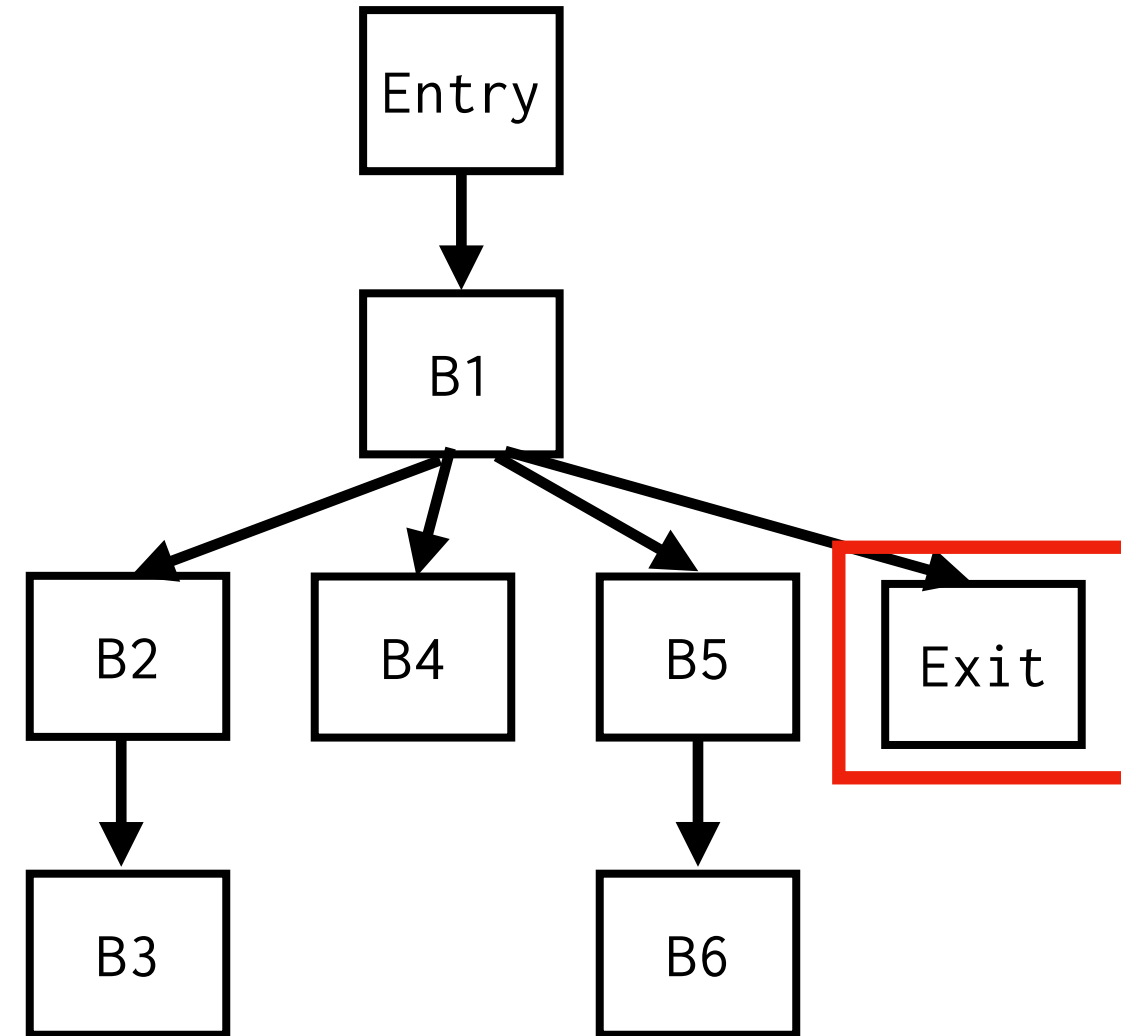
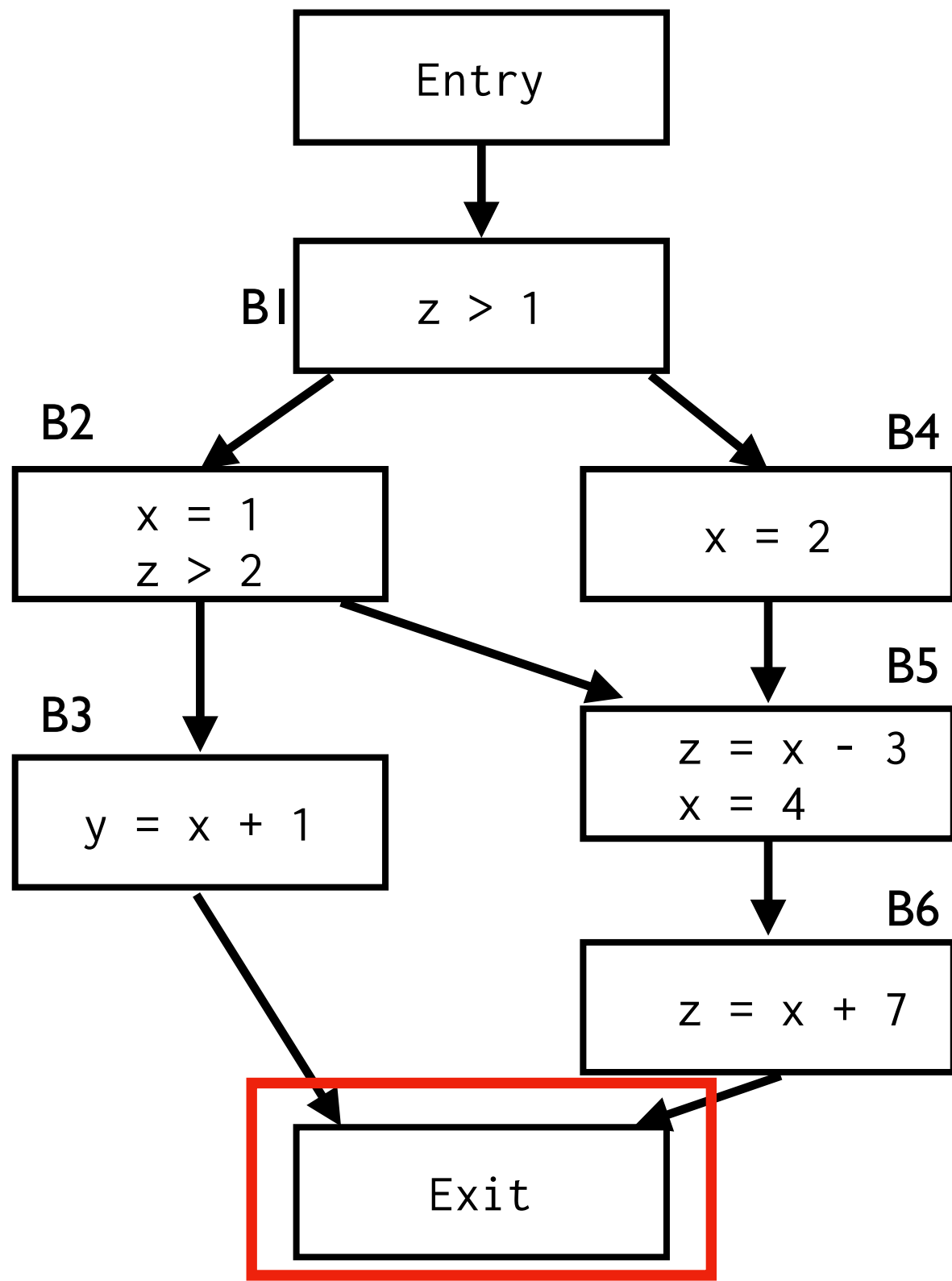
- B5: Dominates B5, B6
- Successor node outside this region is Exit
- $DF(B5) = \{\text{Exit}\}$

Dominance Frontiers



- B6: Dominates B6
- Successor node outside this region is Exit
- $DF(B6) = \{Exit\}$

Dominance Frontiers



- Exit: Dominates Exit
- No successor node outside this region
- $DF(Exit) = \{\}$

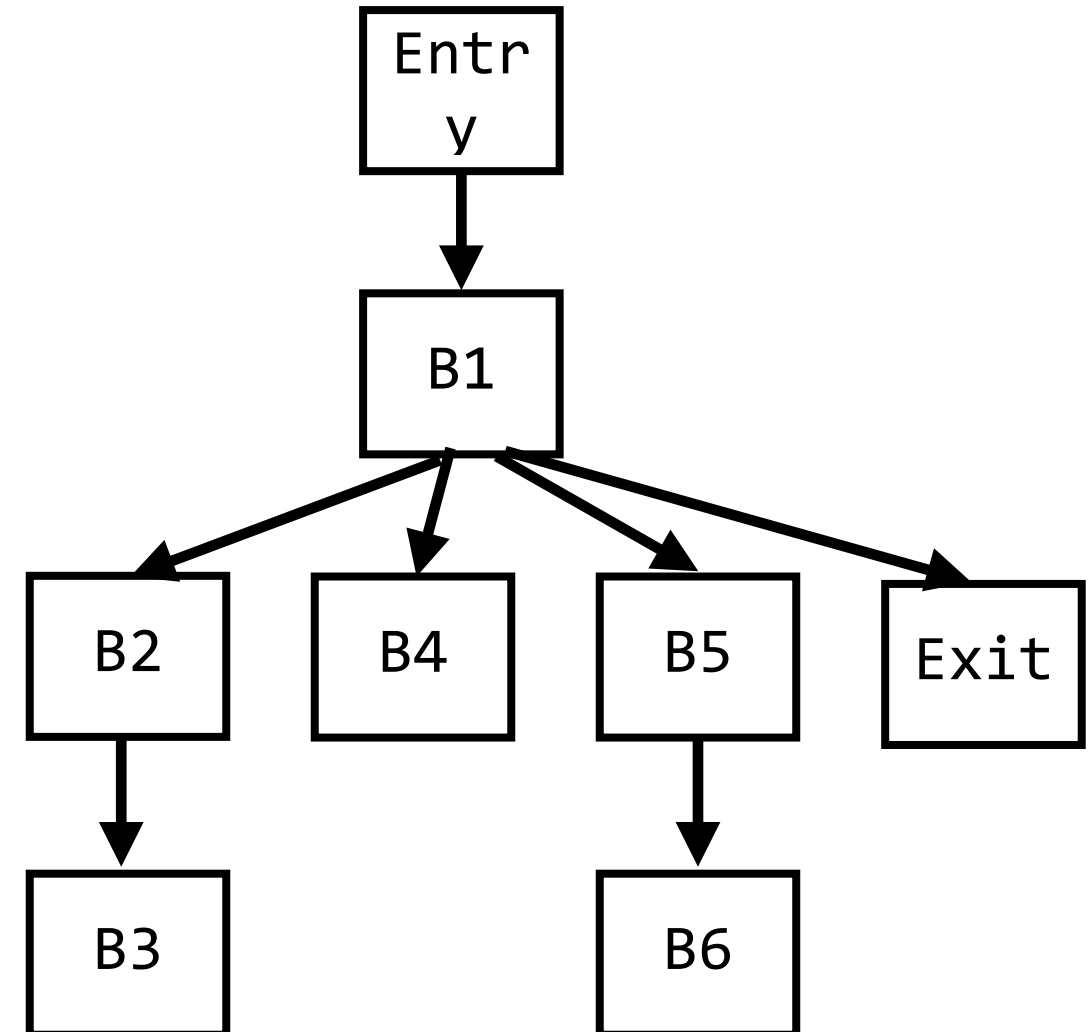
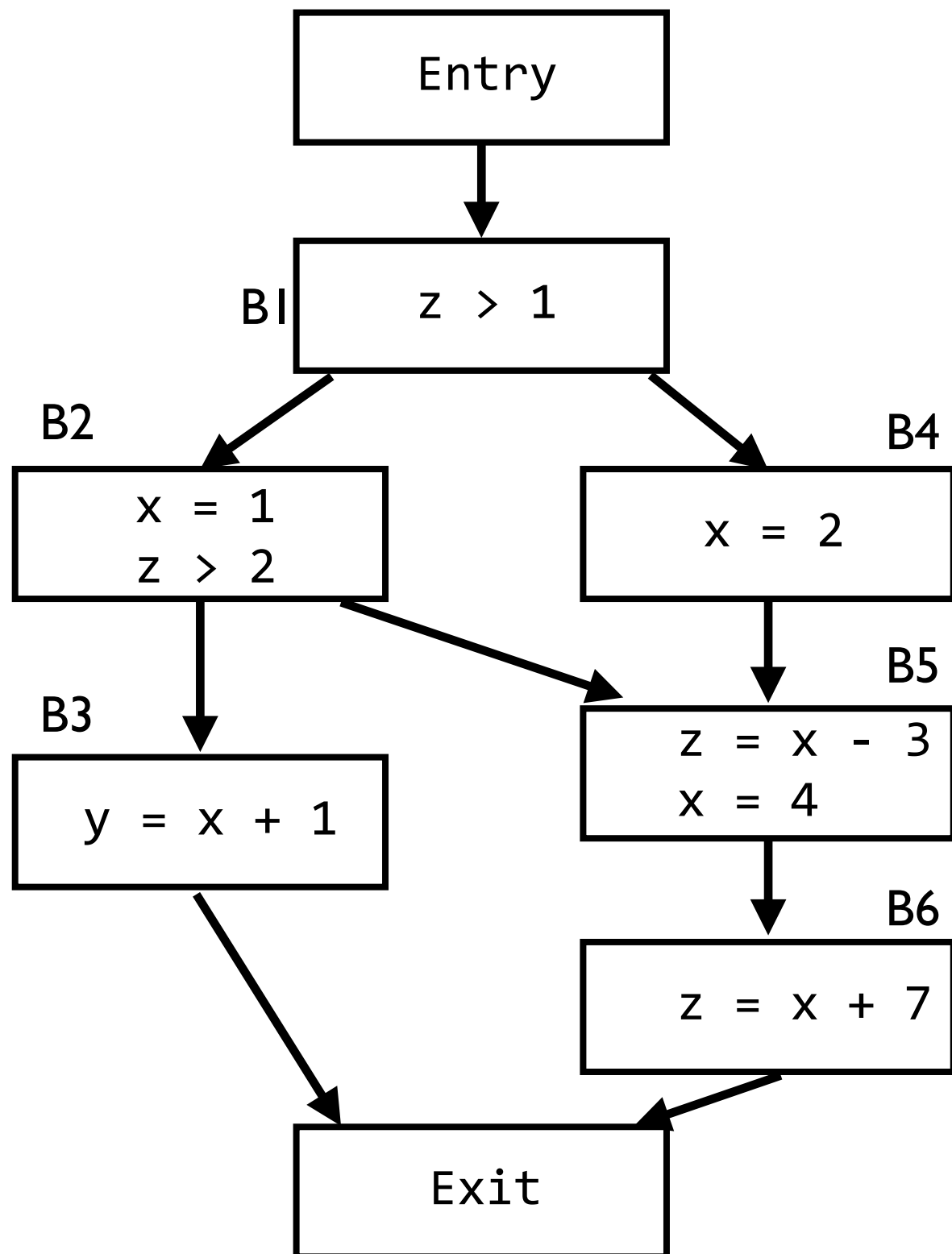
Dominance Frontier

- $DF_{local}[n]$: The successors of n that are not strictly dominated by n
- $DF_{up}[n]$: Nodes in the dominance frontier of n that are not dominated by n 's immediate dominator
- The dominance frontier of n can be computed from DF_{local} and DF_{up} :
 - $DF[n] = DF_{local}[n] \cup \bigcup_{c \in children[n]} DF_{up}[c]$
- where $children[n]$ are the nodes whose immediate dominator (idom) is n

Dominance Frontier

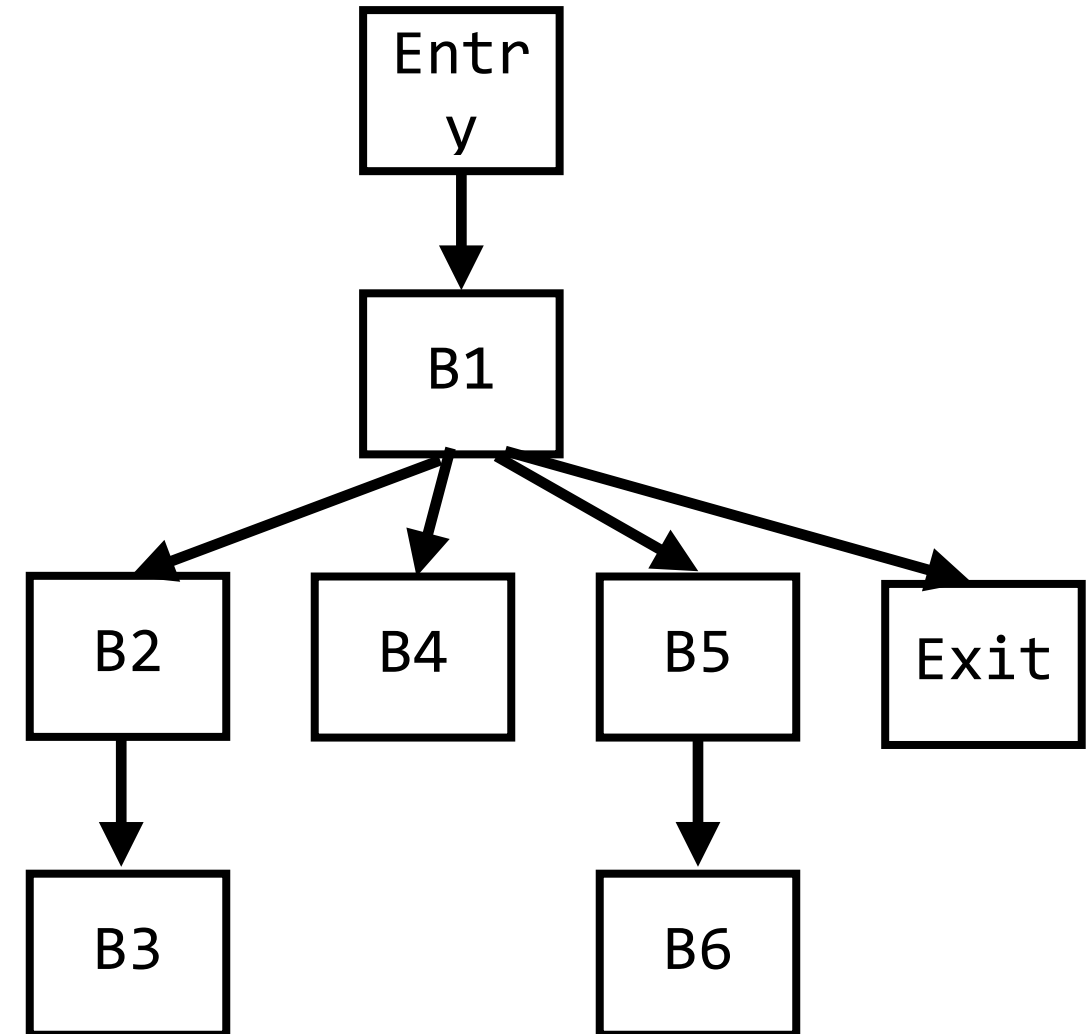
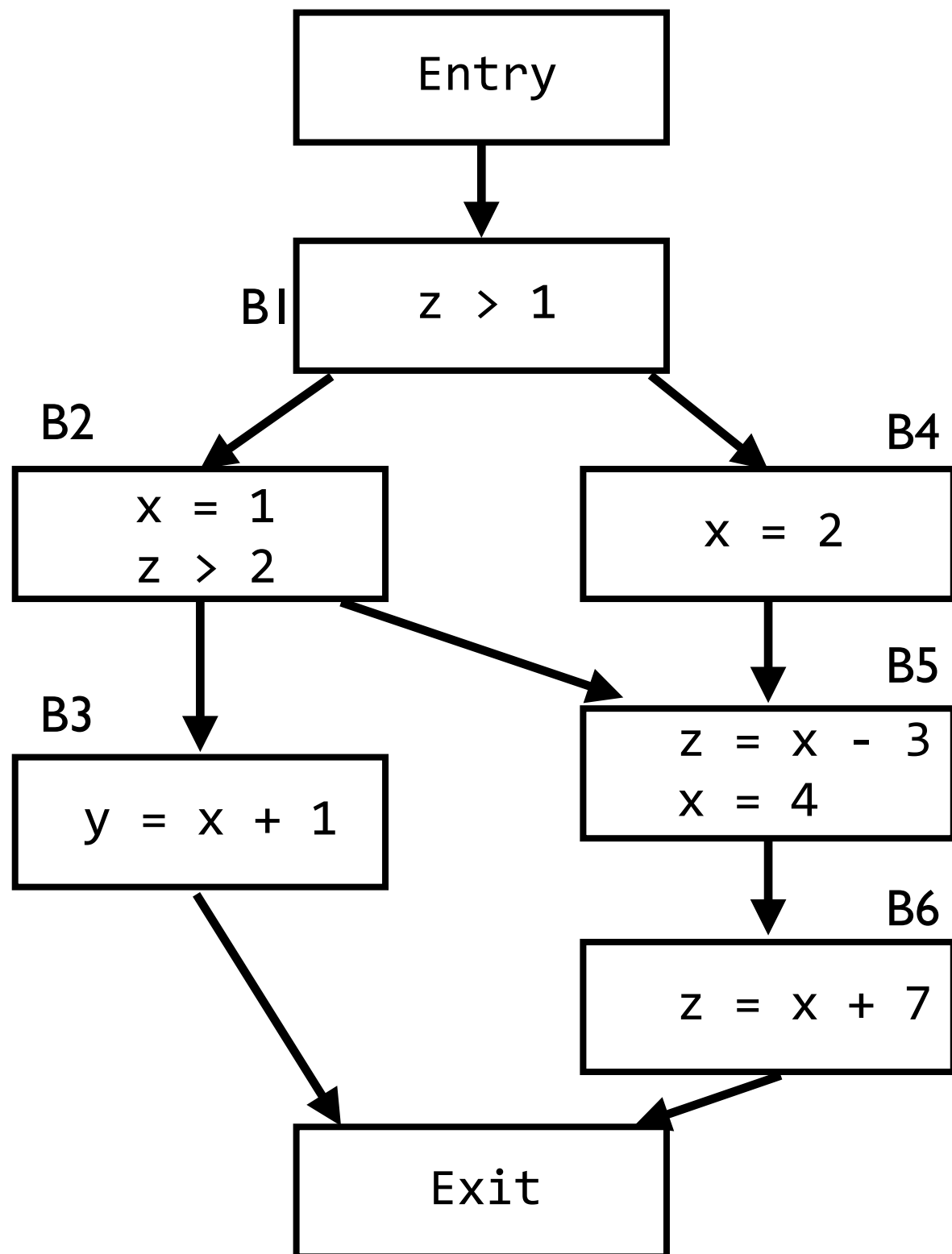
```
computeDF[n] =  
     $S \leftarrow \{\}$   
    for each node  $y$  in  $\text{succ}[n]$            // This loop computes  $DF_{\text{local}}[n]$   
        if  $\text{idom}(y) \neq n$   
             $S \leftarrow S \cup \{y\}$   
    for each child  $c$  of  $n$  in the dominator tree  
        computeDF[c]  
        for each element  $w$  of  $DF[c]$        // This loop computes  $DF_{\text{up}}[c]$   
            if  $n$  does not dominate  $w$   
                 $S \leftarrow S \cup \{w\}$   
 $DF[n] \leftarrow S$ 
```

Dominance Frontiers



- $DF_{\text{local}}[B6] = \{\text{Exit}\}$
- B6 has no children so no DF_{up} to compute
- $DF(B6) = DF_{\text{local}}[B6] = \{\text{Exit}\}$

Dominance Frontiers



- $DF_{\text{local}}[B5] = \{\}$
- B6 children of B5 in dom tree:
 $DF_{\text{up}}[B6] = \{\text{Exit}\}$
- $DF(B5) = DF_{\text{local}}[B5] \cup DF_{\text{up}}[B6] = \{\text{Exit}\}$

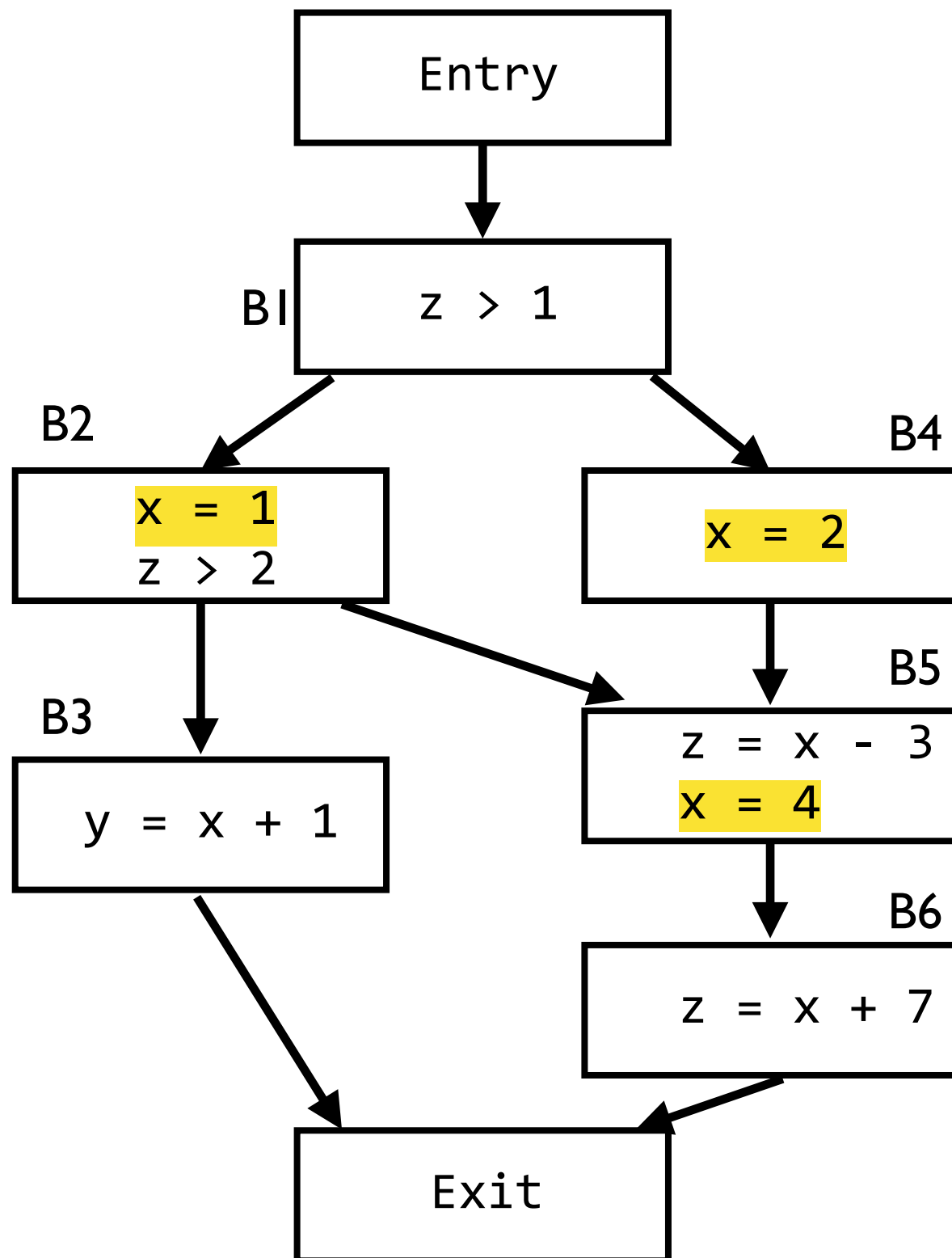
Computing SSA Form

- Step 1: Place Φ nodes
 - 1a: Compute the dominance frontier
 - 1b: Place a Φ function for variable a in dominance frontier of node X if X contains an assignment to a
- Step 2: Rename variables so only one definition per name

Iterated Dominance Frontiers (IDF)

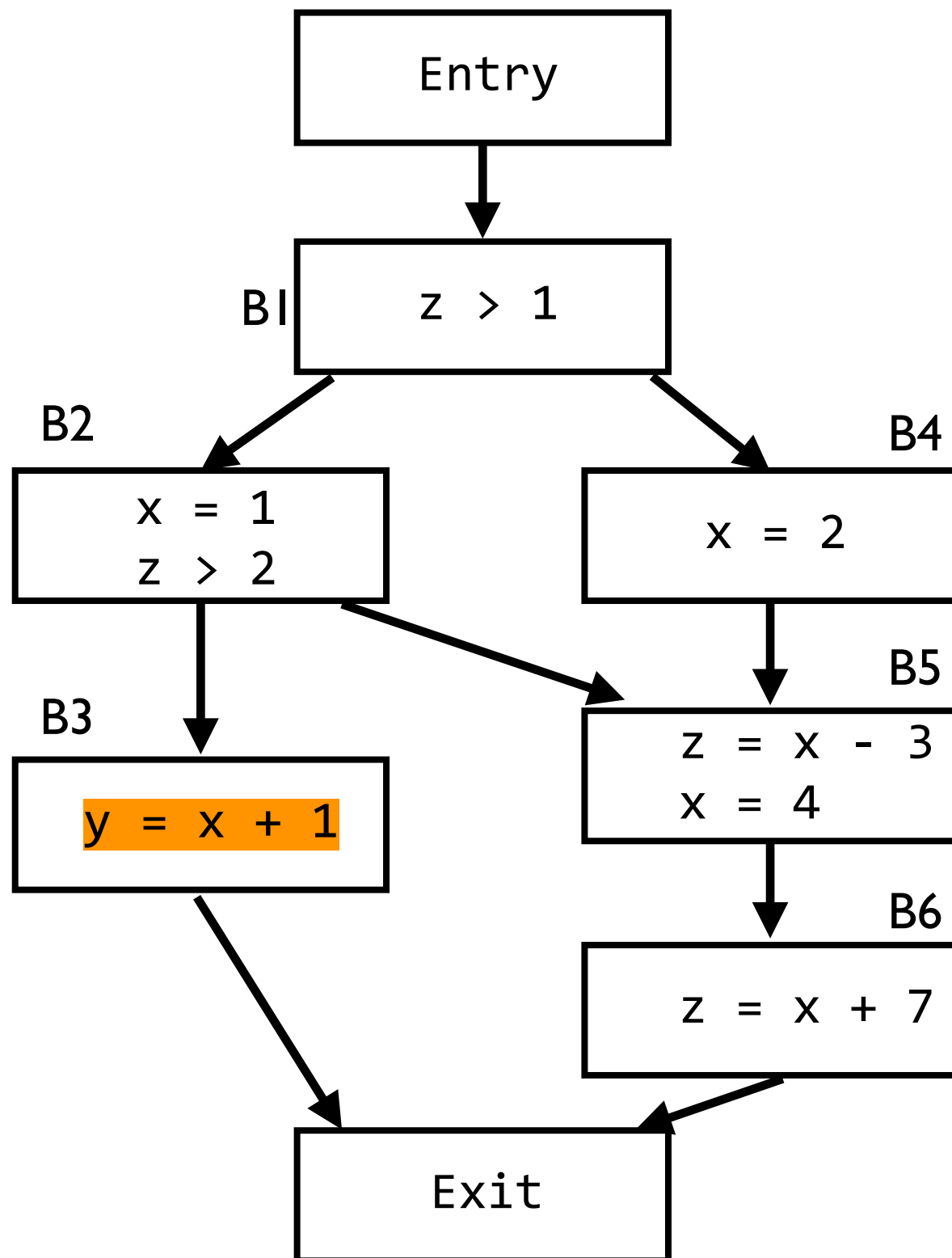
- Since Φ functions themselves are a kind of definition, we must iterate the dominance-frontier criterion until there are no nodes that need Φ functions
- We use the **iterated dominance frontier (IDF)**:
 $IDF(X) = DF^+(X) = \lim DF^i(X)$ where $DF^{i+1} = DF(X \cup DF^i(X))$
- Insert a ϕ function for variable $\underline{a}$ at every node in $IDF(Def(\underline{a}))$, where $Def(\underline{a})$ is the set of nodes in which $\underline{a}$ is defined

Iterated Dominance Frontiers (IDF)



- For variable x : $IDF(Def(x))$
- $Def(x) = \{B2, B4, B5\}$
- $DF^1(B2, B4, B5) = \{B5, Exit\}$
- $DF^2(B2, B4, B5)$
 $= DF(B2, B4, B5 \cup DF^1(B2, B4, B5))$
 $= DF(B2, B4, B5, Exit)$
 $= \{B5, Exit\}$
- *Fixpoint !*

Iterated Dominance Frontiers



- For variable y : $IDF(Def(y))$

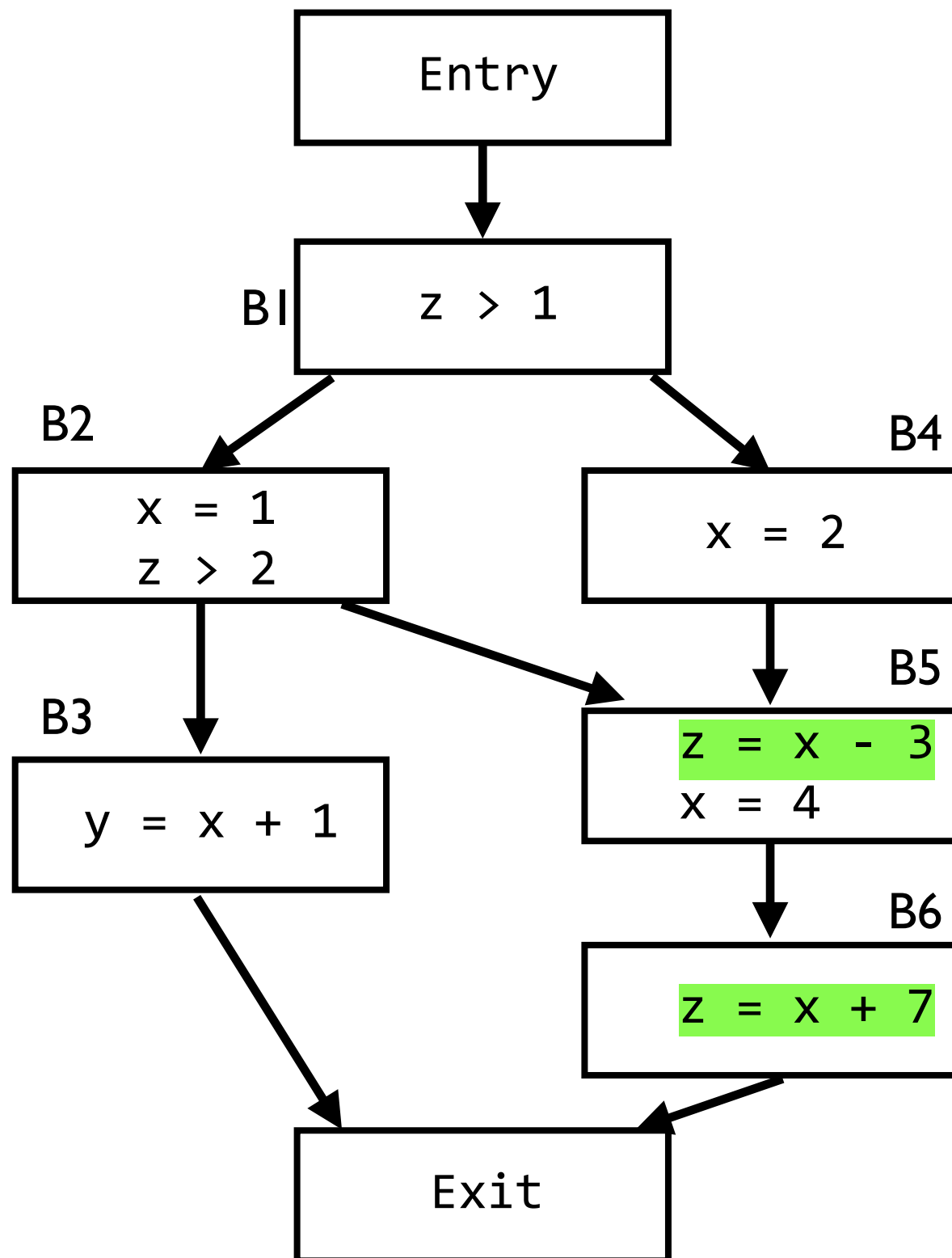
- $Def(y) = \{B3\}$

- $DF^1(B3) = \{Exit\}$

- $DF^2(B3)$
 $= DF(B3 \cup DF^1(B3))$
 $= DF(B3, Exit)$
 $= \{Exit\}$

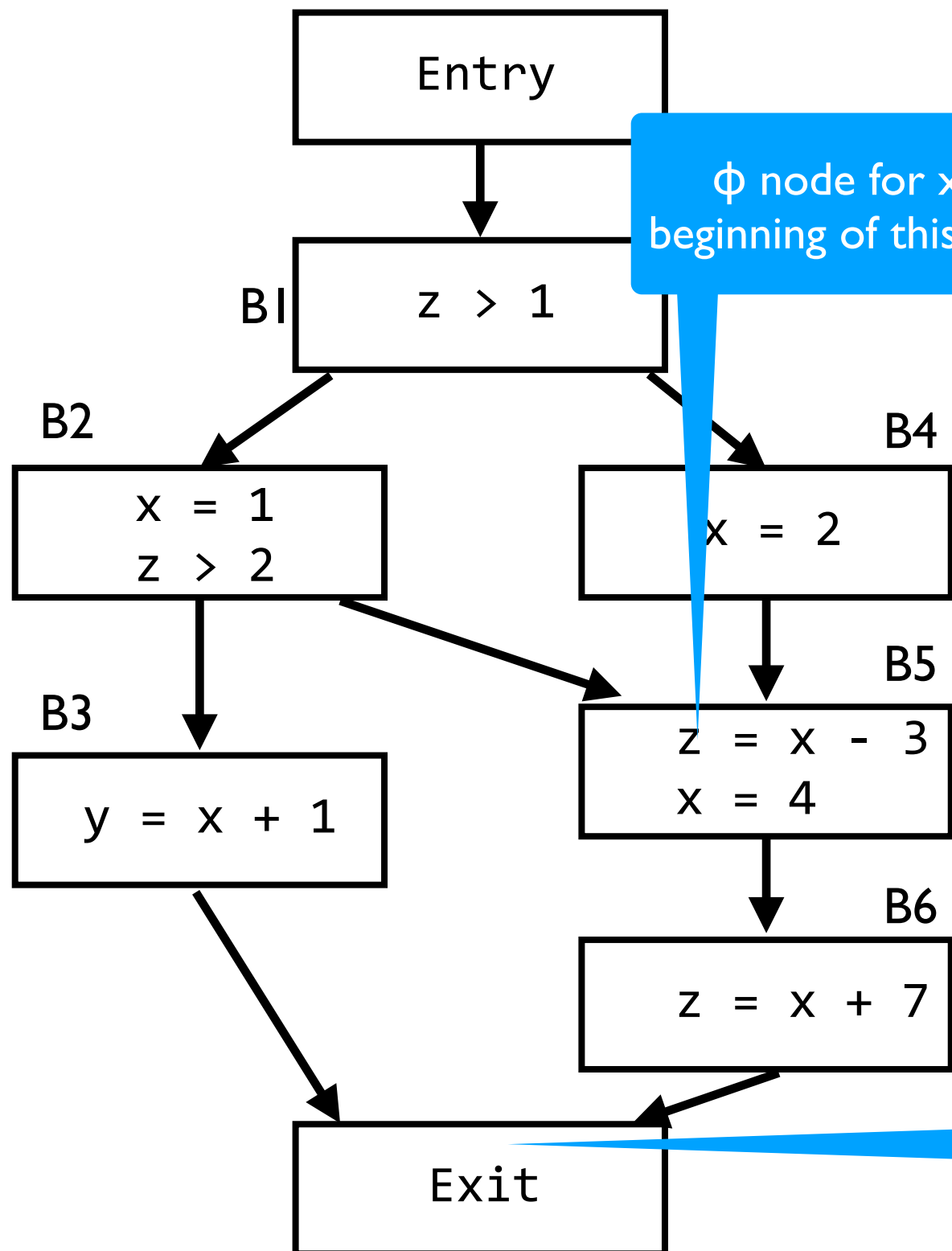
- *Fixpoint!*

Iterated Dominance Frontiers



- For variable z : $IDF(Def(z))$
- $Def(z) = \{B5, B6\}$
- $DF^1(B5, B6) = \{Exit\}$
- $DF^2(B5, B6)$
 $= DF(B5, B6 \cup DF^1(B5, B6))$
 $= DF(B5, B6, Exit)$
 $= \{Exit\}$
- *Fixpoint!*

Iterative Dominance Frontiers



ϕ node for x at beginning of this block

- IDFs

- For x : {B5, Exit}

- For y : {Exit}

- For z : {Exit}

ϕ node for x, y, z at beginning of this block

Computing SSA Form

- Step 1: Place Φ nodes
 - 1a: Compute the dominance frontier
 - 1b: Place a Φ function for $\underline{a}$ in dominance frontier of X if node X contains assignment to $\underline{a}$
- Step 2: Rename variables so only one definition per name

Renaming the Variables

- After the ϕ -functions are placed, we can walk the dominator tree, renaming the different definitions (including ϕ -functions) of variable a to a_1, a_2, a_3 and so on.
- Rename each use of a to use the closest definition d of a that is above a in the dominator tree.

Renaming Variables

Initialization:

```
for each variable  $a$   
   $Count[a] \leftarrow 0$   
   $Stack[a] \leftarrow \text{empty}$   
  push 0 onto  $Stack[a]$ 
```

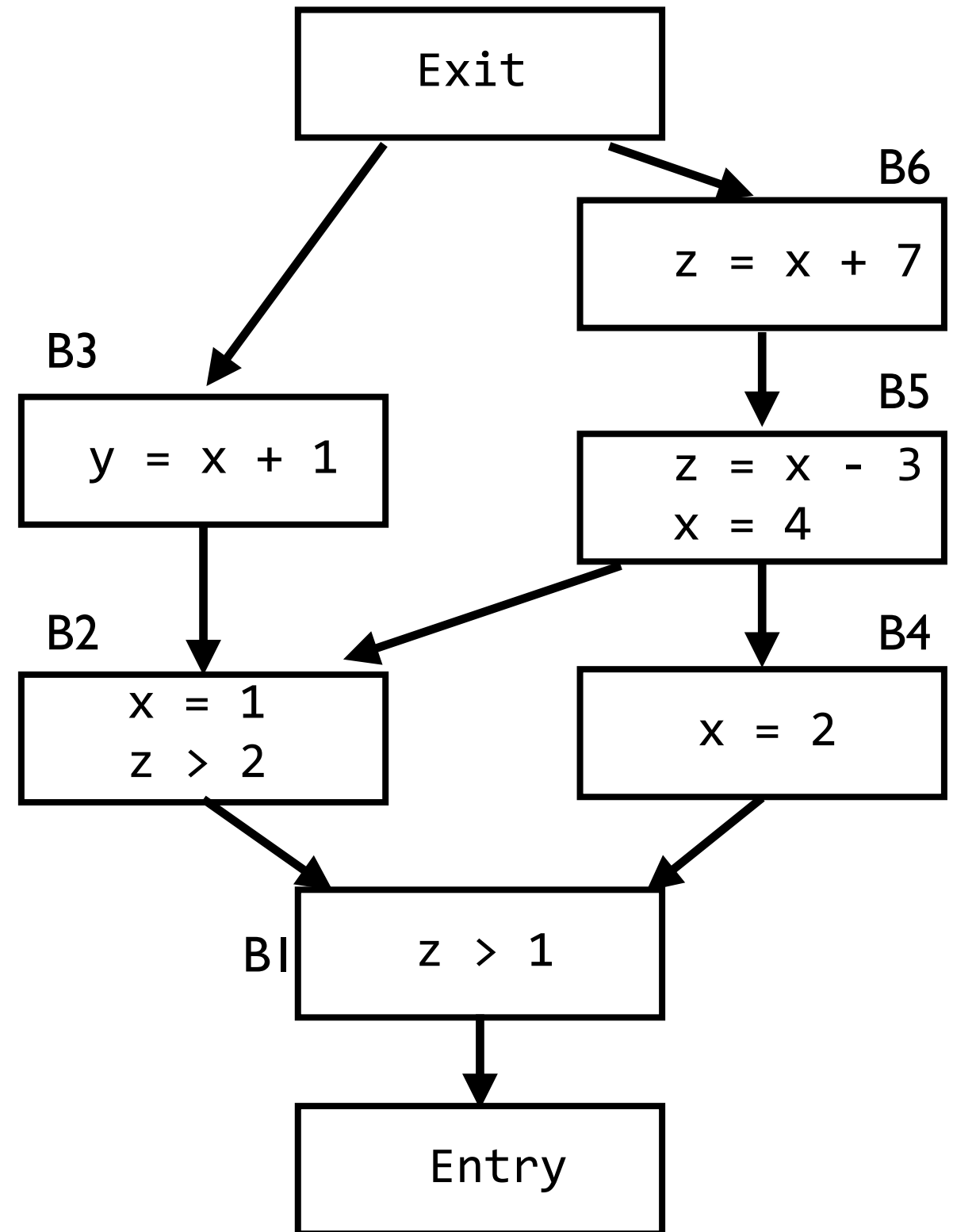
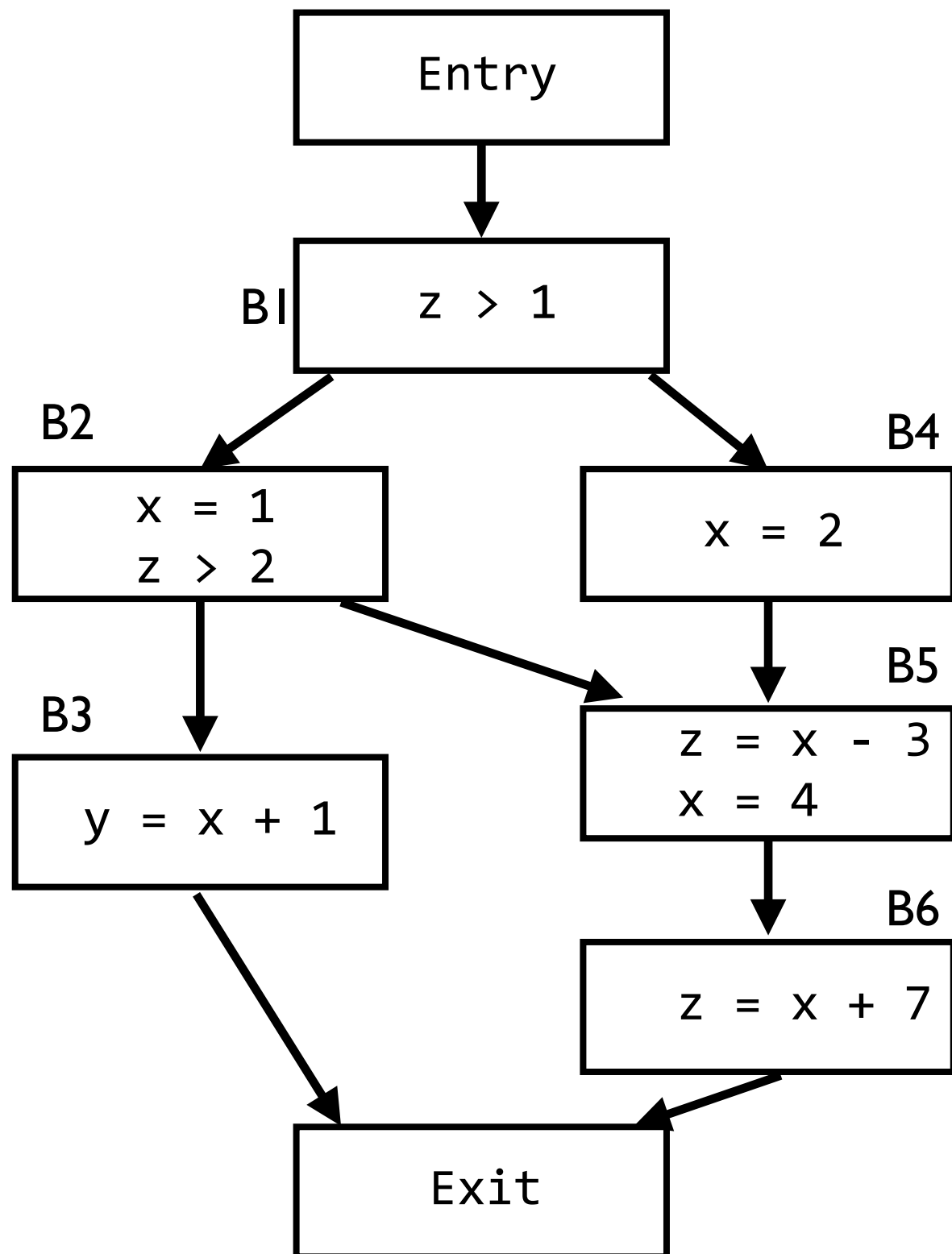
Rename(n) =

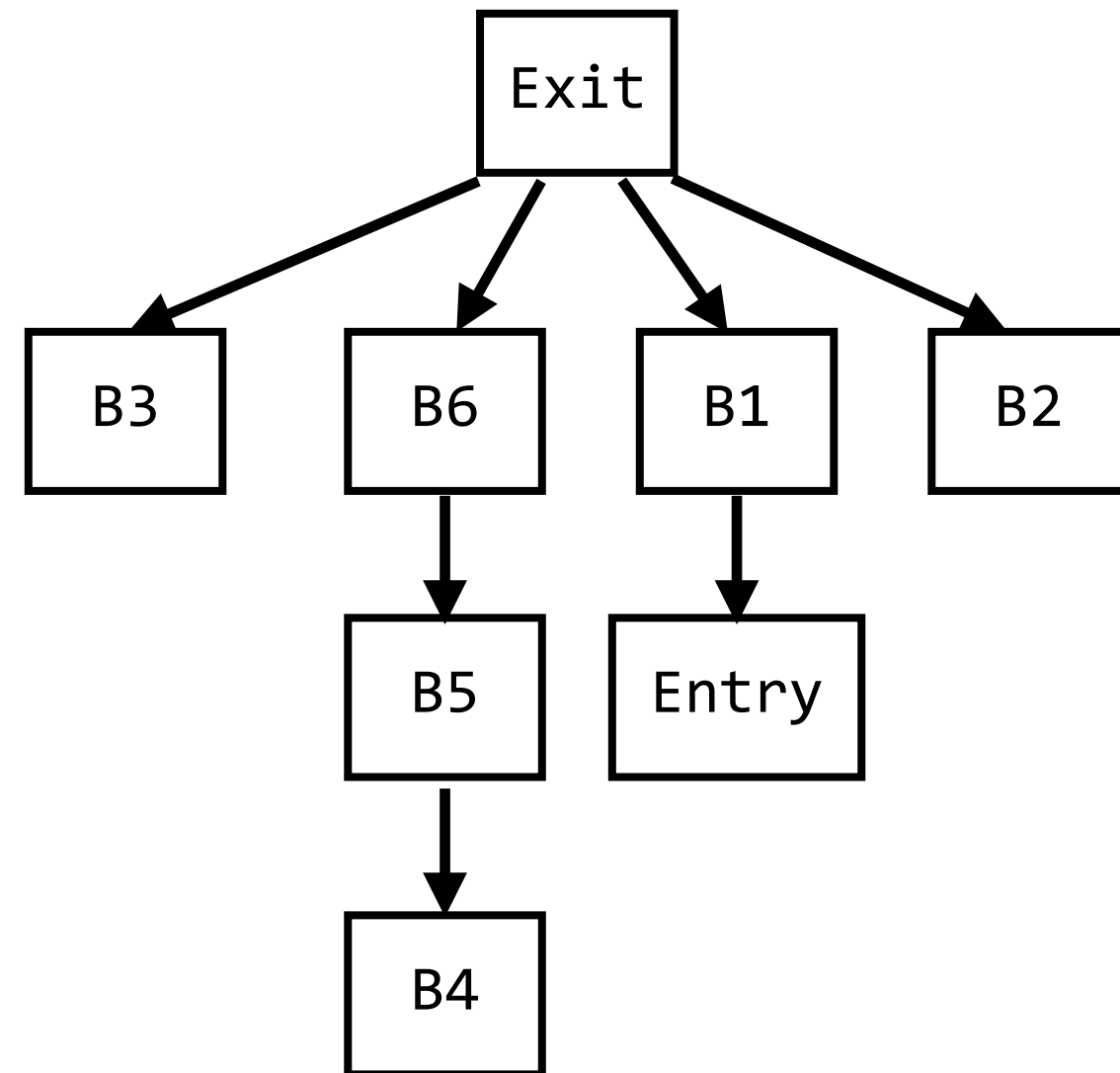
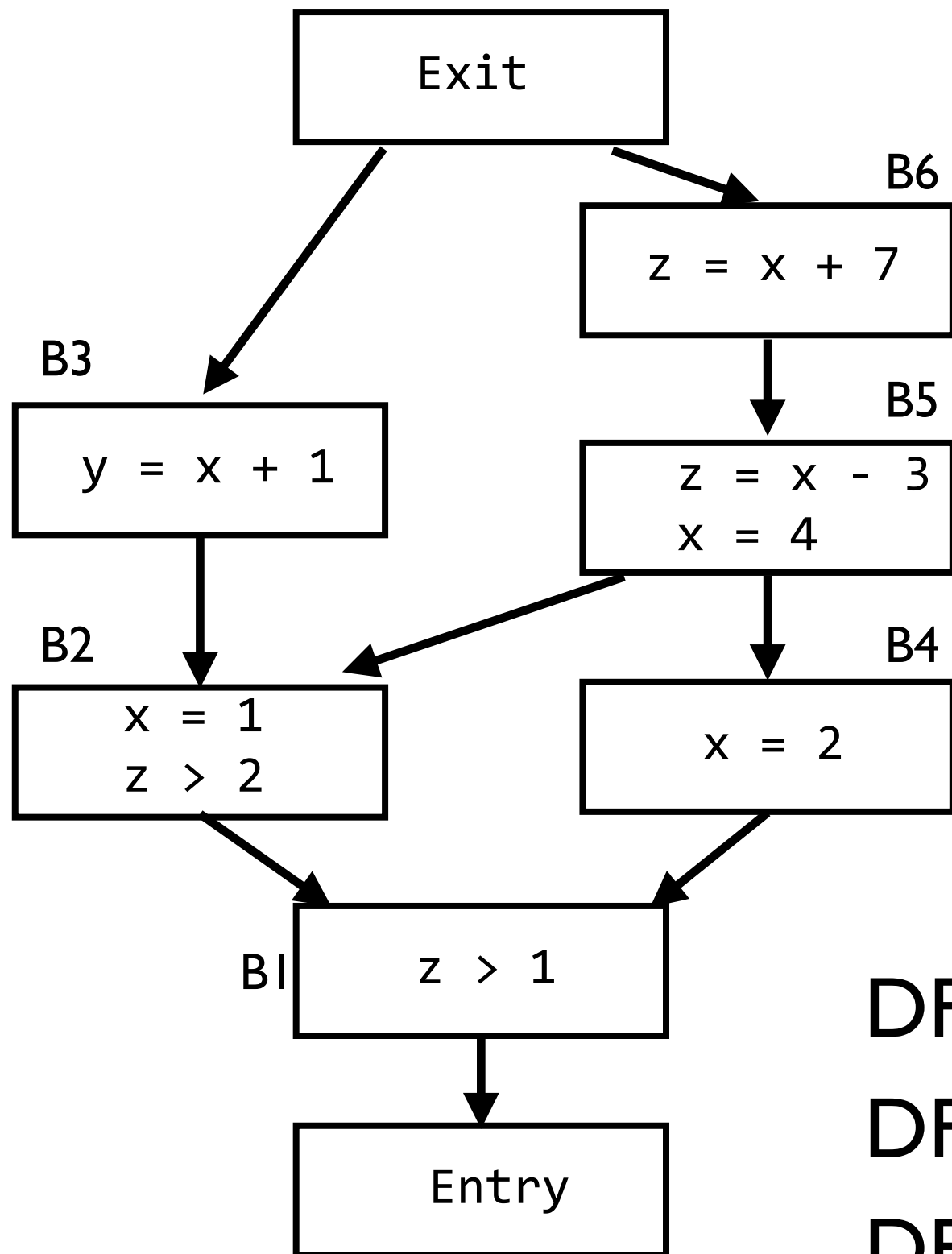
```
for each statement  $S$  in block  $n$   
  if  $S$  is not a  $\phi$ -function  
    for each use of some variable  $x$  in  $S$   
       $i \leftarrow \text{top}(Stack[x])$   
      replace the use of  $x$  with  $x_i$  in  $S$   
    for each definition of some variable  $a$  in  $S$   
       $Count[a] \leftarrow Count[a] + 1$   
       $i \leftarrow Count[a]$   
      push  $i$  onto  $Stack[a]$   
      replace definition of  $a$  with definition of  $a_i$  in  $S$   
  for each successor  $Y$  of block  $n$ ,  
    Suppose  $n$  is the  $j$ th predecessor of  $Y$   
    for each  $\phi$ -function in  $Y$   
      suppose the  $j$ th operand of the  $\phi$ -function is  $a$   
       $i \leftarrow \text{top}(Stack[a])$   
      replace the  $j$ th operand with  $a_i$   
  for each child  $X$  of  $n$   
    Rename( $X$ )  
for each definition of some variable  $a$  in the original  $S$   
  pop  $Stack[a]$ 
```

Control Dependence & Dominance Frontiers

- Control dependence can be computed using dominance frontiers (not data dependence, not program dependence)
- Control dependencies are dominance frontiers on the **reverse control flow graph**

Example





$DF(\text{Entry}) = \{\}$

$DF(B1) = \{\}$

$DF(B2) = \{B1\}$

$DF(B3) = \{B2\}$

$DF(B4) = \{B1\}$

$DF(B5) = \{B1, B2\}$

$DF(B6) = \{B1, B2\}$

$DF(\text{Exit}) = \{\}$

$DF(Entry) = \{\}$

$DF(B4) = \{BI\}$

$DF(B1) = \{\}$

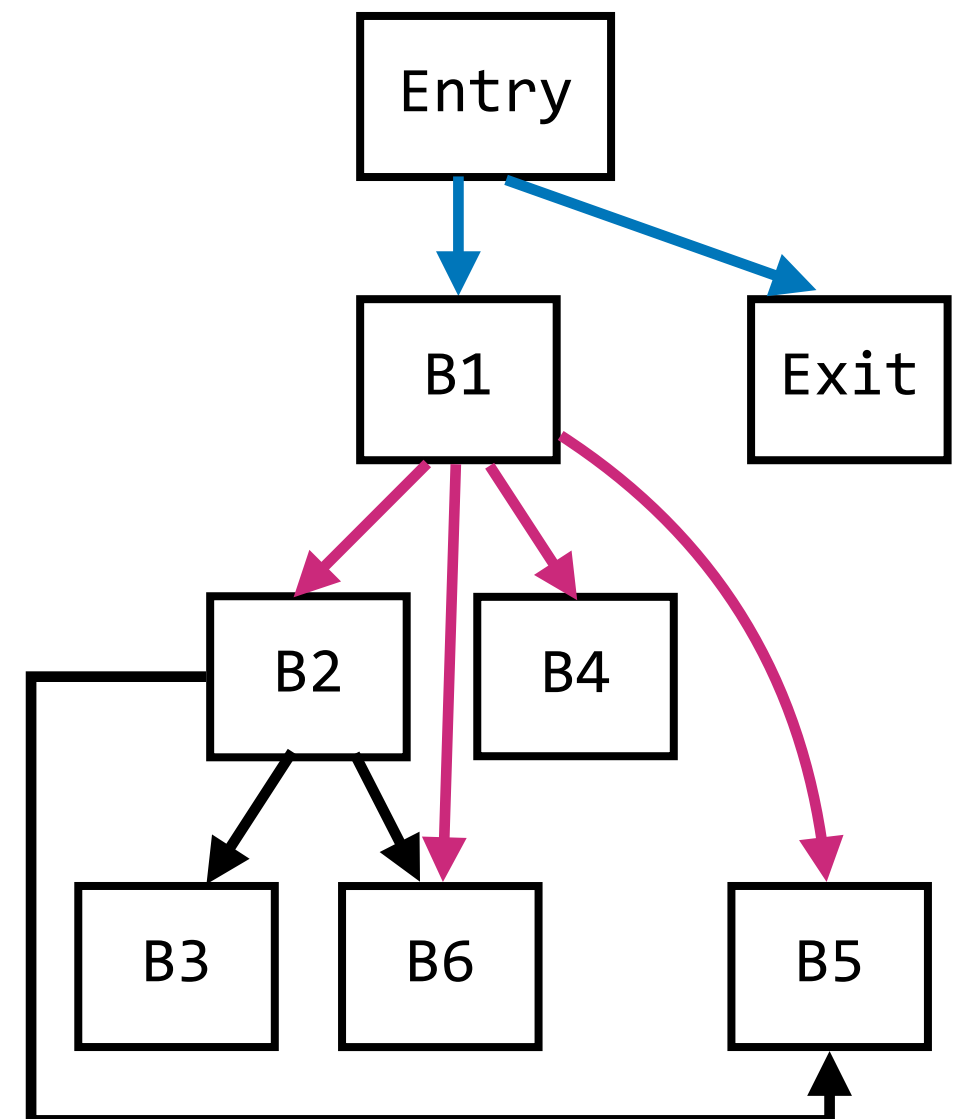
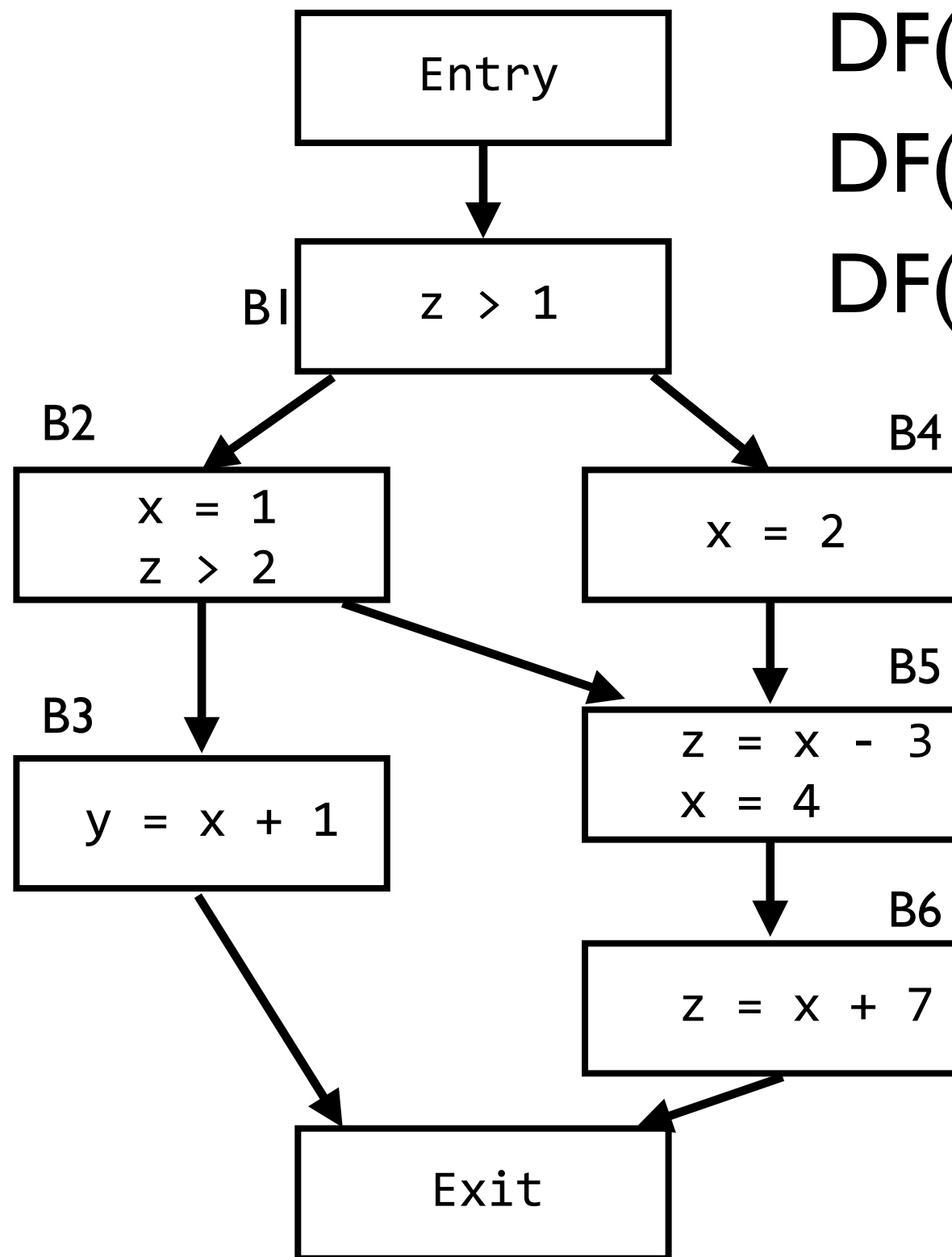
$DF(B5) = \{BI, B2\}$

$DF(B2) = \{BI\}$

$DF(B6) = \{BI, B2\}$

$DF(B3) = \{B2\}$

$DF(Exit) = \{\}$



Outline

1. Static Single Assignment Form
2. Slicing
3. Interprocedural Slicing
4. Dynamic Slicing

Program Dependence Graph

- Nodes: as in the control flow graph (CFG)
- Edges: Control Dependences + Data Dependences
- Complete overview of dependences in a program!

Slicing

Slicing

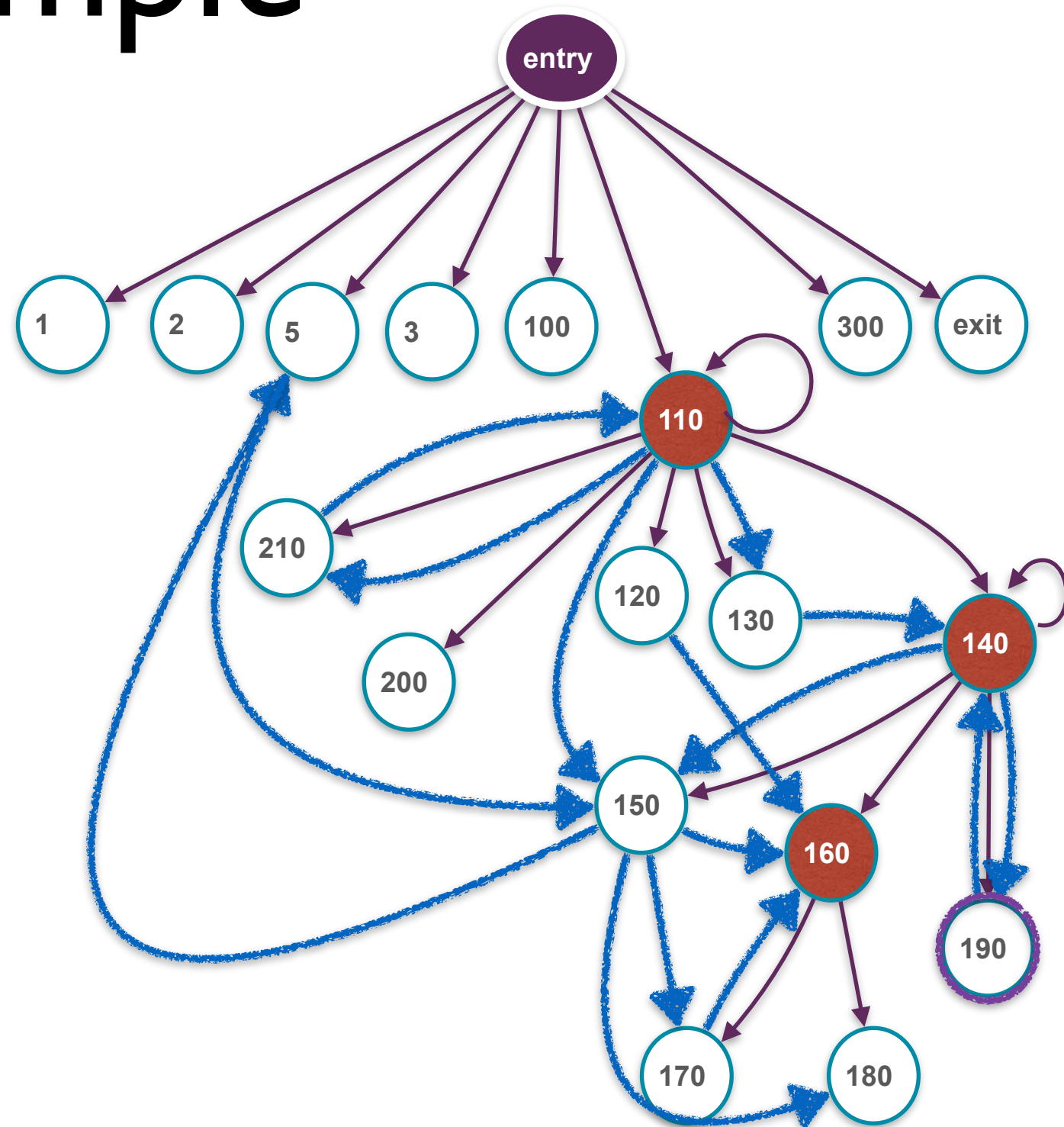
I am concerned about the value of variables V at statement s .

What set of statements could have affected V at s ?

- Backwards slice:
 - Select target node in program dependence graph
 - Slice = all nodes reachable by backwards traversal
- Forward slice:
 - Select target node in program dependence graph
 - Slice = all nodes reachable by forwards traversal

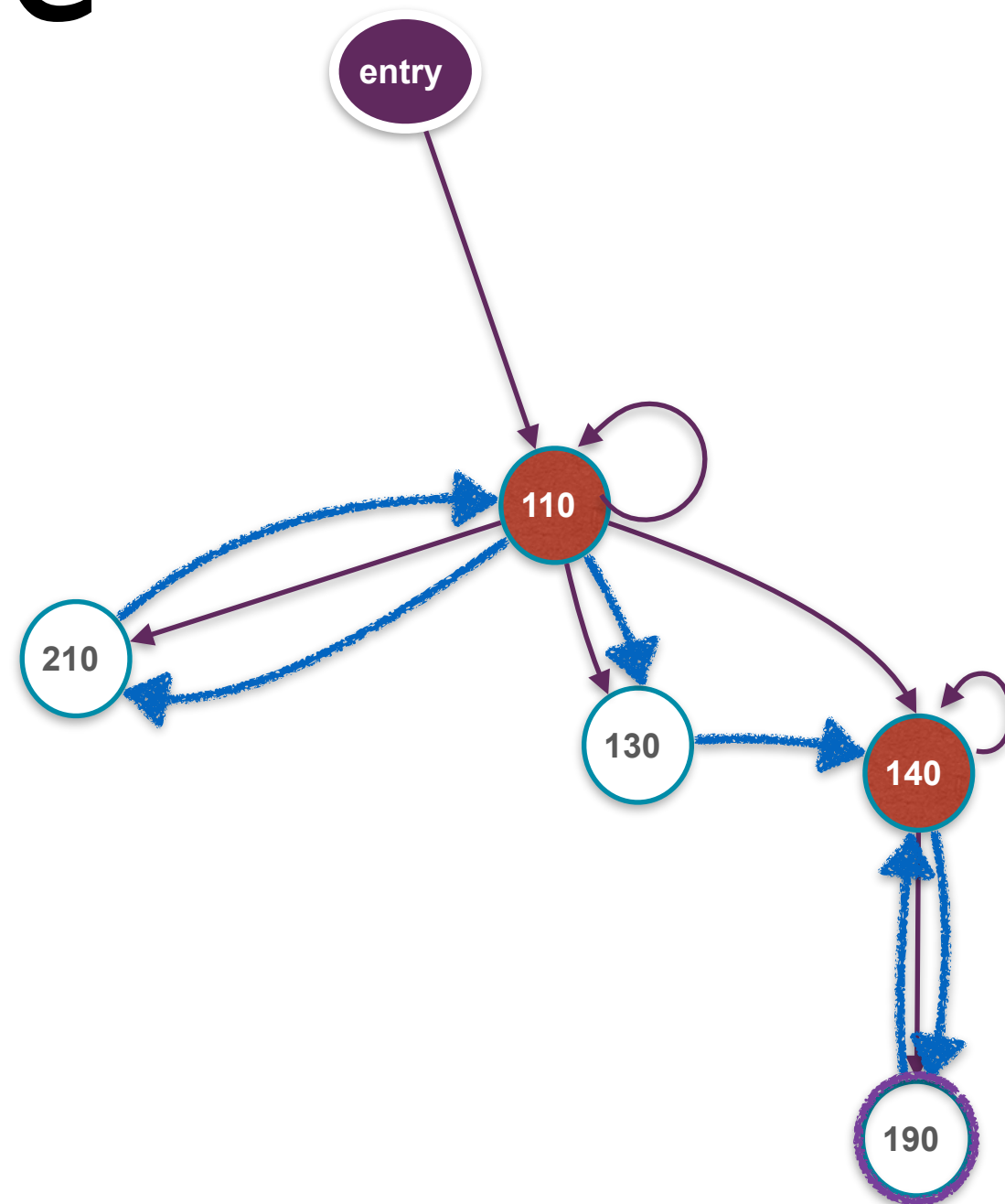
Example

```
1 PRINT TAB(32);"3D PLOT"  
2 PRINT TAB(15);"CREATIVE COMPUTING"  
3 PRINT:PRINT:PRINT  
5 DEF FNA(Z)=30*EXP(-Z*Z/100)  
100 PRINT  
110 FOR X=-30 TO 30 STEP 1.5  
  120 L=0  
  130 Y1=5*INT(SQR(900-X*X)/5)  
  140 FOR Y=Y1 TO -Y1 STEP -5  
    150 Z=INT(25+FNA(SQR(X*X+Y*Y))-.7*Y)  
    160 IF Z<=L THEN 190  
    170 L=Z  
    180 PRINT TAB(Z);"*";  
    190 NEXT Y  
  200 PRINT  
  210 NEXT X  
300 END
```



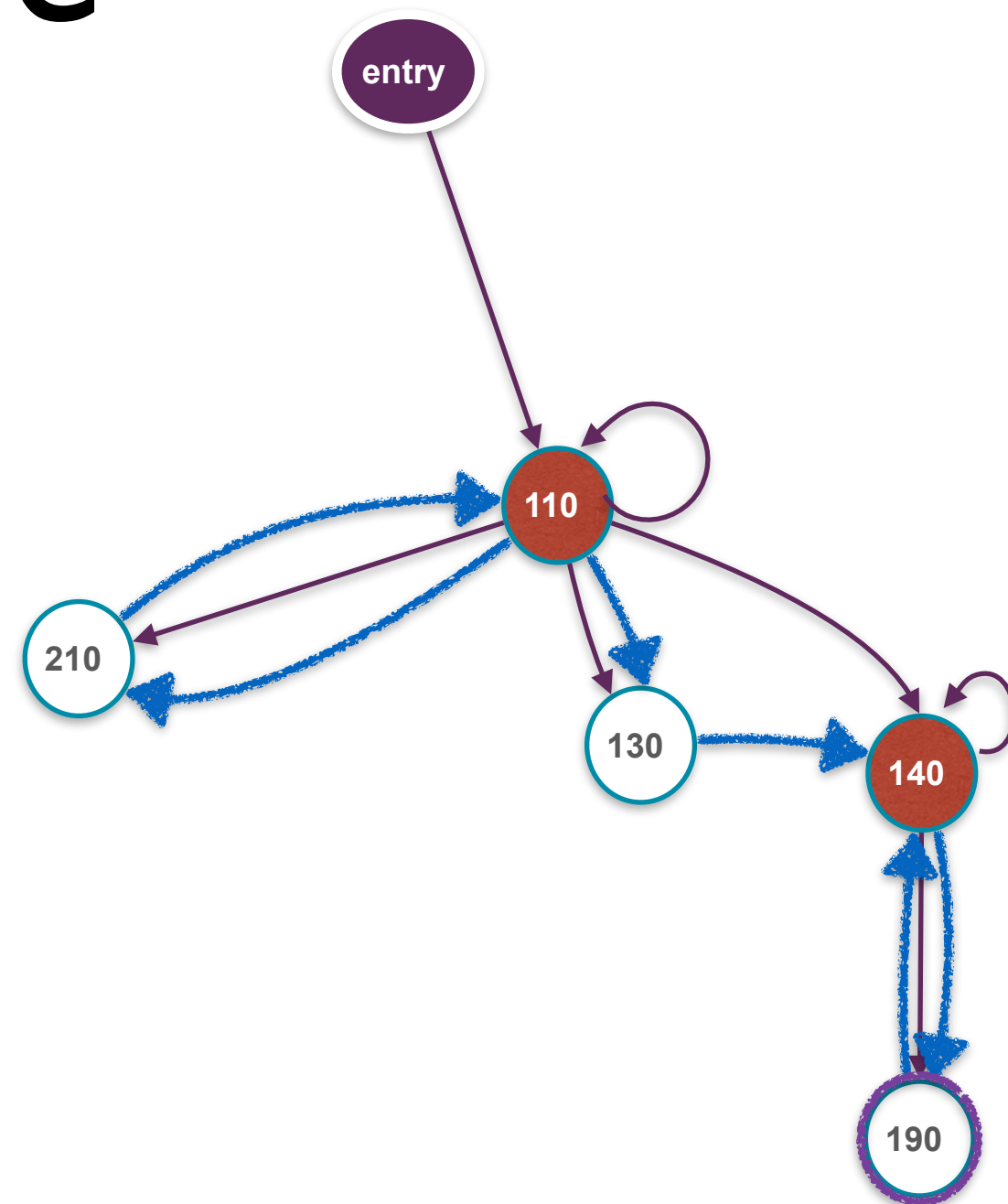
Example

```
1 PRINT TAB(32);"3D PLOT"  
2 PRINT TAB(15);"CREATIVE COMPUTING"  
3 PRINT:PRINT:PRINT  
5 DEF FNA(Z)=30*EXP(-Z*Z/100)  
100 PRINT  
110 FOR X=-30 TO 30 STEP 1.5  
  120 L=0  
  130 Y1=5*INT(SQR(900-X*X)/5)  
  140 FOR Y=Y1 TO -Y1 STEP -5  
    150 Z=INT(25+FNA(SQR(X*X+Y*Y))-.7*Y)  
    160 IF Z<=L THEN 190  
    170 L=Z  
    180 PRINT TAB(Z);"*";  
    190 NEXT Y  
  200 PRINT  
  210 NEXT X  
300 END
```



Example

```
1 PRINT TAB(32);"3D PLOT"  
2 PRINT TAB(15);"CREATIVE COMPUTING"  
3 PRINT:PRINT:PRINT  
5 DEF FNA(Z)=30*EXP(-Z*Z/100)  
100 PRINT  
110 FOR X=-30 TO 30 STEP 1.5  
  120 L=0  
  130 Y1=5*INT(SQR(900-X*X)/5)  
  140 FOR Y=Y1 TO -Y1 STEP -5  
    150 Z=INT(25+FNA(SQR(X*X+Y*Y))-.7*Y)  
    160 IF Z<=L THEN 190  
    170 L=Z  
    180 PRINT TAB(Z);"*";  
    190 NEXT Y  
  200 PRINT  
  210 NEXT X  
300 END
```

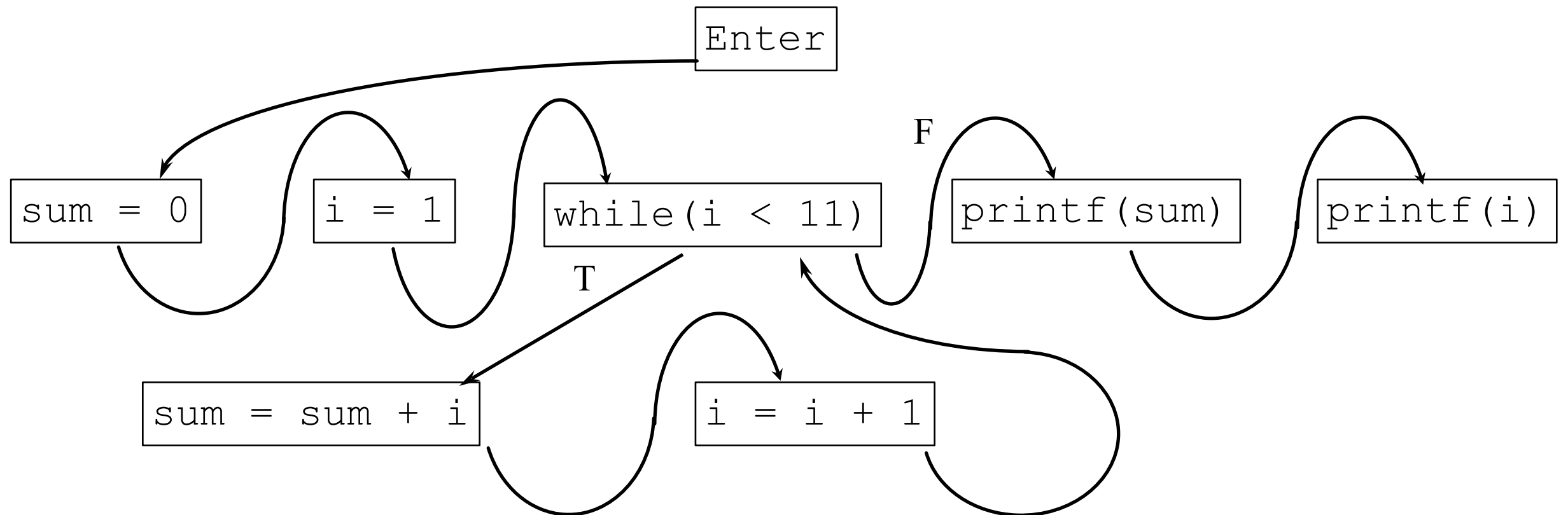


Example

```
int main() {  
    int sum = 0;  
    int i = 1;  
    while (i < 11) {  
        sum = sum + i;  
        i = i + 1;  
    }  
    printf("%d\n", sum);  
    printf("%d\n", i);  
}
```

Control Flow Graph

```
int main() {  
    int sum = 0;  
    int i = 1;  
    while (i < 11) {  
        sum = sum + i;  
        i = i + 1;  
    }  
    printf("%d\n", sum);  
    printf("%d\n", i);  
}
```



Data Dependence Graph

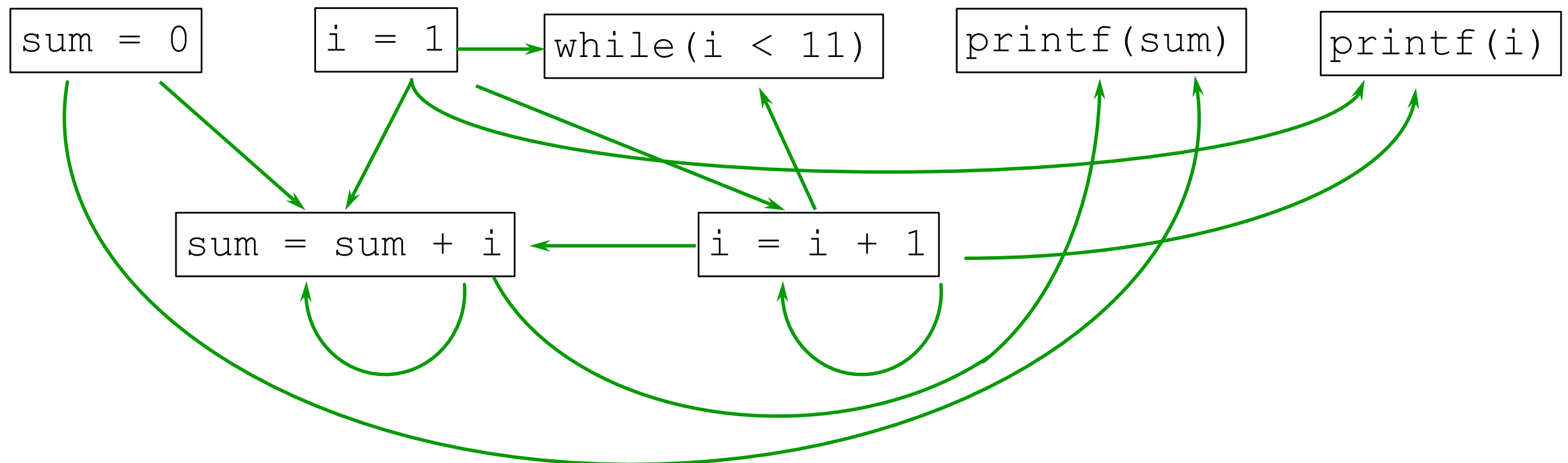
```
int main() {  
    int sum = 0;  
    int i = 1;  
    while (i < 11) {  
        sum = sum + i;  
        i = i + 1;  
    }  
    printf("%d\n", sum);  
    printf("%d\n", i);  
}
```

Data dependence

$p \rightarrow q$

Value of variable
assigned at p may be
used at q .

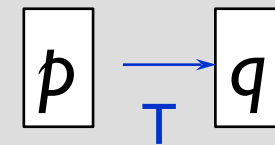
Enter



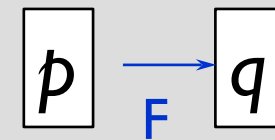
Control Dependence Graph

```
int main() {  
    int sum = 0;  
    int i = 1;  
    while (i < 11) {  
        sum = sum + i;  
        i = i + 1;  
    }  
    printf("%d\n", sum);  
    printf("%d\n", i);  
}
```

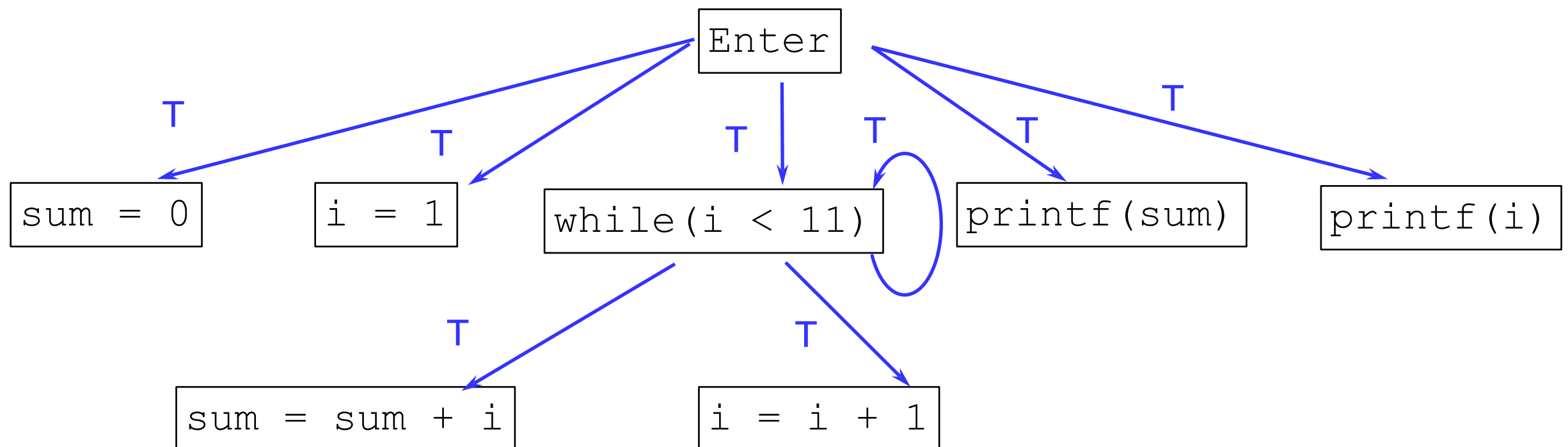
Control dependence



q is reached from p if condition p is true (T), not otherwise.

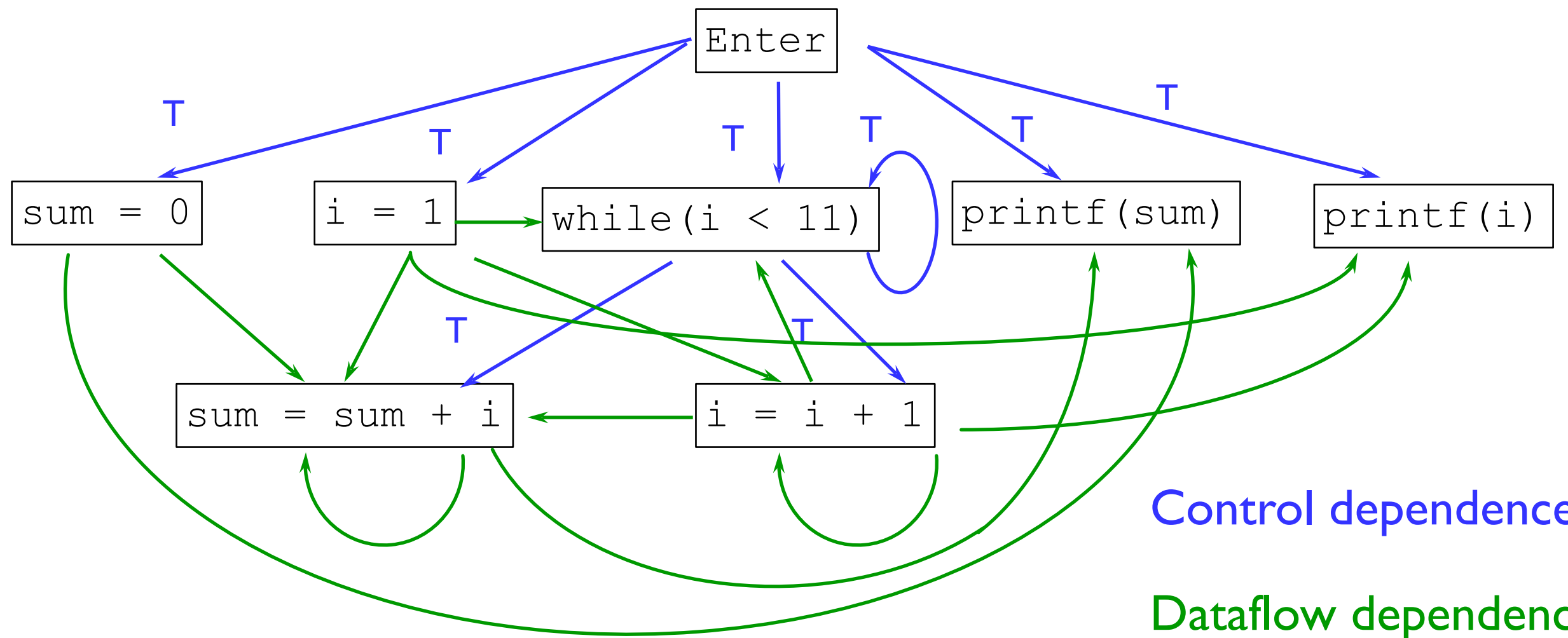


Similar for false (F).



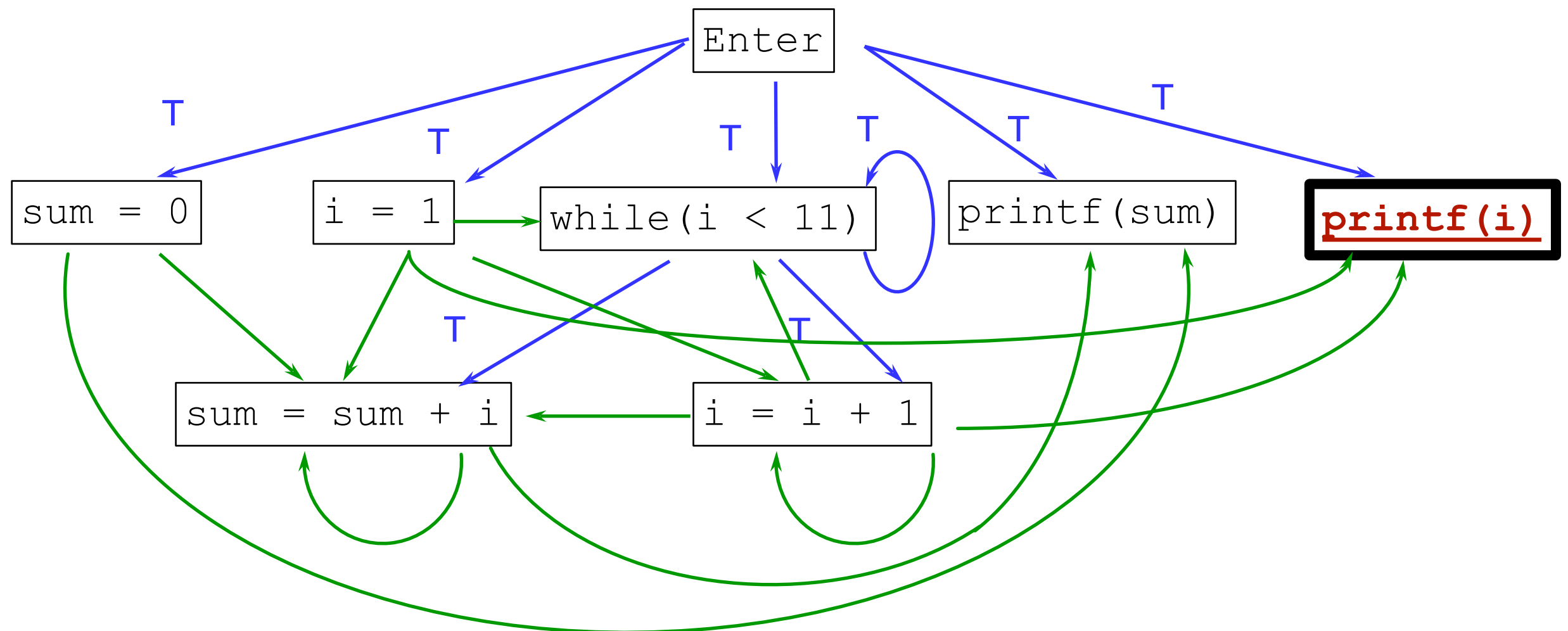
Program Dependence Graph (PDG)

```
int main() {  
    int sum = 0;  
    int i = 1;  
    while (i < 11) {  
        sum = sum + i;  
        i = i + 1;  
    }  
    printf("%d\n", sum);  
    printf("%d\n", i);  
}
```



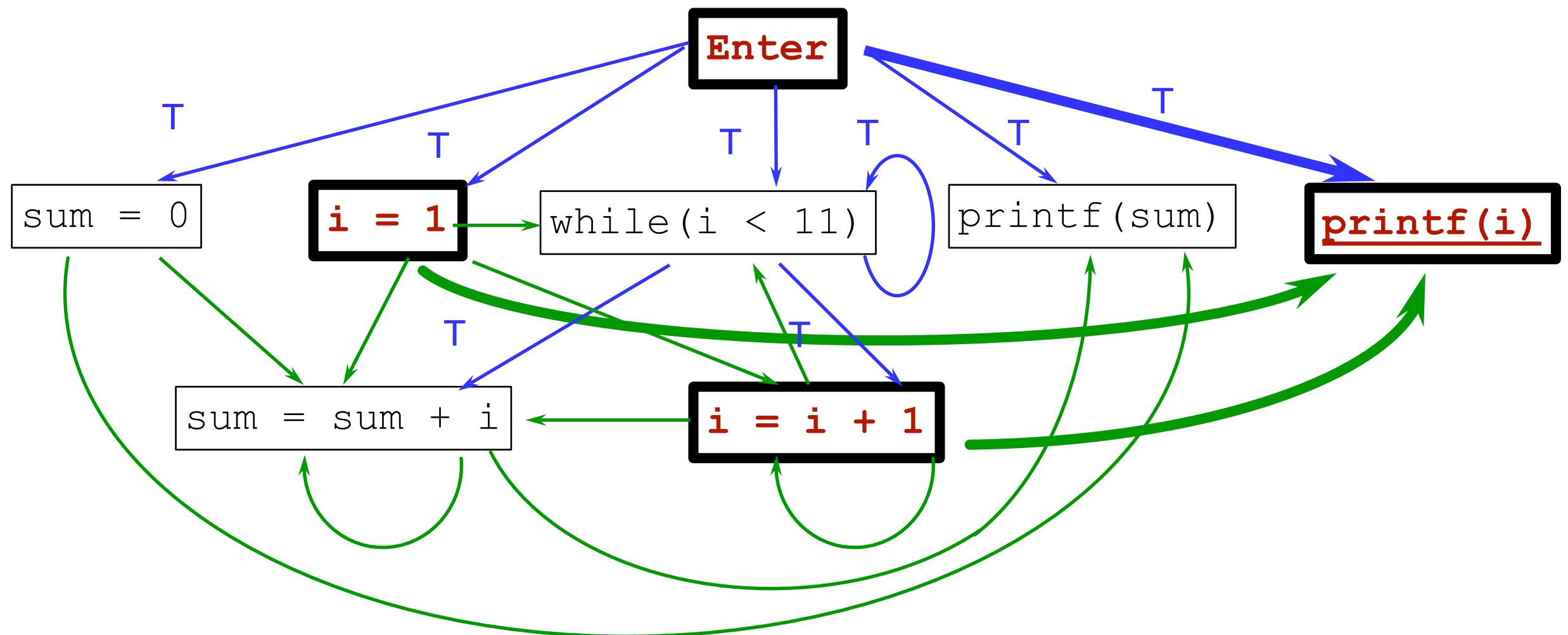
Backward Slice

```
int main() {  
    int sum = 0;  
    int i = 1;  
    while (i < 11) {  
        sum = sum + i;  
        i = i + 1;  
    }  
    printf("%d\n", sum);  
    printf("%d\n", i);  
}
```



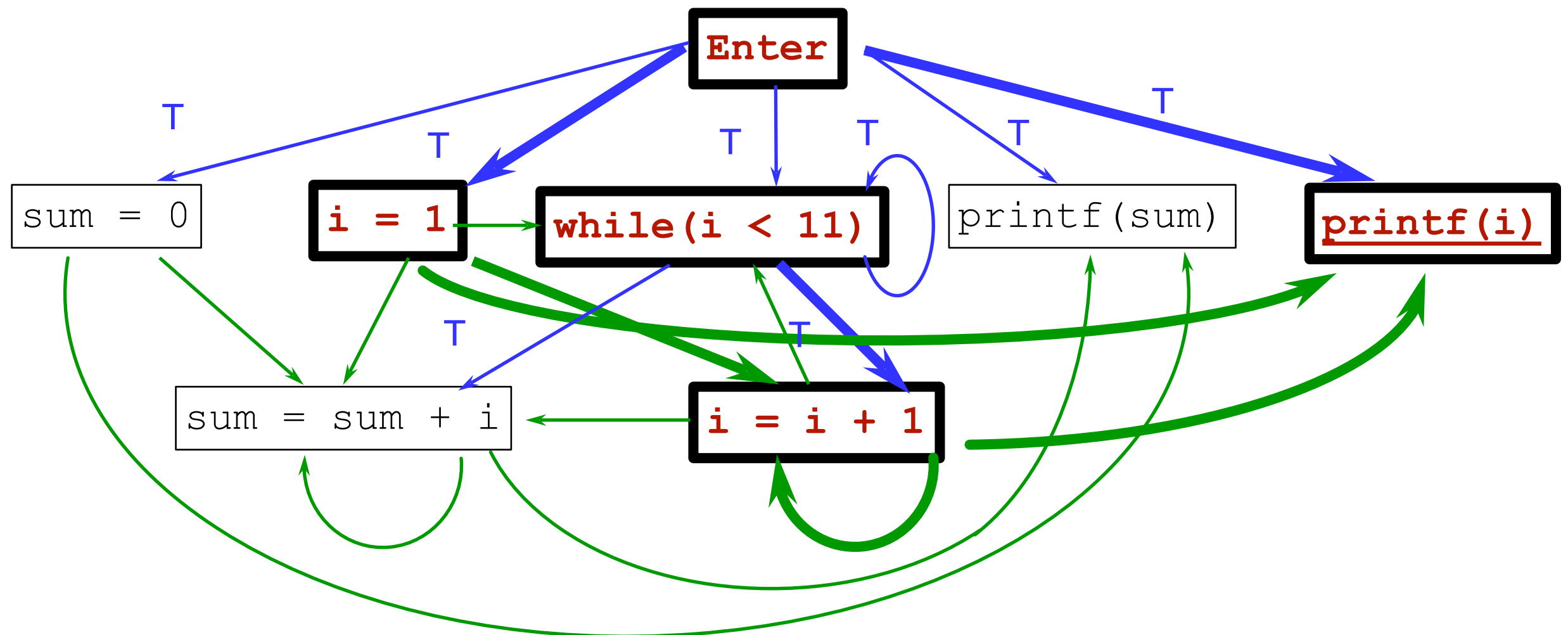
Backward Slice

```
int main() {  
    int sum = 0;  
    int i = 1;  
    while (i < 11) {  
        sum = sum + i;  
        i = i + 1;  
    }  
    printf("%d\n", sum);  
    printf("%d\n", i);  
}
```



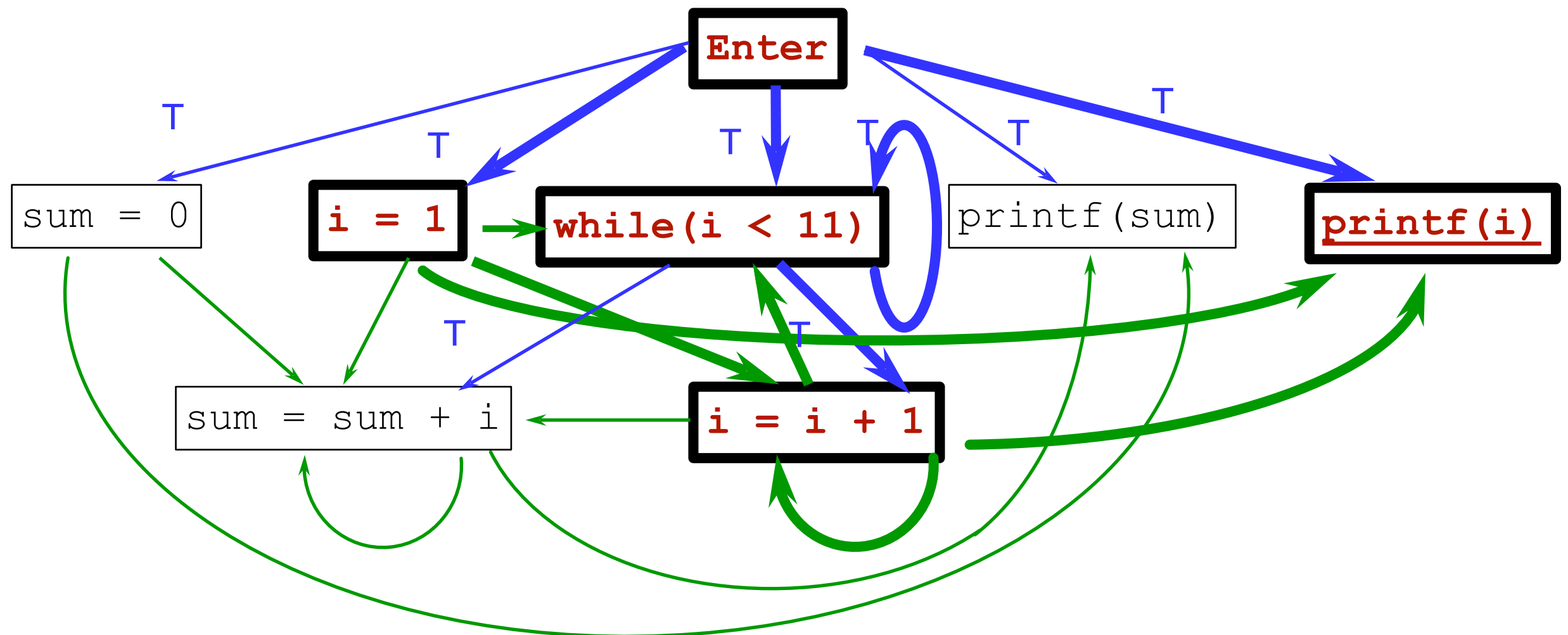
Backward Slice

```
int main() {  
    int sum = 0;  
    int i = 1;  
    while (i < 11) {  
        sum = sum + i;  
        i = i + 1;  
    }  
    printf("%d\n", sum);  
    printf("%d\n", i);  
}
```



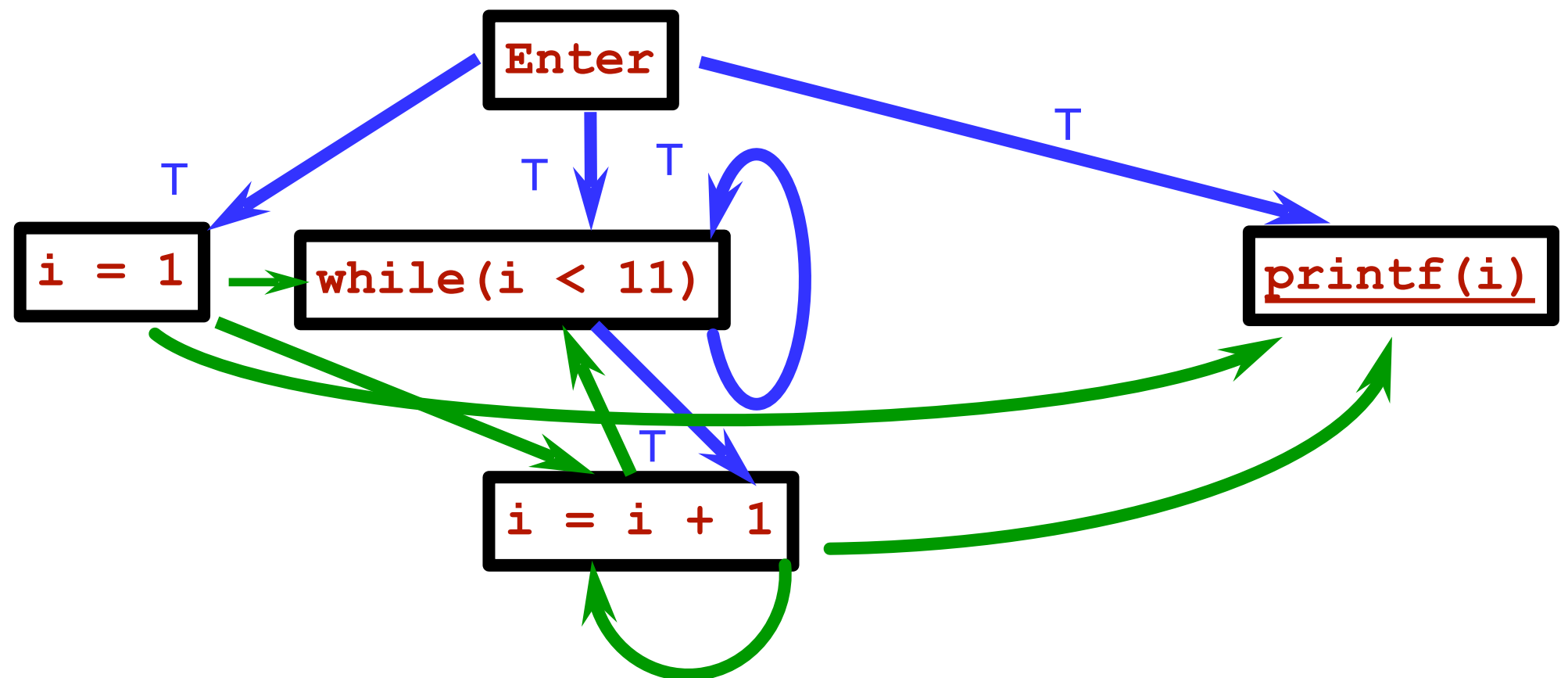
Backward Slice

```
int main() {  
    int sum = 0;  
    int i = 1;  
    while (i < 11) {  
        sum = sum + i;  
        i = i + 1;  
    }  
    printf("%d\n", sum);  
    printf("%d\n", i);  
}
```



Slice Extraction

```
int main() {  
  
    int i = 1;  
    while (i < 11) {  
        i = i + 1;  
    }  
  
    printf("%d\n", i);  
}
```



Interprocedural Slicing

System Dependence Graph

- SDG is a set of connected extended PDGs
- Each PDG is extended to have nodes for procedure parameters and function result
 - Entry node
 - Formal-in nodes
 - Formal-out nodes
- Each call statement is extended with
 - Call-site node
 - Actual-in nodes
 - Actual-out nodes
- Appropriate edges (intra and inter)
 - Call-site node to actual-in/out (control-dependence)
 - Entry node to formal-in/out (control-dependence)
 - Call-site node to entry node (control dependence)
 - Parameter-in edges, from actual-in to formal-in (data-dependence)
 - Parameter-out edges, from formal-out to actual-out (data-dependence)
 - Summary edges, between formal in and formal out (data-dependence)

```

int main() {
    int sum = 0;
    int i = 1;
    while (i < 11) {
        sum = add(sum,i);
        i = add(i,1);
    }
    printf("%d\n",sum);
    printf("%d\n",i);
}

int add(int x, int y) {
    x = x + y;
    return x;
}

```

1. Before the call, the calling procedure copies actual parameters to temporary values
2. Formal parameters of the called procedure are initialized using the corresponding temporary values
3. Before the return, the called procedure copies the final values of the formal parameters to the temporary variables
4. After returning, the calling procedure updates the actual parameters by copying the values of the corresponding temporary variables

```

int main() {
    int sum = 0;
    int i = 1;
    while (i < 11) {
        xin = sum;
        yin = i;
        sum = add(sum,i);
        i = add(i,1);
    }
    printf("%d\n",sum);
    printf("%d\n",i);
}

```

```

int add(int x, int y) {
    x = xin;
    y = yin;
    x = x + y;
    return x;
}

```

1. Before the call, the calling procedure copies actual parameters to temporary values
2. Formal parameters of the called procedure are initialized using the corresponding temporary values
3. Before the return, the called procedure copies the final values of the formal parameters to the temporary variables
4. After returning, the calling procedure updates the actual parameters by copying the values of the corresponding temporary variables

```

int main() {
    int sum = 0;
    int i = 1;
    while (i < 11) {
        xin = sum;
        yin = i;
        sum = add(sum,i);
        xin = i;
        yin = 1;
        i = add(i,1);
    }
    printf("%d\n",sum);
    printf("%d\n",i);
}

```

```

int add(int x, int y) {
    x = xin;
    y = yin;
    x = x + y;
    return x;
}

```

1. Before the call, the calling procedure copies actual parameters to temporary values
2. Formal parameters of the called procedure are initialized using the corresponding temporary values
3. Before the return, the called procedure copies the final values of the formal parameters to the temporary variables
4. After returning, the calling procedure updates the actual parameters by copying the values of the corresponding temporary variables

```

int main() {
    int sum = 0;
    int i = 1;
    while (i < 11) {
        xin = sum;
        yin = i;
        sum = add(sum,i);
        sum = xout;
        xin = i;
        yin = 1;
        i = add(i,1);
    }
    printf("%d\n",sum);
    printf("%d\n",i);
}

int add(int x, int y) {
    x = xin;
    y = yin;
    x = x + y;
    xout = x;
    return;
}

```

1. Before the call, the calling procedure copies actual parameters to temporary values
2. Formal parameters of the called procedure are initialized using the corresponding temporary values
3. Before the return, the called procedure copies the final values of the formal parameters to the temporary variables
4. After returning, the calling procedure updates the actual parameters by copying the values of the corresponding temporary variables

```

int main() {
    int sum = 0;
    int i = 1;
    while (i < 11) {
        xin = sum;
        yin = i;
        sum = add(sum,i);
        sum = xout;
        xin = i;
        yin = 1;
        i = add(i,1);
        i = xout;
    }
    printf("%d\n",sum);
    printf("%d\n",i);
}

int add(int x, int y) {
    x = xin;
    y = yin;
    x = x + y;
    xout = x;
    return;
}

```

1. Before the call, the calling procedure copies actual parameters to temporary values
2. Formal parameters of the called procedure are initialized using the corresponding temporary values
3. Before the return, the called procedure copies the final values of the formal parameters to the temporary variables
4. After returning, the calling procedure updates the actual parameters by copying the values of the corresponding temporary variables

```

int main() {
    int sum = 0;
    int i = 1;
    while (i < 11) {
        xin = sum;
        yin = i;
        call add;
        sum = xout;
        xin = i;
        yin = 1;
        call add;
        i = xout;
    }
    printf("%d\n", sum);
    printf("%d\n", i);
}

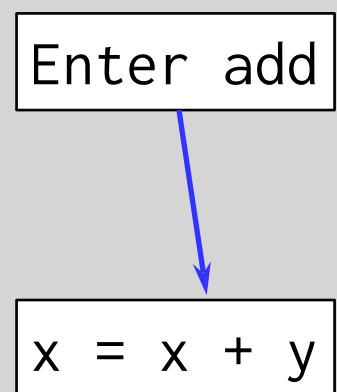
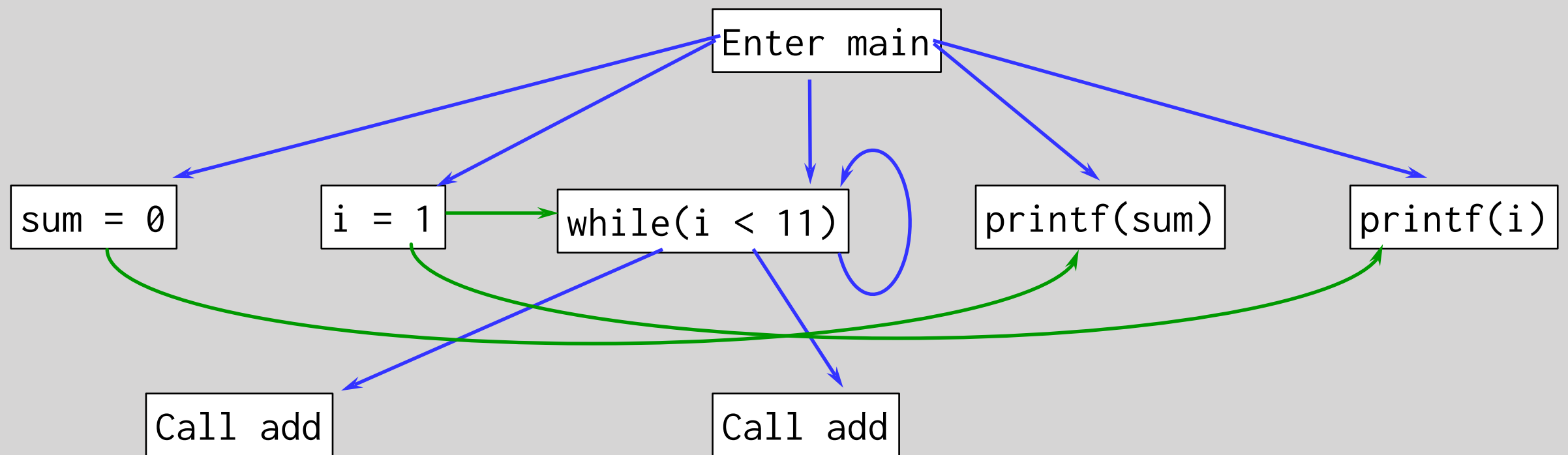
int add(int x, int y) {
    x = xin;
    y = yin;
    x = x + y;
    xout = x;
    return;
}

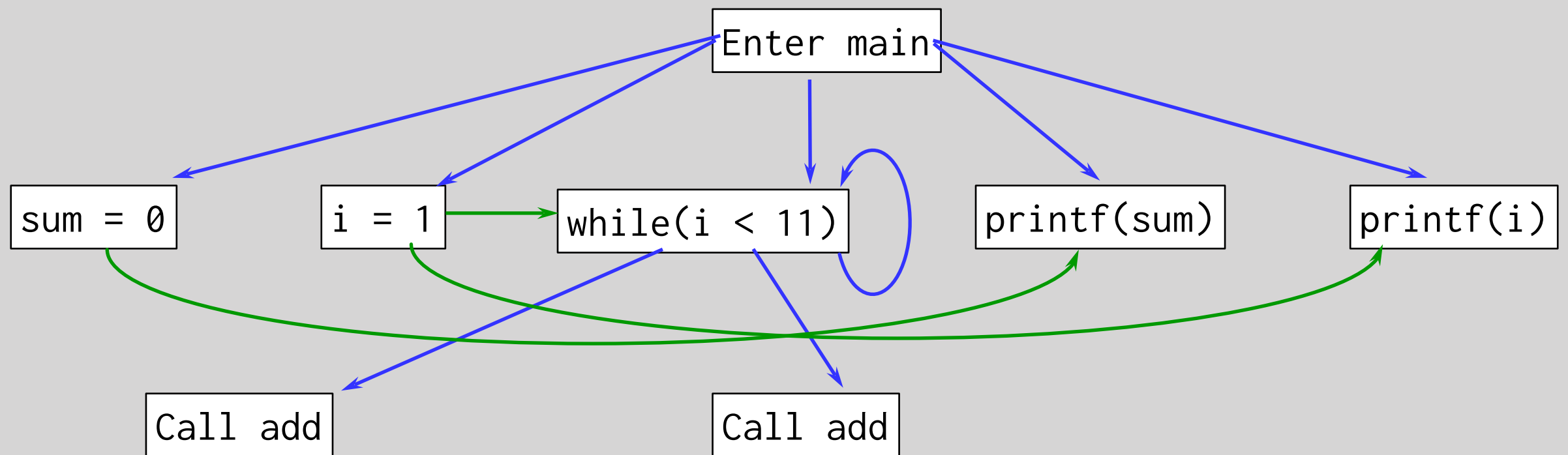
```

1. Before the call, the calling procedure copies actual parameters to temporary values
2. Formal parameters of the called procedure are initialized using the corresponding temporary values
3. Before the return, the called procedure copies the final values of the formal parameters to the temporary variables
4. After returning, the calling procedure updates the actual parameters by copying the values of the corresponding temporary variables

```
int main() {  
    int sum = 0;  
    int i = 1;  
    while (i < 11) {  
        sum = add(sum,i);  
        i = add(i,1);  
    }  
    printf("%d\n",sum);  
    printf("%d\n",i);  
}
```

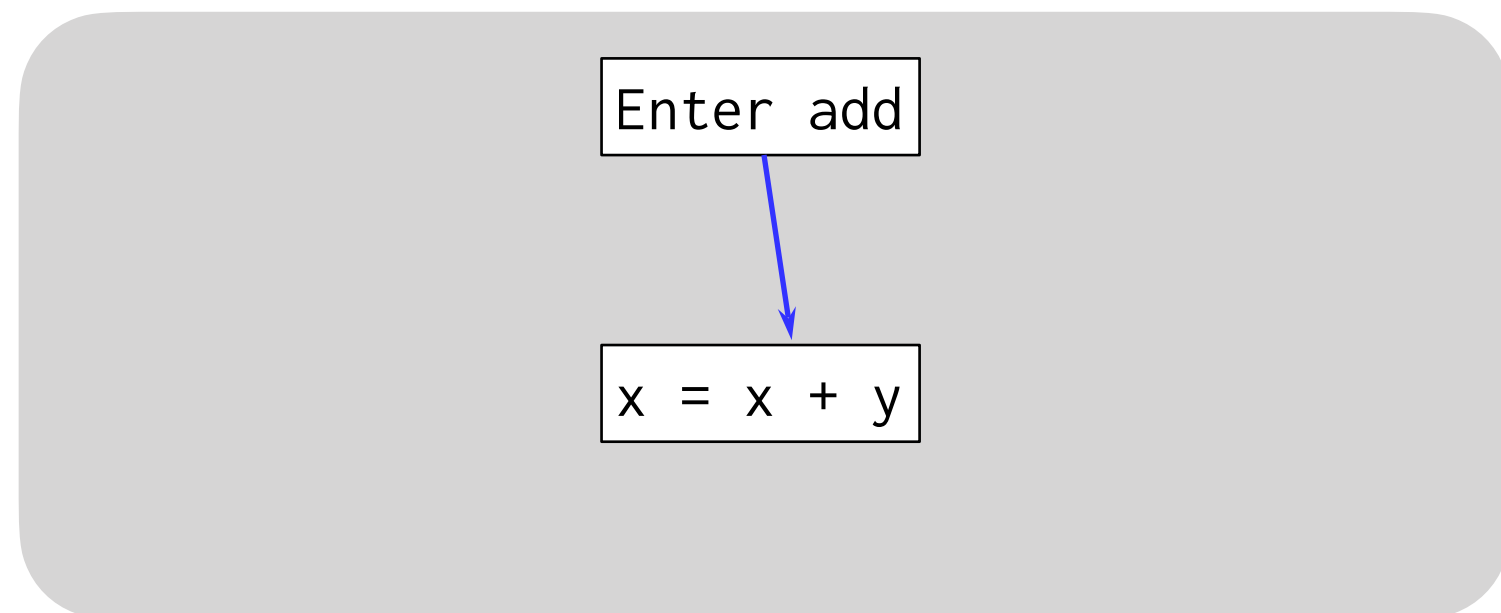
```
int add(int x, int y) {  
    x = x + y;  
    return x;  
}
```

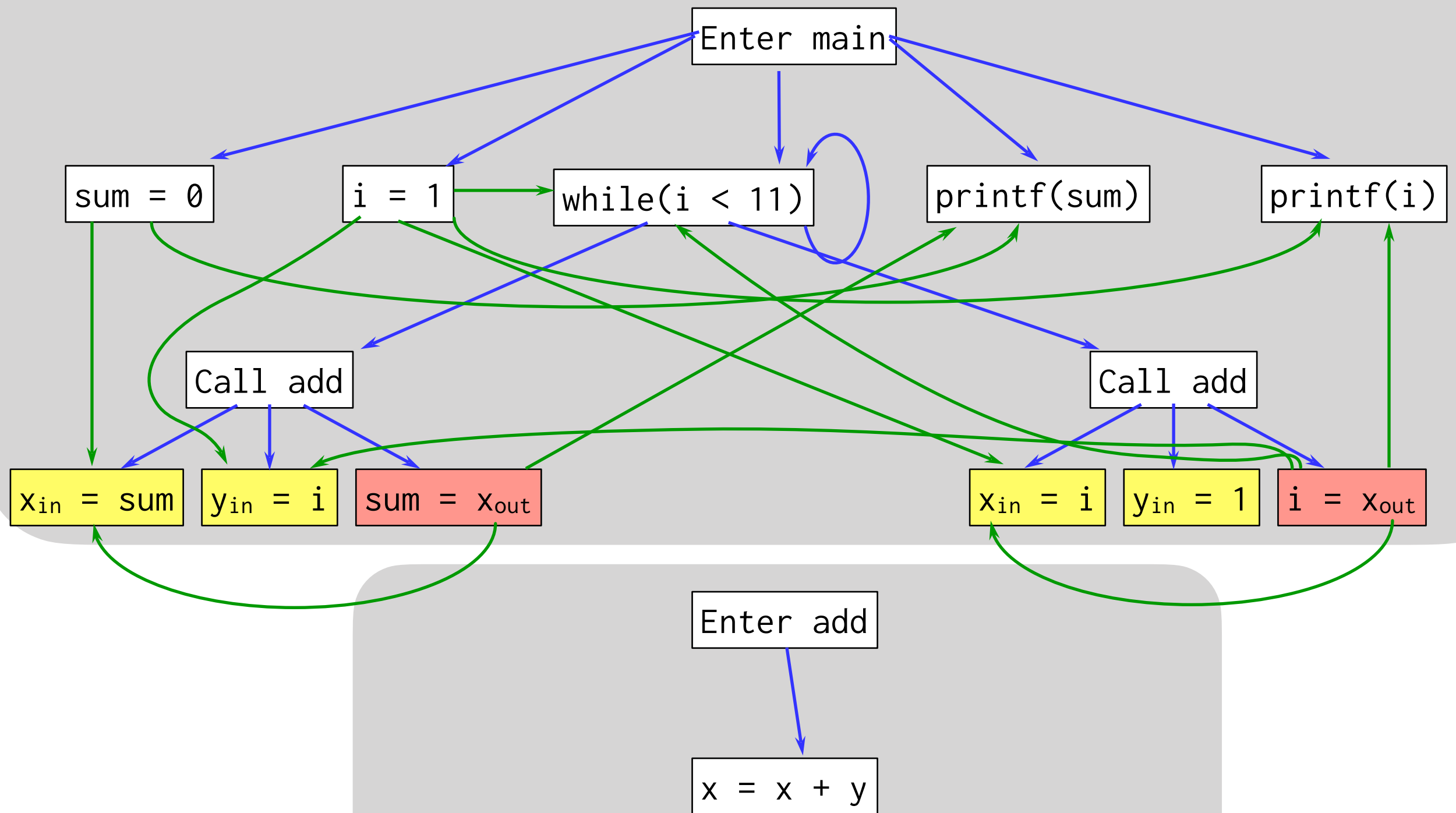


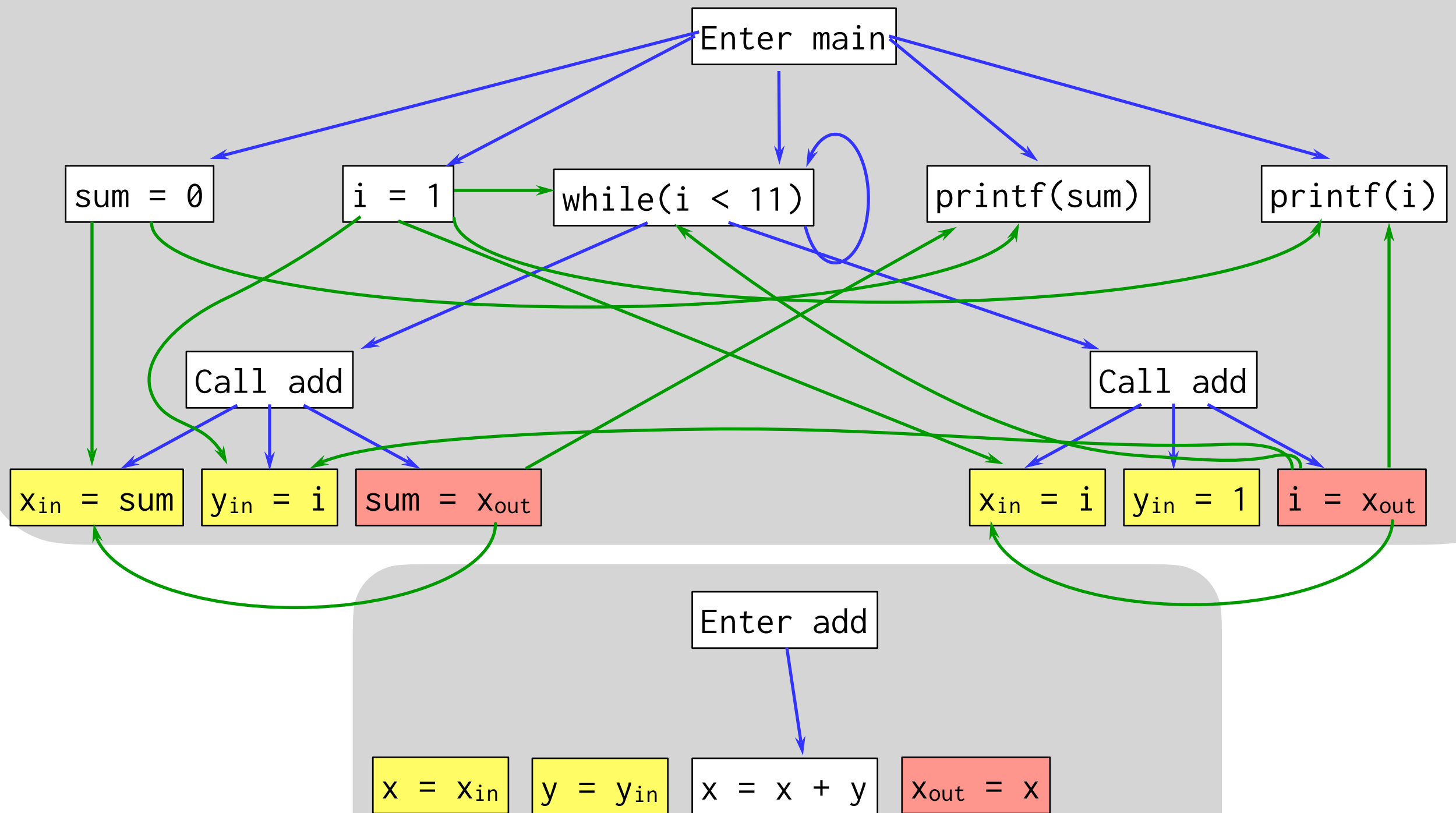


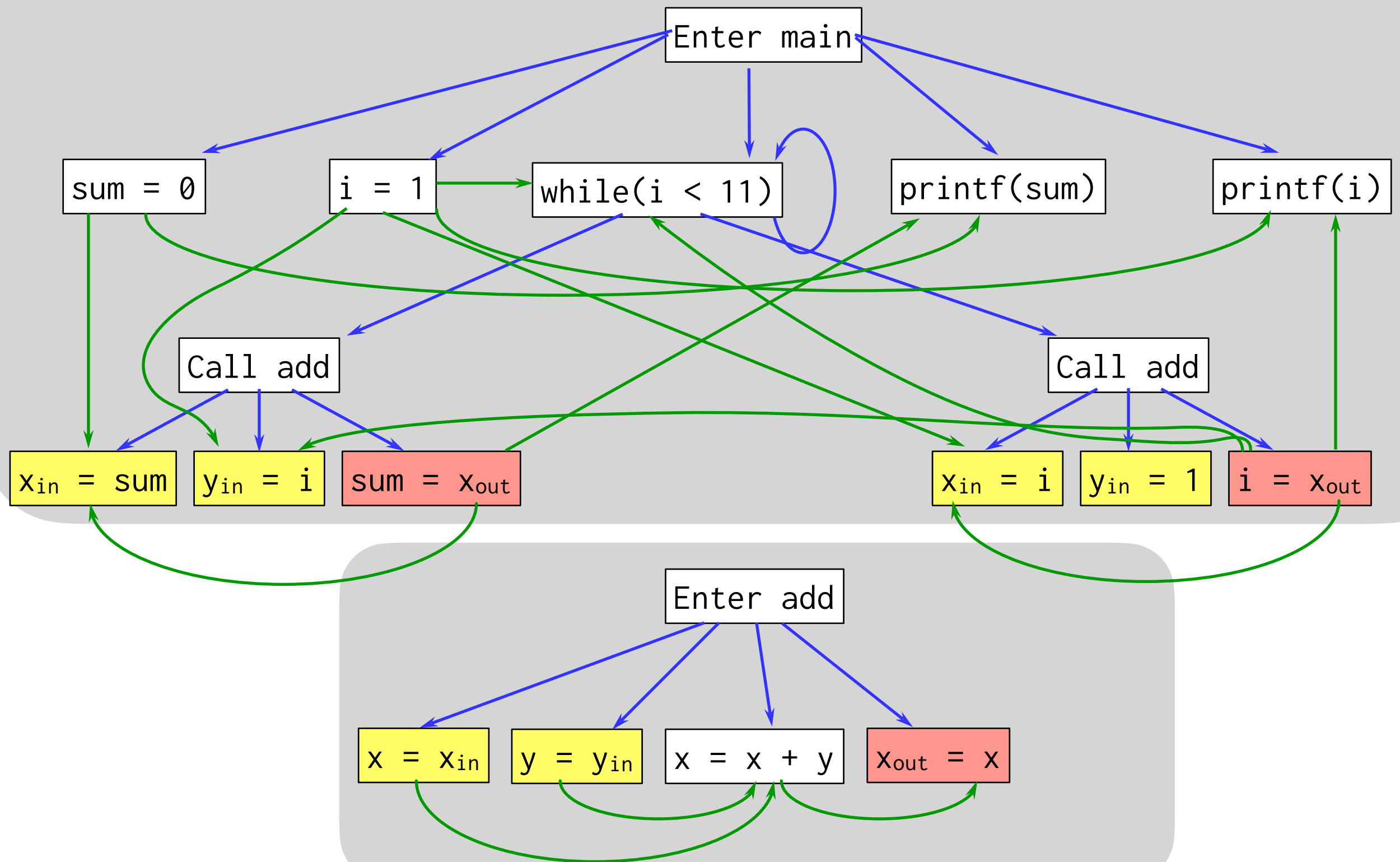
$x_{in} = \text{sum}$ $y_{in} = i$ $\text{sum} = x_{out}$

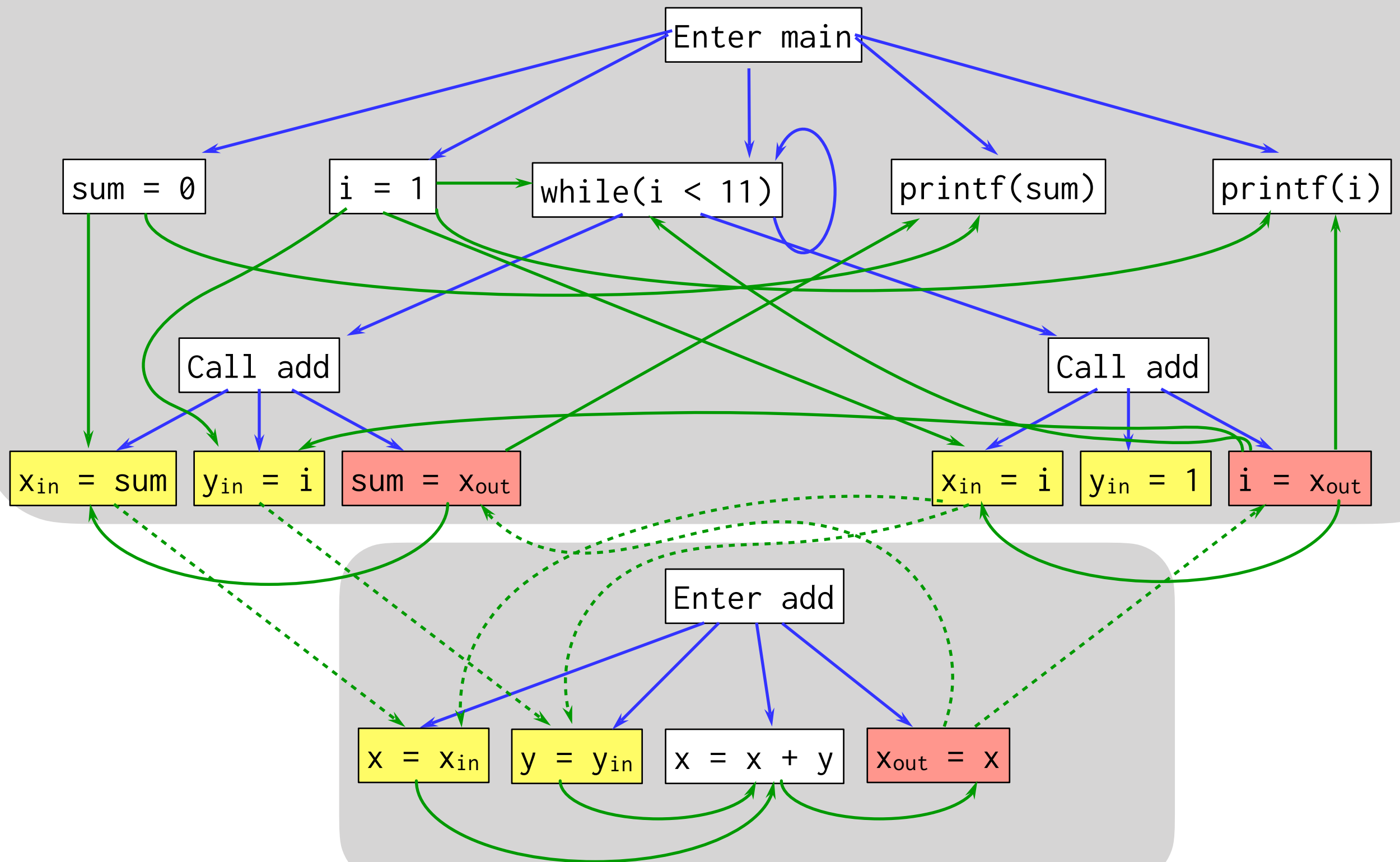
$x_{in} = i$ $y_{in} = 1$ $i = x_{out}$

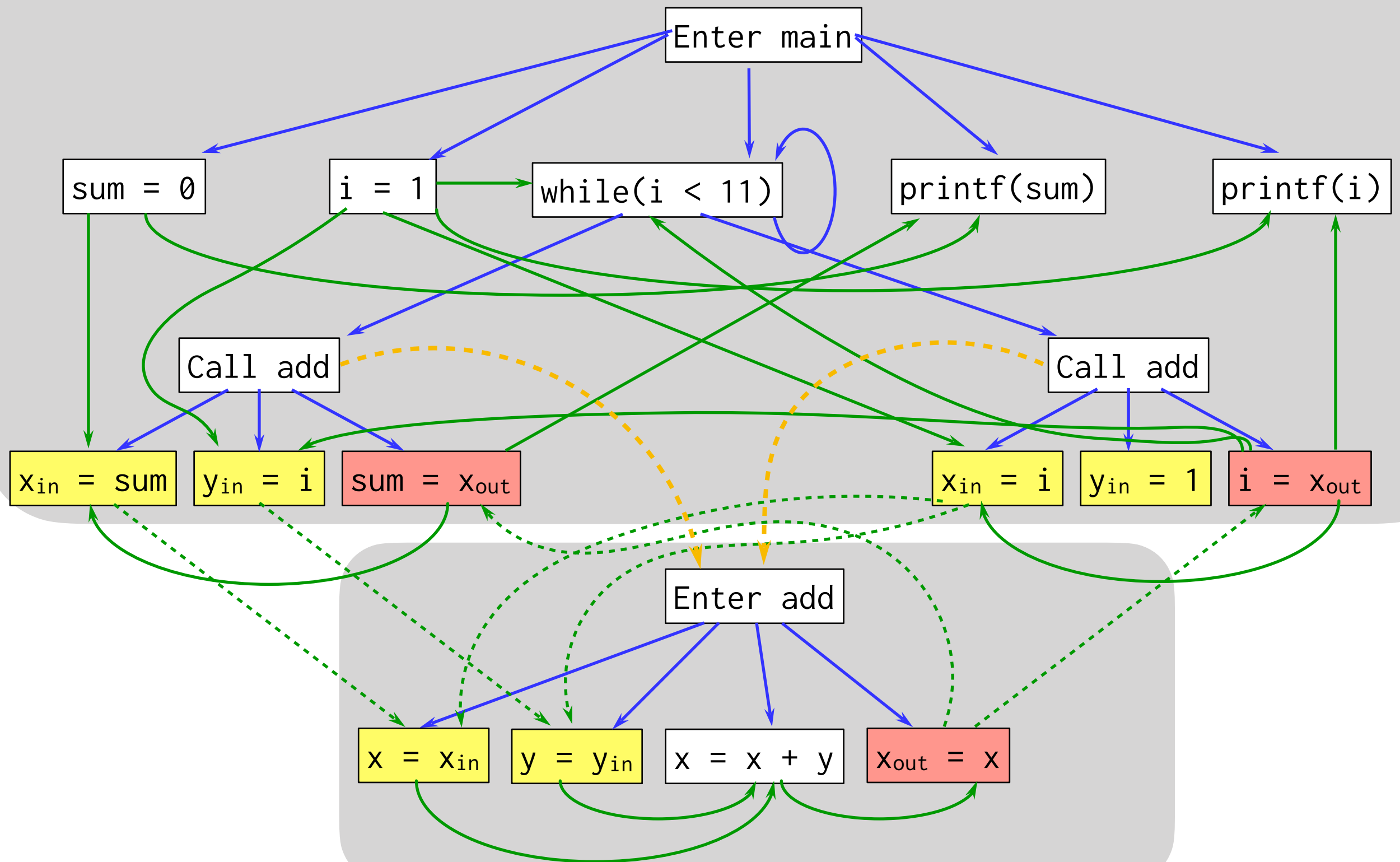








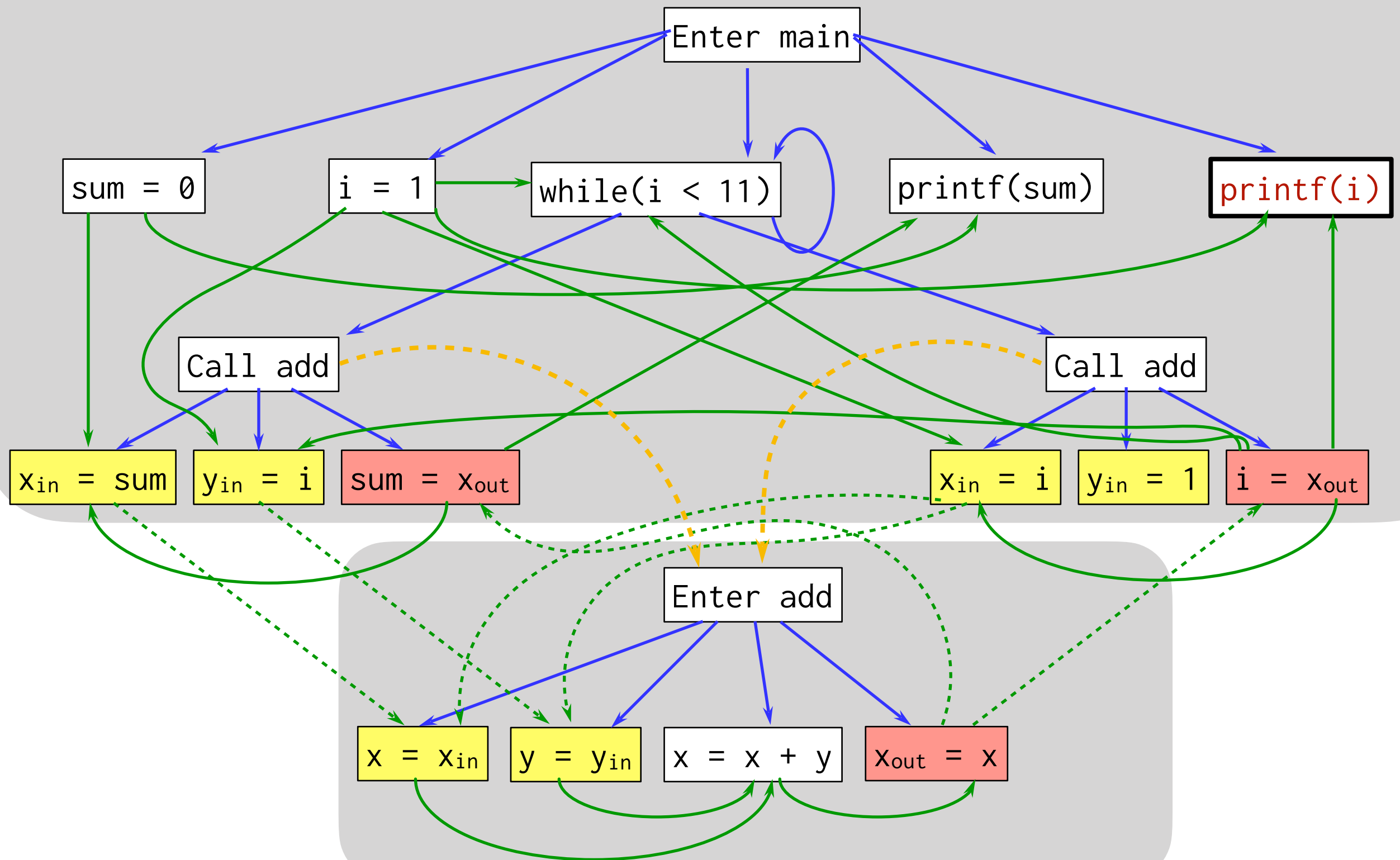


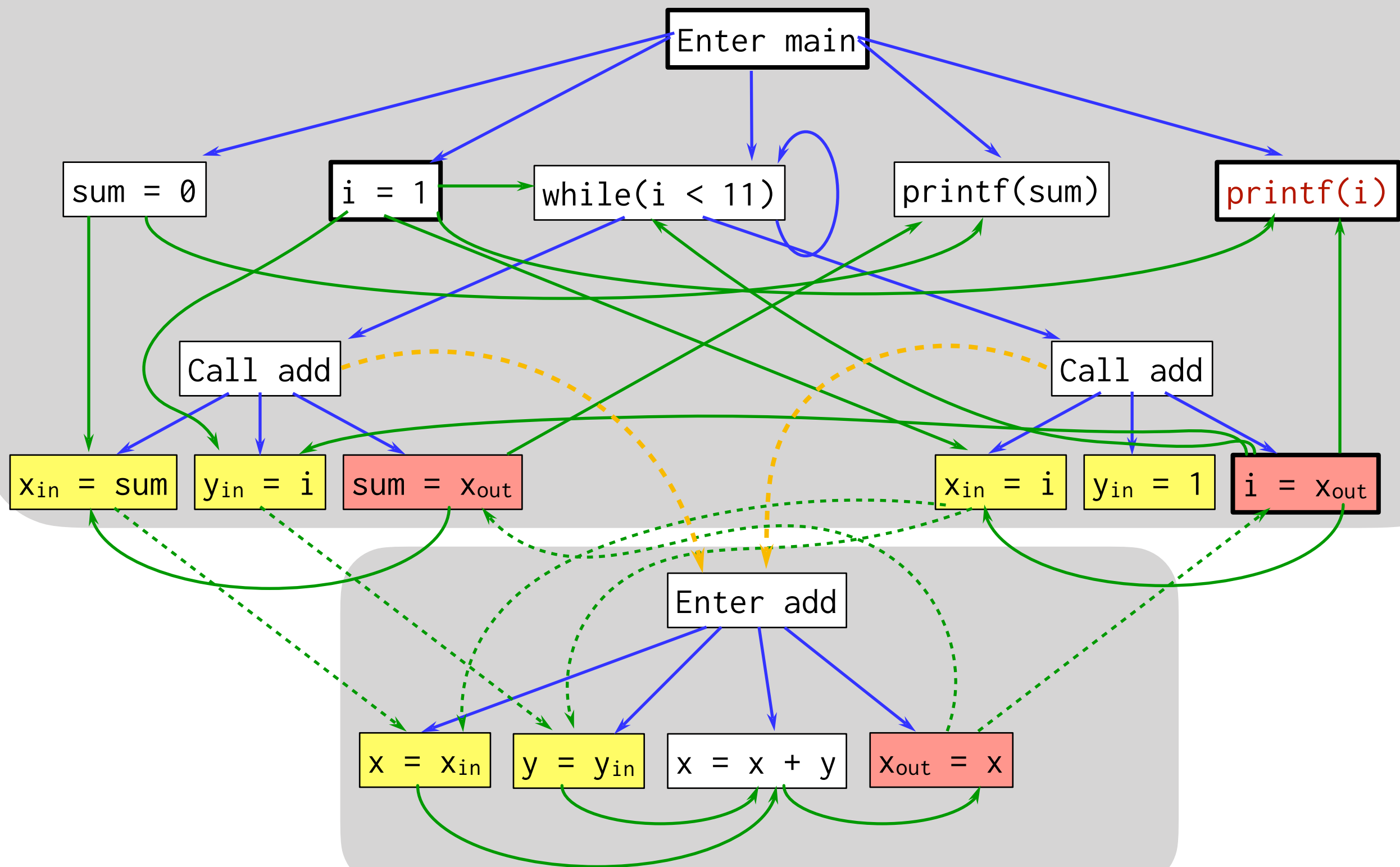


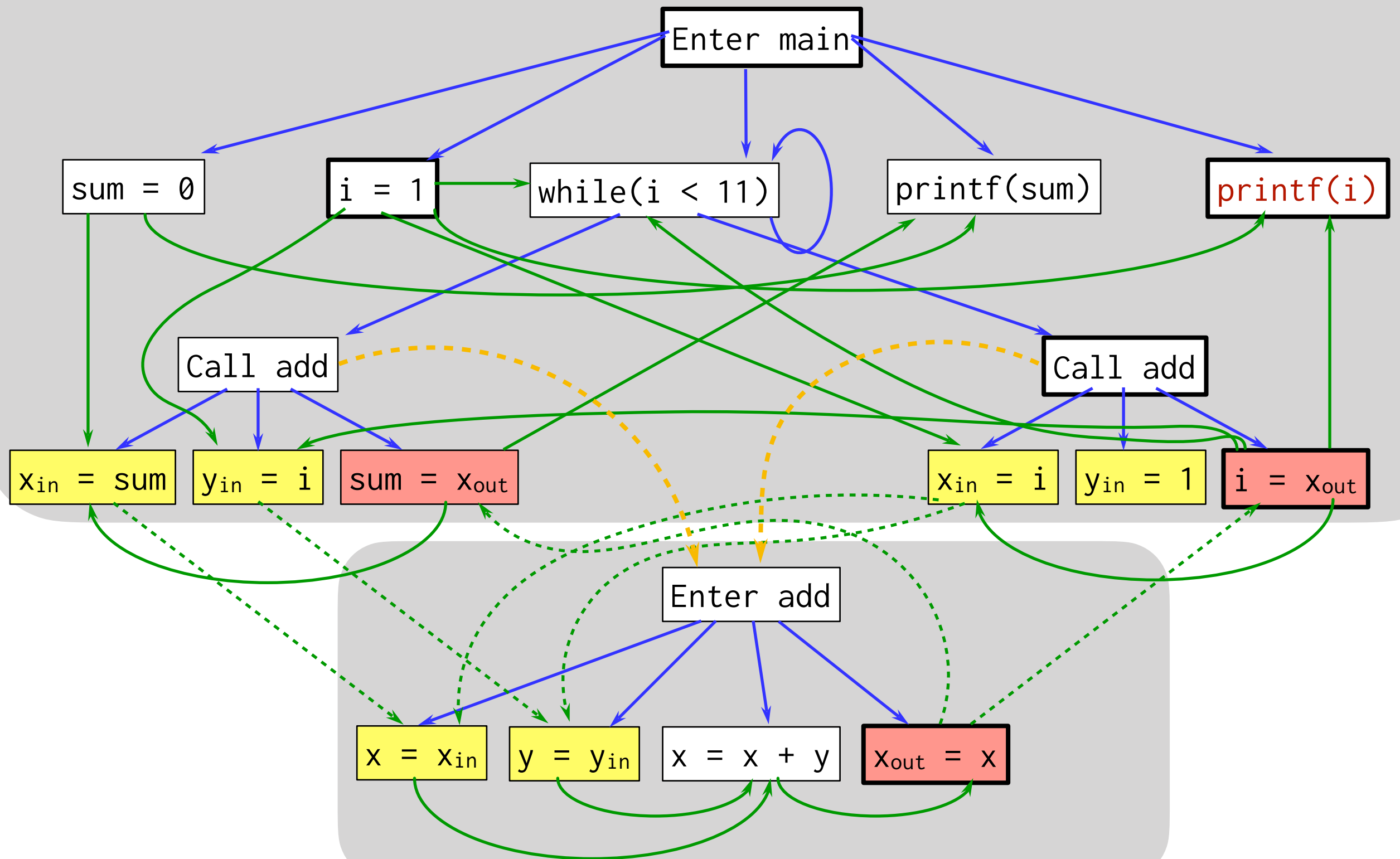
```
int main() {  
    int sum = 0;  
    int i = 1;  
    while (i < 11) {  
        sum = add(sum,i);  
        i = add(i,1);  
    }  
    printf("%d\n",sum);  
    printf("%d\n",i);  
}
```

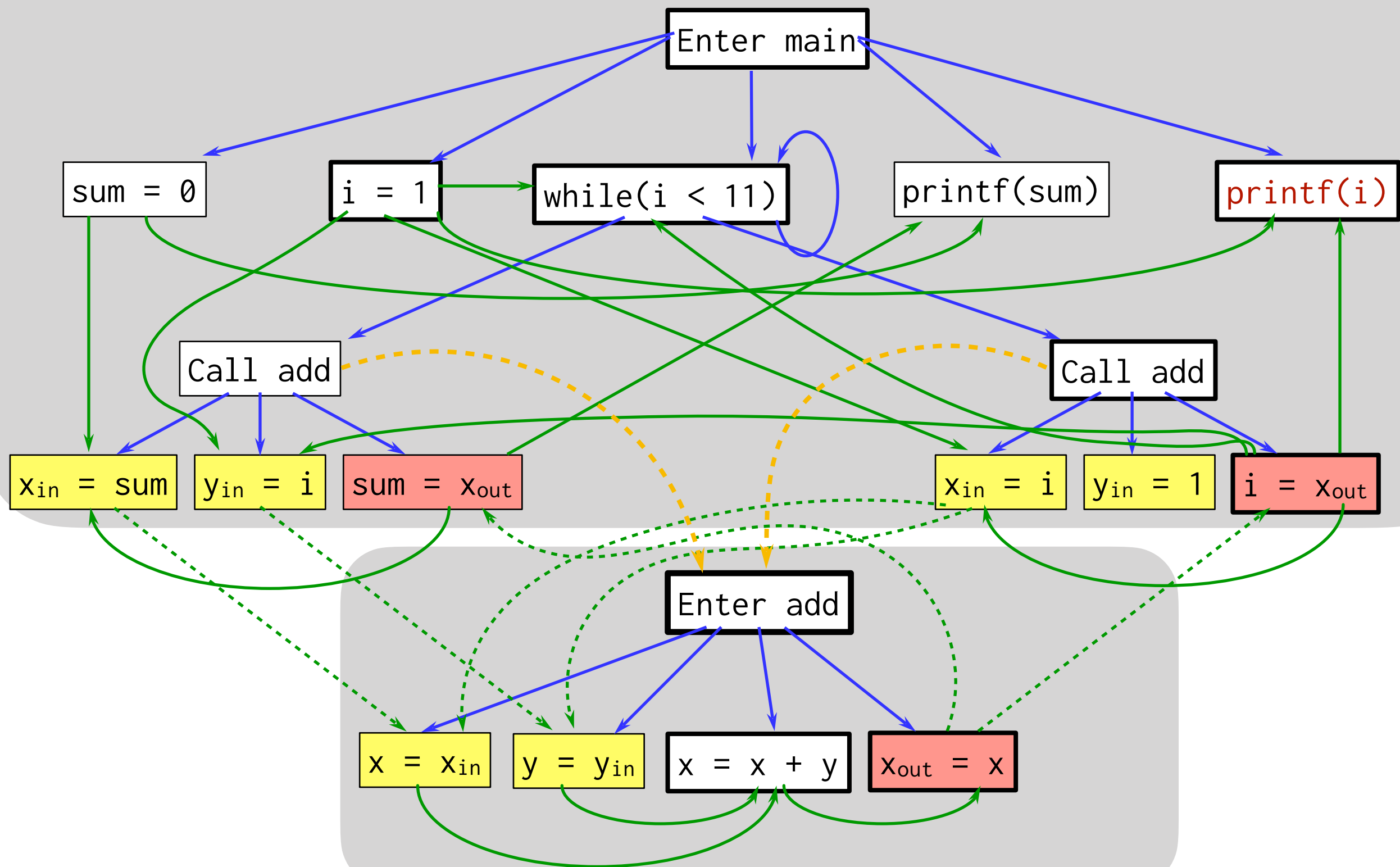
```
int add(int x, int y) {  
    x = x + y;  
    return x;  
}
```

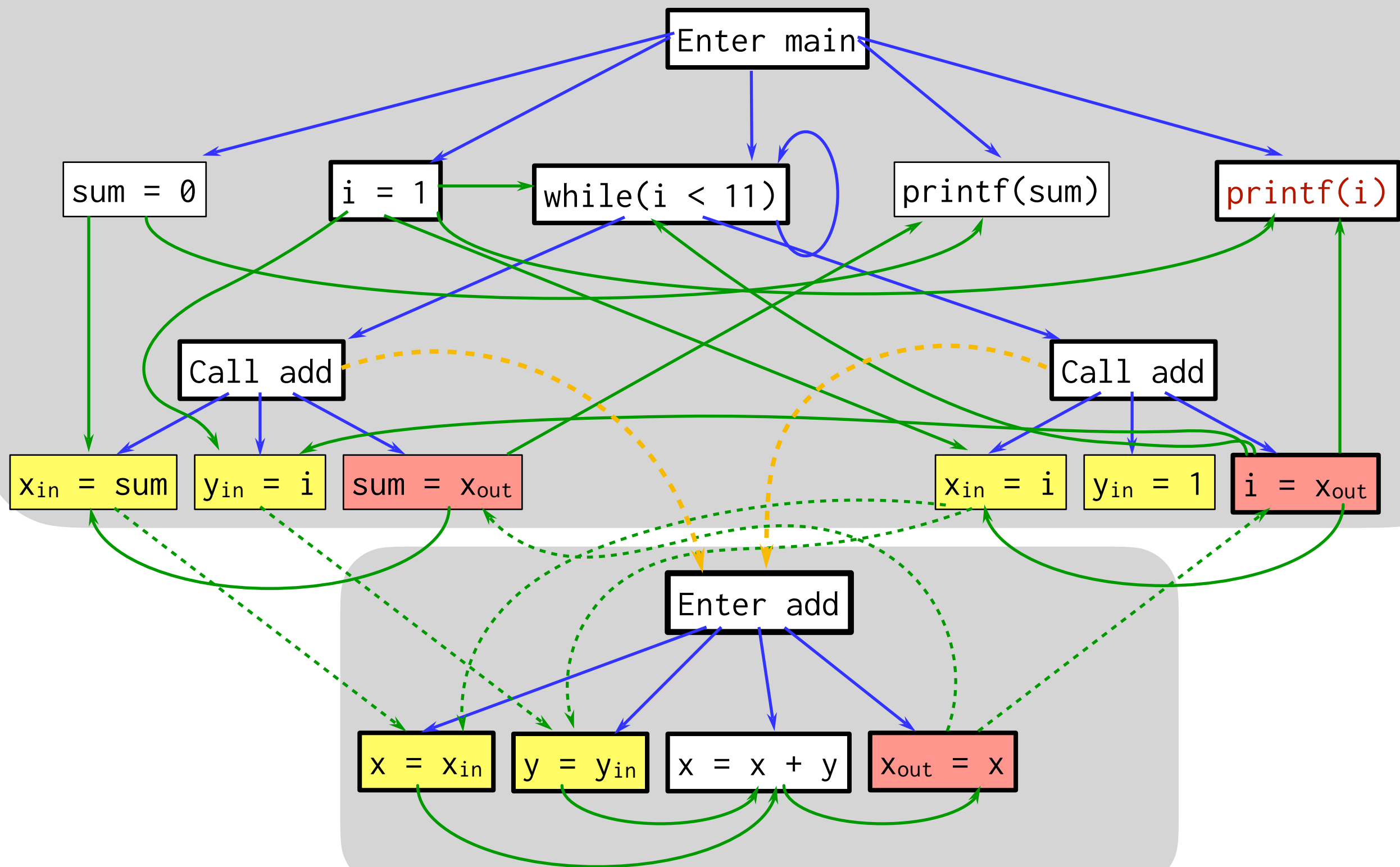
← Slicing target

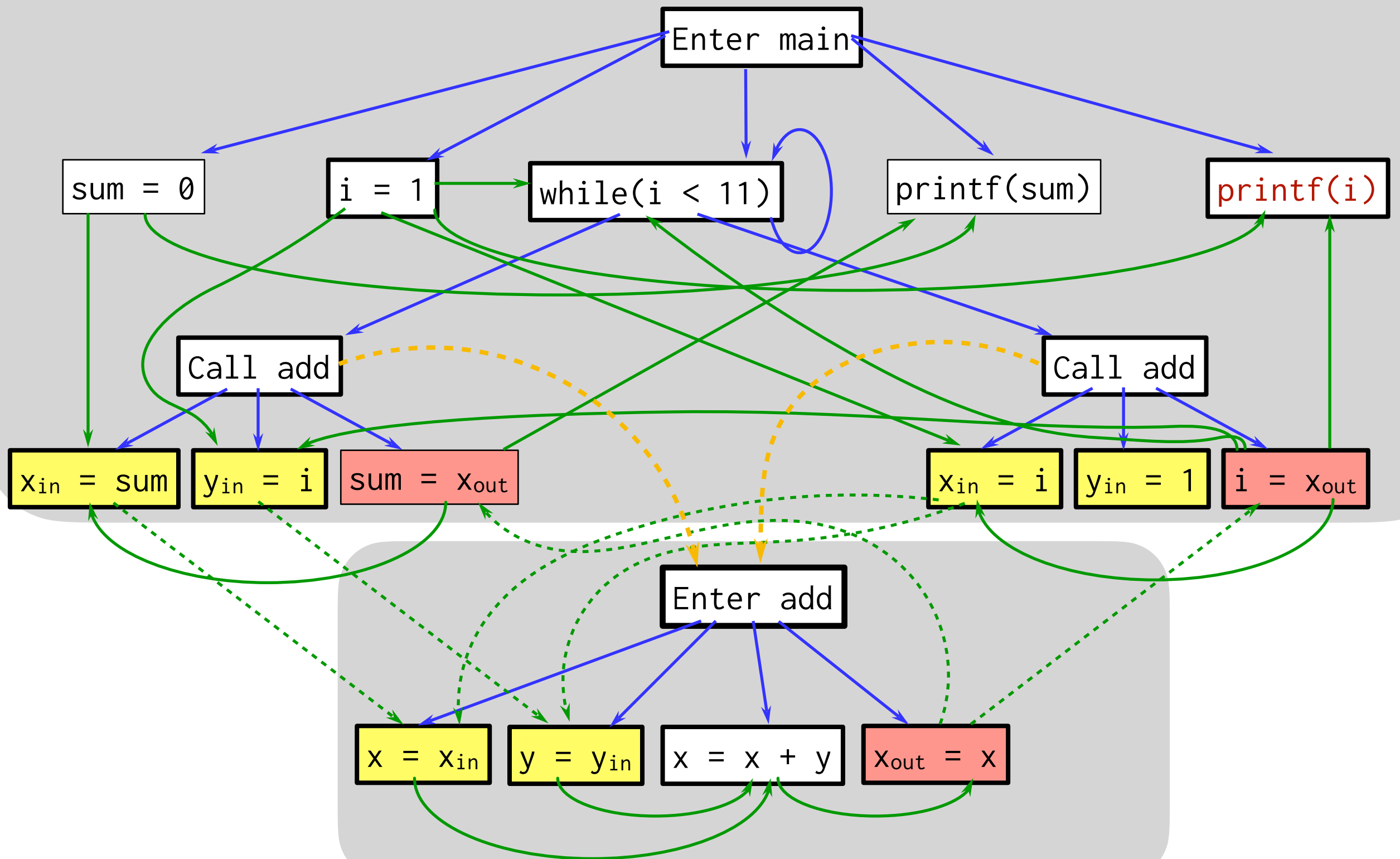


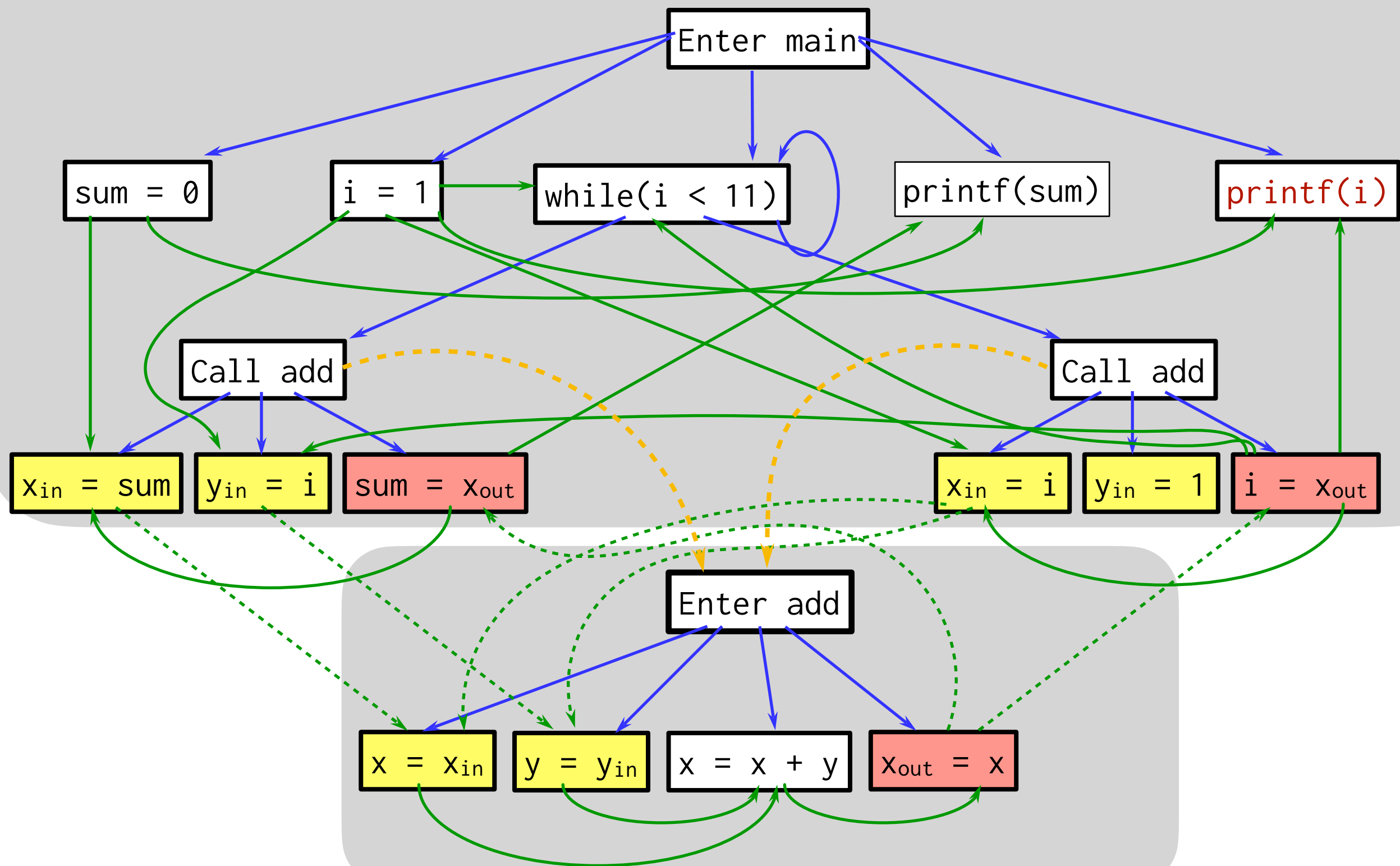


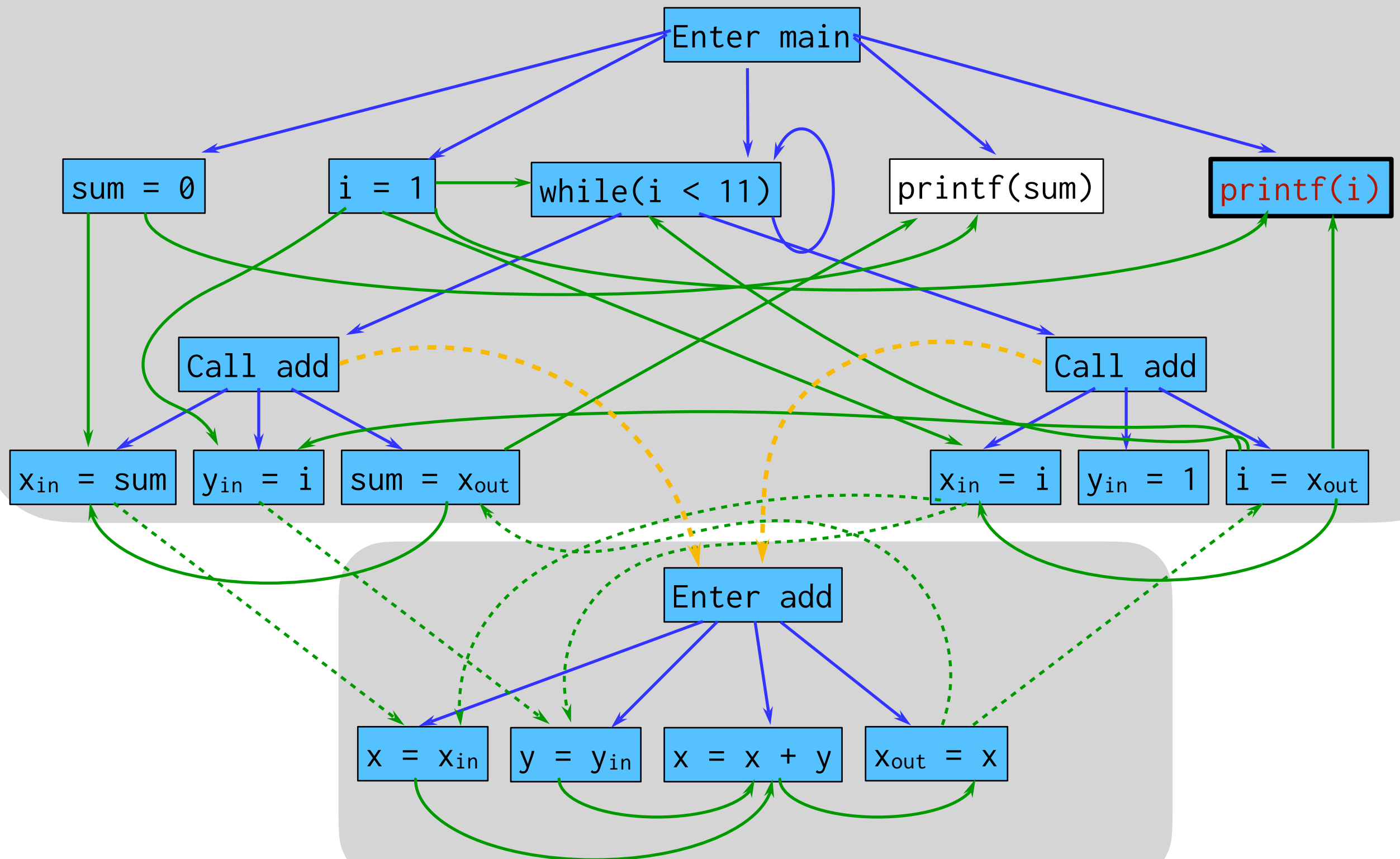








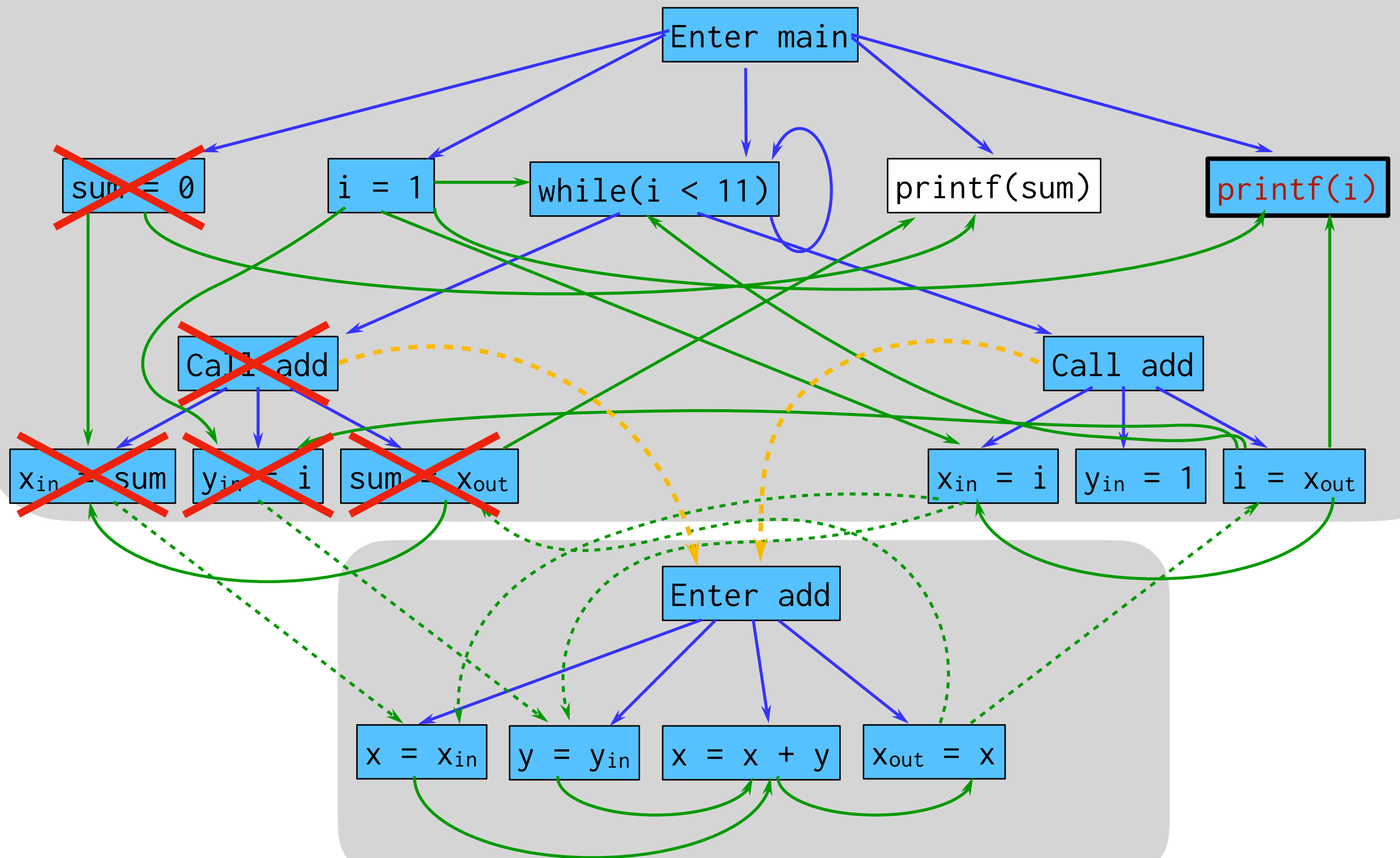


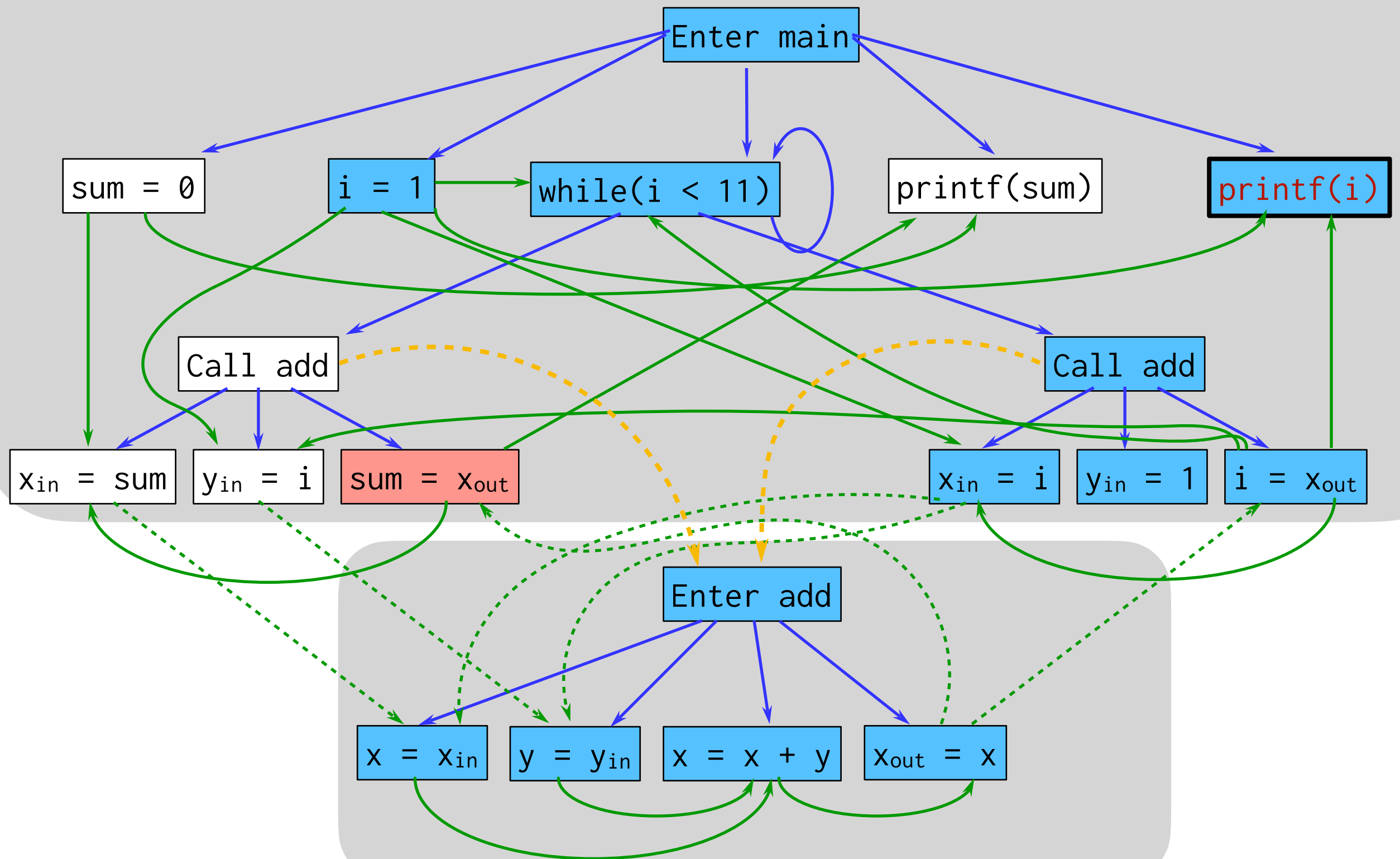


```
int main() {  
    int sum = 0;  
    int i = 1;  
    while (i < 11) {  
        sum = add(sum,i);  
        i = add(i,1);  
    }  
    printf("%d\n",sum);  
    printf("%d\n",i);  
}
```

```
int add(int x, int y) {  
    x = x + y;  
    return x;  
}
```

← Slicing target



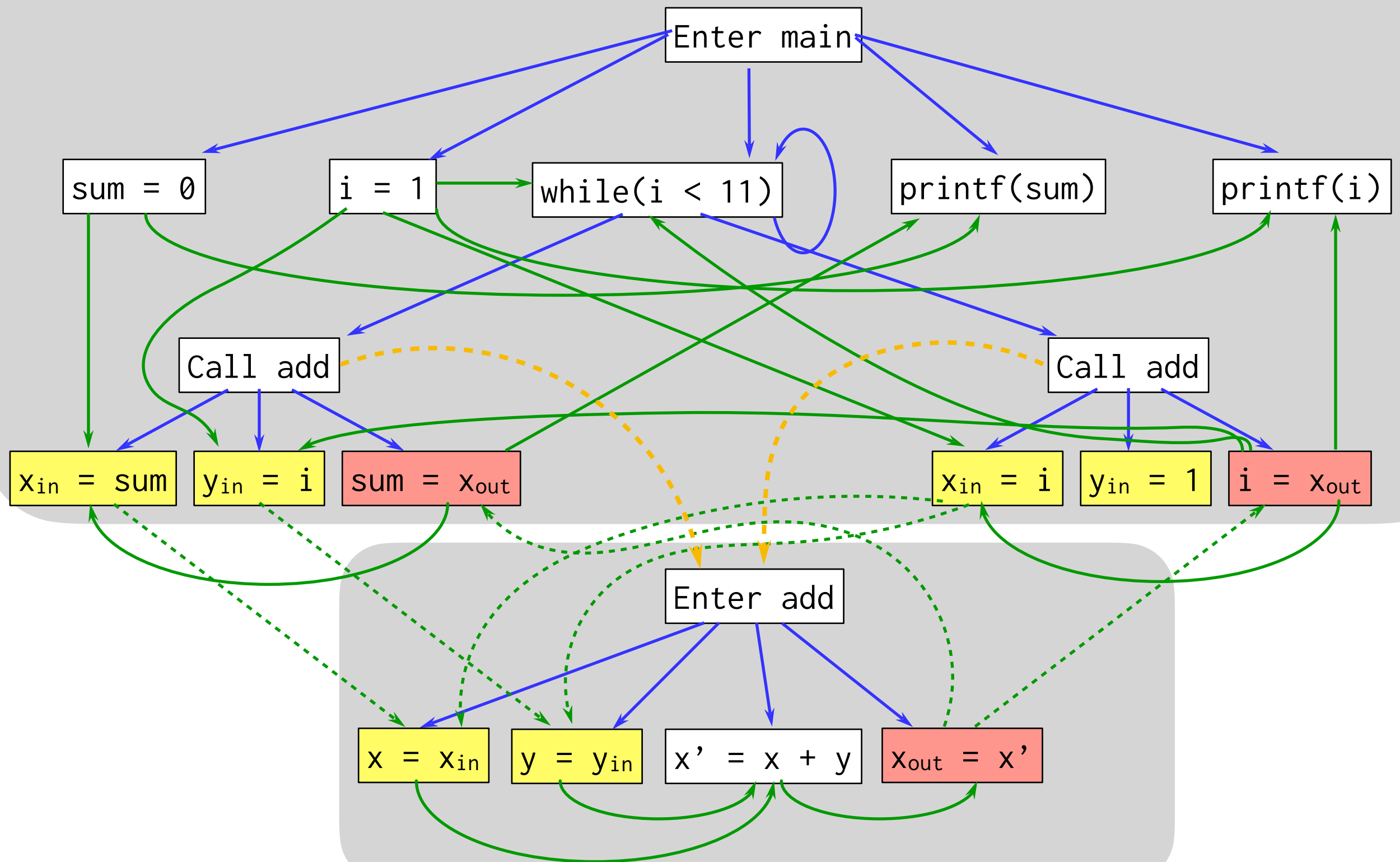


Two-Phase Slicing Algorithm

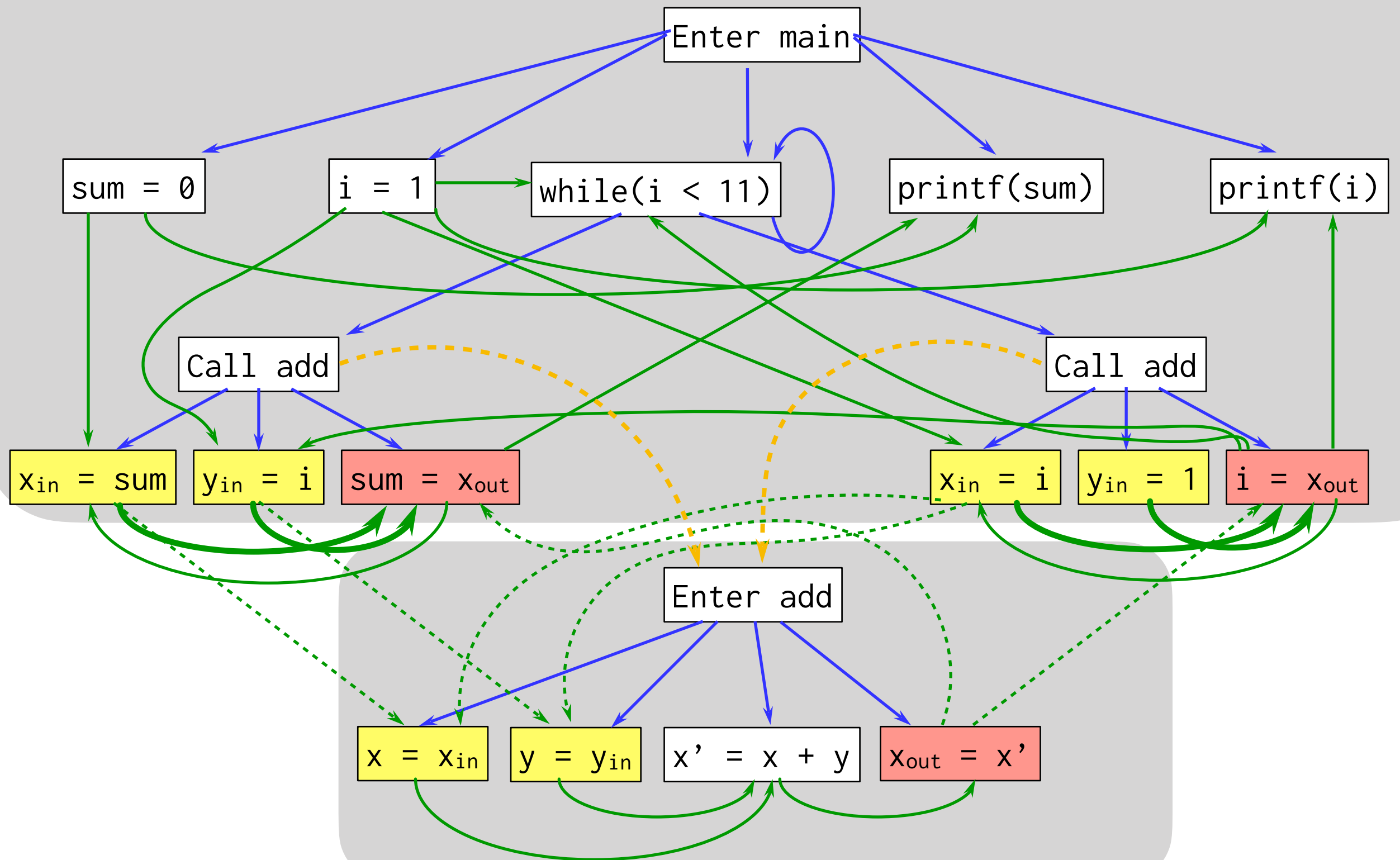
- Horwitz, S., Reps, T., & Binkley, D. (1990). Interprocedural slicing using dependence graphs. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 12(1), 26-60.
- To avoid the mismatches of procedure returns and procedure calls when traversing the graph
 - Phase I: find the statements in the current procedure and the callers of the current procedure that may affect the slicing criterion
 - Do not traverse return edges
 - Use summary information to continue the slicing at each callsite
 - Phase II: Find the statements in the callees of the current procedure that may affect the slicing criterion
 - Do not traverse call edges

Computing Summary Edges

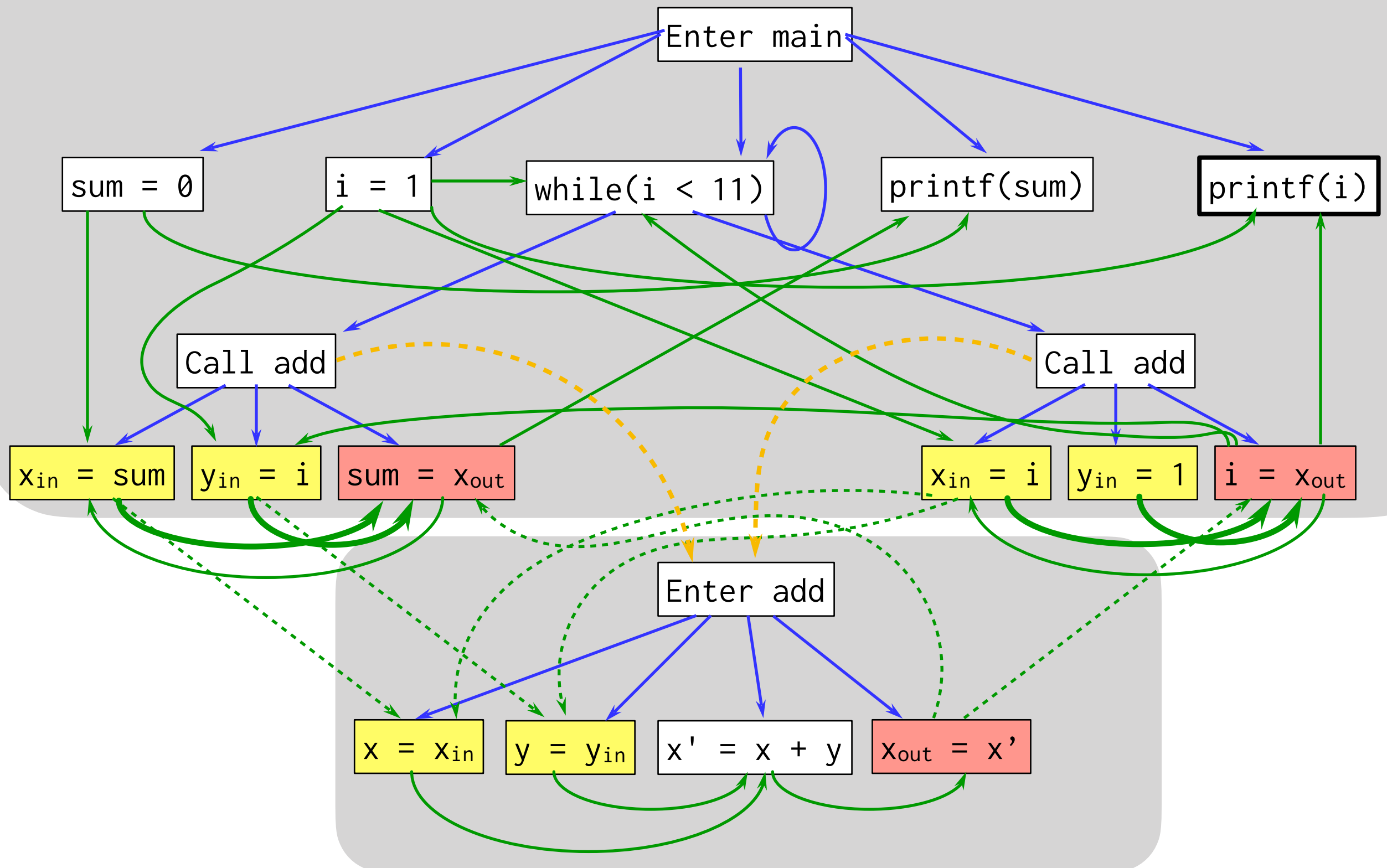
- Step 1: compute the (transitive) reachability from formal-in nodes to formal-out nodes in each procedure
- Step 2: create the summary edges in each caller according to the reachability from formal-in nodes to formal-out nodes in a procedure



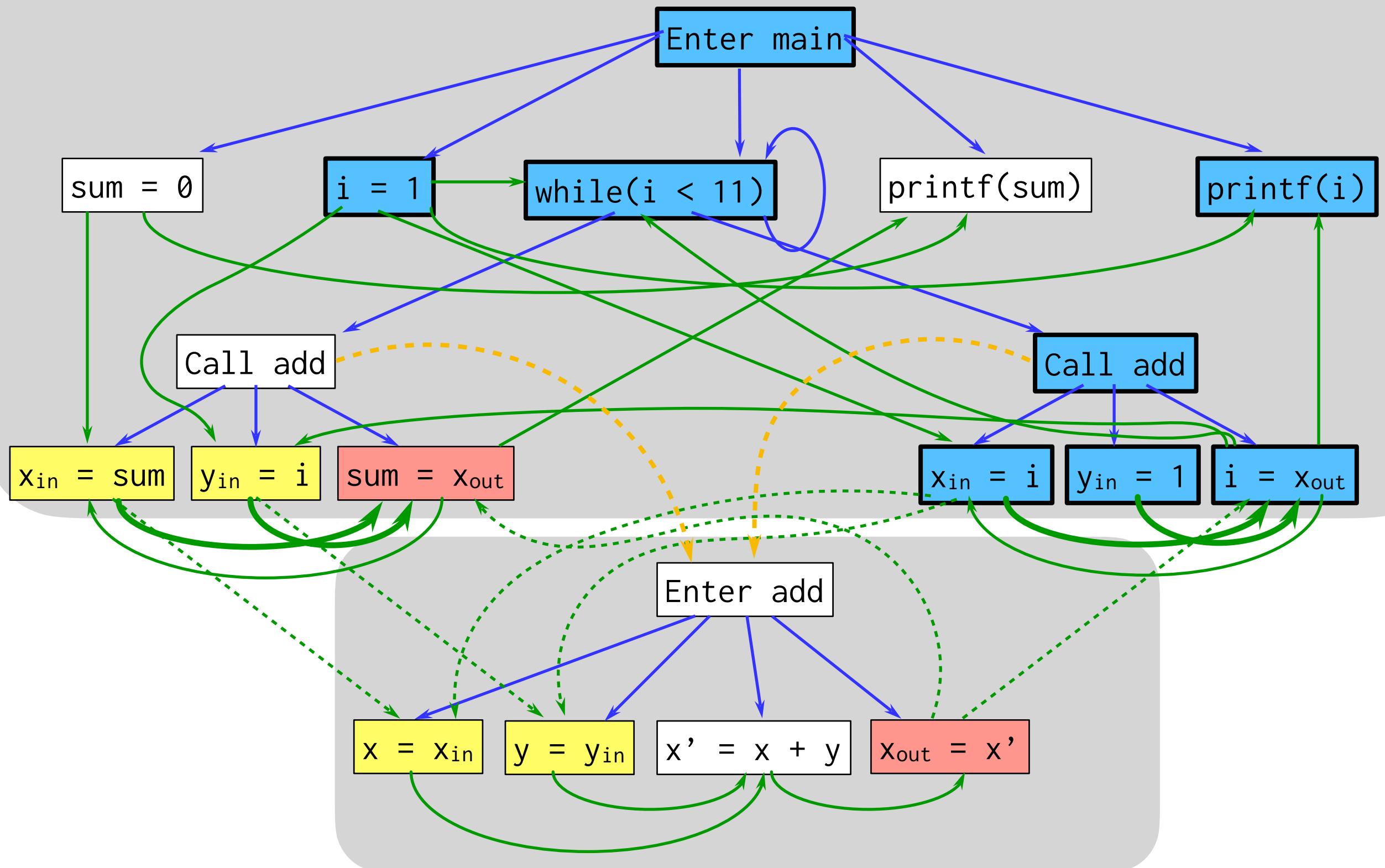
Summary Edges



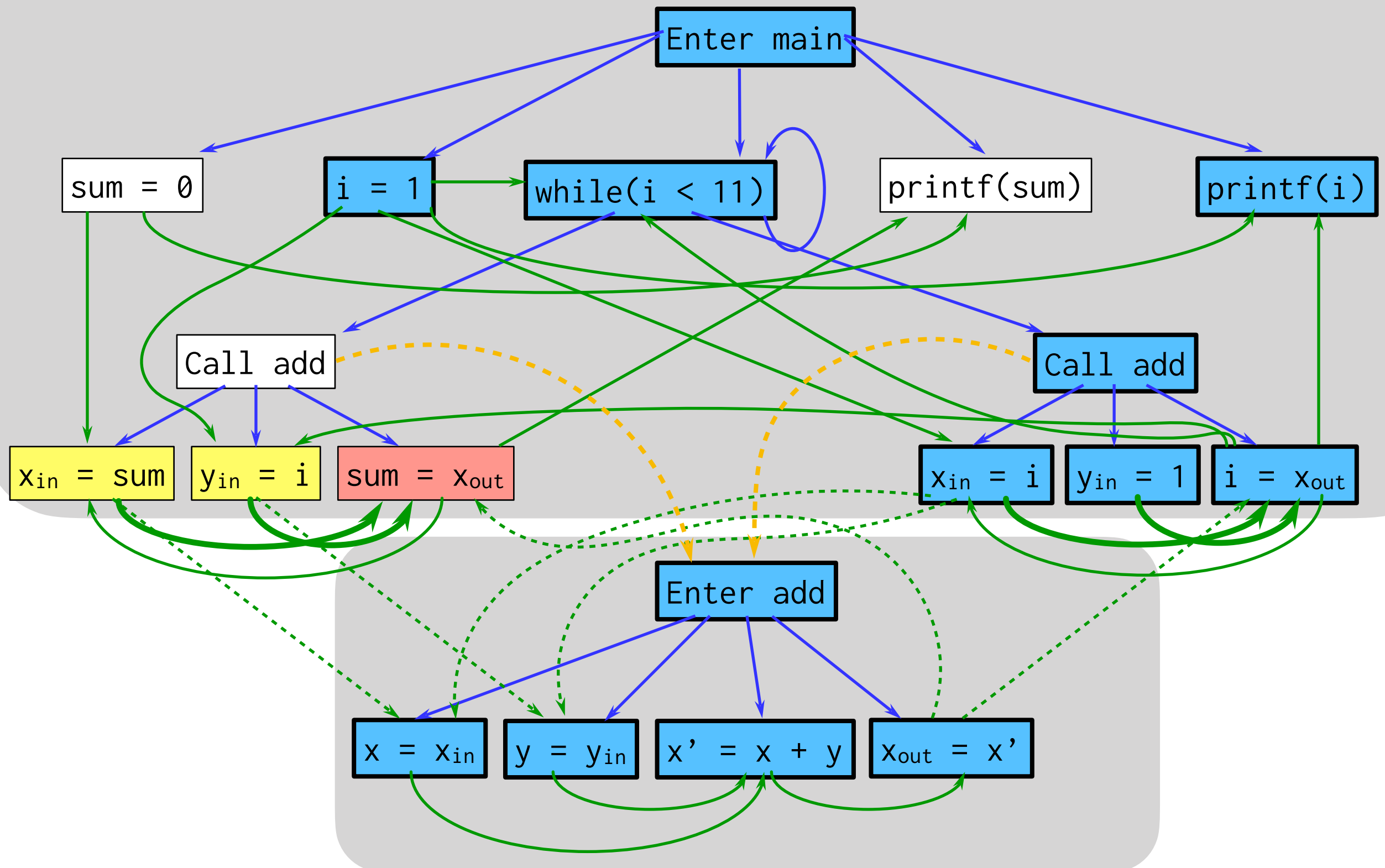
Two-Phase Slicing: Phase I



Two-Phase Slicing: Phase I



Two-Phase Slicing: Phase 2



```
int main() {  
    int sum = 0;  
    int i = 1;  
    while (i < 11) {  
        sum = add(sum,i);  
        i = add(i,1);  
    }  
    printf("%d\n",sum);  
    printf("%d\n",i);  
}
```

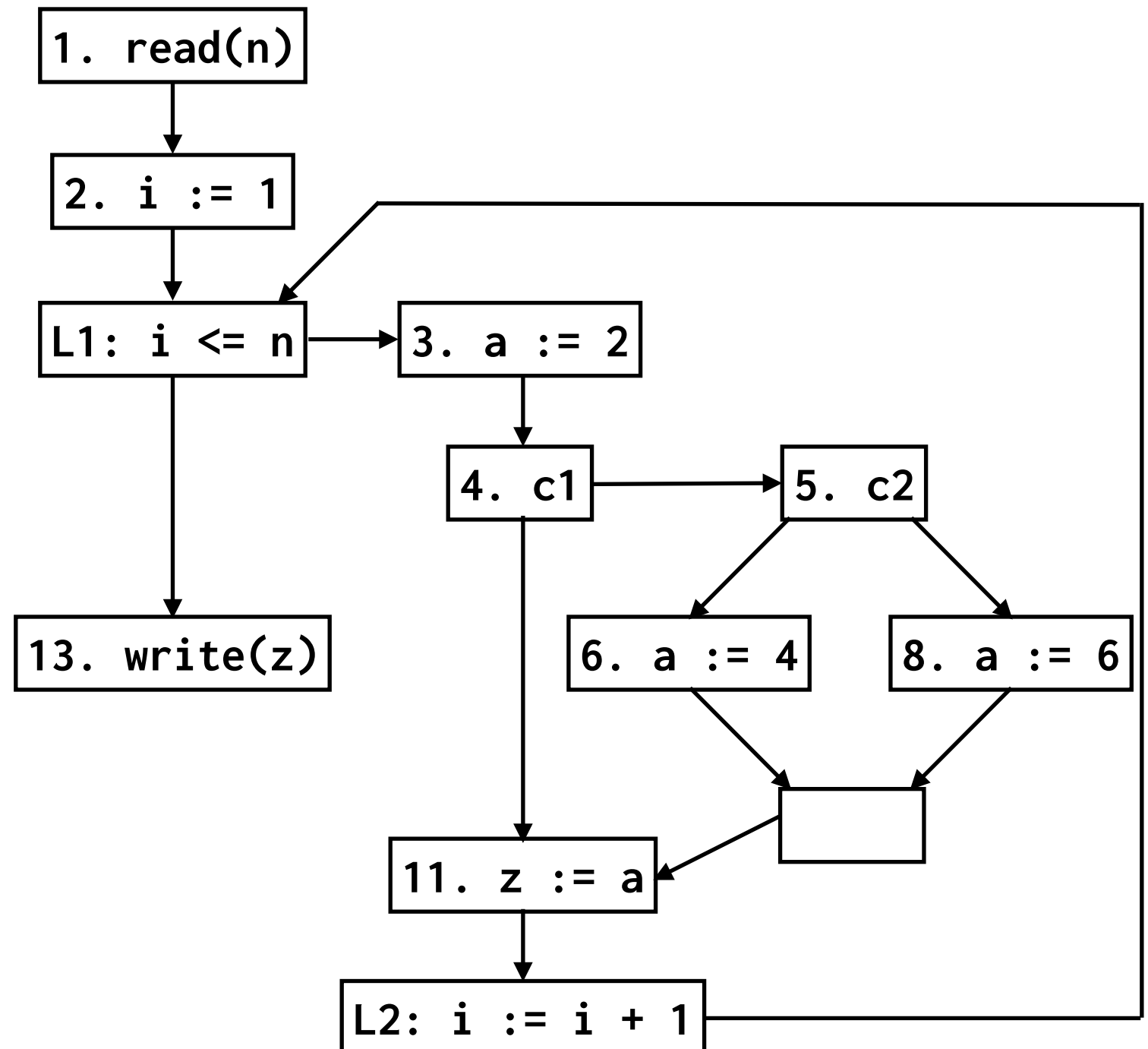
```
int add(int x, int y) {  
    x = x + y;  
    return x;  
}
```

← Slicing target

Dynamic Slicing

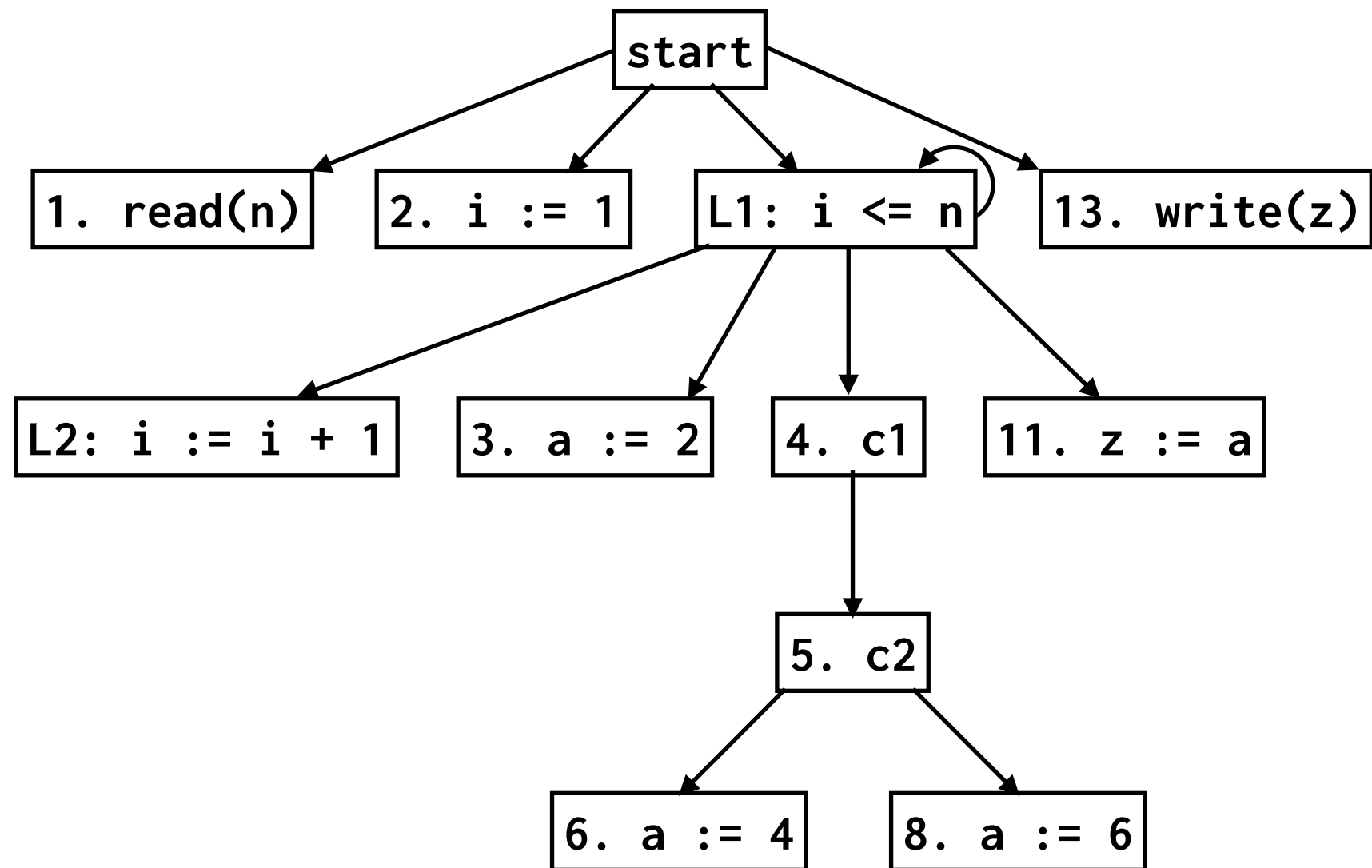
Slicing

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```



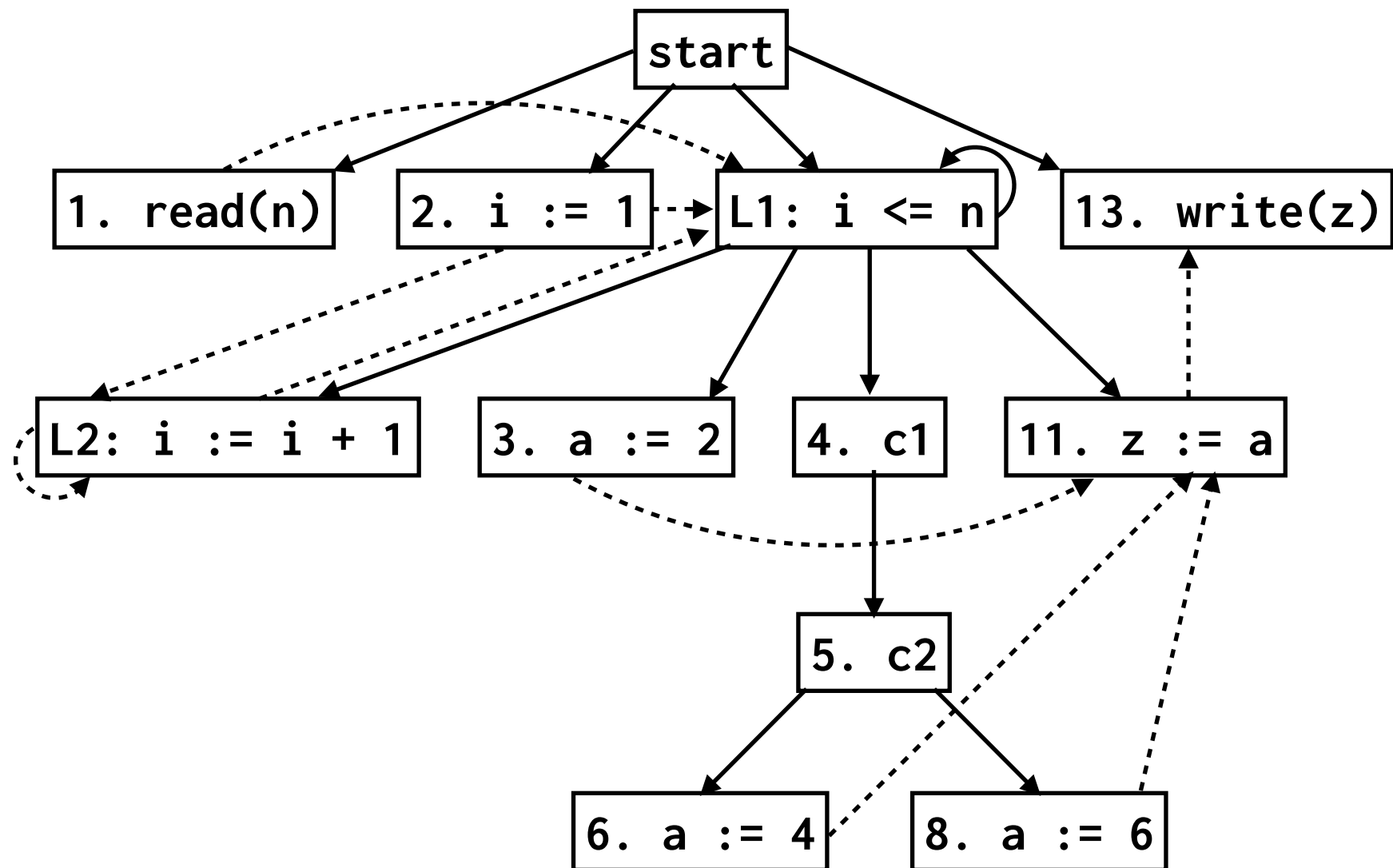
Slicing

```
1.  read(n)
2.  for i := 1 to n do
3.      a := 2
4.      if c1 then
5.          if c2 then
6.              a := 4
7.          else
8.              a := 6
9.          endif
10.     endif
11.     z := a
12. endfor
13. write(z)
```



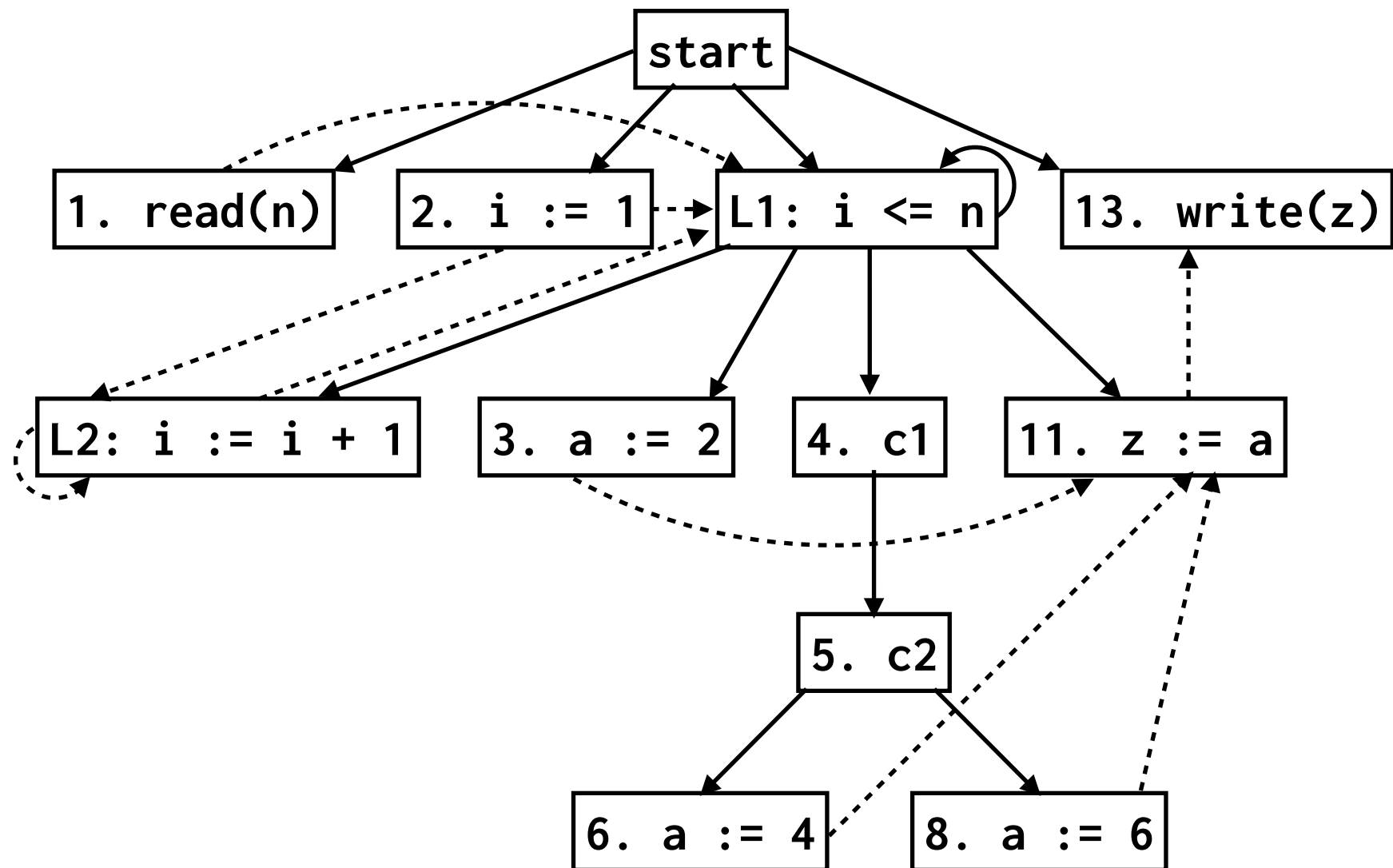
Slicing

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```



Slicing

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

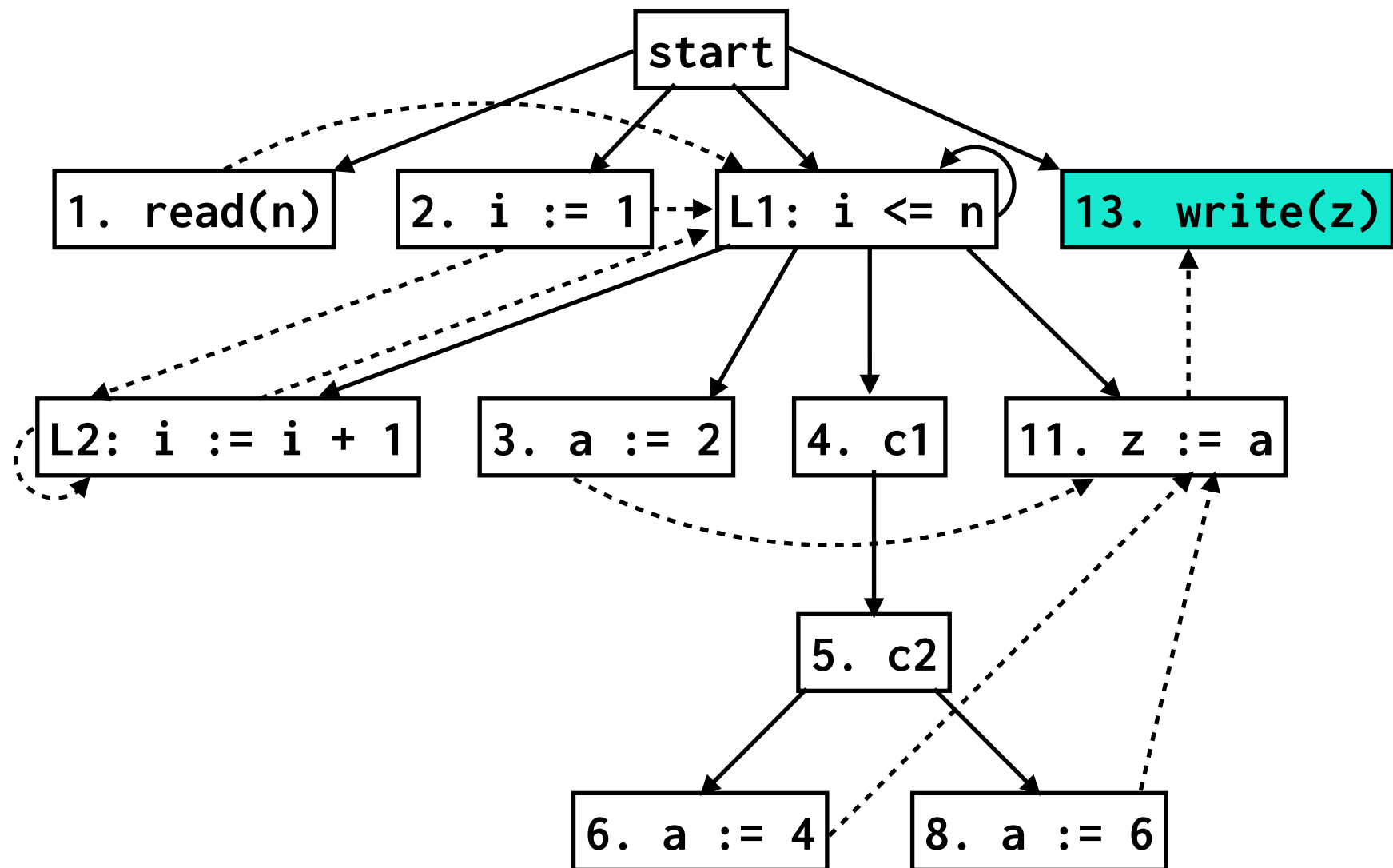


Static Slice

Criterion <13, z>

Slicing

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

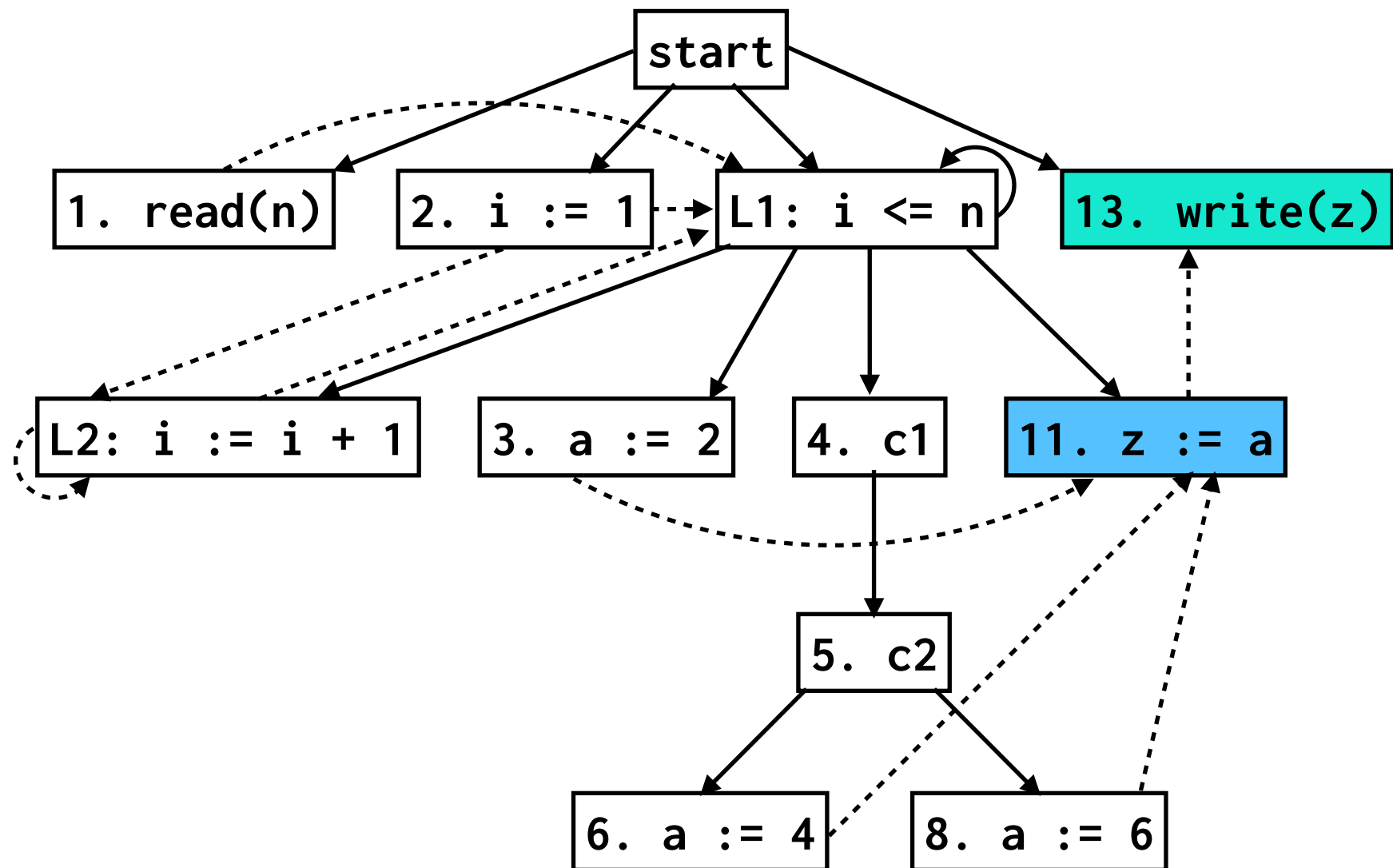


Static Slice

Criterion <13, z>

Slicing

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

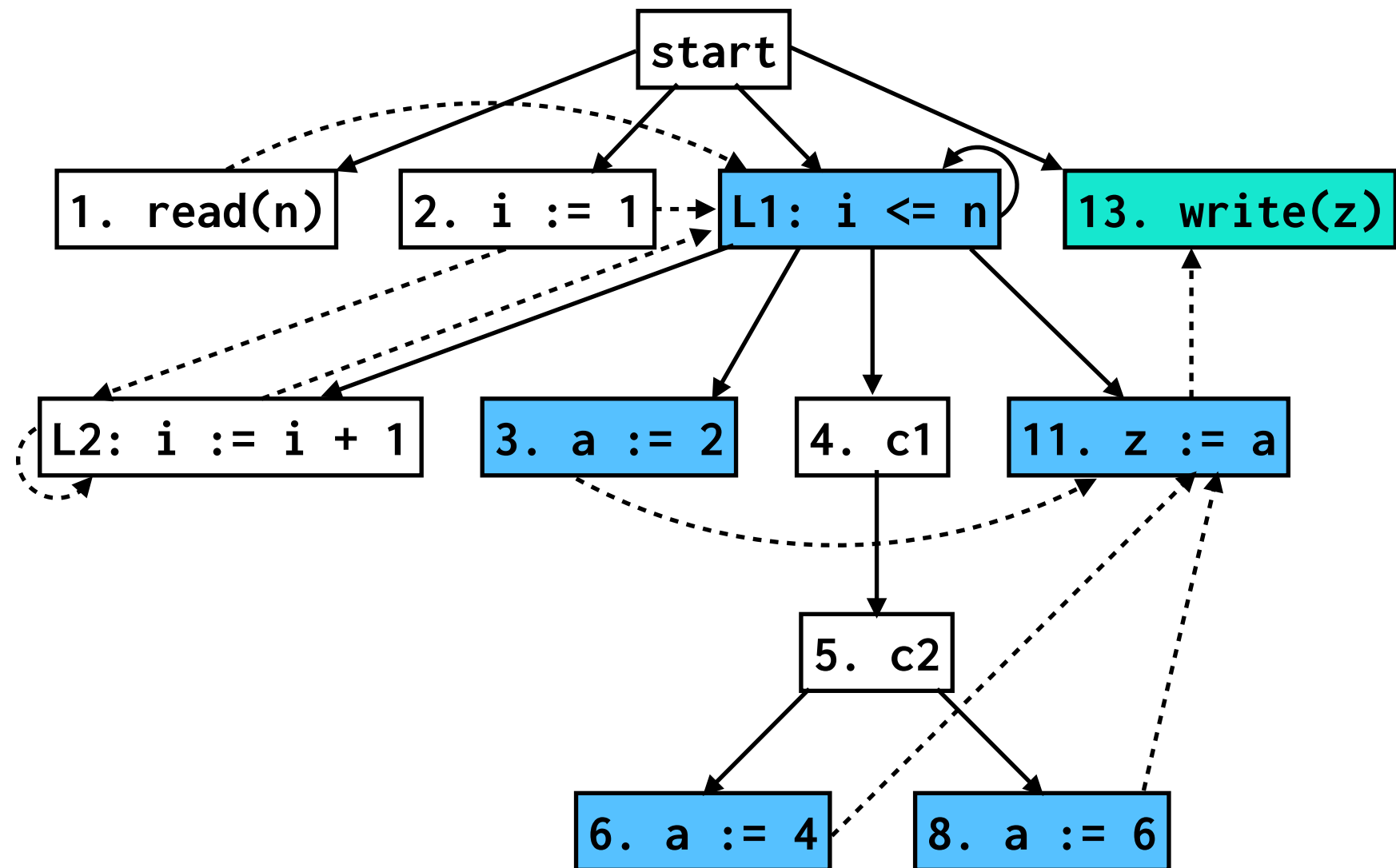


Static Slice

Criterion <13, z>

Slicing

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

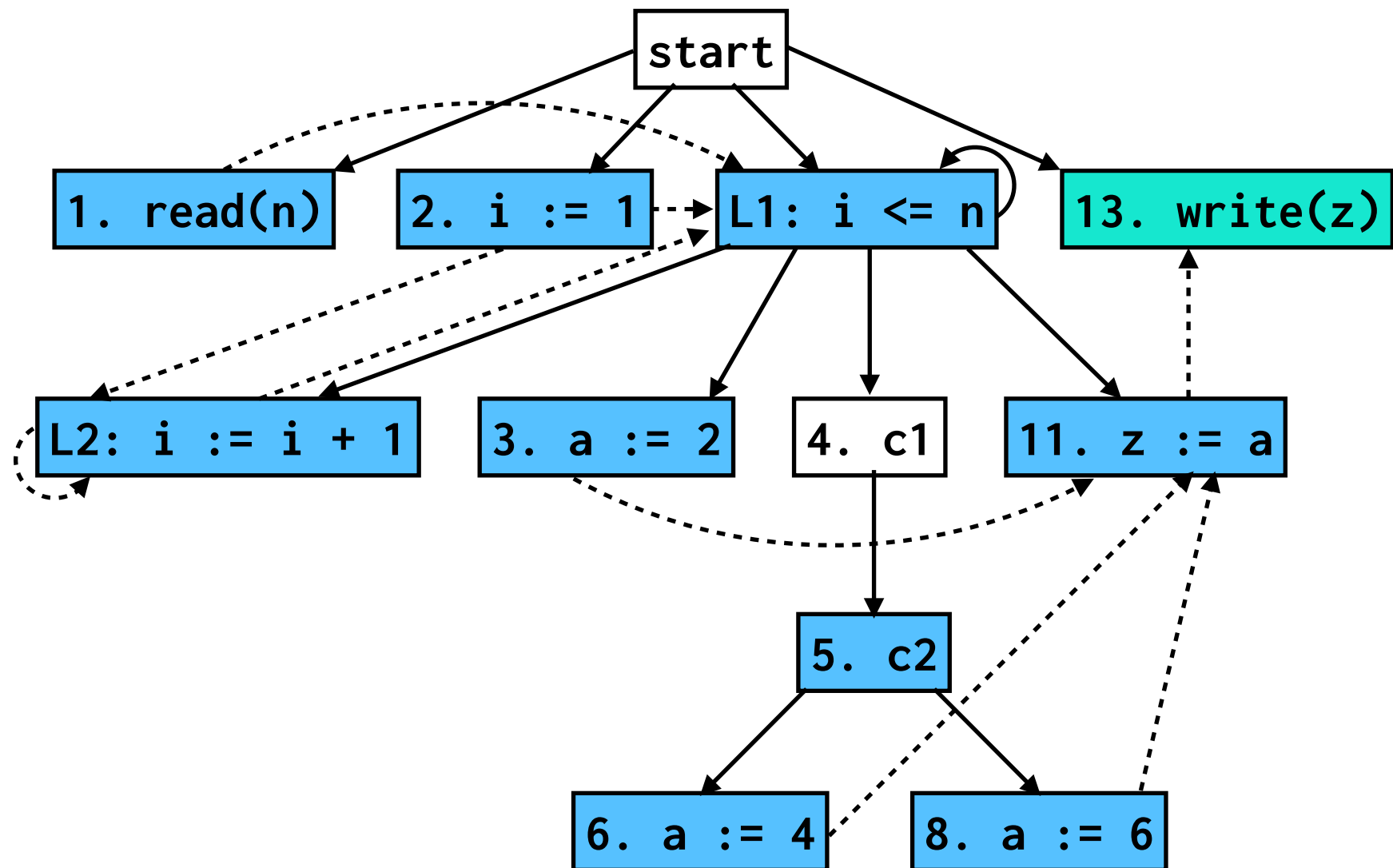


Static Slice

Criterion <13, z>

Slicing

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

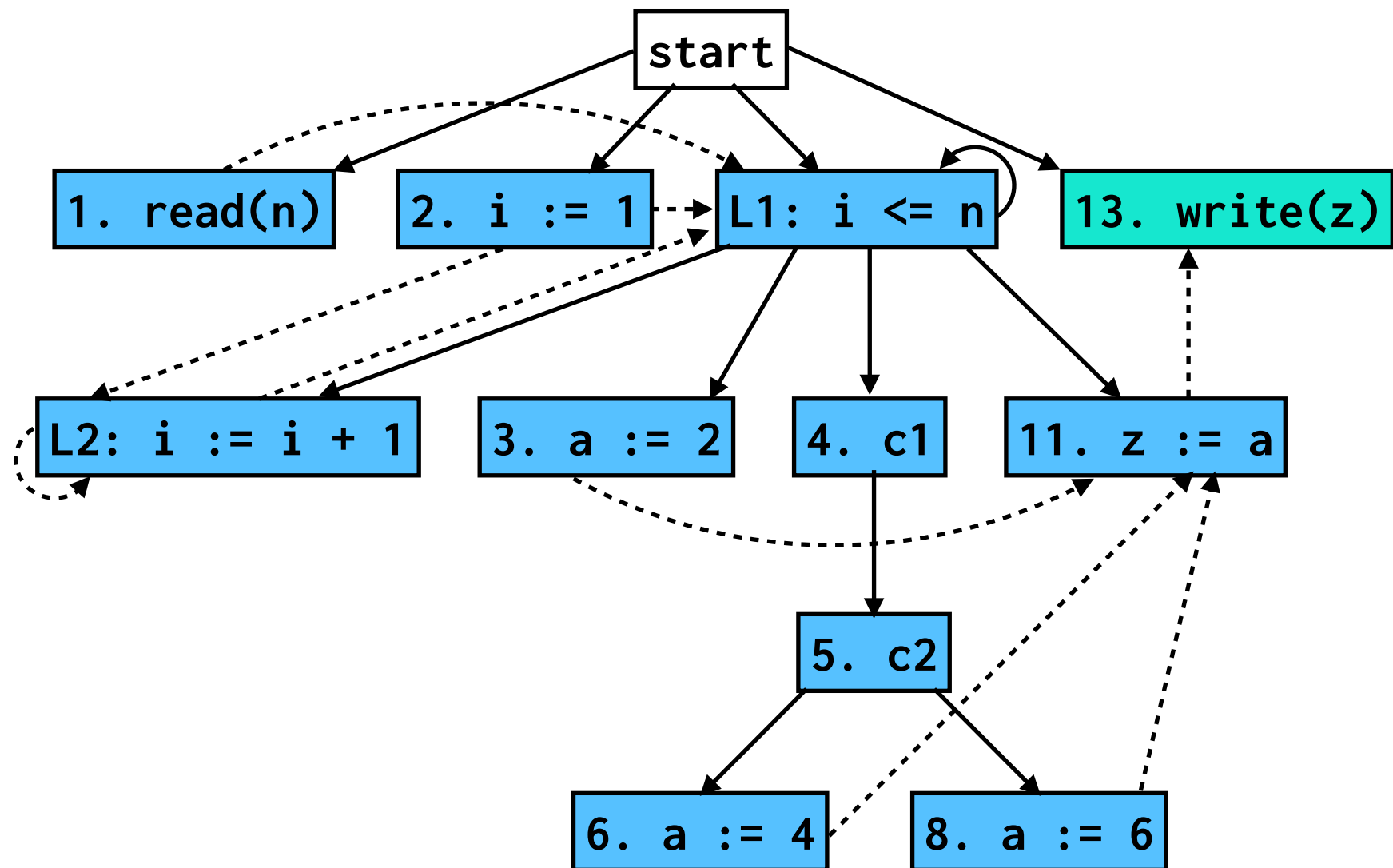


Static Slice

Criterion $\langle 13, z \rangle$

Slicing

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```



Static Slice

Criterion <13, z>

Dynamic Slicing

- Mark **nodes** in PDG that are executed to produce the **reduced PDG**
- Static backward slice on reduced PDG

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

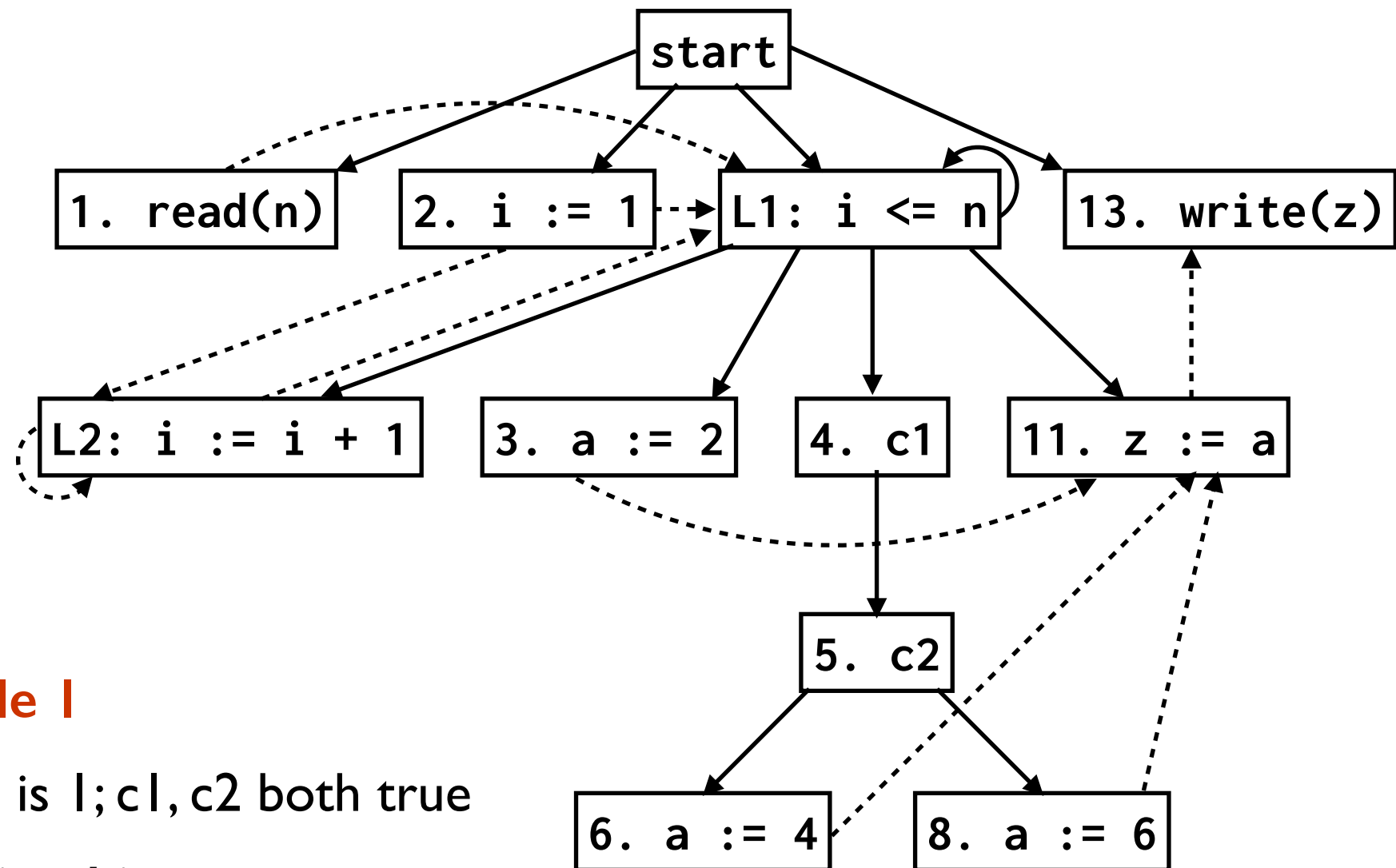
Example 1

Input n is 1; c1, c2 both true

Execution history is:

1, 2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion <13, z>



Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

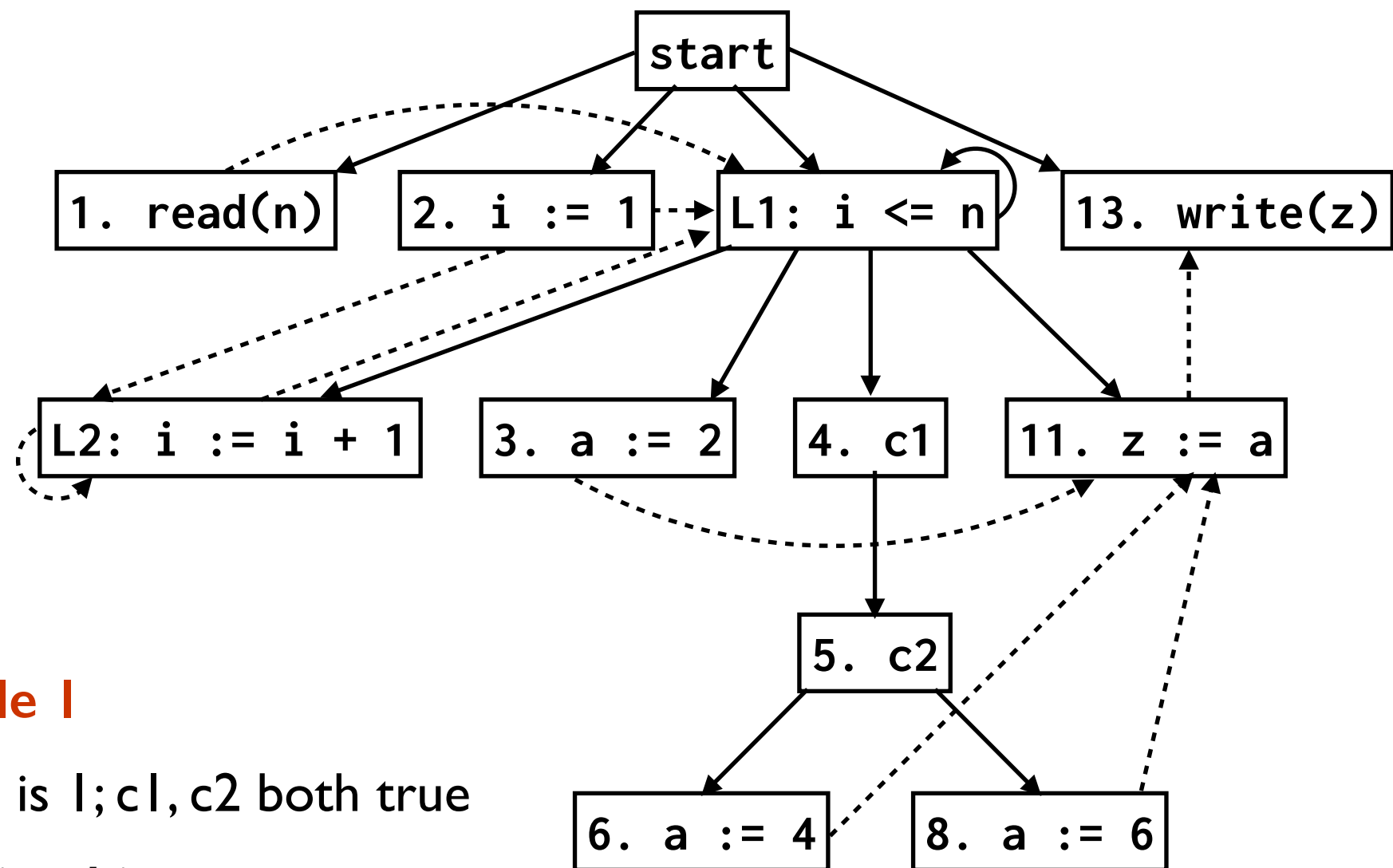
Example 1

Input n is 1; c1, c2 both true

Execution history is:

1, 2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion $\langle 13, z \rangle$



Mark nodes in PDG that are
executed

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

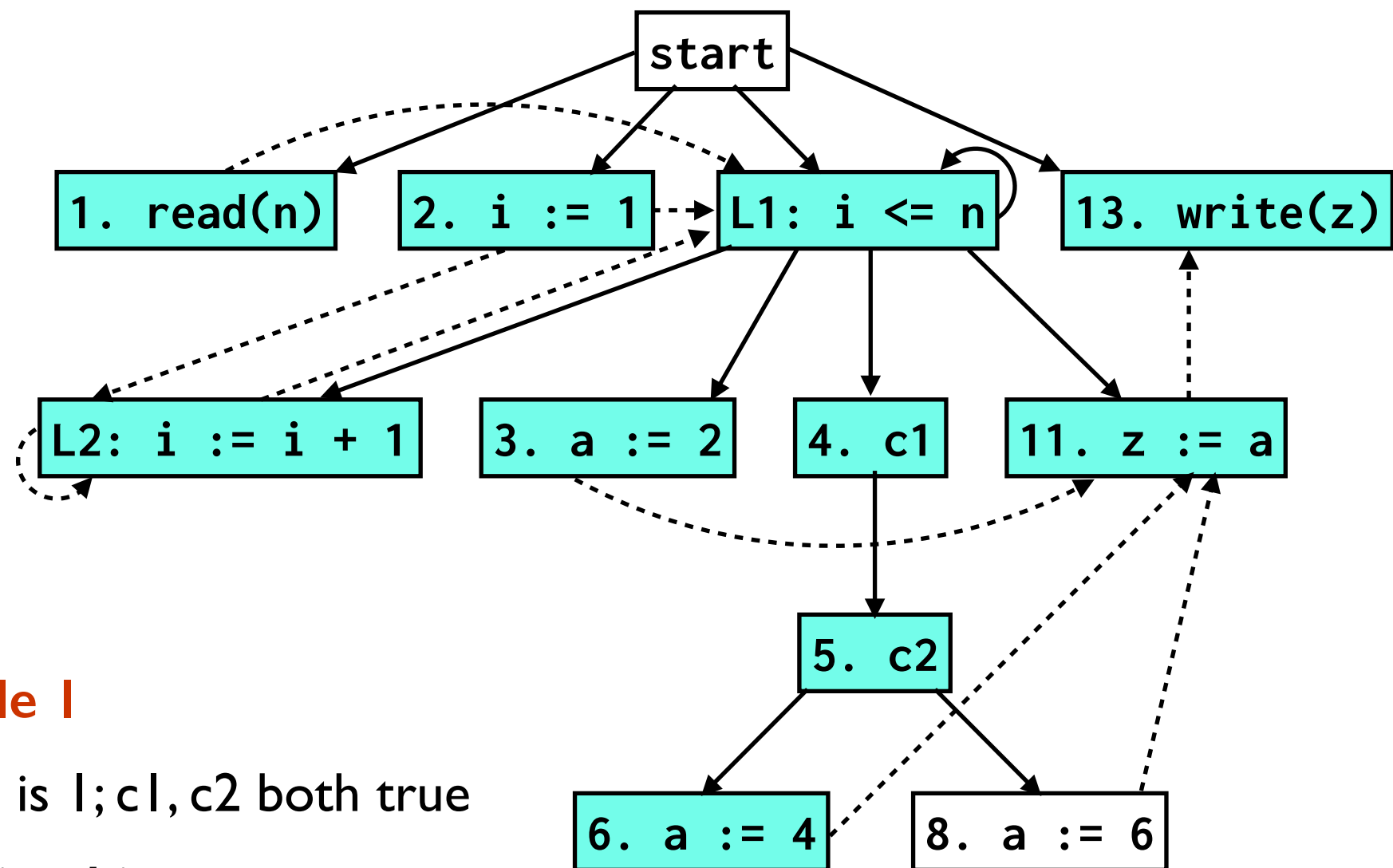
Example 1

Input n is 1; c1, c2 both true

Execution history is:

1, 2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion $\langle 13, z \rangle$



Mark nodes in PDG that are
executed

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

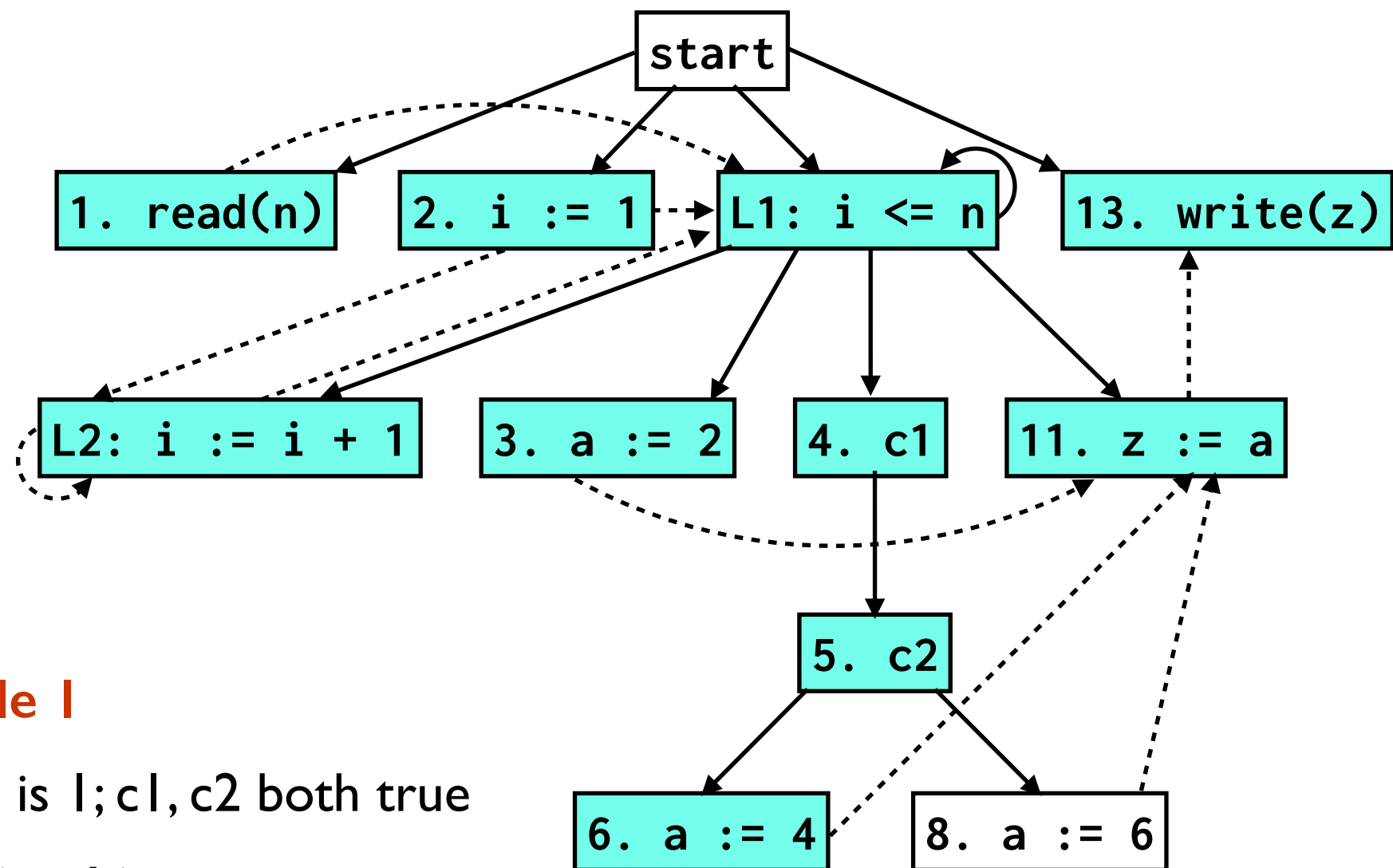
Example 1

Input n is 1; c1, c2 both true

Execution history is:

1, 2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion $\langle 13, z \rangle$



Create the reduced PDG

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

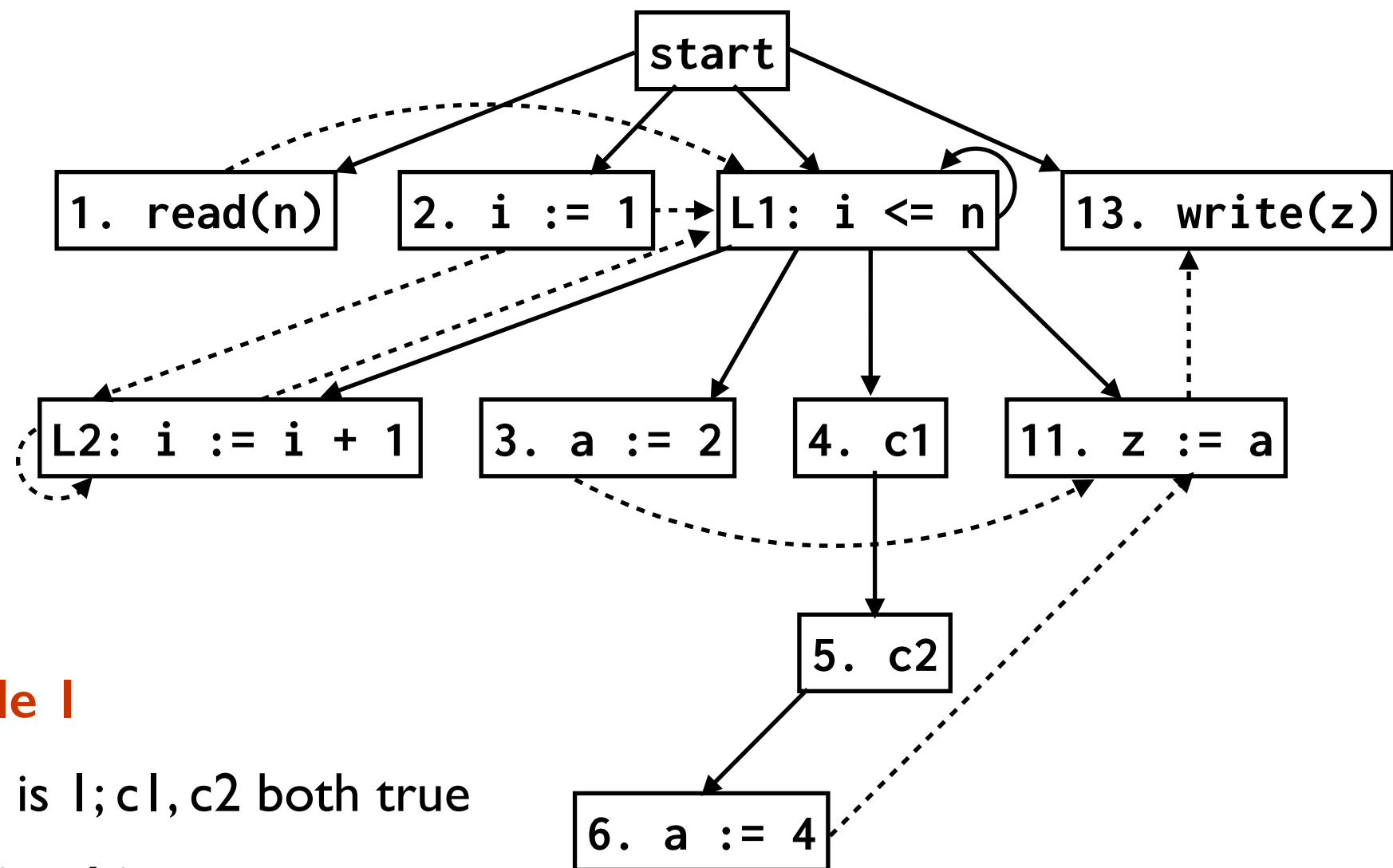
Example 1

Input n is 1; c1, c2 both true

Execution history is:

1, 2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion $\langle 13, z \rangle$



Create the reduced PDG

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

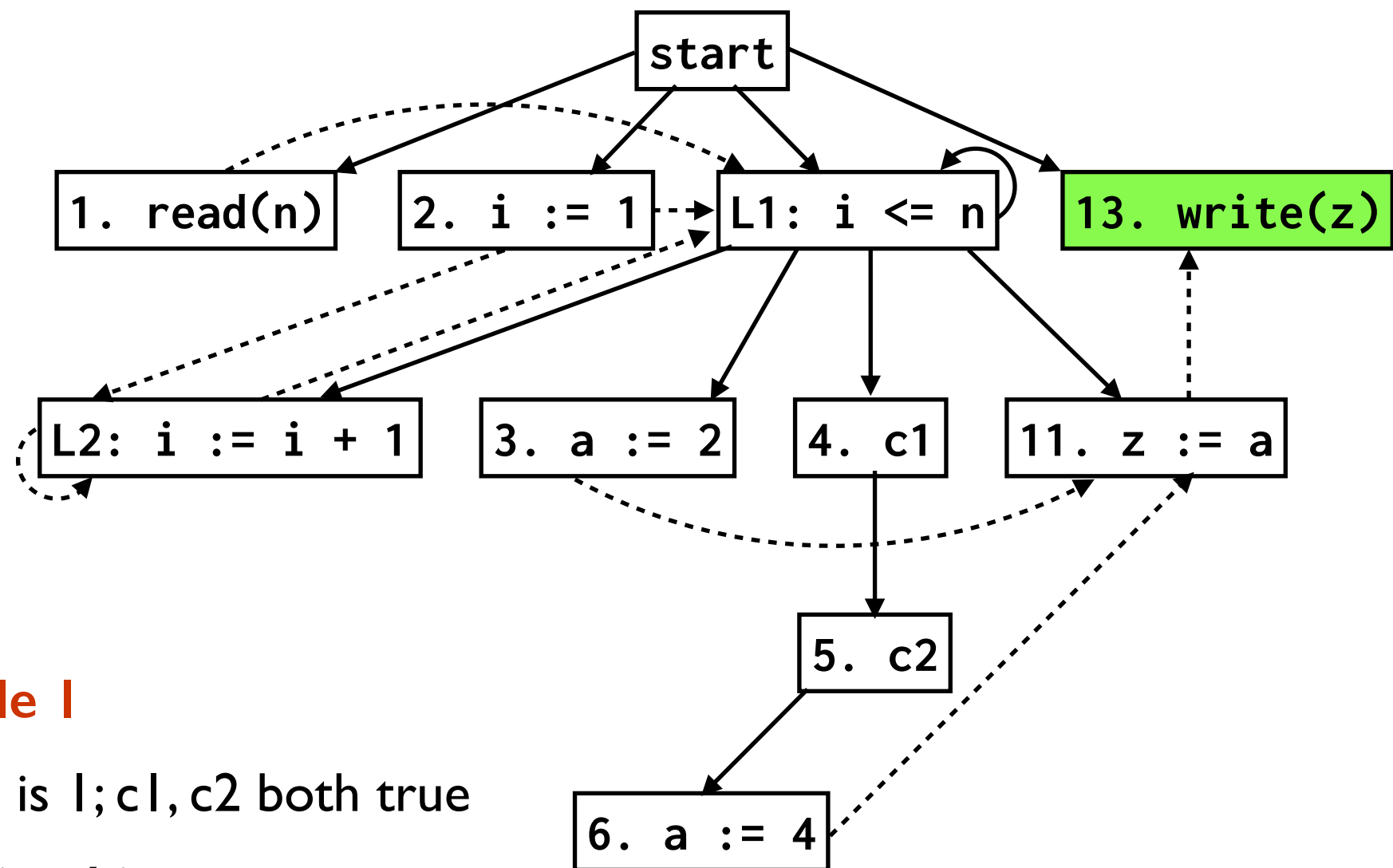
Example 1

Input n is 1; c1, c2 both true

Execution history is:

1, 2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion $\langle 13, z \rangle$



Compute the static slice for

$\langle 13, z \rangle$

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

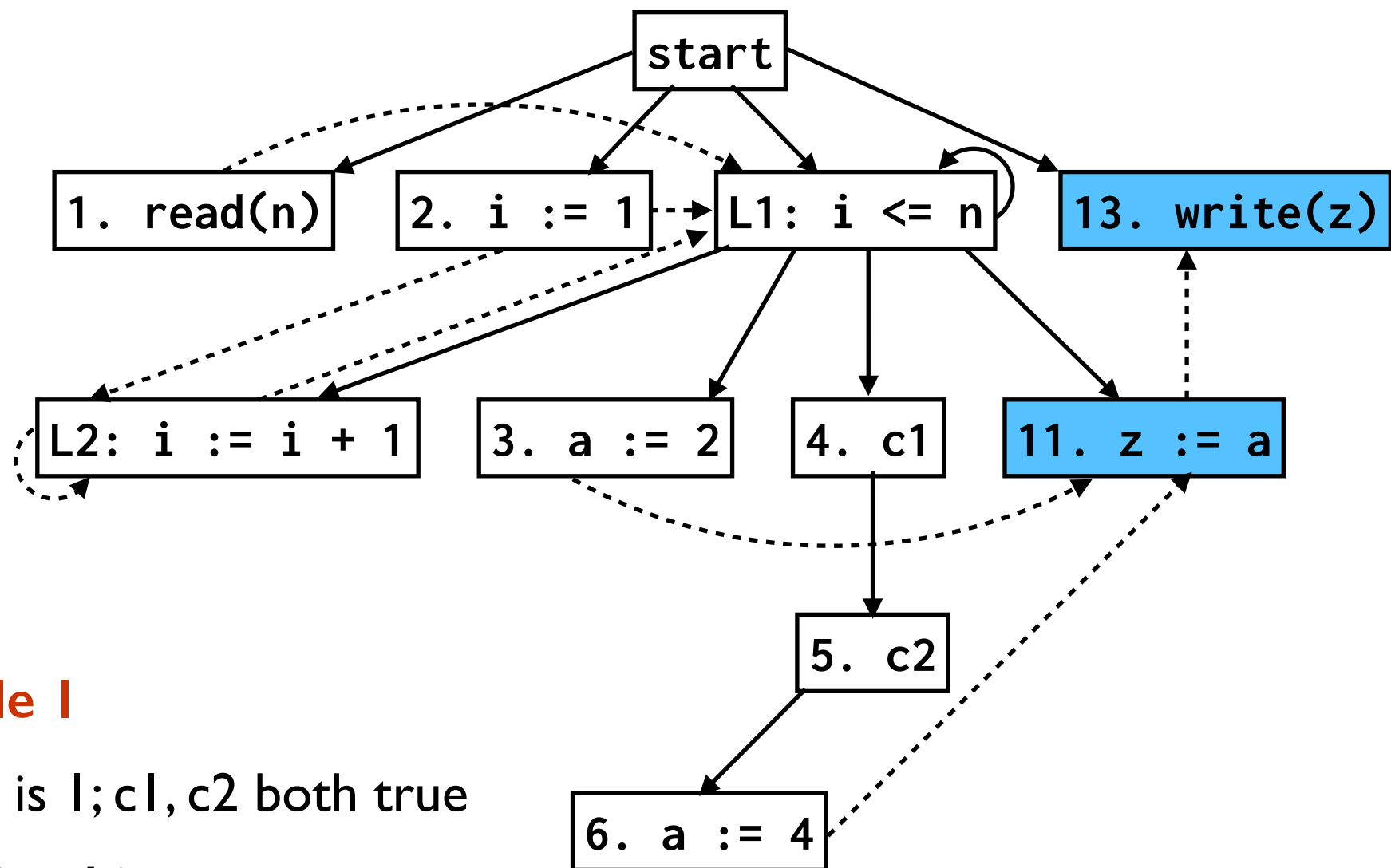
Example 1

Input n is 1; c1, c2 both true

Execution history is:

1, 2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion $\langle 13, z \rangle$



Compute the static slice for
 $\langle 13, z \rangle$

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

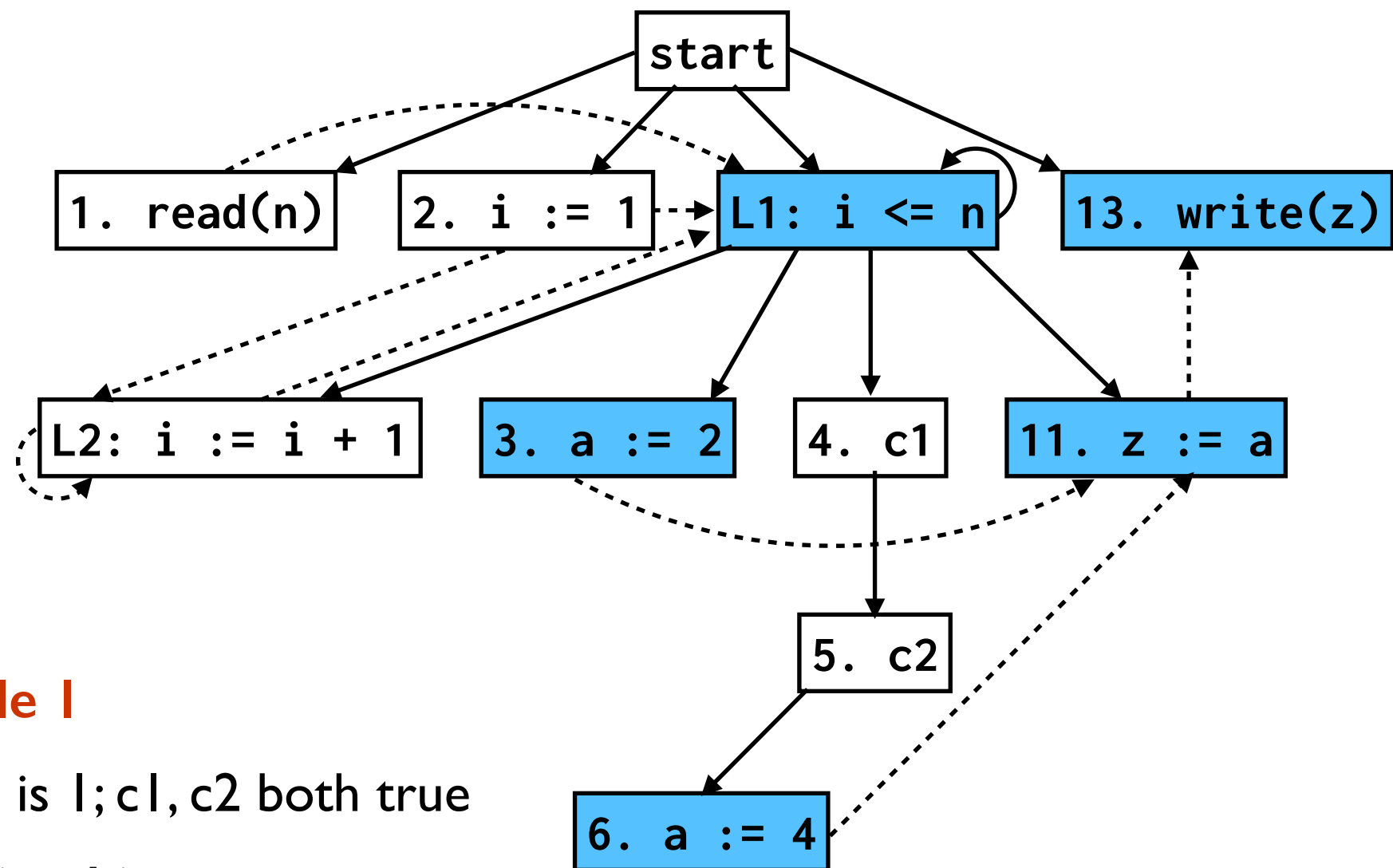
Example 1

Input n is 1; c1, c2 both true

Execution history is:

1, 2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion $\langle 13, z \rangle$



Compute the static slice for
 $\langle 13, z \rangle$

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

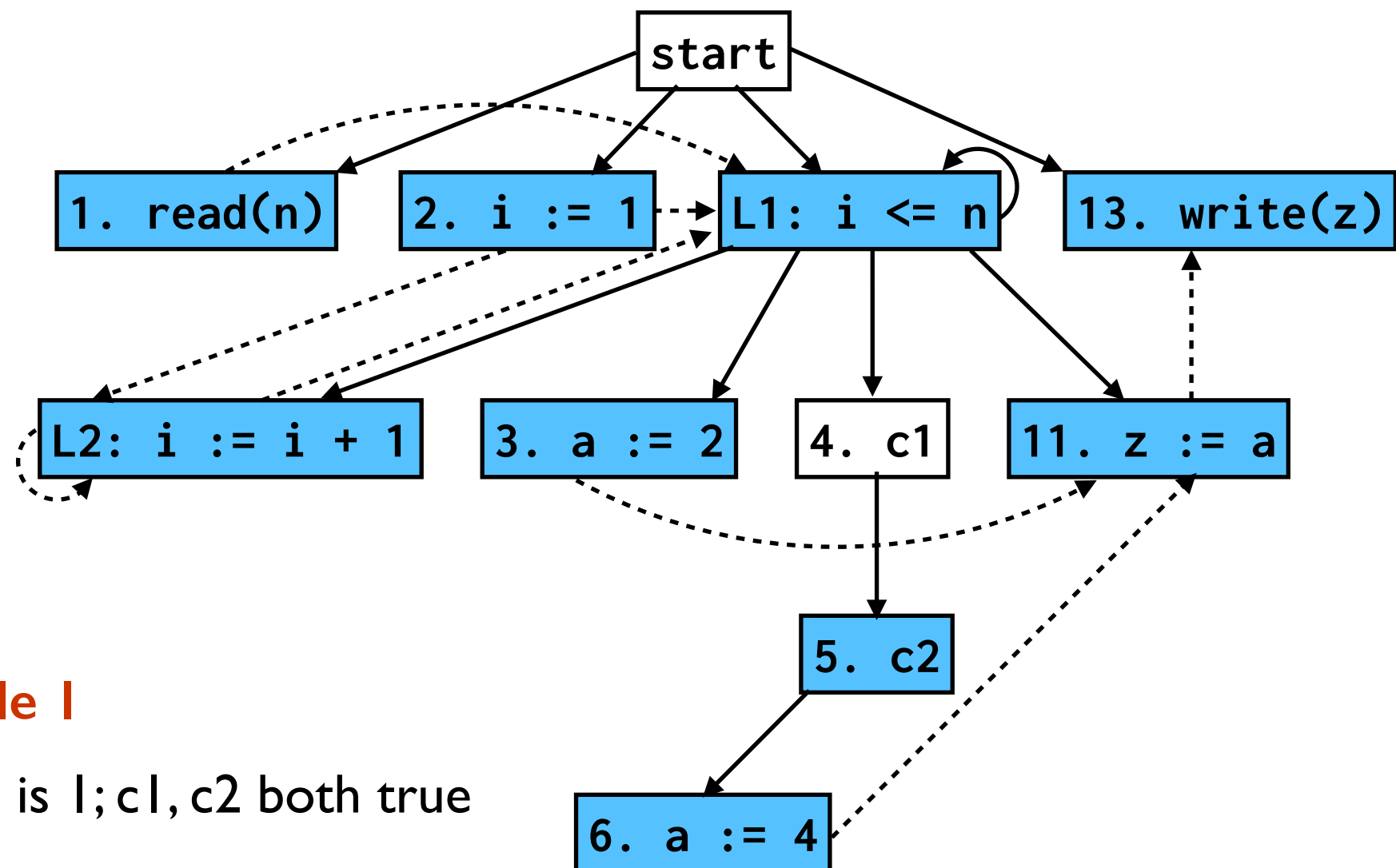
Example 1

Input n is 1; c1, c2 both true

Execution history is:

1, 2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion $\langle 13, z \rangle$



Compute the static slice for
 $\langle 13, z \rangle$

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

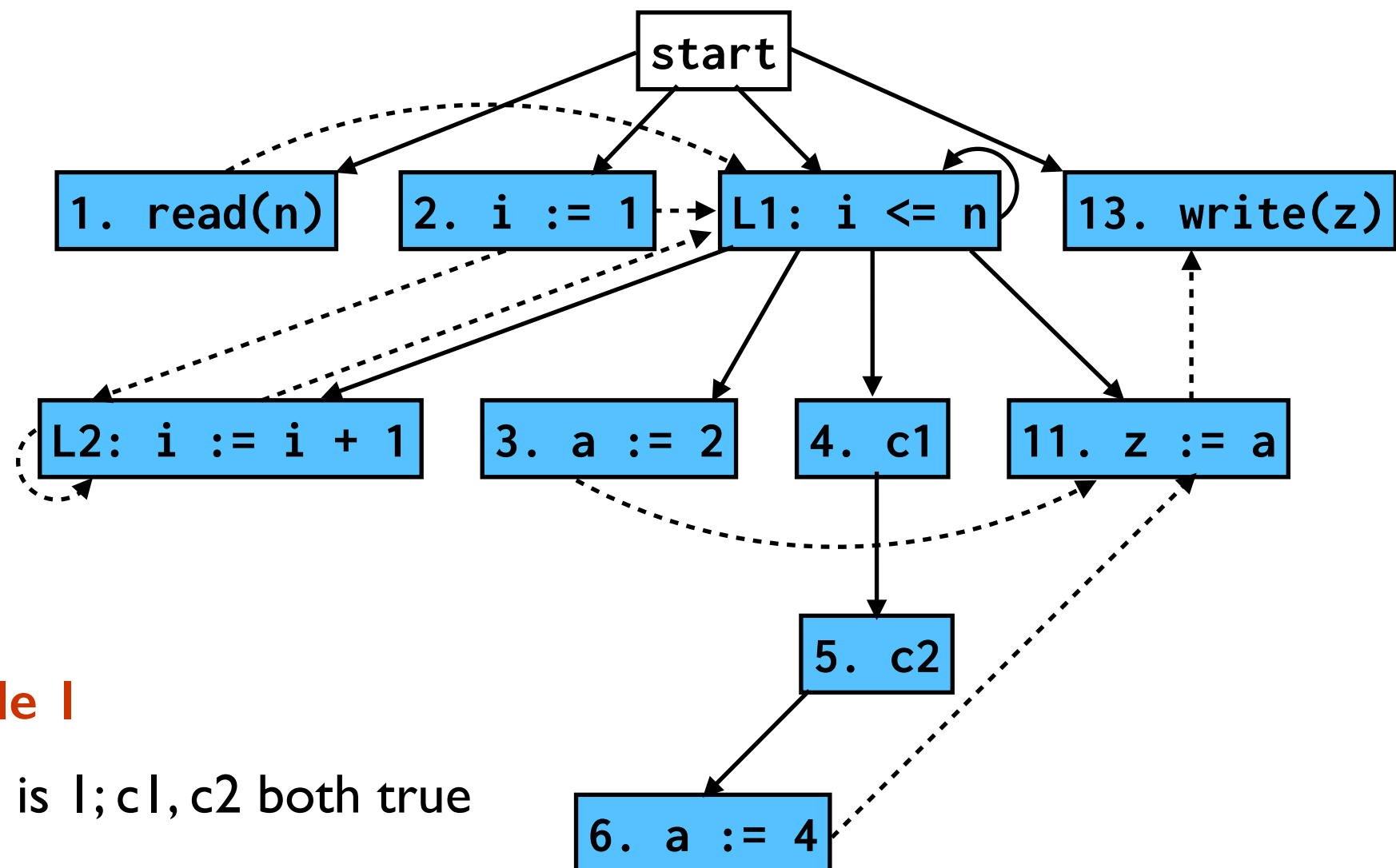
Example 1

Input n is 1; c1, c2 both true

Execution history is:

1, 2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion $\langle 13, z \rangle$



Slice includes all nodes reachable
in the reduced PDG:
1, 2, 3, 4, 5, 6, 11, 13

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.      a := 2
4.      if c1 then
5.          if c2 then
6.              a := 4
7.          else
8.              a := 6
9.          endif
10.     endif
11.     z := a
12. endfor
13. write(z)
```

Example 1

Input n is 1; c1, c2 both true

Execution history is:

1, 2, L1, 3, 4, 5, 6, 11, L2, L1, 13

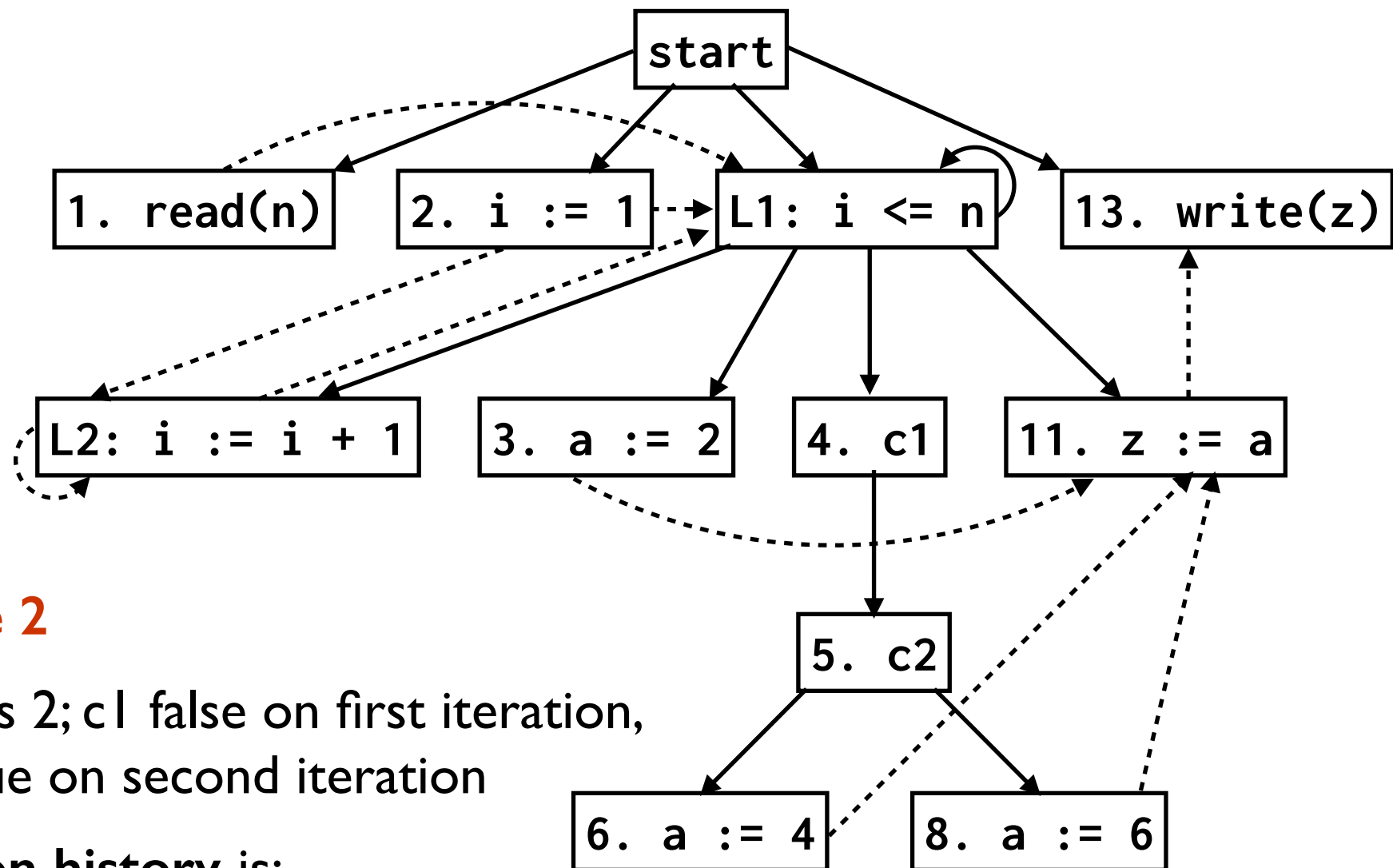
Criterion <13, z>

```
1.  read(n)
2.  for i := 1 to n do
3.      a := 2
4.      if c1 then
5.          if c2 then
6.              a := 4
7.          else
8.              a := 6
9.          endif
10.     endif
11.     z := a
12. endfor
13. write(z)
```

Slice includes all nodes reachable
in the reduced PDG:
1, 2, 3, 4, 5, 6, 11, 13

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```



Example 2

Input n is 2; c1 false on first iteration,
c1, c2 true on second iteration

Execution history is:

1, 2, L1, 3, 4, 11, L2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion <13, z>

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

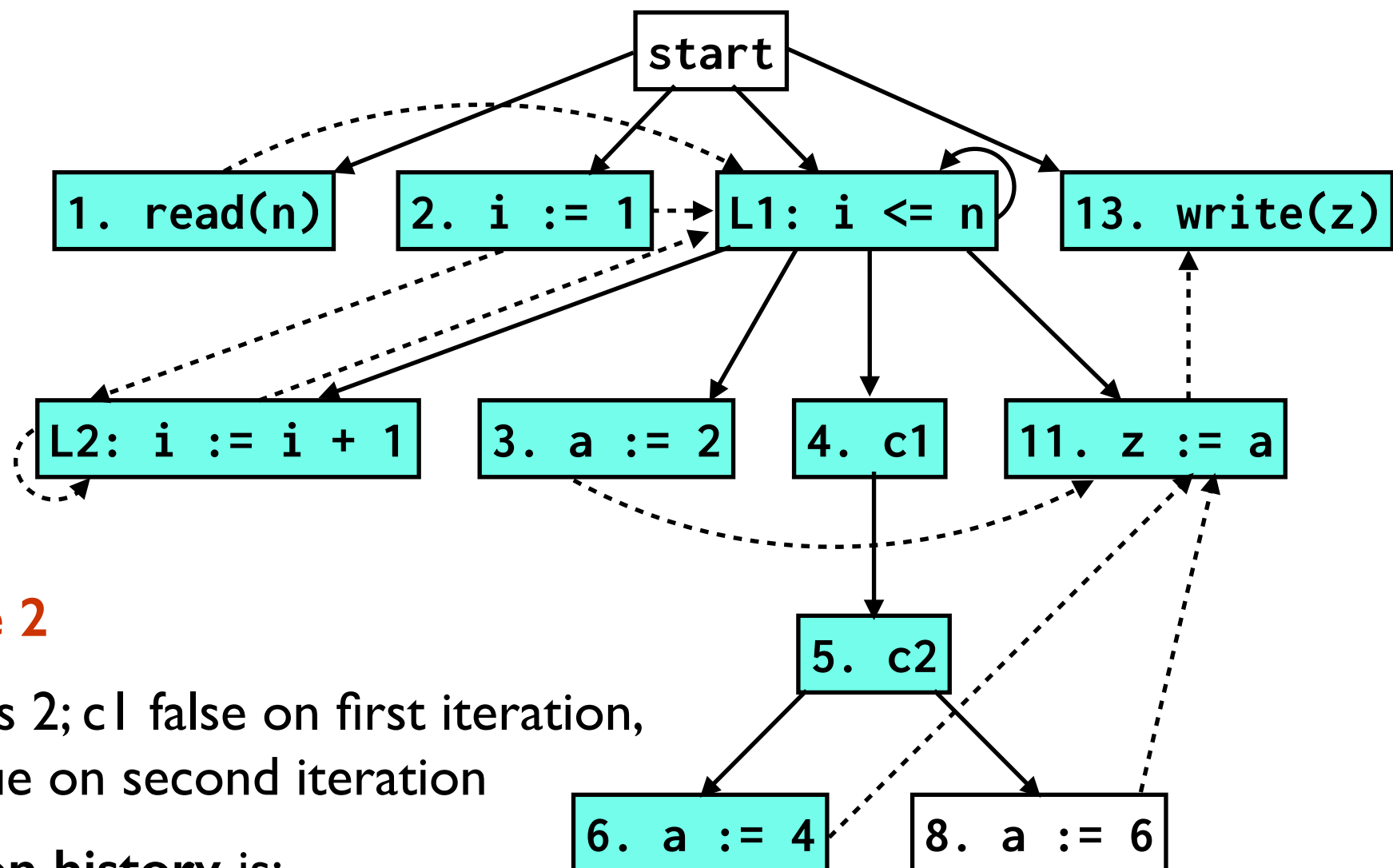
Example 2

Input n is 2; c1 false on first iteration,
c1, c2 true on second iteration

Execution history is:

1, 2, L1, 3, 4, 11, L2, L1, 3, 4, 5, 6, 11, L2, L1, 13

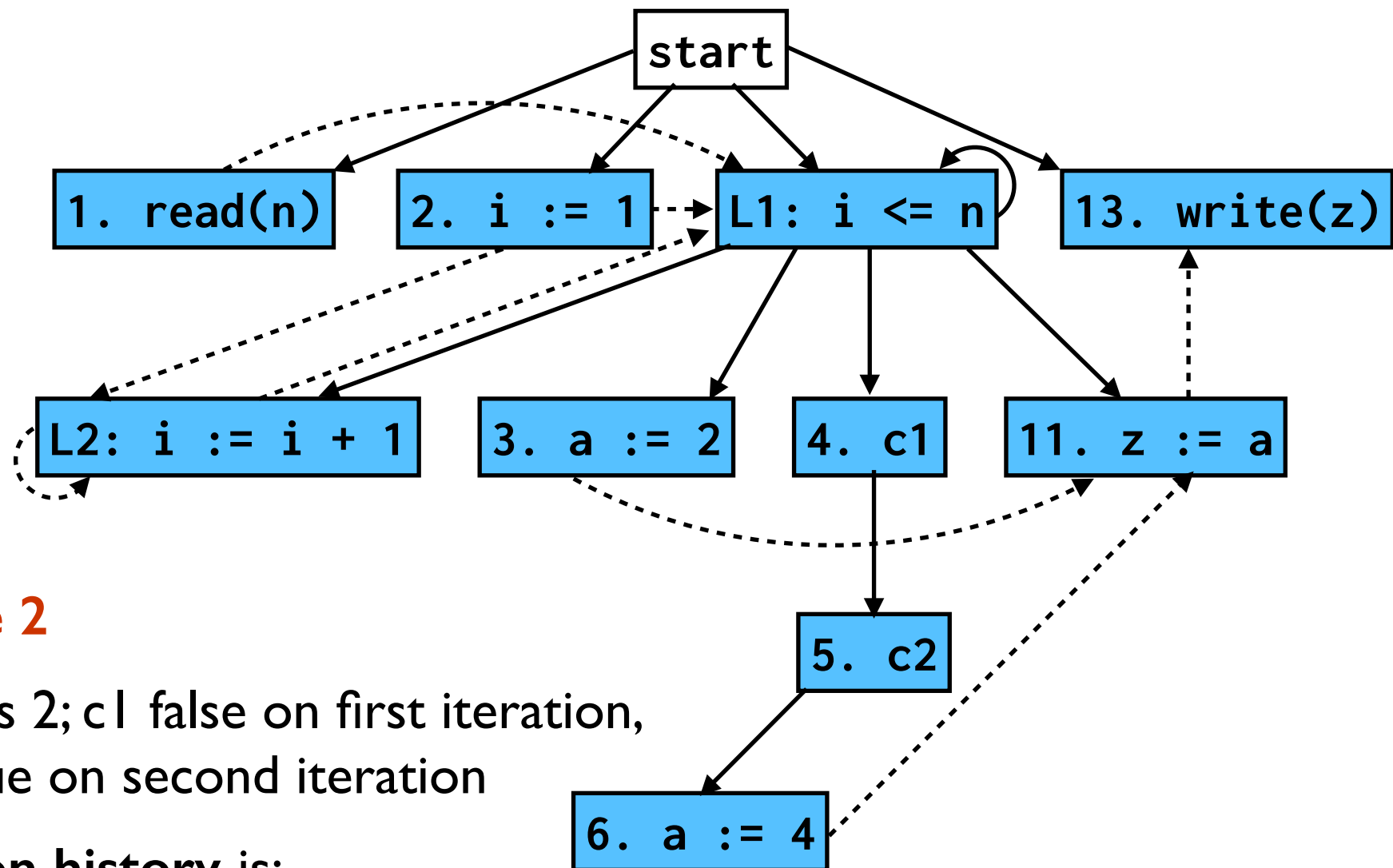
Criterion <13, z>



Mark nodes in PDG that are
executed → reduced PDG

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```



Example 2

Input n is 2; $c1$ false on first iteration,
 $c1, c2$ true on second iteration

Execution history is:

1, 2, L1, 3, 4, 11, L2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion $\langle 13, z \rangle$

Slice includes all nodes reachable
in reduced PDG— 1, 2, 3, 4, 5, 6, 11, 13

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

Example 2

Input n is 2; c1 false on first iteration,
c1, c2 true on second iteration

Execution history is:

1, 2, L1, 3, 4, 11, L2, L1, 3, 4, 5, 6, 11, L2, L1, 13

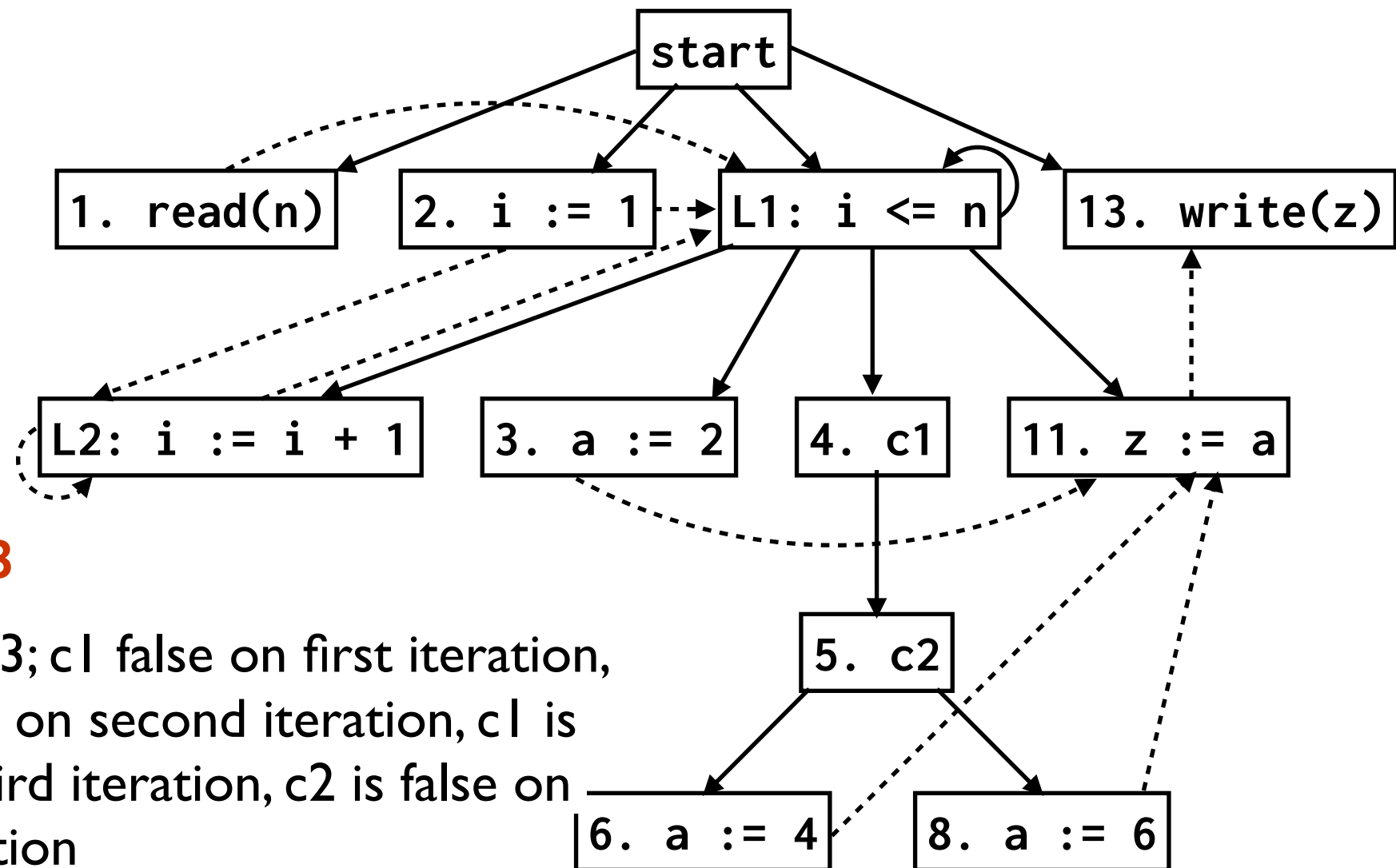
Criterion <13, z>

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

Slice includes all nodes reachable
in reduced PDG— 1,2,3,4,5,6,11,13

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```



Example 3

Input n is 3; c1 false on first iteration, c1, c2 true on second iteration, c1 is true on third iteration, c2 is false on third iteration

Execution history is:

1, 2, L1, 3, 4, 11, L2, L1, 3, 4, 5, 6, 11, L2,
L1, 3, 4, 5, 8, 11, L2, L1, 13

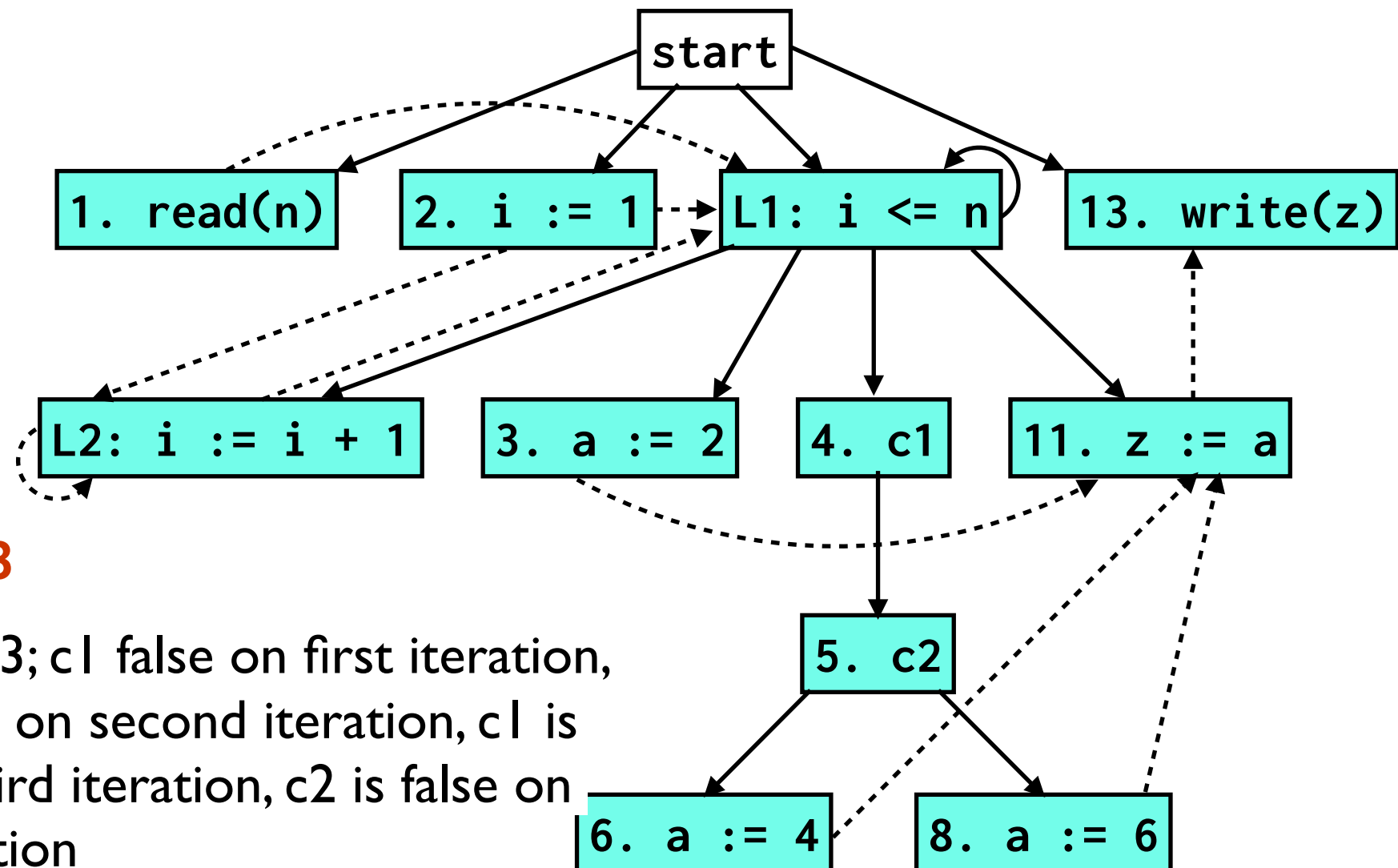
Criterion <13, z>

Dynamic Slicing Dependence Graphs

```

1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)

```



Example 3

Input n is 3; c1 false on first iteration, c1, c2 true on second iteration, c1 is true on third iteration, c2 is false on third iteration

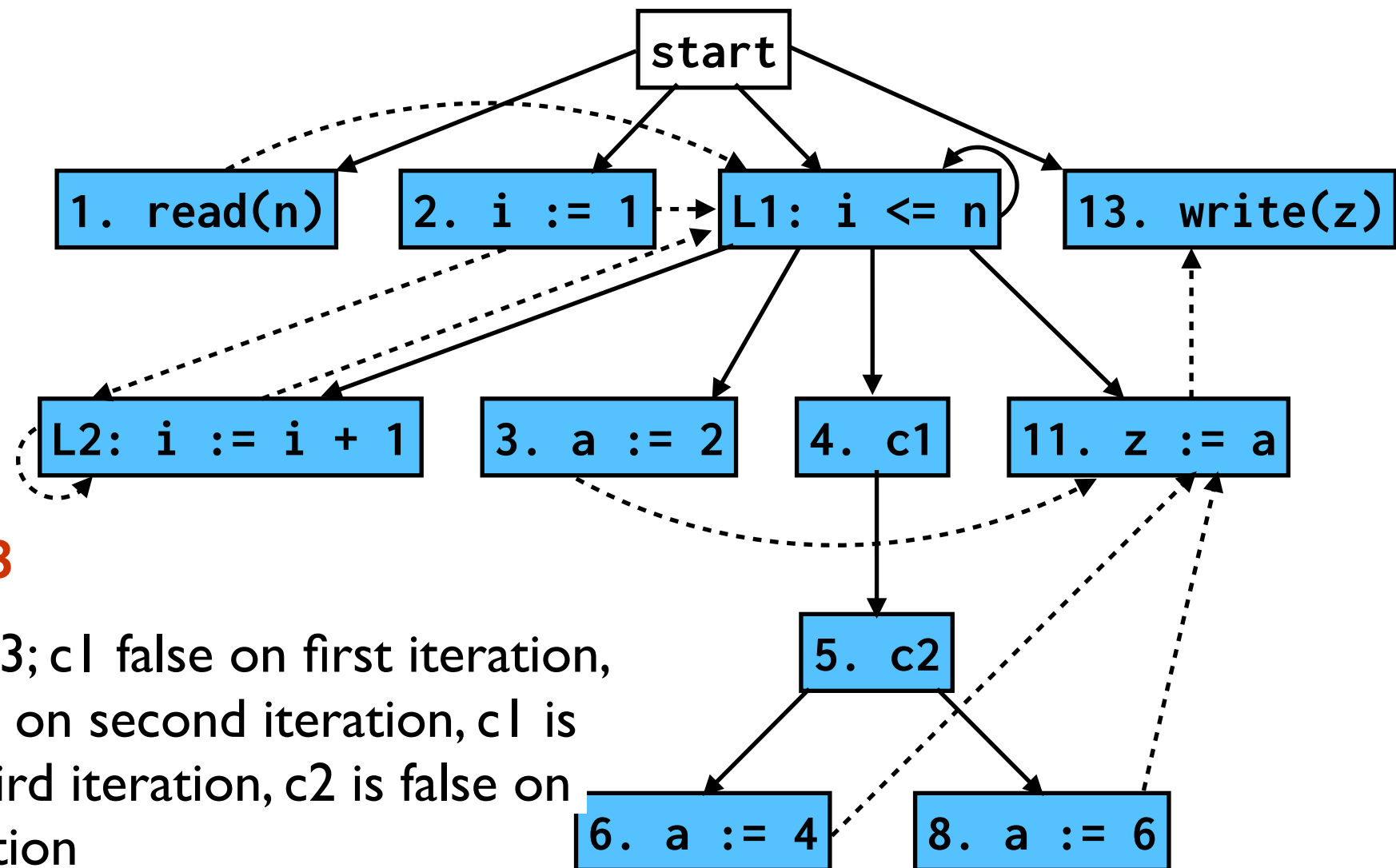
Execution history is:

1, 2, L1, 3, 4, 11, L2, L1, 3, 4, 5, 6, 11, L2,
L1, 3, 4, 5, 8, 11, L2, L1, 13

Criterion <13, z>

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```



Example 3

Input n is 3; c1 false on first iteration, c1, c2 true on second iteration, c1 is true on third iteration, c2 is false on third iteration

Execution history is:

1, 2, L1, 3, 4, 11, L2, L1, 3, 4, 5, 6, 11, L2,
L1, 3, 4, 5, 8, 11, L2, L1, 13

Slice includes all nodes reachable
in reduced PDG: 1, 2, 3, 4, 5, 6, 8, 11, 13

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.      a := 2
4.      if c1 then
5.          if c2 then
6.              a := 4
7.          else
8.              a := 6
9.          endif
10.     endif
11.     z := a
12. endfor
13. write(z)
```

Example 3

Input n is 3; c1 false on first iteration, c1, c2 true on second iteration, c1 is true on third iteration, c2 is false on third iteration

Execution history is:

1, 2, L1, 3, 4, 11, L2, L1, 3, 4, 5, 6, 11, L2,
L1, 3, 4, 5, 8, 11, L2, L1, 13

```
1.  read(n)
2.  for i := 1 to n do
3.      a := 2
4.      if c1 then
5.          if c2 then
6.              a := 4
7.          else
8.              a := 6
9.          endif
10.     endif
11.     z := a
12. endfor
13. write(z)
```

Slice includes all nodes reachable
in reduced PDG: 1,2,3,4,5,6,8,11,13

Dynamic Slicing: Issues

- Multiple occurrences of a single statement are represented as a single node in PDG
- Different occurrences of a statement may have different dependences
- All occurrences get conflated, i.e., slices may be larger than necessary

Dynamic Slicing

- Mark **nodes** in PDG that are executed to produce the **reduced PDG**
- Static backward slice on reduced PDG

Dynamic Slicing

- Mark **edges** in PDG that are executed to produce the **reduced PDG**
- Static backward slice on reduced PDG

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

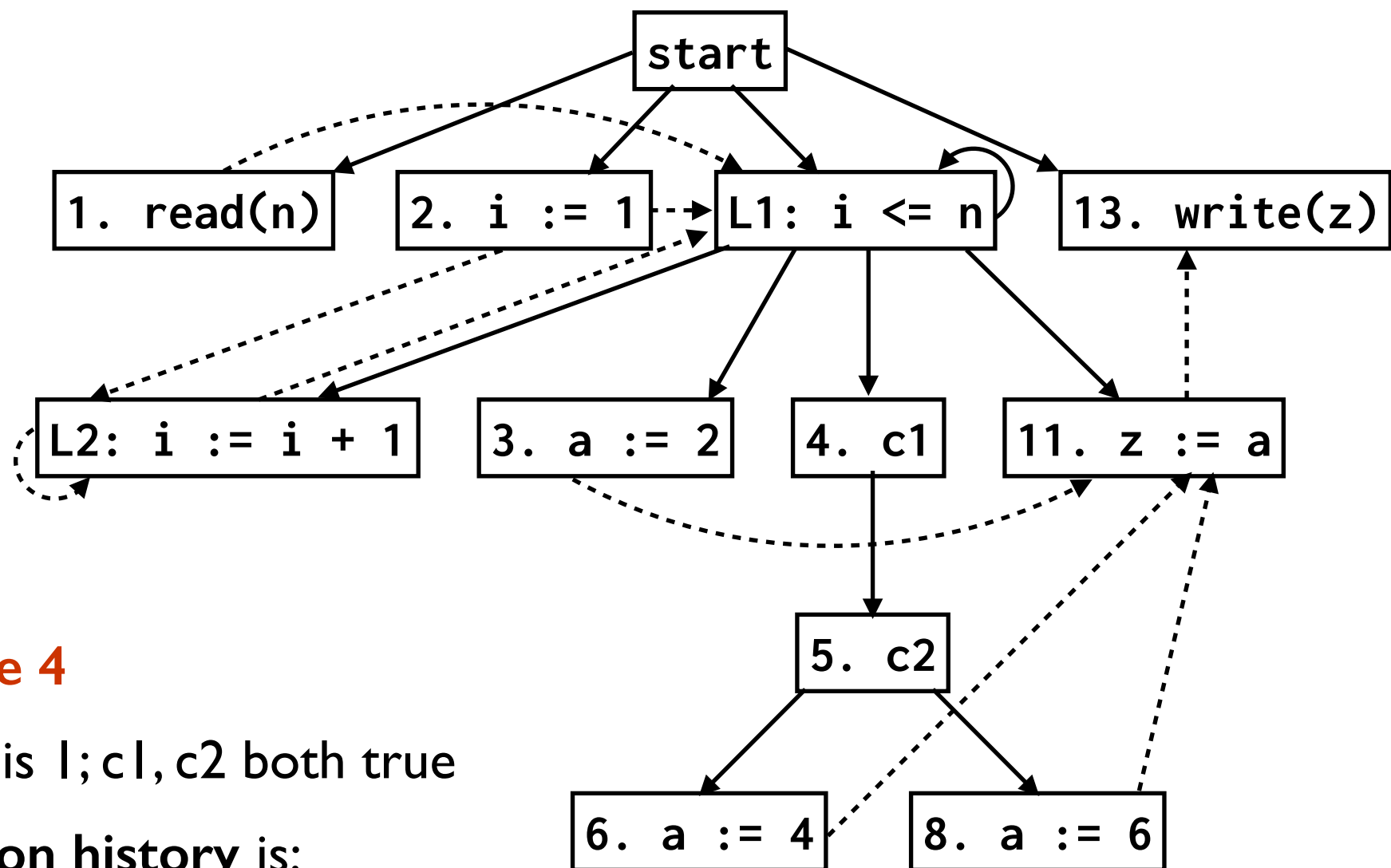
Example 4

Input n is 1; c1, c2 both true

Execution history is:

1, 2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion <13, z>



Mark edges in PDG that are executed

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

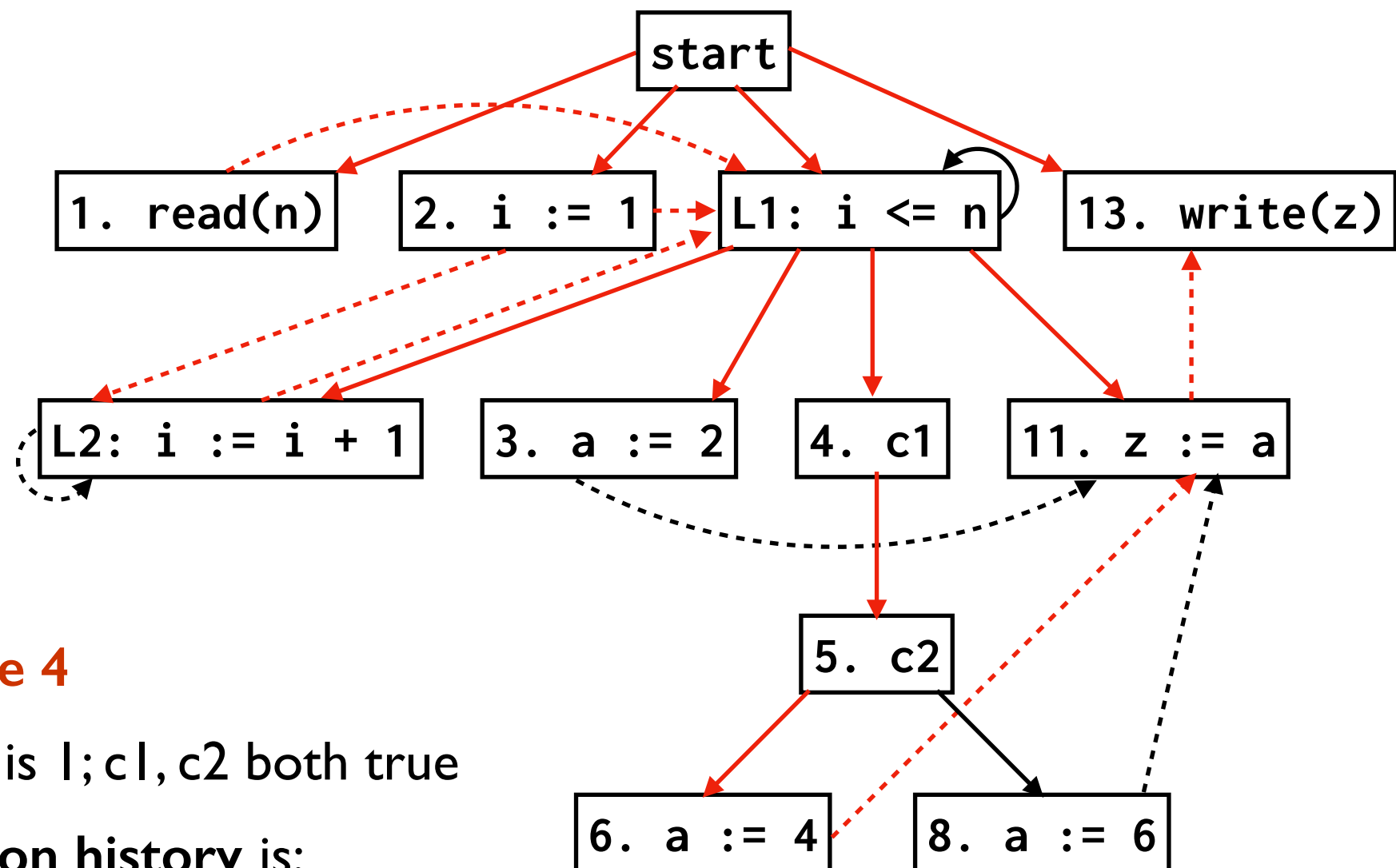
Example 4

Input n is 1; c1, c2 both true

Execution history is:

1, 2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion $\langle 13, z \rangle$



Mark edges in PDG that are executed

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

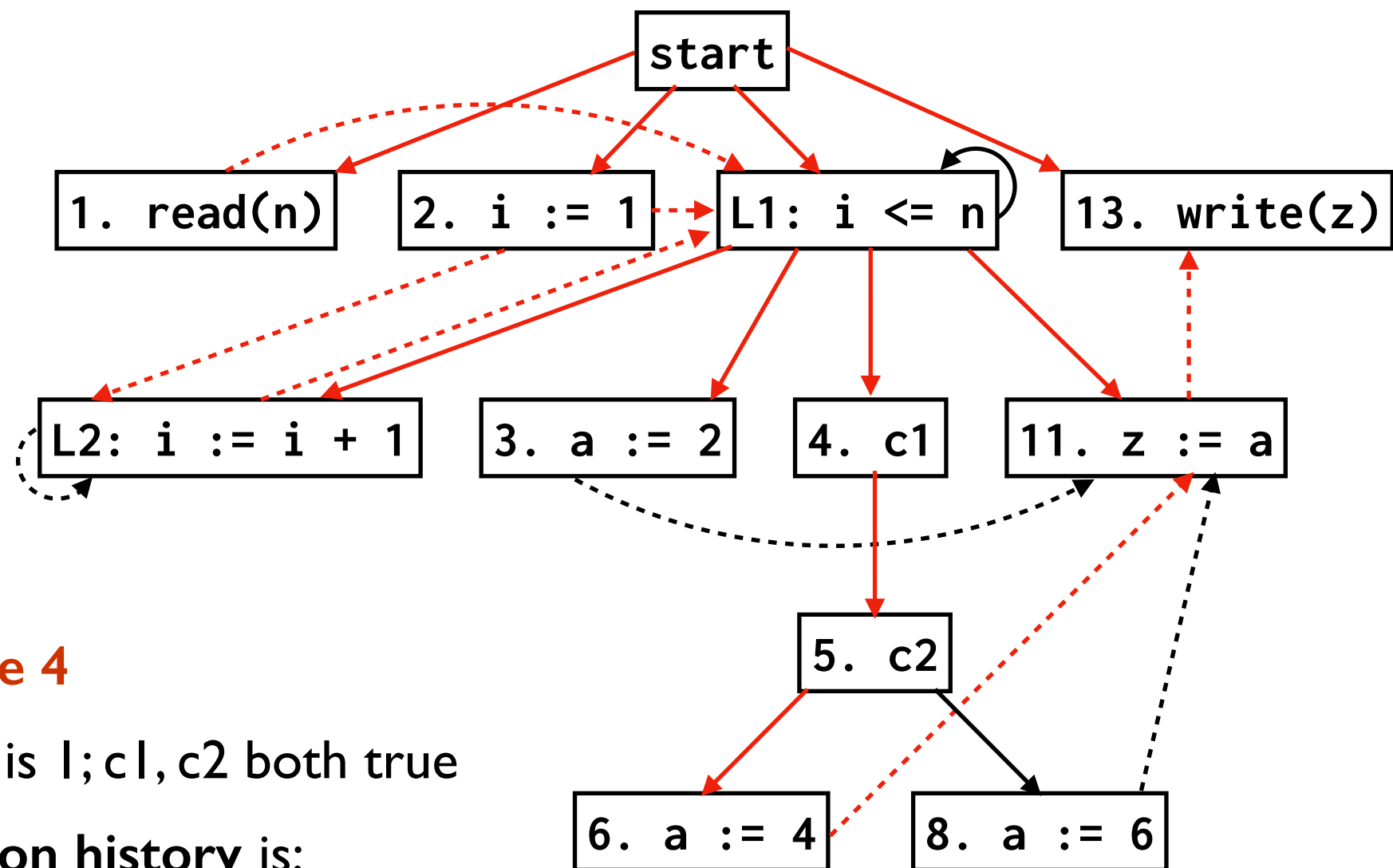
Example 4

Input n is 1; c1, c2 both true

Execution history is:

1, 2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion $\langle 13, z \rangle$



Compute the reduced PDG

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

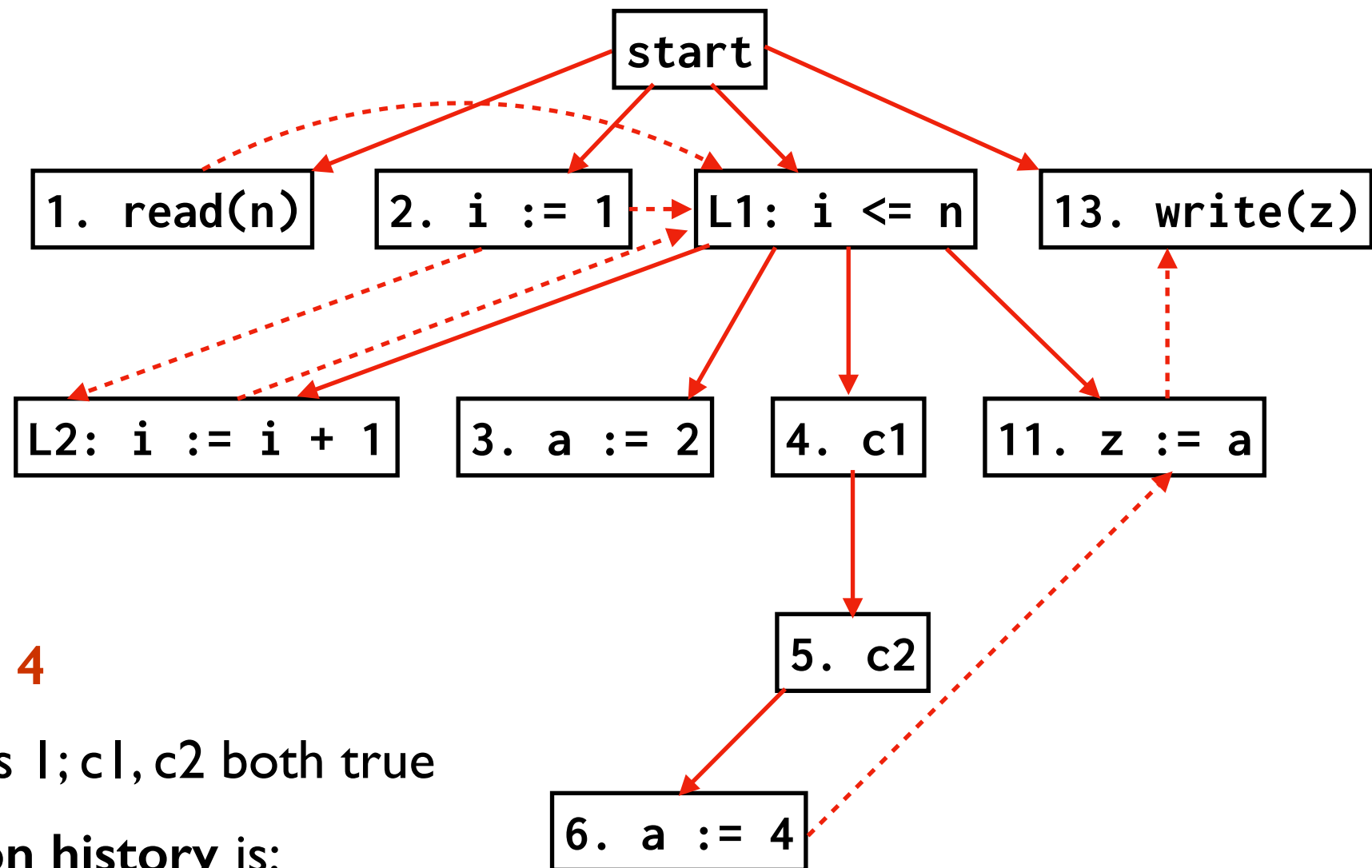
Example 4

Input n is 1; c1, c2 both true

Execution history is:

1, 2, L1, 3, 4, 5, 6, 11, L2, L1, 13

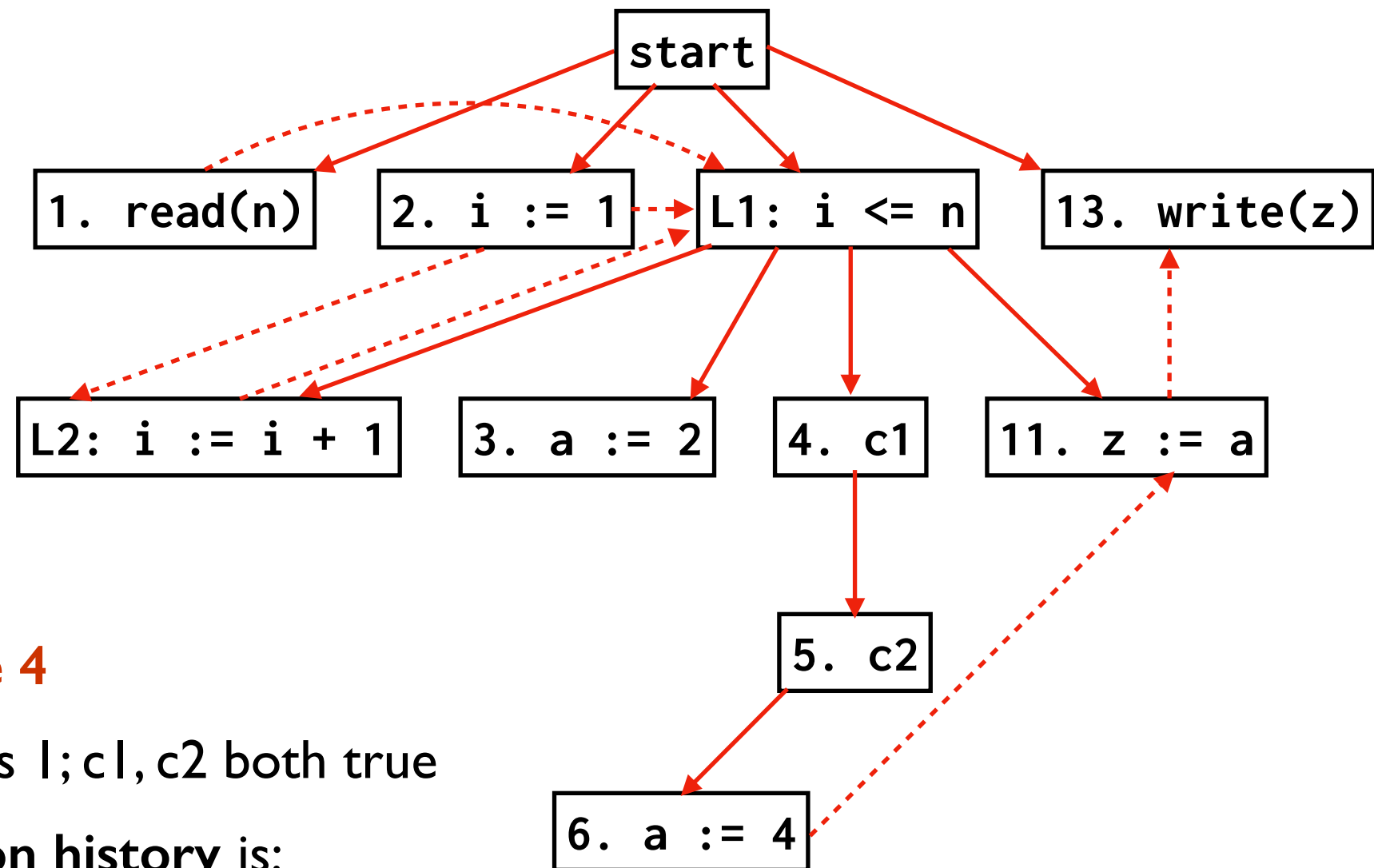
Criterion <13, z>



Compute the reduced PDG

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```



Example 4

Input n is 1; c1, c2 both true

Execution history is:

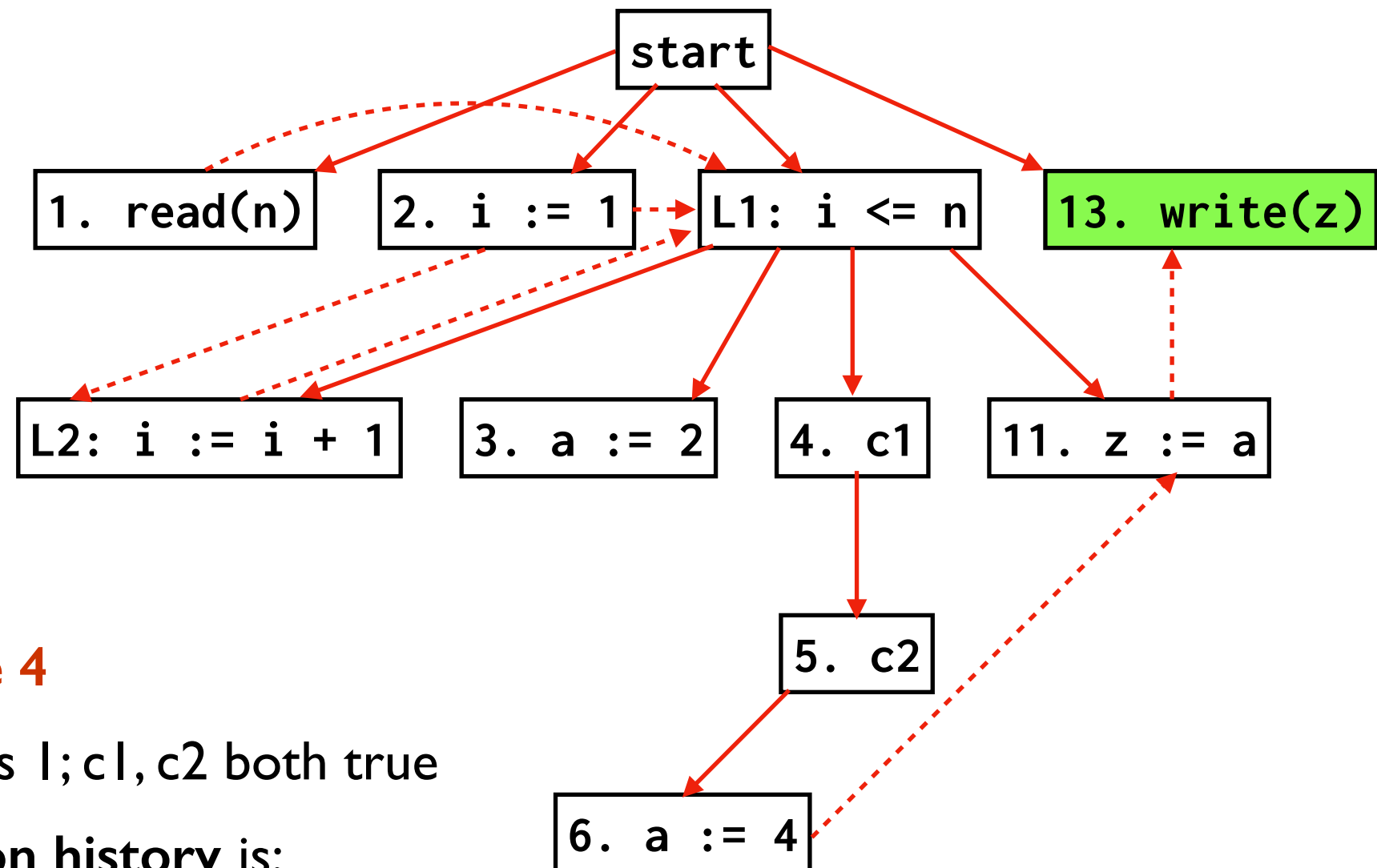
1, 2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion $\langle 13, z \rangle$

Reduced PDG still has node 3 but the data-flow edge from 3 to 11 is not in the reduced graph

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```



Example 4

Input n is 1; c1, c2 both true

Execution history is:

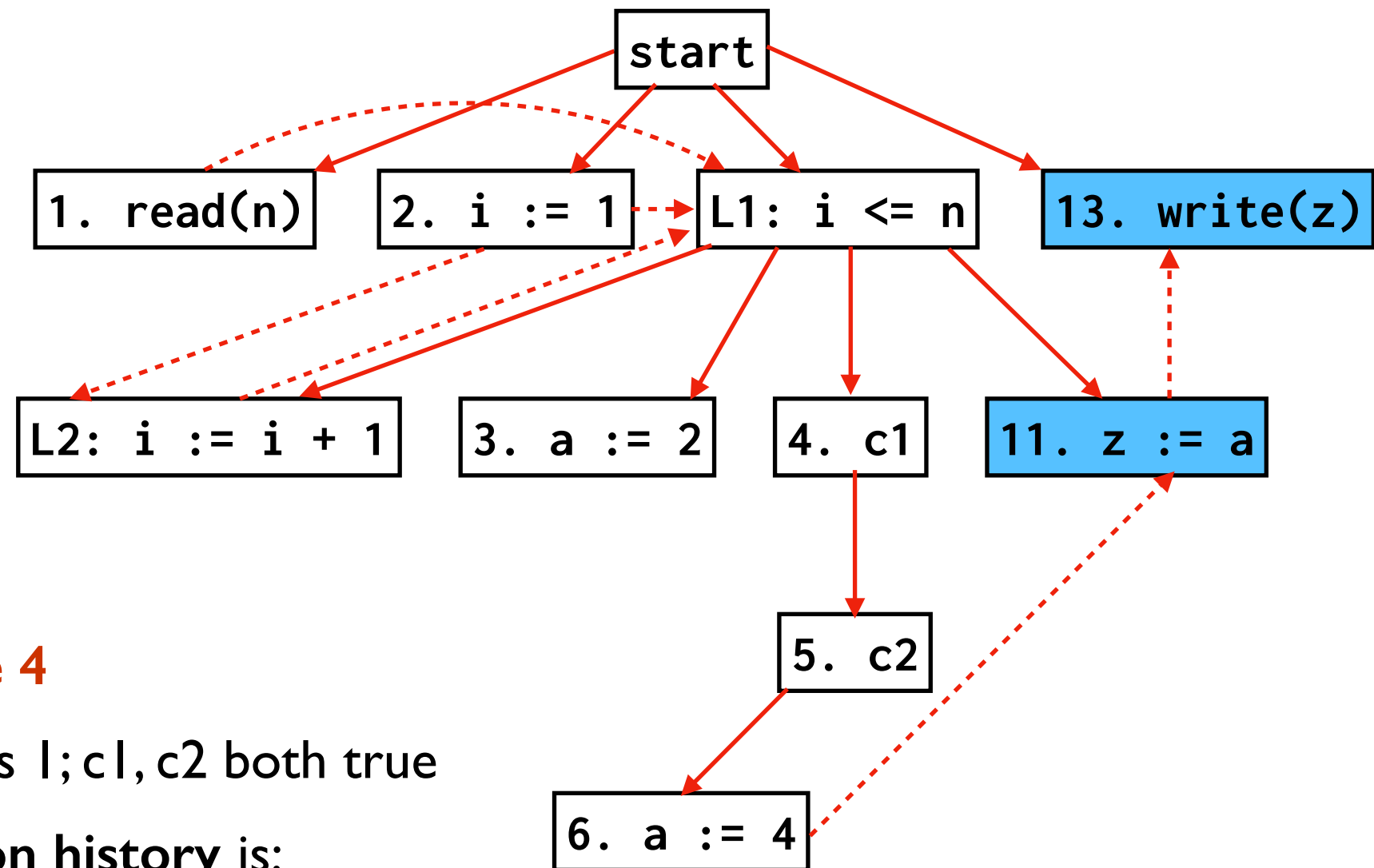
1, 2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion <13, z>

Reduced PDG still has node 3 but the data-flow edge from 3 to 11 is not in the reduced graph

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```



Example 4

Input n is 1; c1, c2 both true

Execution history is:

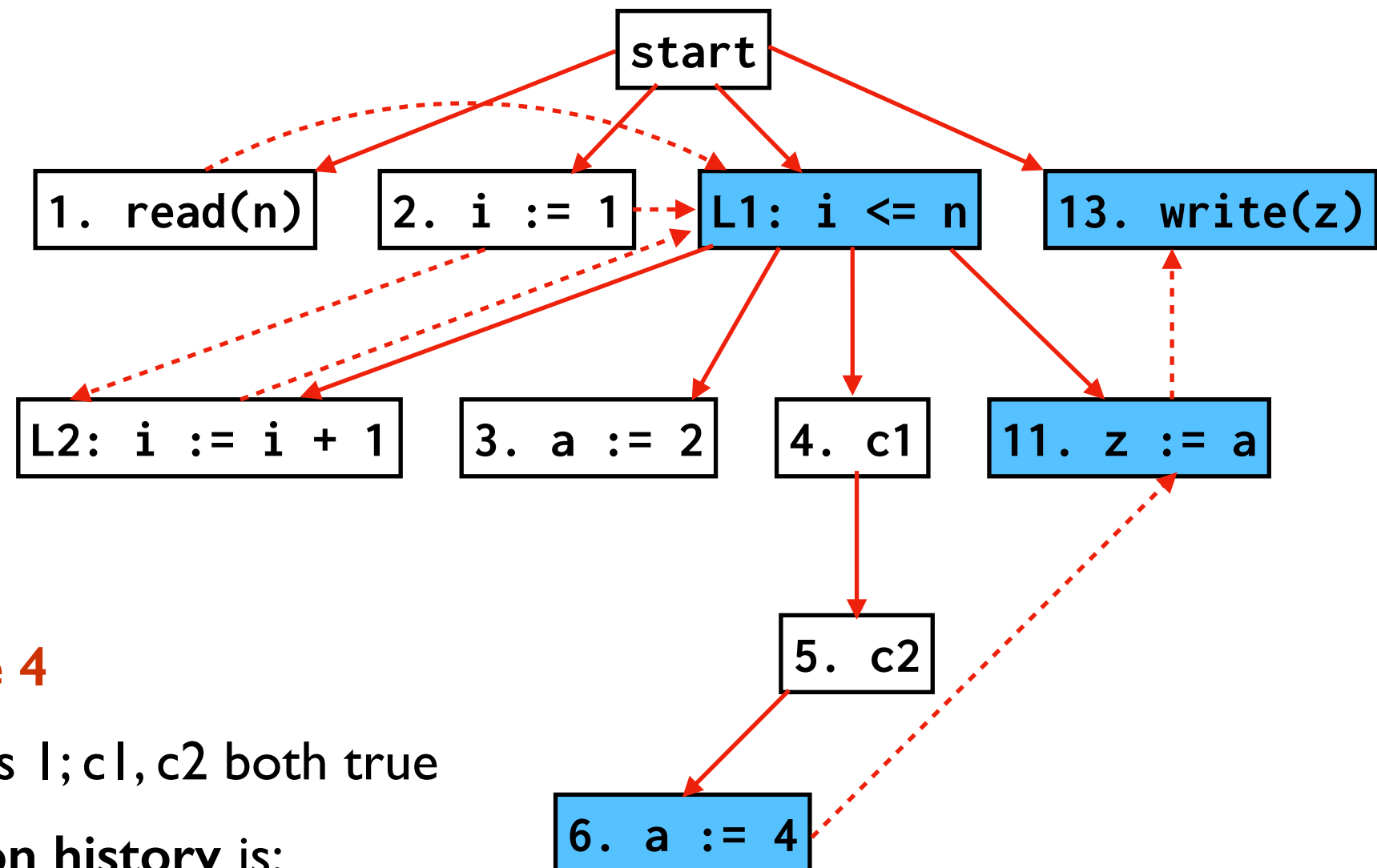
1, 2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion $\langle 13, z \rangle$

Reduced PDG still has node 3 but the data-flow edge from 3 to 11 is not in the reduced graph

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```



Example 4

Input n is 1; c1, c2 both true

Execution history is:

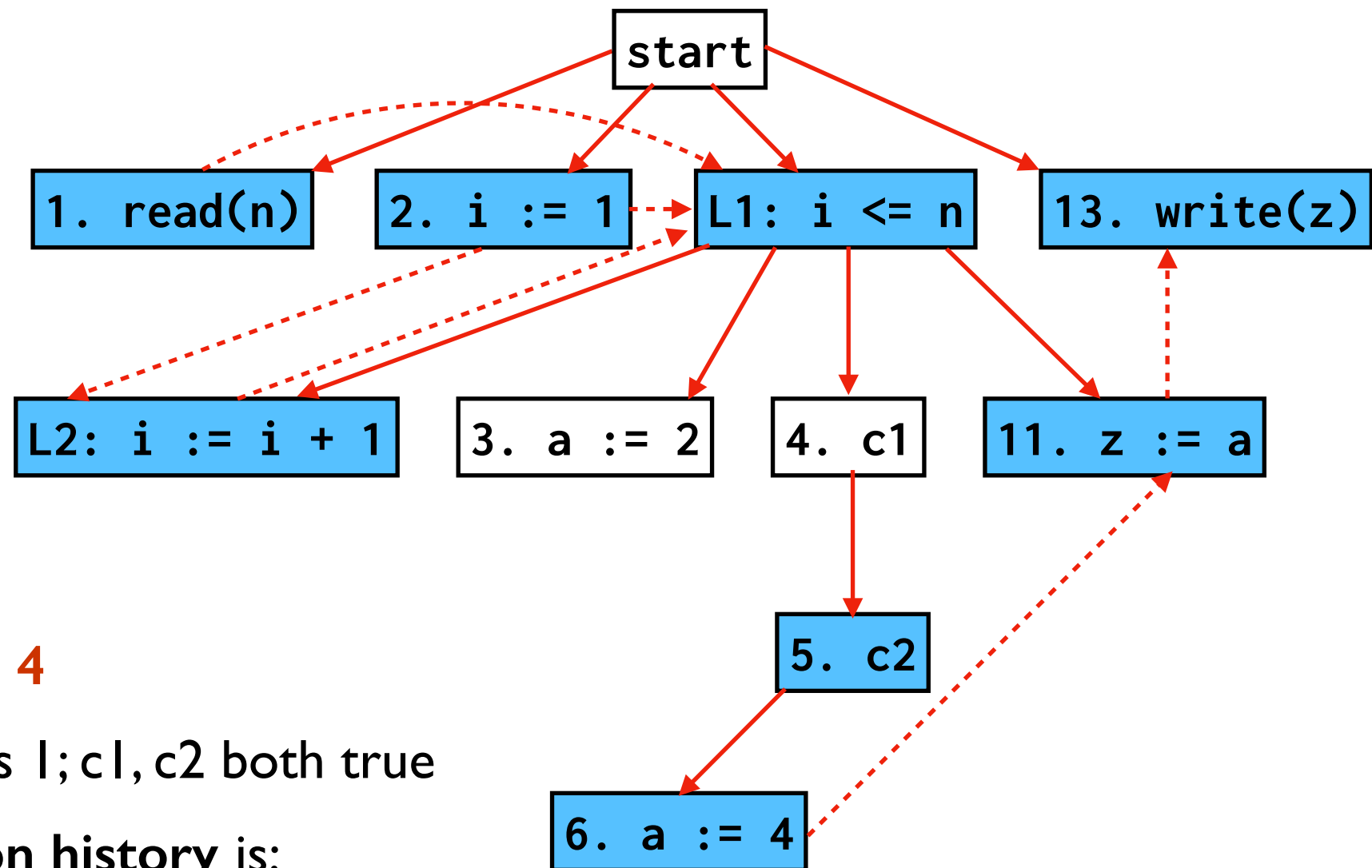
1, 2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion $\langle 13, z \rangle$

Reduced PDG still has node 3 but the data-flow edge from 3 to 11 is not in the reduced graph

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```



Example 4

Input n is 1; c1, c2 both true

Execution history is:

1, 2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion $\langle 13, z \rangle$

Reduced PDG still has node 3 but the data-flow edge from 3 to 11 is not in the reduced graph

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

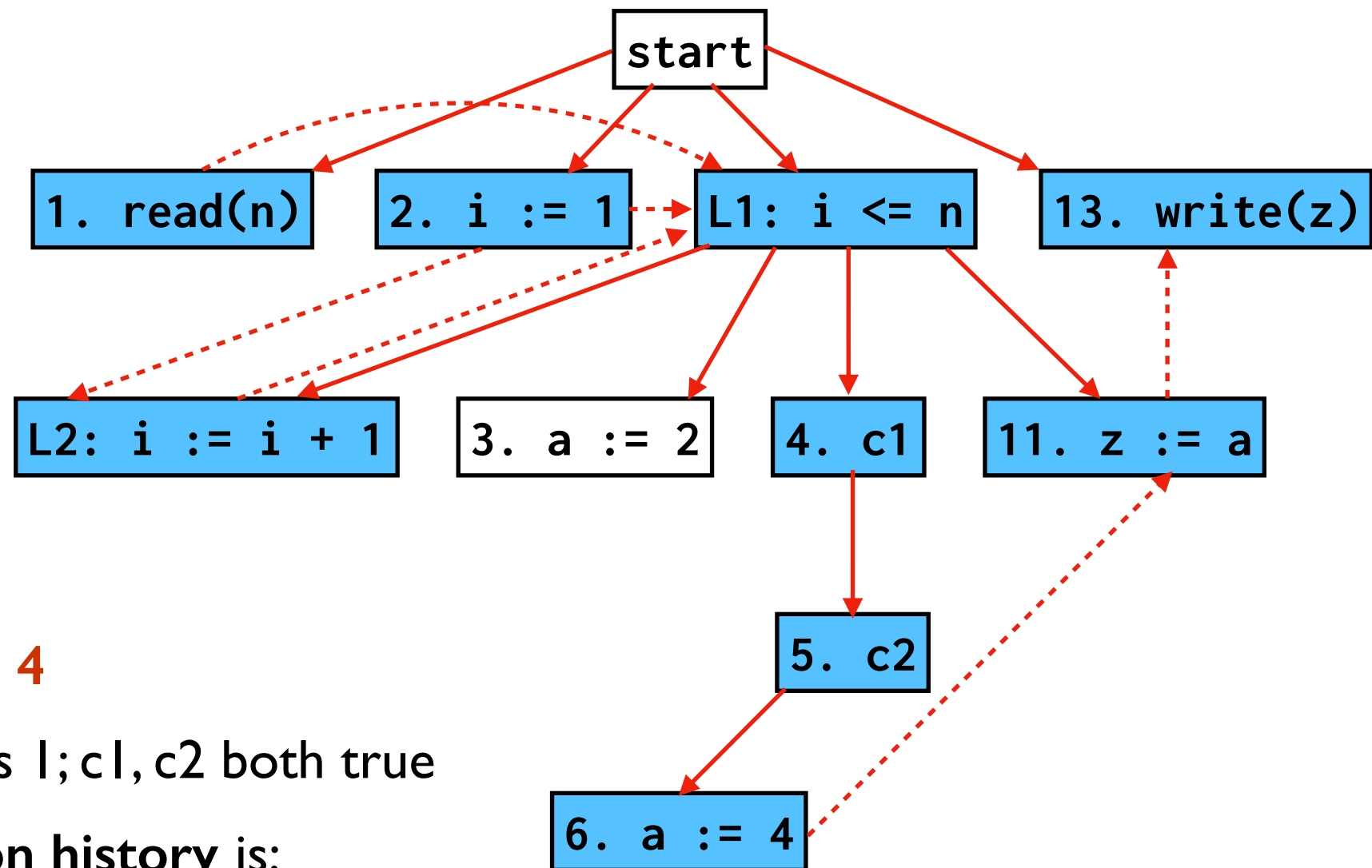
Example 4

Input n is 1; c1, c2 both true

Execution history is:

1, 2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion <13, z>



Slice includes all nodes reachable
in reduced PDG: 1, 2, 4, 5, 6, 11, 13

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.      a := 2
4.      if c1 then
5.          if c2 then
6.              a := 4
7.          else
8.              a := 6
9.          endif
10.     endif
11.     z := a
12. endfor
13. write(z)
```

Example 4

Input n is 1; c1, c2 both true

Execution history is:

1, 2, L1, 3, 4, 5, 6, 11, L2, L1, 13

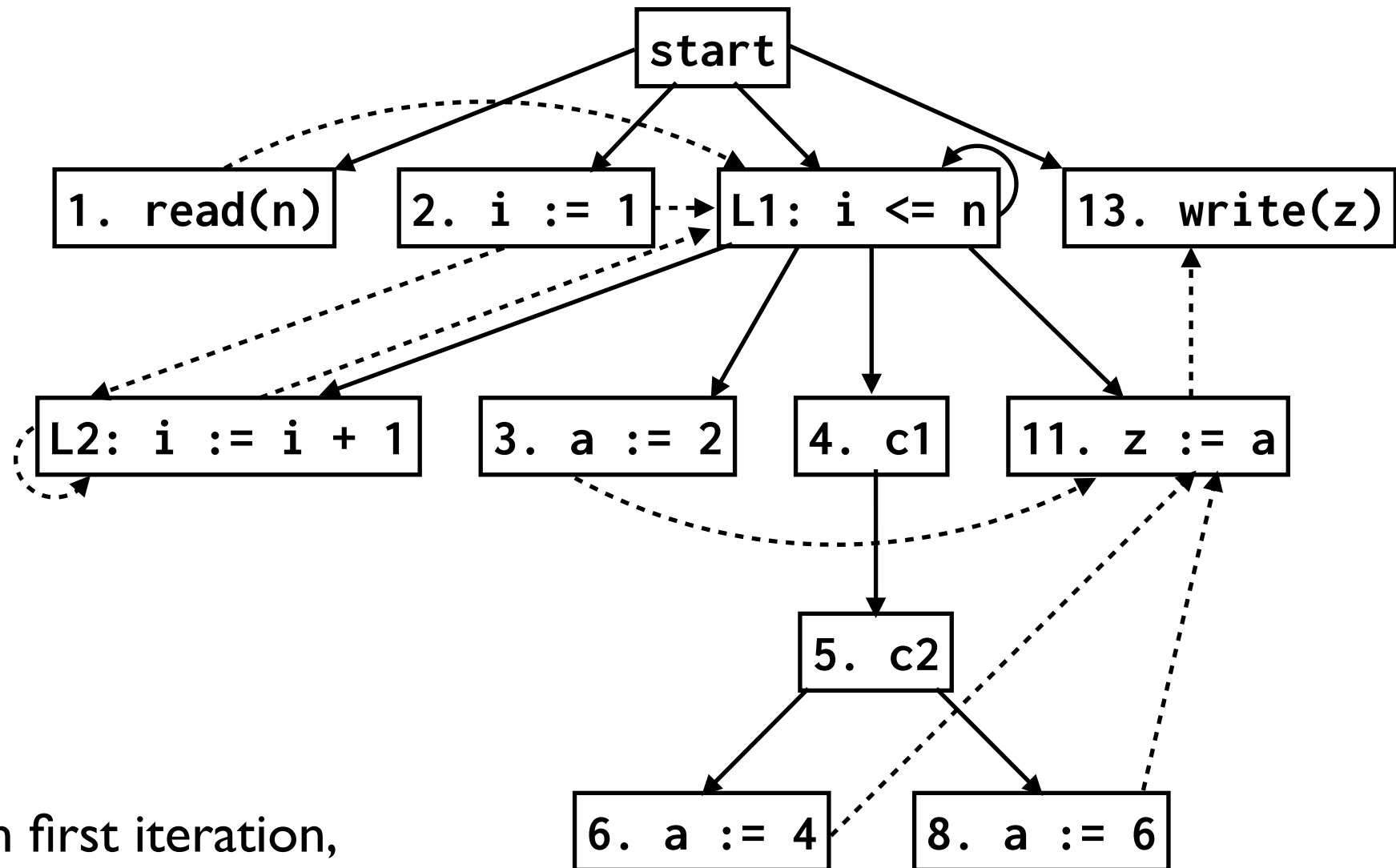
Criterion <13, z>

```
1.  read(n)
2.  for i := 1 to n do
3.      a := 2
4.      if c1 then
5.          if c2 then
6.              a := 4
7.          else
8.              a := 6
9.          endif
10.     endif
11.     z := a
12. endfor
13. write(z)
```

By marking edges executed instead of nodes executed, the slice excludes node 3

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```



Example 5

Input n is 2; $c1$ true, $c2$ false on first iteration,
 $c1$, $c2$ true on second iteration

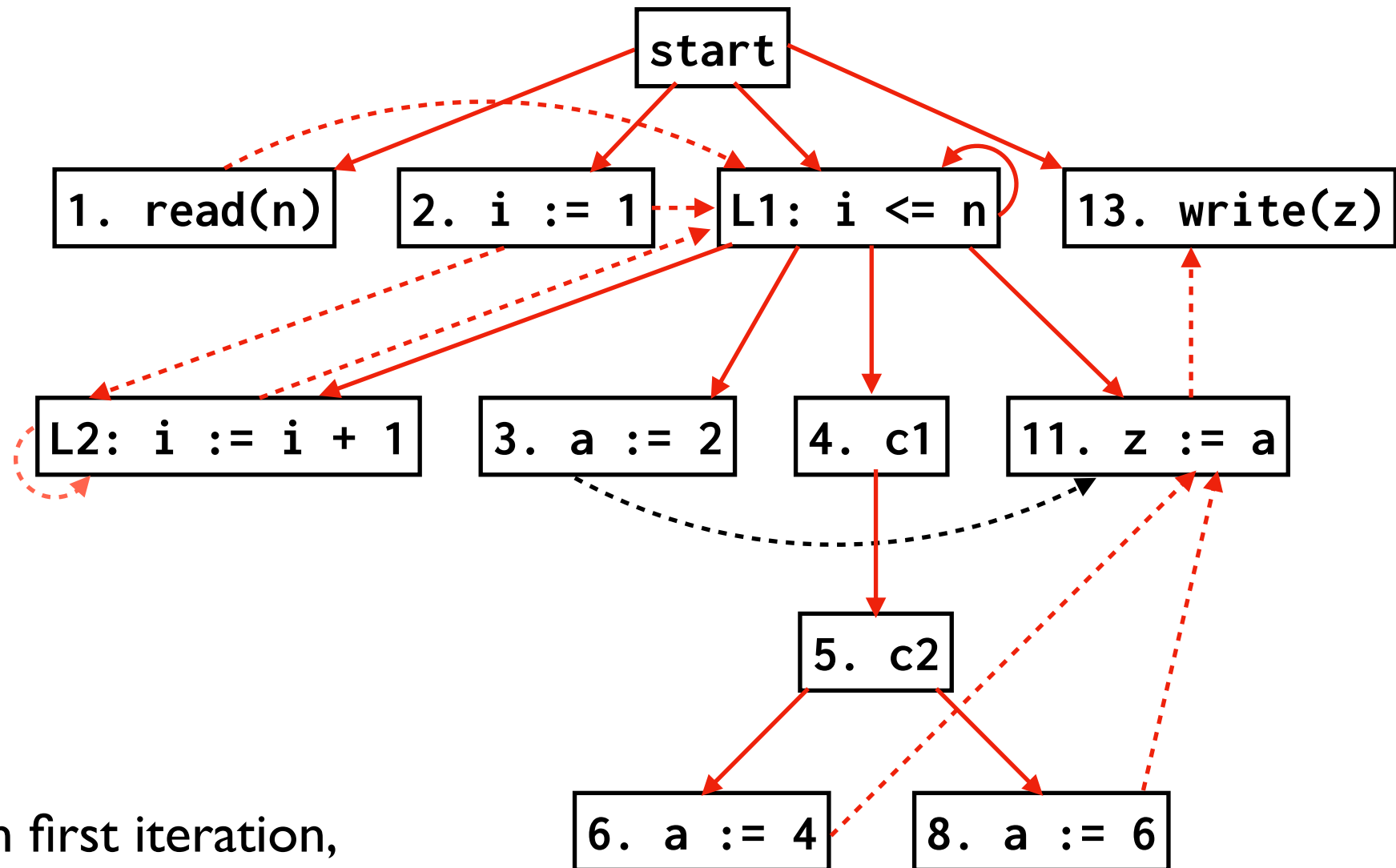
Execution history is:

1, 2, L1, 3, 4, 5, 8, 11, L2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion $\langle 13, z \rangle$

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```



Example 5

Input n is 2; $c1$ true, $c2$ false on first iteration,
 $c1$, $c2$ true on second iteration

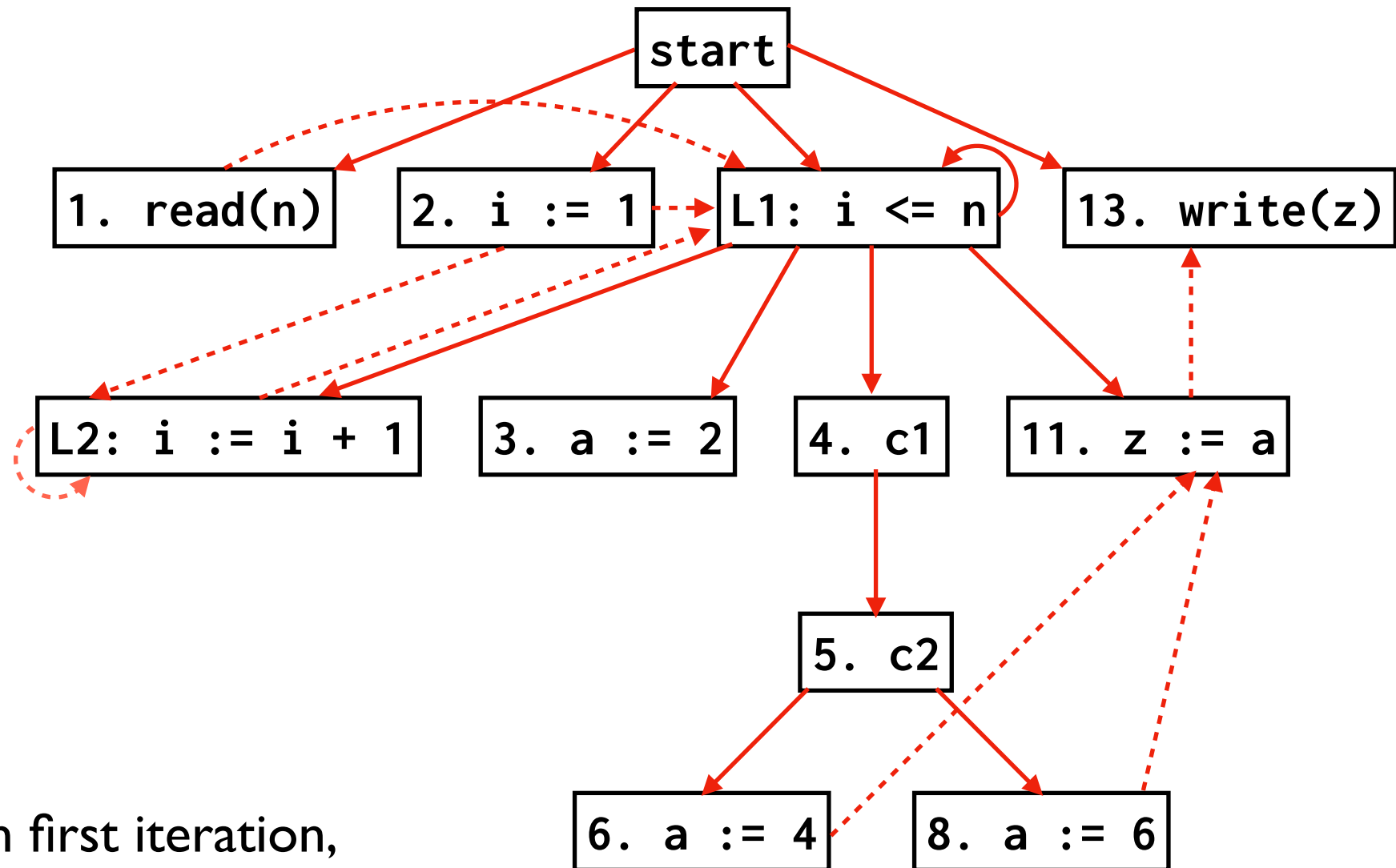
Execution history is:

1, 2, L1, 3, 4, 5, 8, 11, L2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion $\langle 13, z \rangle$

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```



Example 5

Input n is 2; $c1$ true, $c2$ false on first iteration,
 $c1$, $c2$ true on second iteration

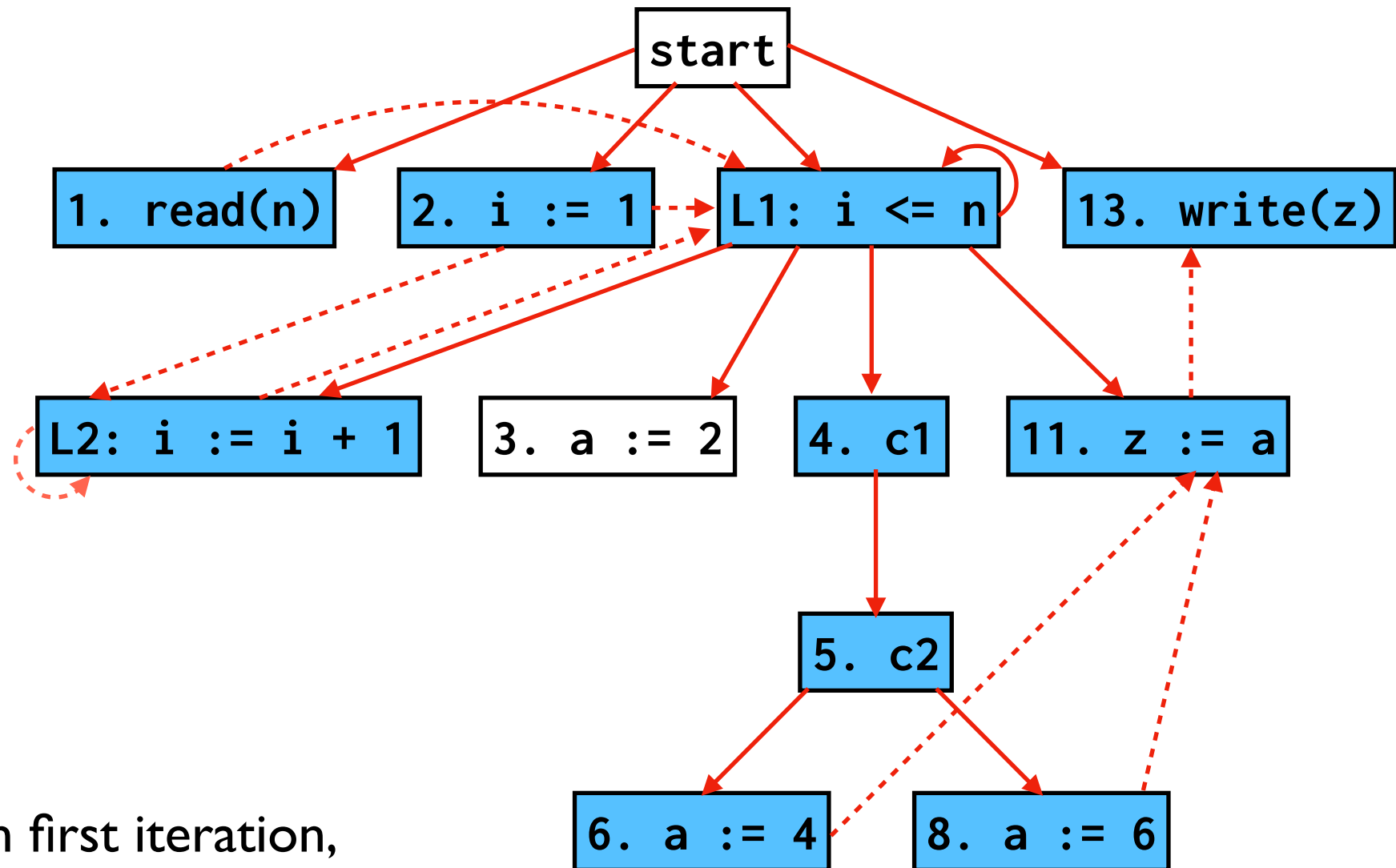
Execution history is:

1, 2, L1, 3, 4, 5, 8, 11, L2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion $\langle 13, z \rangle$

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```



Example 5

Input n is 2; $c1$ true, $c2$ false on first iteration,
 $c1$, $c2$ true on second iteration

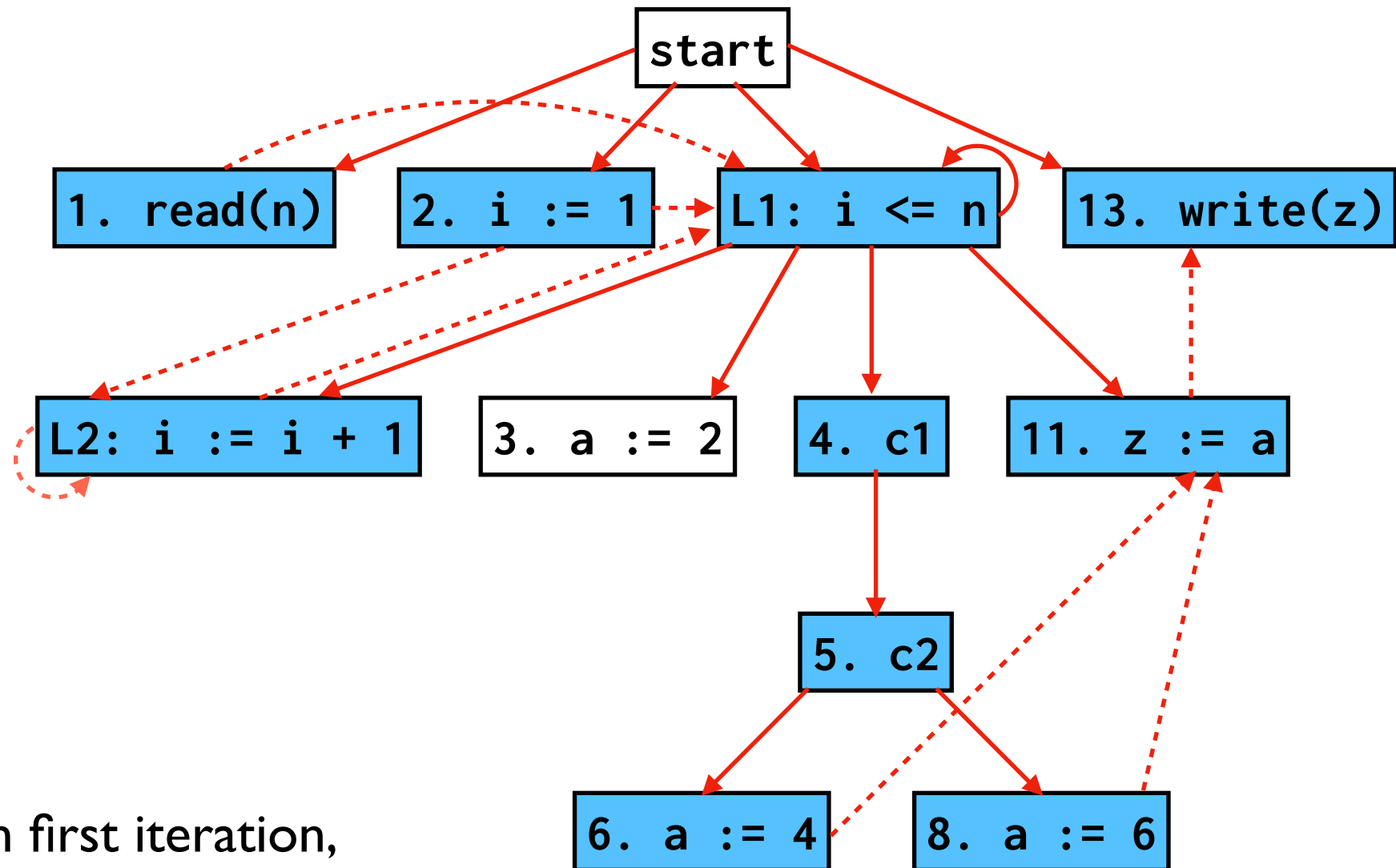
Execution history is:

1, 2, L1, 3, 4, 5, 8, 11, L2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Slice includes all nodes reachable
in reduced PDG: 1, 2, 4, 5, 6, 8, 11, 13

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```



Example 5

Input n is 2; $c1$ true, $c2$ false on first iteration,
 $c1$, $c2$ true on second iteration

Execution history is:

1, 2, L1, 3, 4, 5, 8, 11, L2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Because loop iterations are collapsed, both nodes 6 and 8 are in slice but only one can actually reach node 11 during execution

Dynamic Slicing

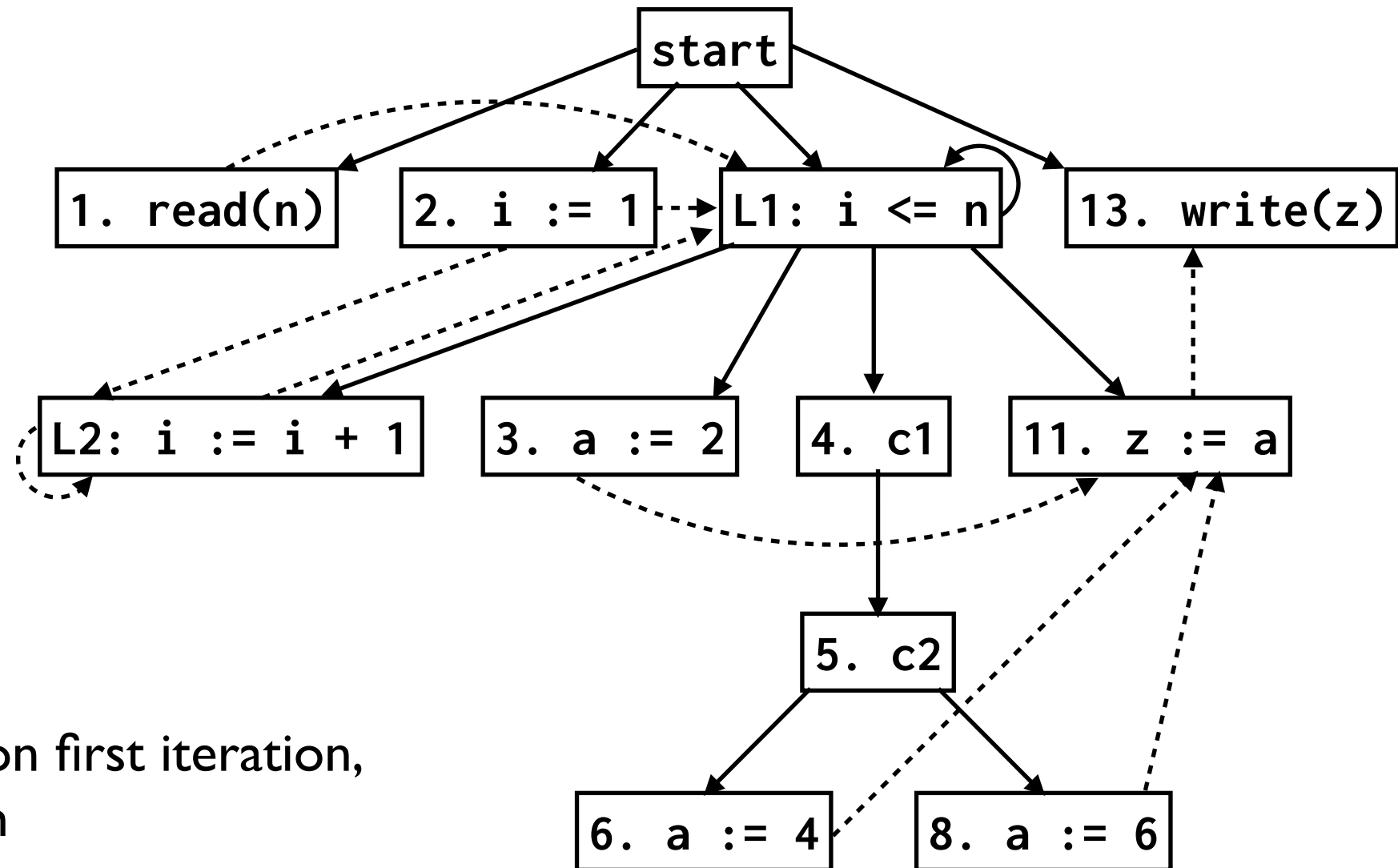
- **Create dynamic dependence graph (DDG)**
- **"Static" slice on the DDG**

Dynamic Dependence Graph

- Create a separate node for each occurrence of a statement in the execution history with dependence edges to only those statements on which this statement occurrence is dependent
- Every node in the DDG has at most one outgoing edge for each variable used in a statement

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```



Example 6

Input: n is 2; c1 true, c2 false on first iteration,
c1, c2 true on second iteration

Execution history: 1, 2, L1, 3, 4, 5, 8, 11, L2, L1,
3, 4, 5, 6, 11, L2, L1, 13

Criterion: <13, z>

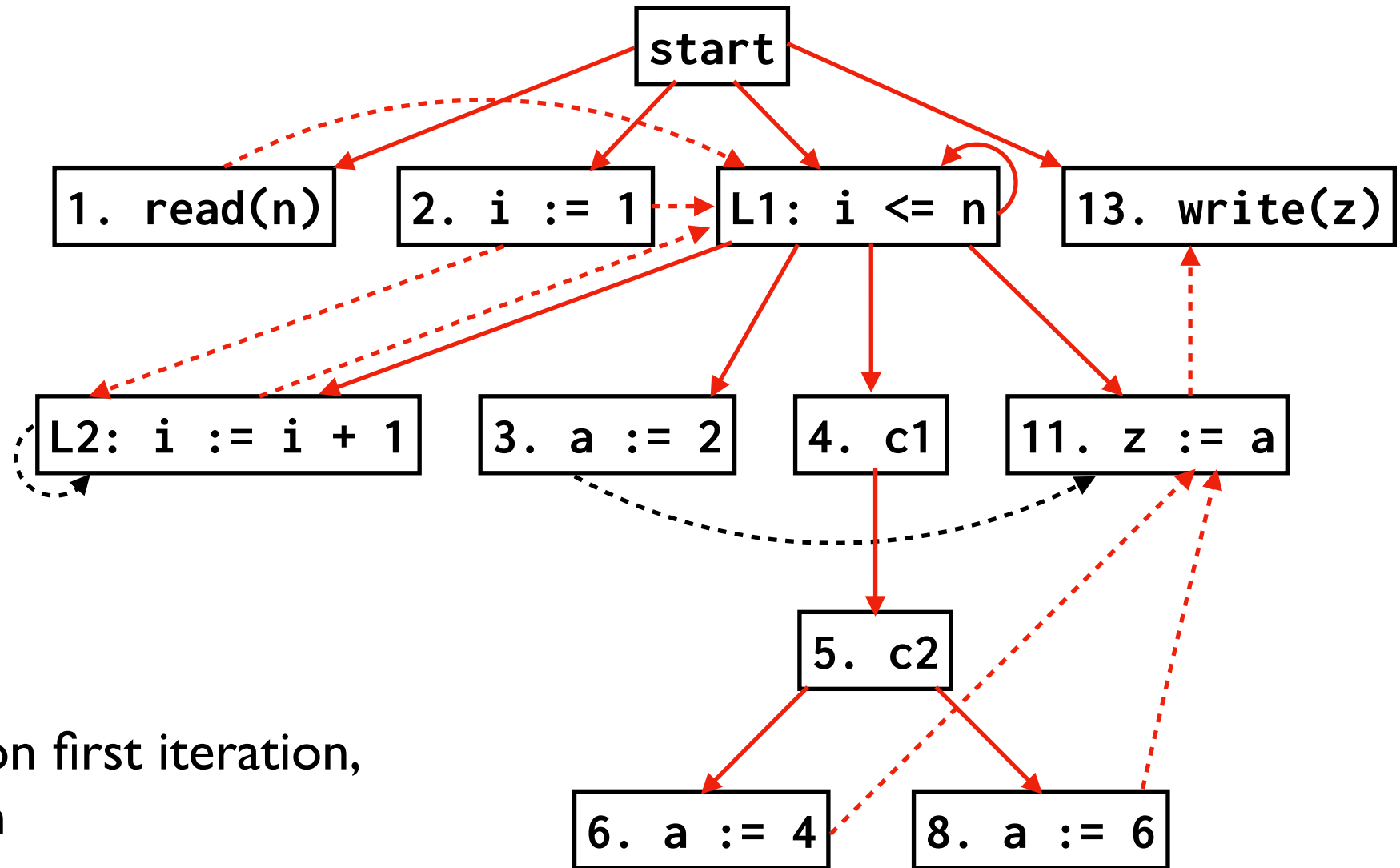
Mark **edges** in PDG that are
executed → **reduced PDG**

Dynamic Slicing Dependence Graphs

```

1.      read(n)
2.      for i := 1 to n do
3.          a := 2
4.          if c1 then
5.              if c2 then
6.                  a := 4
7.              else
8.                  a := 6
9.              endif
10.         endif
11.         z := a
12.     endfor
13.     write(z)

```



Example 6

Input: n is 2; c1 true, c2 false on first iteration,
c1, c2 true on second iteration

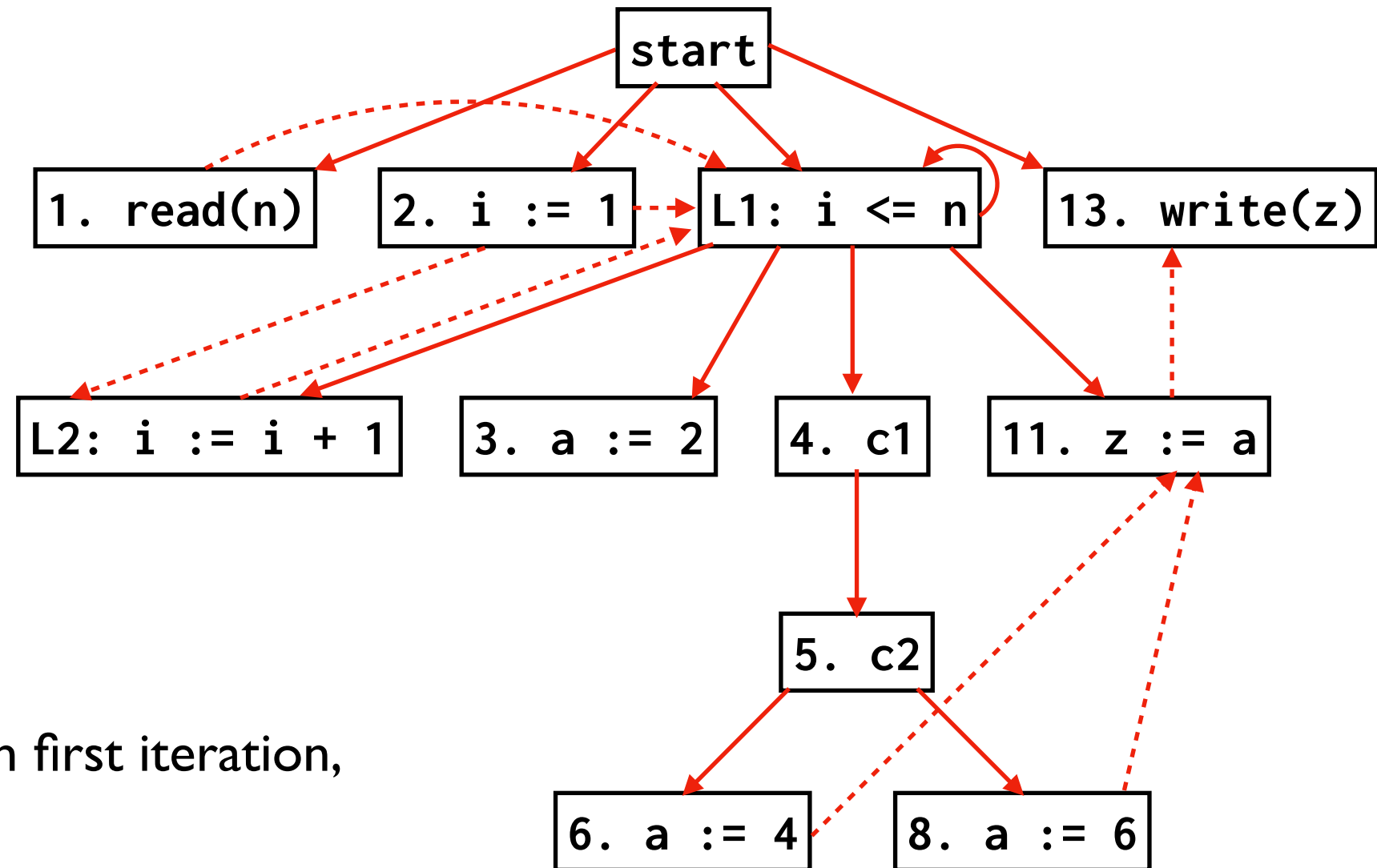
Execution history: I, 2, LI, 3, 4, 5, 8, II, L2, LI, 3, 4, 5, 6, II, L2, LI, I3

Criterion: $\langle 13, z \rangle$

Mark edges in PDG that are executed → reduced PDG

Dynamic Slicing Dependence Graphs

```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```



Example 6

Input: n is 2; c1 true, c2 false on first iteration,
c1, c2 true on second iteration

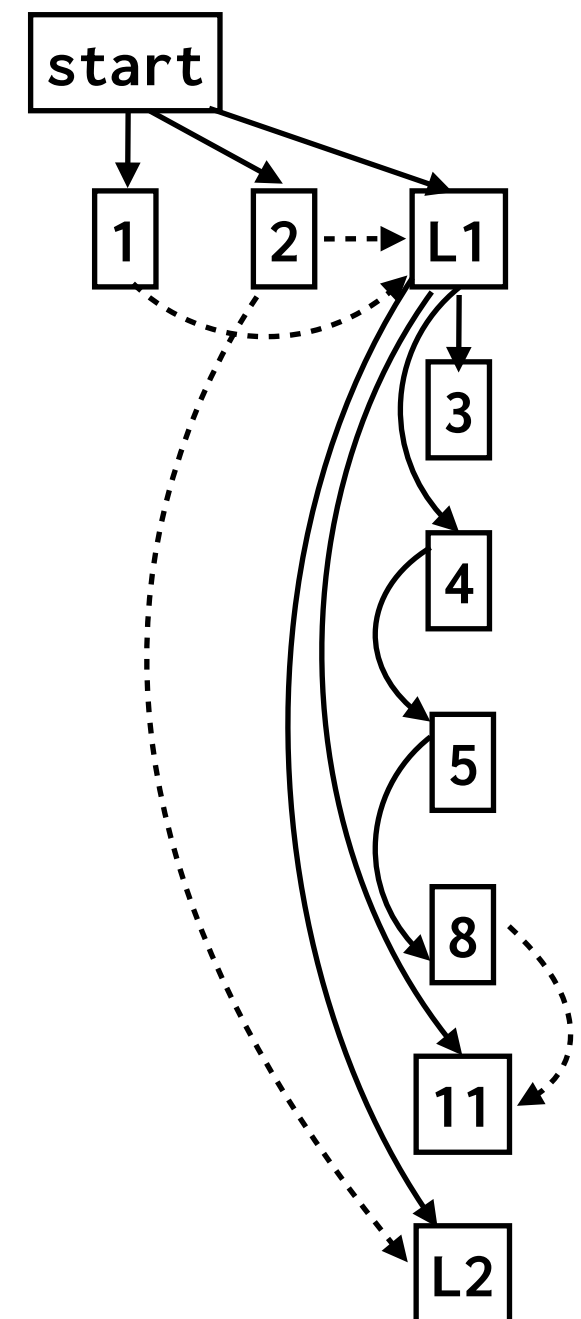
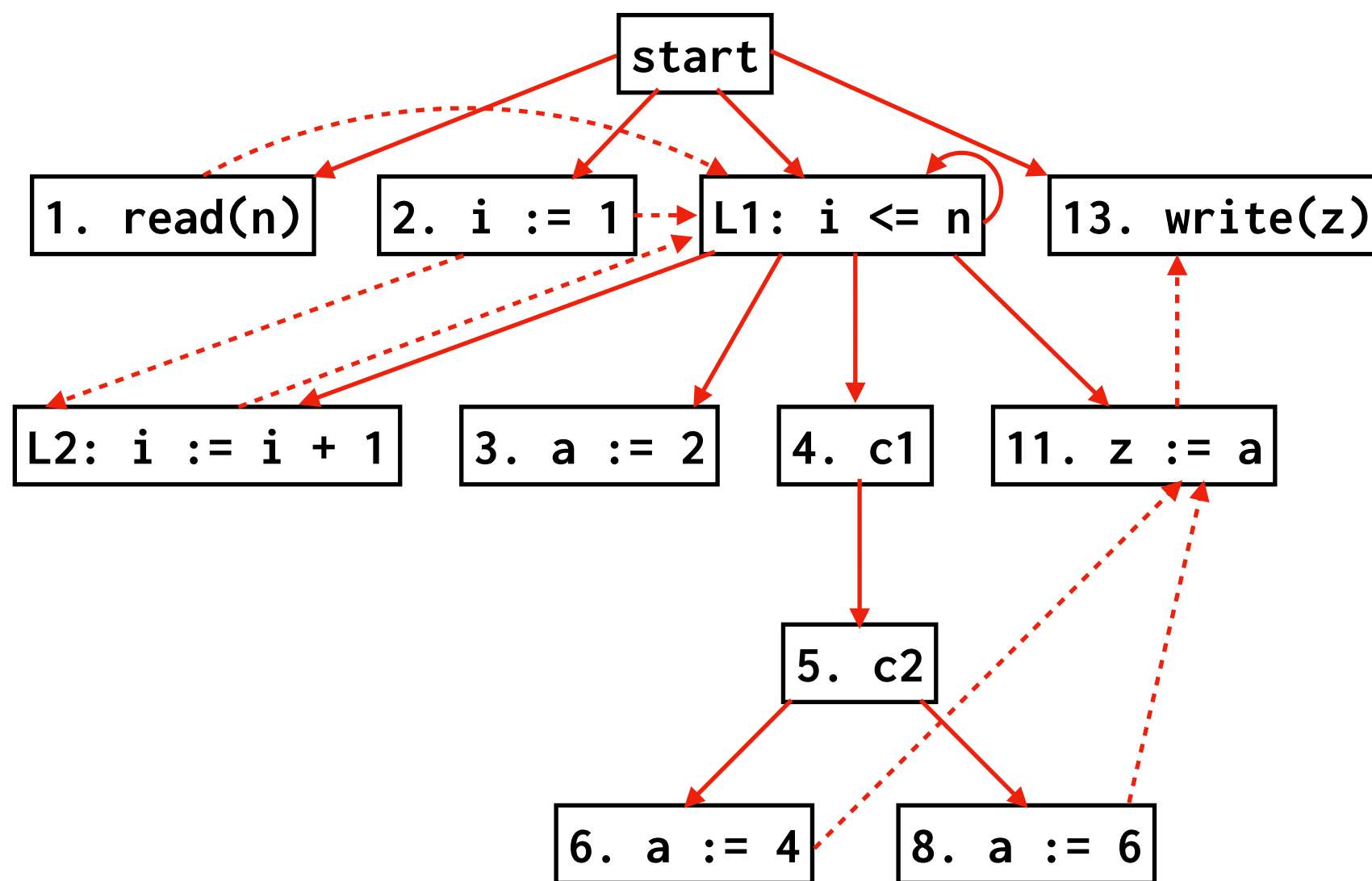
Execution history: 1, 2, L1, 3, 4, 5, 8, 11, L2, L1,
3, 4, 5, 6, 11, L2, L1, 13

Criterion: <13, z>

Mark edges in PDG that are
executed → reduced PDG

Dynamic Slicing Dependence Graphs

Create a separate node for each occurrence of a statement in the execution history, with outgoing dependence edges to only those statements on which this statement occurrence is dependent



I, 2, L1, 3, 4, 5, 8, 11, L2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Dynamic Slicing Dependence Graphs

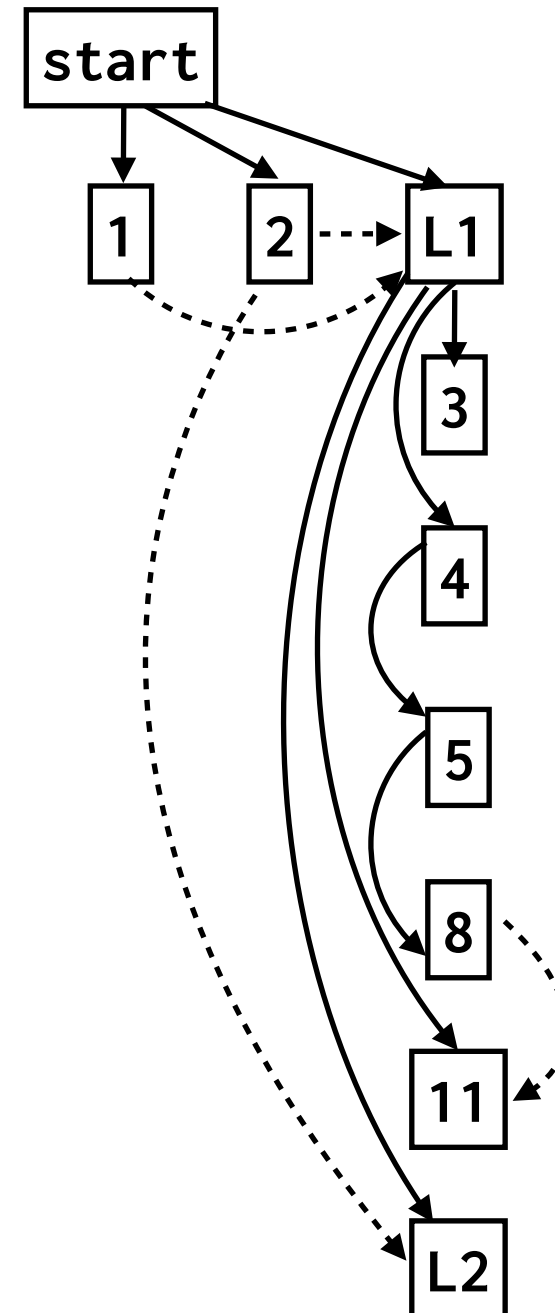
```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

Example 6

Input: n is 2; c1 true, c2 false on first iteration,
c1, c2 true on second iteration

Execution history: 1, 2, L1, 3, 4, 5, 8, 11, L2, L1,
3, 4, 5, 6, 11, L2, L1, 13

Criterion: <13, z>



Dynamic Slicing Dependence Graphs

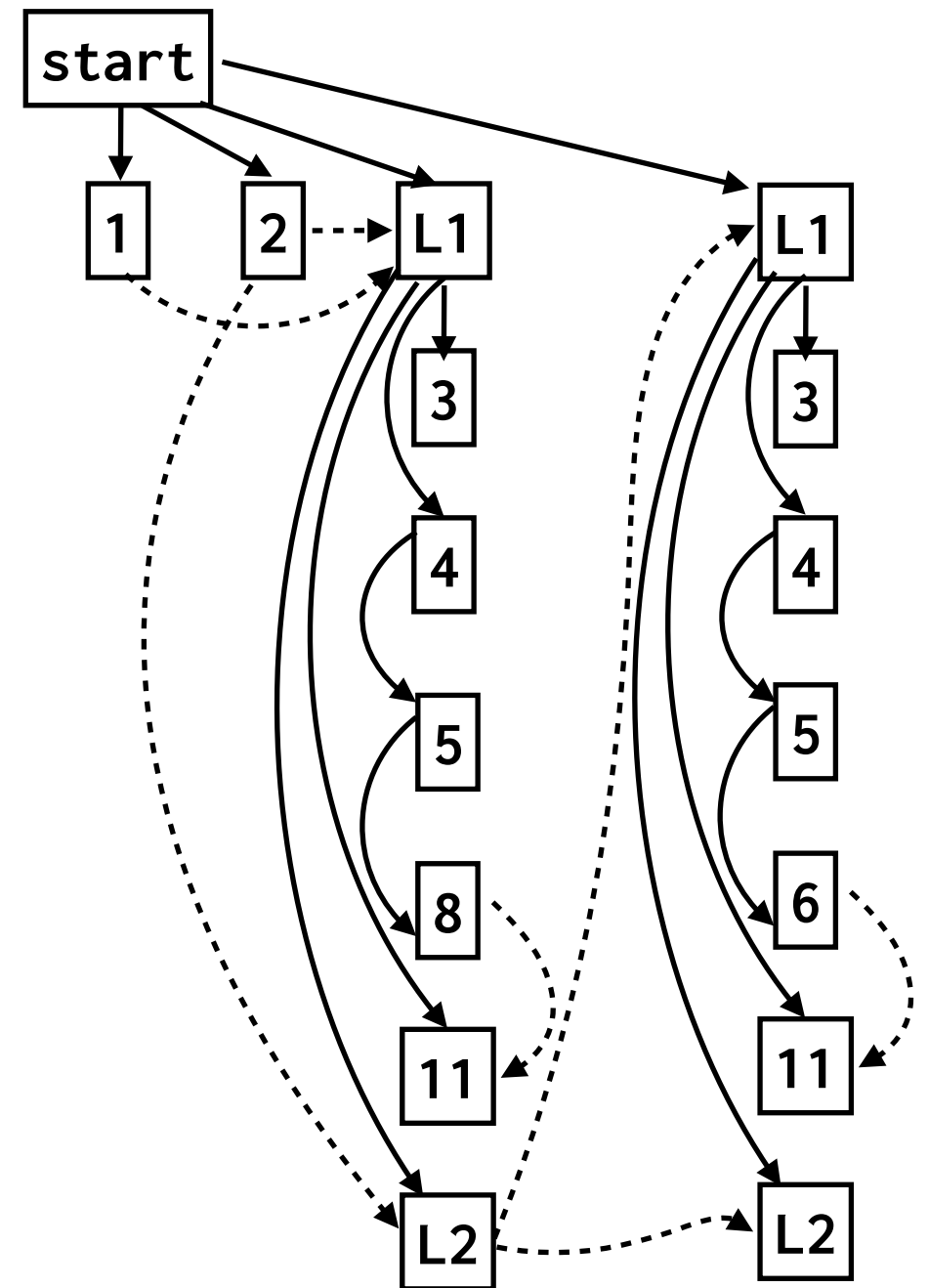
```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

Example 6

Input: n is 2; c1 true, c2 false on first iteration,
c1, c2 true on second iteration

Execution history: 1, 2, L1, 3, 4, 5, 8, 11, L2, L1,
3, 4, 5, 6, 11, L2, L1, 13

Criterion: <13, z>



Dynamic Slicing Dependence Graphs

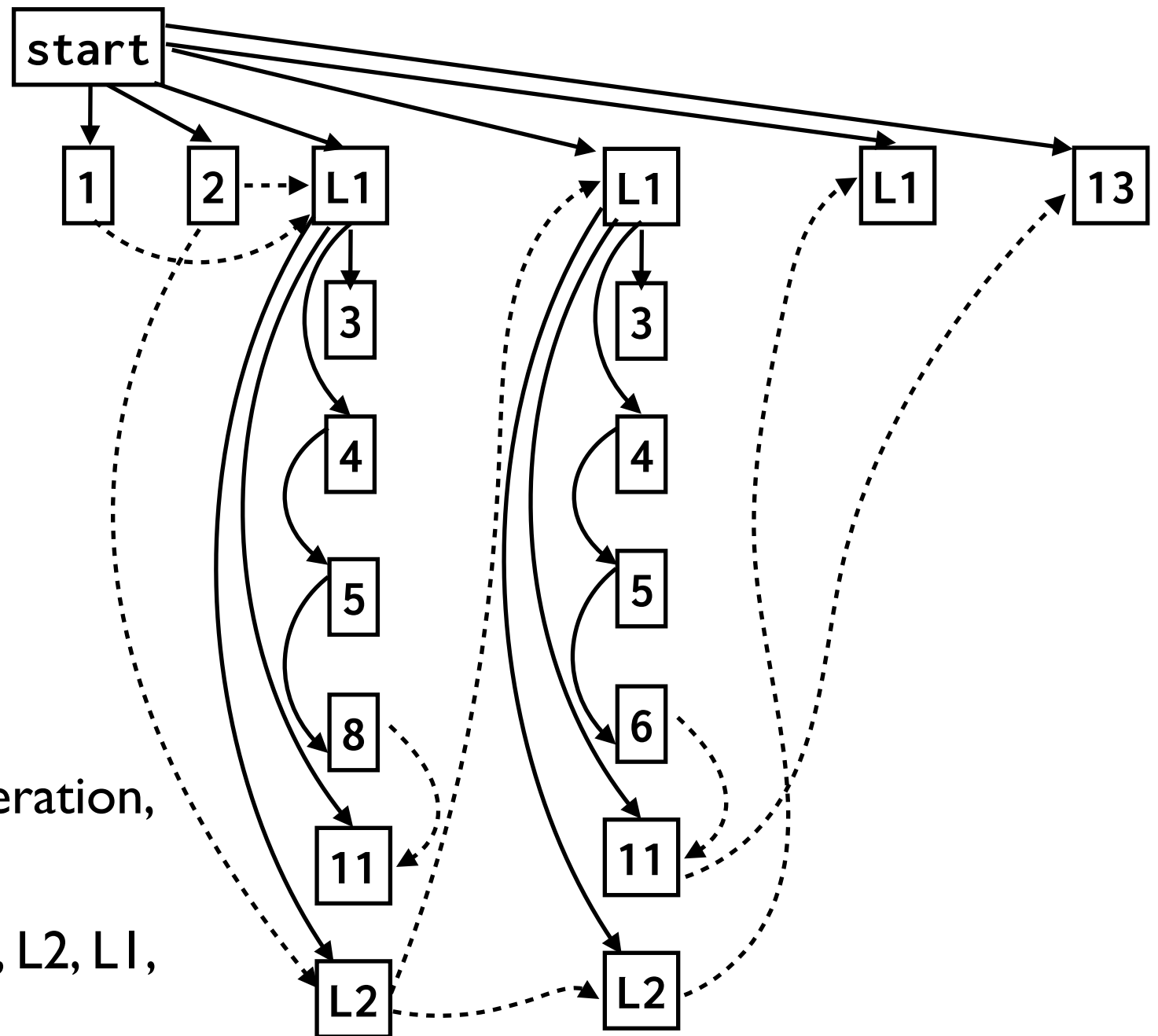
```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

Example 6

Input: n is 2; c1 true, c2 false on first iteration,
c1, c2 true on second iteration

Execution history: 1, 2, L1, 3, 4, 5, 8, 11, L2, L1,
3, 4, 5, 6, 11, L2, L1, 13

Criterion: <13, z>



Dynamic Slicing Dependence Graphs

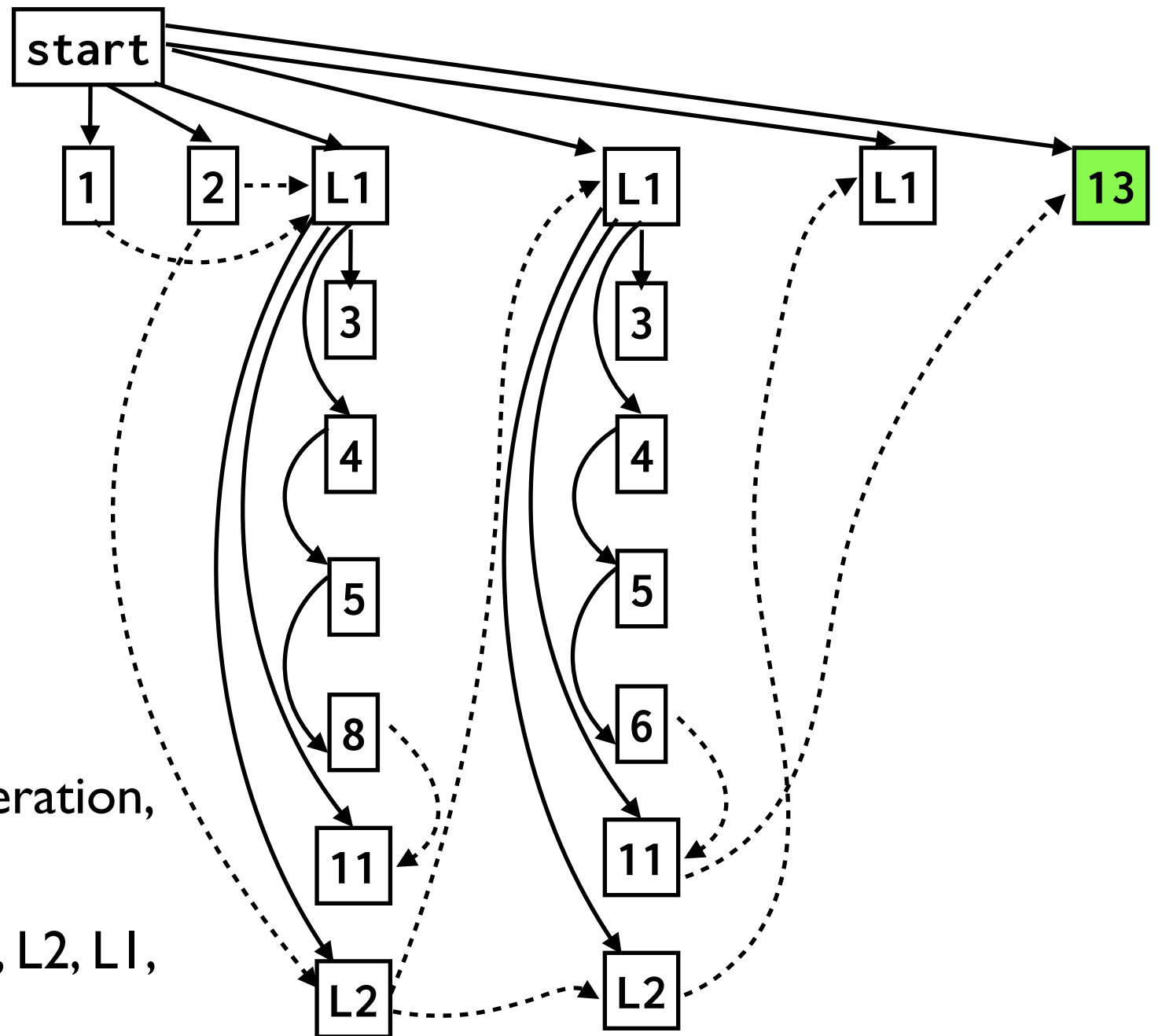
```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

Example 6

Input: n is 2; c1 true, c2 false on first iteration,
c1, c2 true on second iteration

Execution history: 1, 2, L1, 3, 4, 5, 8, 11, L2, L1,
3, 4, 5, 6, 11, L2, L1, 13

Criterion: <13, z>



Dynamic Slicing Dependence Graphs

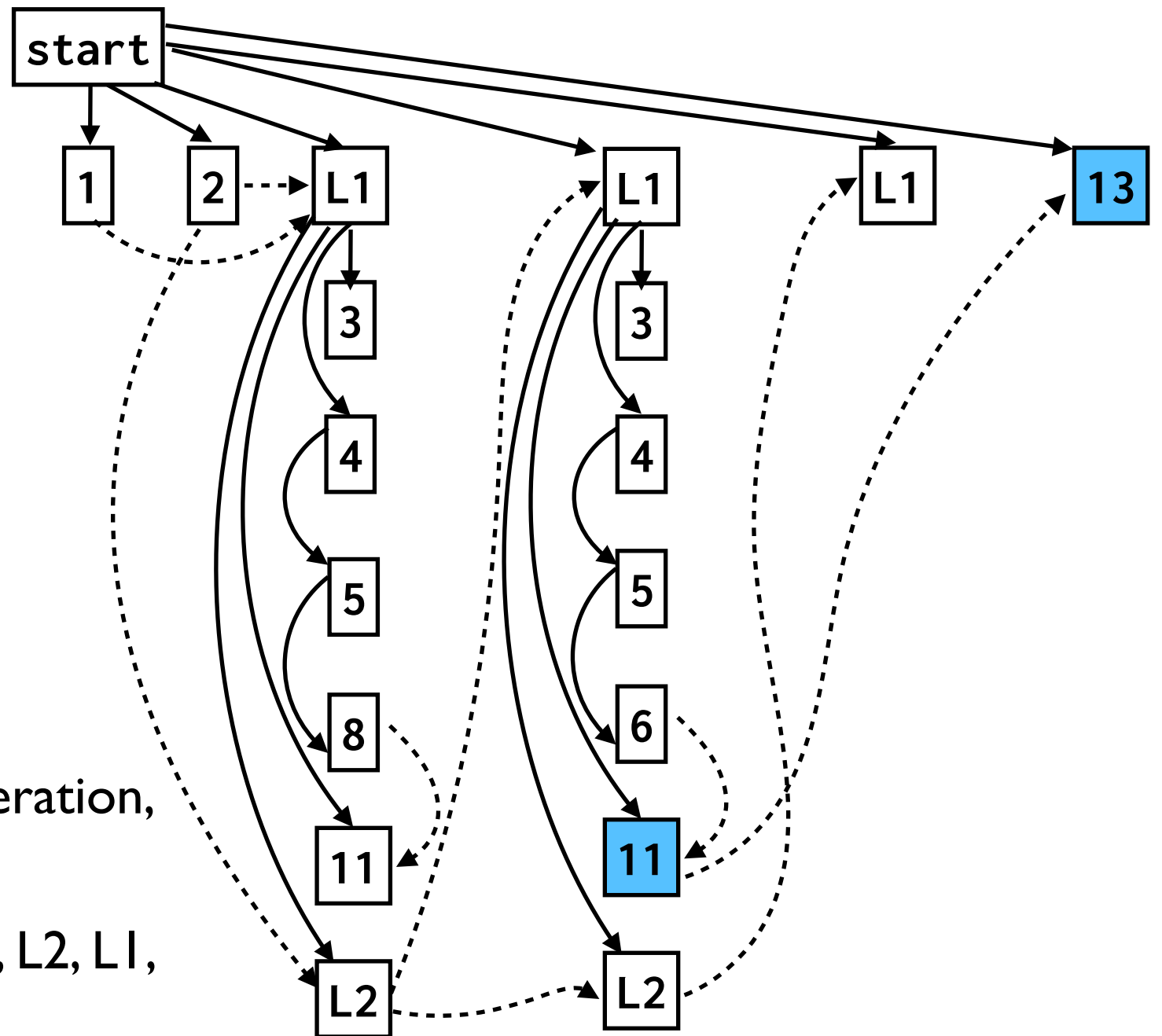
```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

Example 6

Input: n is 2; c1 true, c2 false on first iteration,
c1, c2 true on second iteration

Execution history: 1, 2, L1, 3, 4, 5, 8, 11, L2, L1,
3, 4, 5, 6, 11, L2, L1, 13

Criterion: <13, z>



Dynamic Slicing Dependence Graphs

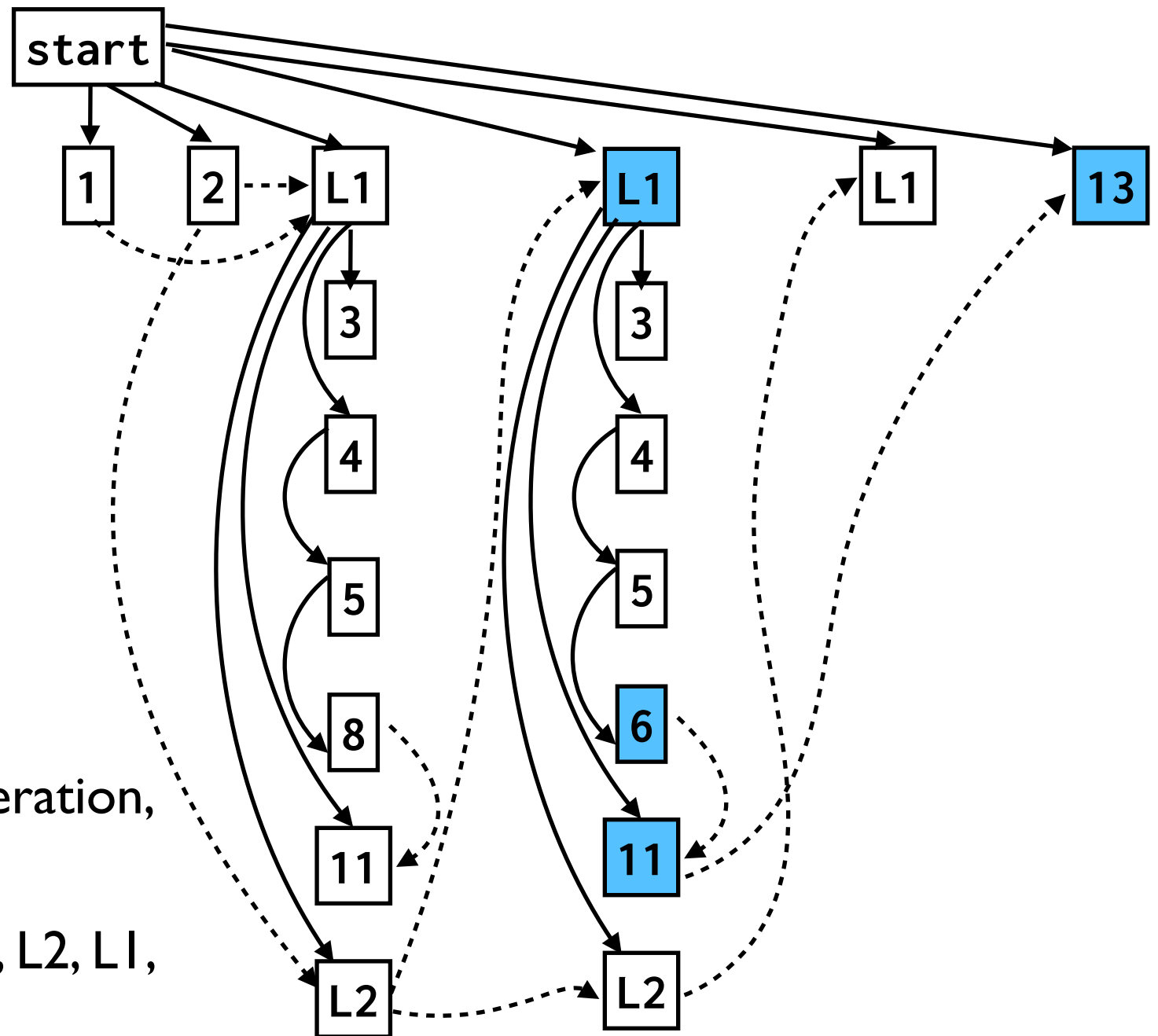
```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

Example 6

Input: n is 2; c1 true, c2 false on first iteration,
c1, c2 true on second iteration

Execution history: 1, 2, L1, 3, 4, 5, 8, 11, L2, L1,
3, 4, 5, 6, 11, L2, L1, 13

Criterion: <13, z>



Dynamic Slicing Dependence Graphs

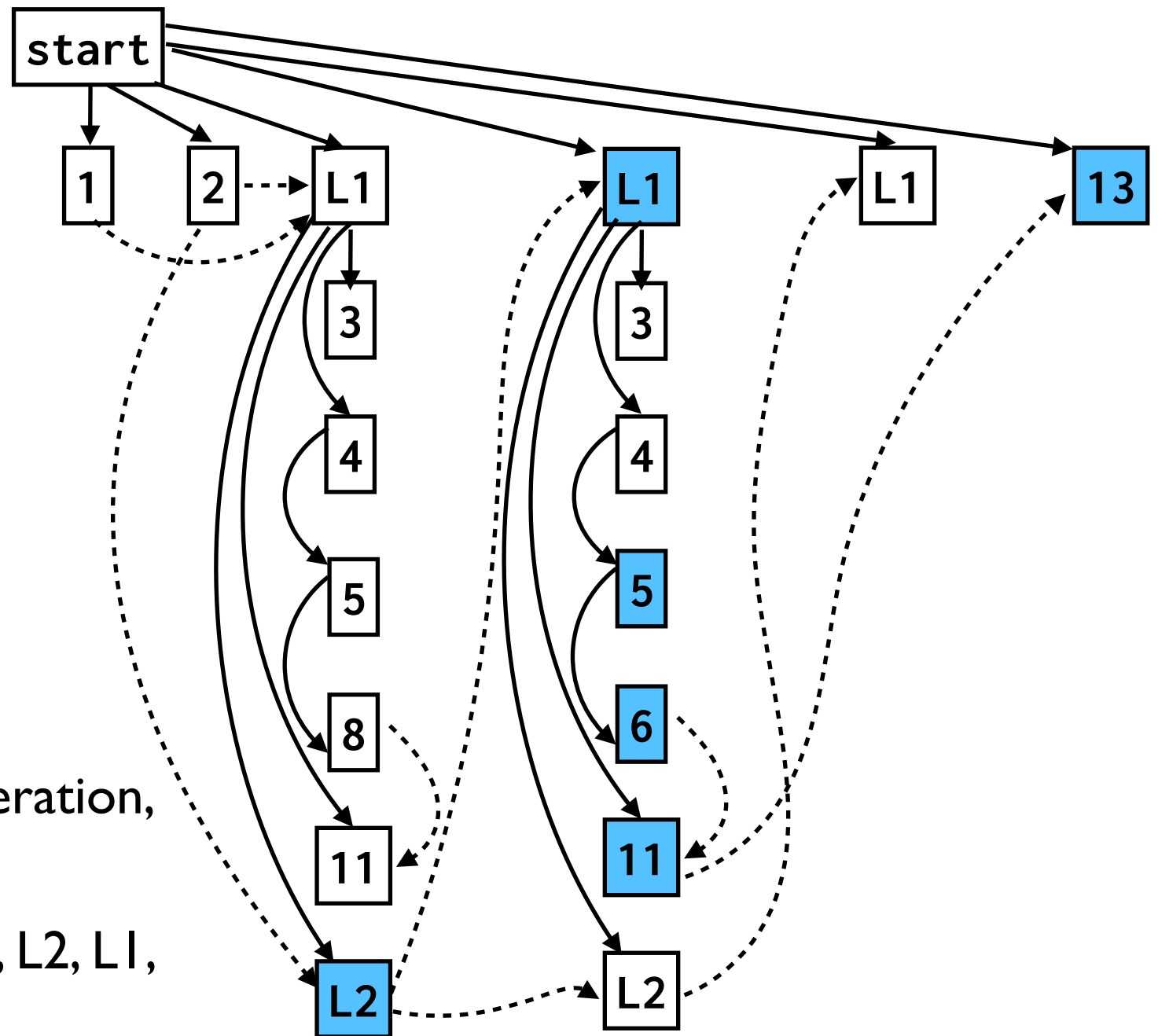
```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

Example 6

Input: n is 2; c1 true, c2 false on first iteration, c1, c2 true on second iteration

Execution history: 1, 2, L1, 3, 4, 5, 8, 11, L2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion: <13, z>



Dynamic Slicing Dependence Graphs

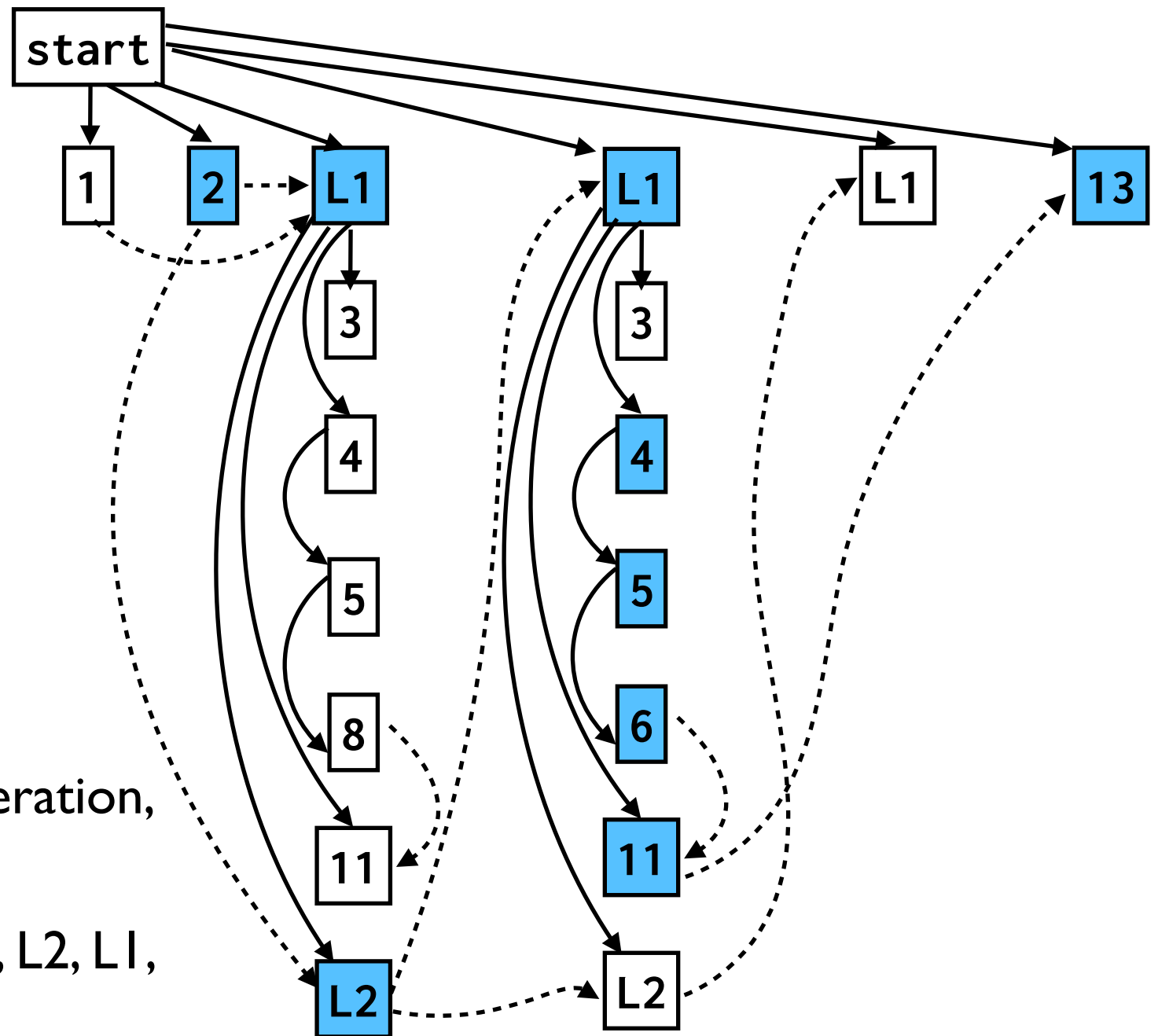
```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

Example 6

Input: n is 2; c1 true, c2 false on first iteration,
c1, c2 true on second iteration

Execution history: 1, 2, L1, 3, 4, 5, 8, 11, L2, L1,
3, 4, 5, 6, 11, L2, L1, 13

Criterion: <13, z>



Dynamic Slicing Dependence Graphs

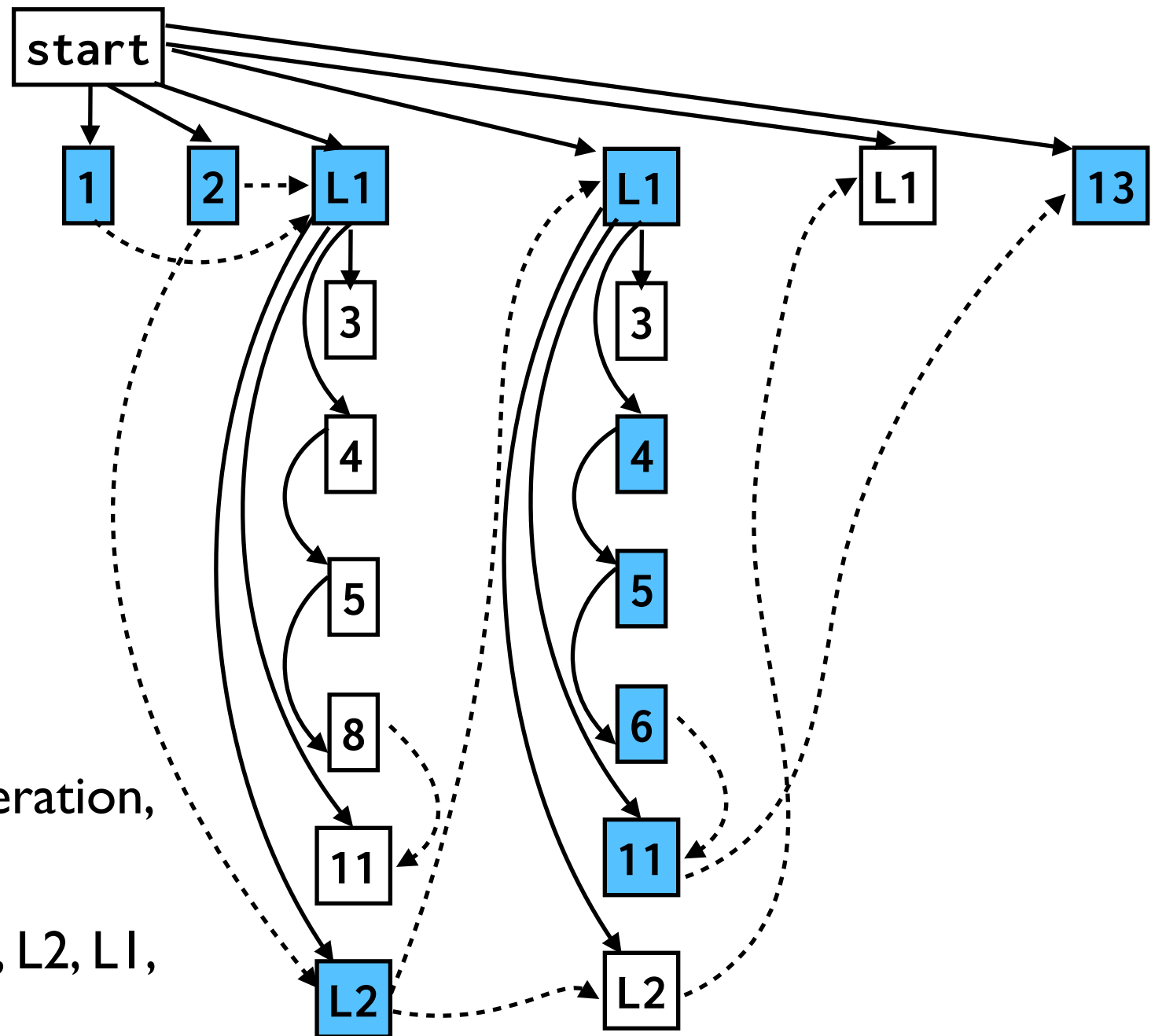
```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.   z := a
12. endfor
13. write(z)
```

Example 6

Input: n is 2; c1 true, c2 false on first iteration, c1, c2 true on second iteration

Execution history: 1, 2, L1, 3, 4, 5, 8, 11, L2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion: <13, z>



Dynamic Slicing Dependence Graphs

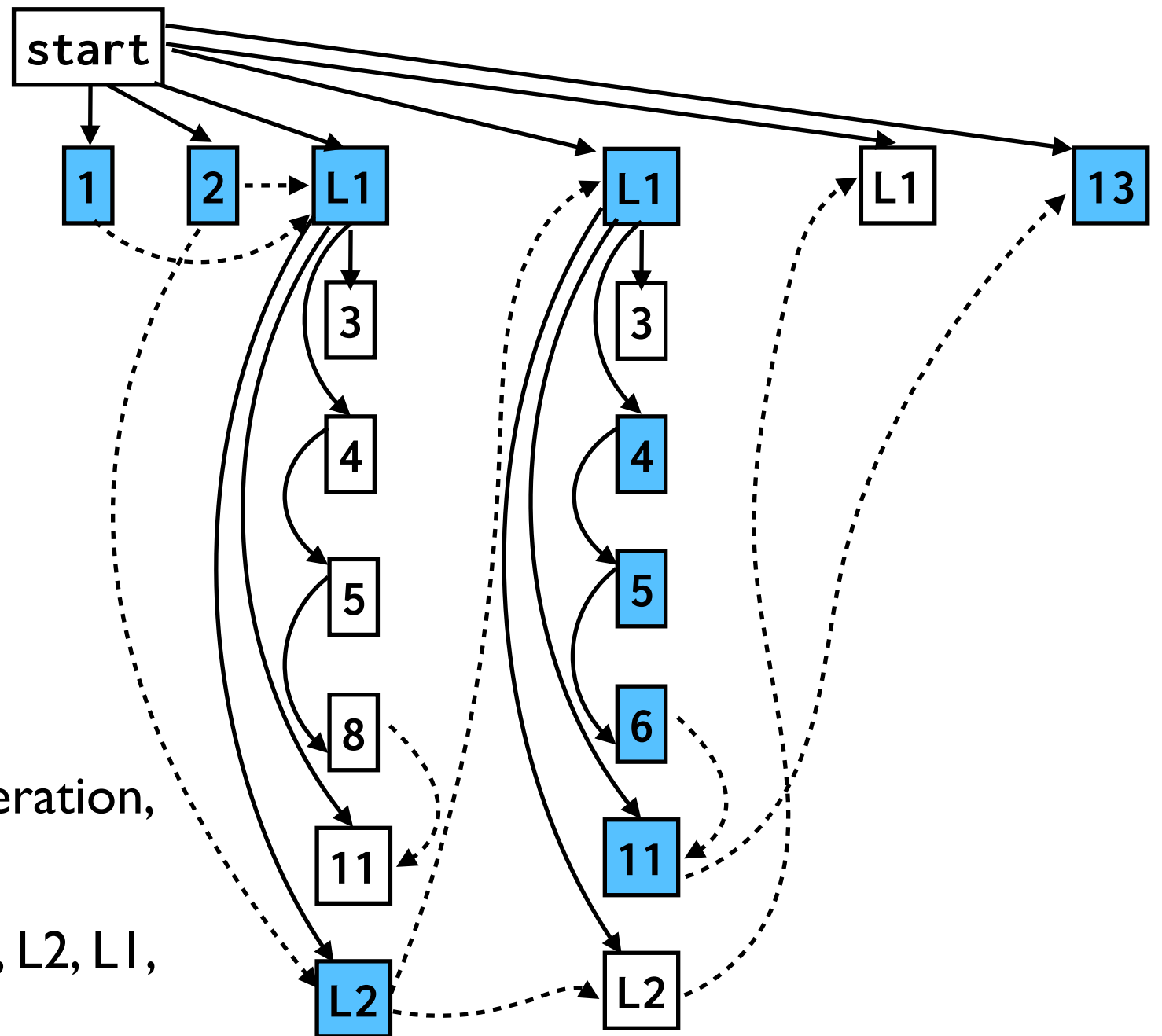
```
1.  read(n)
2.  for i := 1 to n do
3.    a := 2
4.    if c1 then
5.      if c2 then
6.        a := 4
7.      else
8.        a := 6
9.      endif
10.   endif
11.  z := a
12. endfor
13. write(z)
```

Example 6

Input: n is 2; c1 true, c2 false on first iteration, c1, c2 true on second iteration

Execution history: 1, 2, L1, 3, 4, 5, 8, 11, L2, L1, 3, 4, 5, 6, 11, L2, L1, 13

Criterion: <13, z>



Outline

1. Static Single Assignment Form
2. Slicing
3. Interprocedural Slicing
4. Dynamic Slicing